目录

第一章 线性方程组的求解 **3**

1.1 问题背景与数学模型 3

1.2 三角线性方程组 4

1.2.1 下三角方程组与前代法 4

1.2.2 上三角方程组与回代法 5

1.3 高斯消元法 6

1.3.1 方法思想 6

1.3.2 消元过程的代数描述 6

1.3.3 算法实现 7

1.3.4 主元与数值稳定性 8

1.3.5 算法复杂度 8

1.4 Gauss 消元法的矩阵表示 9

1.4.1 初等变换矩阵 9

1.4.2 消去矩阵 10

1.4.3 消元过程的矩阵表示 13

1.4.4 LU 分解的推导 14

1.4.5 LU 分解的时间复杂度 15

1.4.6 直接 LU 分解 16

1.5 带主元的 LU 分解 19

1.5.1 LU 分解的可行性条件 19

1.5.2 零主元与行置换的必要性 20

1.5.3 主元选择与数值稳定性：小主元的危害 21

1.5.4 部分主元 LU 分解：$\boldsymbol{PA} = \boldsymbol{LU}$ 22

1.5.5 部分主元 LU 分解的算法实现 26

1.5.6 求解线性方程组 $\boldsymbol{Ax} = \boldsymbol{b}$ 27

1.5.7 完全主元法 28

1.6 对称正定矩阵的 Cholesky 分解 29

1.6.1 对称矩阵的 $\mathrm{LDL}^{\mathrm{T}}$ 分解 29

1.6.2 Cholesky 分解：存在性与唯一性 30

1.6.3 Cholesky 算法 31

1.6.4 改进的平方根法（$\mathrm{LDL}^{\mathrm{T}}$ 分解） 33

1.6.5 Cholesky 分解的实现 33

1.6.6 应用：求解线性方程组 $\boldsymbol{Ax} = \boldsymbol{b}$ 33

第二章 最小二乘问题的求解 **35**
2.1 最小二乘问题的一般表述 . . . 35
2.1.1 问题背景与数学模型 . . . 35
2.1.2 矩阵形式与理论框架 . . . 36
2.1.3 法方程理论及其几何解释 . . . 37
2.2 法方程方法的算法实现与应用 . . . 39
2.2.1 算法描述与数值稳定性 . . . 39
2.2.2 应用实例 . . . 40

第三章 迭代方法 **43**
3.1 迭代解法的基本理论 . . . 43
3.1.1 基本概念 . . . 43
3.1.2 收敛性理论 . . . 46
3.2 经典迭代法：Jacobi 与 Gauss-Seidel . . . 52
3.2.1 矩阵分裂 . . . 52
3.2.2 Jacobi 迭代法 . . . 53
3.2.3 Gauss-Seidel 迭代法 . . . 55
3.3 收敛性判定与特殊矩阵 . . . 57
3.3.1 严格对角占优矩阵 . . . 57
3.3.2 对称正定矩阵 . . . 60
3.4 单变量非线性方程的求解 . . . 63
3.4.1 二分法 . . . 63
3.4.2 不动点迭代法 . . . 66
3.4.3 Newton 方法 . . . 70
3.4.4 割线法 . . . 74
3.5 非线性方程组的求解 . . . 81
3.5.1 问题背景与数学模型 . . . 81
3.5.2 两个非线性方程的 Newton 方法 . . . 82
3.5.3 不动点方法 . . . 85
3.5.4 非线性方程组的 Newton 方法 . . . 88

第四章 插值法 **91**
4.1 引言 . . . 91
4.1.1 函数的逼近 . . . 91
4.1.2 插值问题的表述 . . . 92
4.2 多项式插值 . . . 93
4.2.1 Lagrange 基中的插值多项式 . . . 93
4.2.2 插值误差的估计 . . . 95
4.2.3 Newton 基中的插值多项式 . . . 97
4.2.4 差商 . . . 97
4.2.5 差商的实际计算 . . . 101
4.2.6 Horner 算法 . . . 102
4.2.7 插值多项式的收敛性 . . . 104
4.3 三次样条插值 * . . . 105
4.3.1 问题的提出 . . . 105
4.3.2 三次样条插值的定义 . . . 106
4.3.3 三次样条插值的构造方法 . . . 107

第五章 积分 **111**
5.1 数值积分方法的动机和原理 . . . 111
5.1.1 问题的提出 . . . 111
5.1.2 数值积分公式的一般形式 . . . 112
5.2 使用插值多项式的方法 . . . 113
5.2.1 方法的原理 . . . 113
5.2.2 参考区间 . . . 114
5.2.3 求积误差的上界与阶数 . . . 115
5.2.4 Newton-Cotes 公式 . . . 115
5.2.5 复合数值积分 . . . 120
5.3 求积误差的先验和后验估计 . . . 121
5.3.1 误差估计的基本思想 . . . 121

第六章 微分方程 **123**
6.1 一般求解原理 . . . 124
6.1.1 微分方程理论要素 . . . 124
6.1.2 数值方法的原理 . . . 128
6.2 单步格式 . . . 129
6.2.1 从数值积分出发的 Euler 格式 . . . 129
6.2.2 单步预测-校正格式 . . . 131
6.2.3 Euler 方法的初步研究：稳定性与误差传播 . . . 131
6.2.4 单步格式的阶和一致性 . . . 133
6.2.5 单步格式的稳定性和收敛性 . . . 135
6.2.6 Runge-Kutta 格式 . . . 137
6.3 多步格式 . . . 139
6.3.1 Adams-Bashforth 格式 . . . 139
6.3.2 Adams-Bashforth 格式的一些例子 . . . 143
6.3.3 Adams-Moulton 隐式格式 . . . 143
6.3.4 Adams-Moulton 格式的一些例子 . . . 144
6.4 Runge-Kutta 方法的理论基础与自适应步长 . . . 144
6.4.1 Runge-Kutta 方法的定义 . . . 144
6.4.2 Lipschitz 条件与解的存在唯一性 . . . 145

第七章 特征值和特征向量的确定 **149**
7.1 回顾和记号 . . . 149
7.1.1 问题陈述 . . . 149
7.1.2 记号约定 . . . 150
7.1.3 特征值计算与多项式求根的关系 . . . 150
7.2 幂迭代方法 . . . 151
7.2.1 方法陈述 . . . 151
7.2.2 假设条件 . . . 151
7.2.3 收敛性定理 . . . 152
7.2.4 收敛性证明 . . . 152
7.2.5 收敛速度与注意事项 . . . 153
7.3 逆幂迭代方法 . . . 154
7.3.1 方法原理 . . . 154
7.3.2 算法描述 . . . 155

7.3.3 收敛性定理 155
7.4 收缩方法 156
7.4.1 对称矩阵的情况 156
7.4.2 一般矩阵的情况：Duncan 和 Collar 方法 157
7.5 方法总结与比较 158
7.5.1 方法特点总结 158
7.5.2 实际应用建议 158
7.5.3 数值稳定性考虑 159

Chapter 1

线性方程组的求解

线性方程组的数值解法是数值分析的核心内容之一，也是后续数值方法（如最小二乘问题、特征值计算以及偏微分方程的数值离散）的基础。本章在有限精度计算的背景下，系统介绍线性方程组的直接解法，重点讨论高斯消元法的思想、算法结构及其数值特性。

本章研究有限维线性方程组的数值求解问题。线性方程组在科学与工程建模中处于基础地位，许多看似不同的数值算法（最小二乘拟合、特征值计算、有限元离散等）在实现层面都要反复求解线性系统。本章将依次讨论：三角线性方程组及前代 / 回代方法；高斯消元及其矩阵表达；不带主元与带主元的 LU 分解；对称（正定）矩阵的专门分解（Cholesky 与 $\mathrm{LDL}^{\mathrm{T}}$ 分解）。本章给出的分解理论和算法将作为全书的线性代数基础，在插值、最小二乘、特征值与偏微分方程数值解等章节中多次被调用。

1.1 问题背景与数学模型

本节的目标是从数值计算的角度重新表述线性方程组问题，明确本章将要处理的数学对象及其基本性质，并区分“理论上可解”与“数值上可行”之间的差异。

考虑一般形式的线性方程组

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}, \quad \boldsymbol{A} \in \mathbb{R}^{n\times n},\ \boldsymbol{b} \in \mathbb{R}^{n} \tag{1.1}$$

其中 $\boldsymbol{A}$ 为系数矩阵，$\boldsymbol{b}$ 为已知向量，$\boldsymbol{x}$ 为待求解的未知向量。

在线性代数理论中，矩阵 $\boldsymbol{A}$ 可逆是该方程组存在唯一解的充分必要条件。然而在数值计算中，问题并不仅限于“是否存在解”，而更关注以下因素：

- 算法在有限精度下的稳定性；
- 舍入误差在计算过程中的传播与放大；
- 随着问题规模增大，算法的计算复杂度与存储需求。

这些问题构成数值线性代数研究的核心。后续各节将围绕这些问题，逐步引入适合计算机实现的分解方法与误差分析工具。

1.2　三角线性方程组

本节的目标是说明三角线性方程组在数值线性代数中的地位，并给出前代与回代两种基本求解过程，为后续高斯消元和 LU 分解提供“终点形式”的参照。

在介绍一般线性方程组的求解方法之前，先讨论一类具有特殊结构、但在数值算法中极其重要的方程组——三角线性方程组。三角方程组之所以在数值计算中占据核心地位，是因为许多复杂算法（如 Gauss 消元法、LU 分解等）最终都可归结为“构造一个三角系统并通过前代或回代求解”。此外，三角方程组的求解过程简单直接，计算复杂度仅为 $\mathscr{O}(n^2)$，远低于一般线性方程组的 $\mathscr{O}(n^3)$。

1.2.1　下三角方程组与前代法

定义 1.2.1 (零矩阵). 设 $\boldsymbol{O}=(o_{ij})\in\mathbb{R}^{m\times n}$，若对所有 $i=1,2,\ldots,m$ 和 $j=1,2,\ldots,n$ 都有 $o_{ij}=0$，则称 $\boldsymbol{O}$ 为零矩阵，记作 $\boldsymbol{O}$ 或 $\mathbf{0}$。特别地，当 $m=n$ 时，称为 n 阶零方阵。

定义 1.2.2 (下三角矩阵). 设 $\boldsymbol{L}=(\ell_{ij})\in\mathbb{R}^{n\times n}$，若当 $i<j$ 时 $\ell_{ij}=0$，则称 $\boldsymbol{L}$ 为下三角矩阵。

下三角方程组的系数矩阵具有如下形式：

$$\boldsymbol{L}=\begin{pmatrix}\ell_{11} & 0 & \cdots & 0\\ \ell_{21} & \ell_{22} & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{nn}\end{pmatrix},$$

对应的线性方程组为

$$\boldsymbol{Lx}=\boldsymbol{b}.$$

由于第 i 个方程中仅包含未知量 $x_1,\ldots,x_i$，在假设 $\ell_{ii}\neq 0$ 的前提下，可按顺序逐个求解未知量。该过程称为**前代法**（forward substitution）。其基本思想是：第一个方程只包含 x_1，可直接求解；得到 x_1 后，第二个方程只包含 x_1 和 x_2，而 x_1 已知，因此可以求解 x_2；依此类推，可以按顺序逐个求解所有未知量。

定理 1.2.1 (前代公式). 若 $\boldsymbol{L}=(\ell_{ij})\in\mathbb{R}^{n\times n}$ 为对角元均非零的下三角矩阵，则方程组 $\boldsymbol{Lx}=\boldsymbol{b}$ 有唯一解，且解由递推公式

$$x_i=\frac{1}{\ell_{ii}}\left(b_i-\sum_{j=1}^{i-1}\ell_{ij}x_j\right),\quad i=1,2,\ldots,n \tag{1.2}$$

唯一确定。

证明. 由于 $\ell_{ii}\neq 0$（$i=1,2,\ldots,n$），矩阵 $\boldsymbol{L}$ 非奇异，因此方程组 $\boldsymbol{Lx}=\boldsymbol{b}$ 有唯一解。

对于 $i=1$，第一个方程为 $\ell_{11}x_1=b_1$，由于 $\ell_{11}\neq 0$，可得

$$x_1=\frac{b_1}{\ell_{11}}=\frac{1}{\ell_{11}}\left(b_1-\sum_{j=1}^{0}\ell_{1j}x_j\right)$$

其中约定空和为零，因此公式对 $i=1$ 成立。

假设公式对 $i=1,2,\ldots,k-1$ 成立，考虑第 k 个方程：

$$\ell_{k1}x_1+\ell_{k2}x_2+\cdots+\ell_{k,k-1}x_{k-1}+\ell_{kk}x_k=b_k$$

由于 $x_1,x_2,\ldots,x_{k-1}$ 已由前 $k-1$ 个方程确定，且 $\ell_{kk}\neq 0$，可得

$$x_k=\frac{1}{\ell_{kk}}\left(b_k-\sum_{j=1}^{k-1}\ell_{kj}x_j\right)$$

由数学归纳法，公式对所有 $i = 1, 2, \ldots, n$ 成立。 □

注. 前代法的计算复杂度为 $\mathscr{O}(n^2)$。具体而言，计算 x_i 需要 $i-1$ 次乘法和 $i-1$ 次减法，以及 1 次除法，因此总运算次数为

$$\sum_{i=1}^{n}(2(i-1)+1) = 2 \cdot \frac{n(n-1)}{2} + n = n^2$$

注意. 从数值分析的角度看，本节的结论可以概括为：一旦能够通过若干变换把一般线性方程组等价地化为三角系统，求解阶段的代价始终是 $\mathscr{O}(n^2)$，主要误差来源在前期的变换过程而非前代 / 回代本身。后续关于高斯消元和矩阵分解的方法，正是围绕“如何以数值稳定的方式构造这样的三角系统”这一问题展开的。

1.2.2 上三角方程组与回代法

定义 1.2.3 (上三角矩阵). 设 $\boldsymbol{U} = (u_{ij}) \in \mathbb{R}^{n\times n}$，若当 $i > j$ 时 $u_{ij} = 0$，则称 $\boldsymbol{U}$ 为上三角矩阵。

上三角方程组的系数矩阵为

$$\boldsymbol{U} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}.$$

对应的线性方程组为

$$\boldsymbol{U}\boldsymbol{x} = \boldsymbol{c}.$$

当 $u_{ii} \neq 0$ 时，未知量可自下而上依次求出，这一过程称为**回代法**（back substitution）。与前代法相反，回代法从最后一个方程开始，逆序求解未知量。

定理 1.2.2 (回代公式). 设 $\boldsymbol{U} = (u_{ij}) \in \mathbb{R}^{n\times n}$ 为对角元均非零的上三角矩阵，则方程组 $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{c}$ 有唯一解，且解满足

$$x_i = \frac{1}{u_{ii}}\left(c_i - \sum_{j=i+1}^{n} u_{ij}x_j\right), \quad i = n, n-1, \ldots, 1. \tag{1.3}$$

证明. 由于 $u_{ii} \neq 0$（$i = 1, 2, \ldots, n$），矩阵 $\boldsymbol{U}$ 非奇异，因此方程组 $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{c}$ 有唯一解。

对于 $i = n$，第 n 个方程为 $u_{nn}x_n = c_n$，由于 $u_{nn} \neq 0$，可得

$$x_n = \frac{c_n}{u_{nn}} = \frac{1}{u_{nn}}\left(c_n - \sum_{j=n+1}^{n} u_{nj}x_j\right)$$

其中约定空和为零，因此公式对 $i = n$ 成立。

假设公式对 $i = n, n-1, \ldots, k+1$ 成立，考虑第 k 个方程：

$$u_{kk}x_k + u_{k,k+1}x_{k+1} + \cdots + u_{kn}x_n = c_k$$

由于 $x_{k+1}, x_{k+2}, \ldots, x_n$ 已由后 $n-k$ 个方程确定，且 $u_{kk} \neq 0$，可得

$$x_k = \frac{1}{u_{kk}}\left(c_k - \sum_{j=k+1}^{n} u_{kj}x_j\right)$$

由数学归纳法（逆序），公式对所有 $i = n, n-1, \ldots, 1$ 成立。 □

注. 回代法的计算复杂度同样为 $\mathscr{O}(n^2)$。计算 x_i 需要 $n-i$ 次乘法和 $n-i$ 次减法，以及 1 次除法，因此总运算次数为

$$\sum_{i=1}^{n}(2(n-i)+1) = 2 \cdot \frac{n(n-1)}{2} + n = n^2$$

例 1.2.1. 求解上三角方程组

$$\begin{cases} 2x_1 + x_2 + x_3 = 4, \\ x_2 + x_3 = 3, \\ 2x_3 = -2. \end{cases}$$

解：从最后一个方程开始回代。

步骤 1：由第 3 个方程 $2x_3 = -2$，得

$$x_3 = \frac{-2}{2} = -1$$

步骤 2：将 $x_3 = -1$ 代入第 2 个方程 $x_2 + x_3 = 3$，得

$$x_2 = 3 - x_3 = 3 - (-1) = 4$$

步骤 3：将 $x_2 = 4$ 和 $x_3 = -1$ 代入第 1 个方程 $2x_1 + x_2 + x_3 = 4$，得

$$x_1 = \frac{1}{2}(4 - x_2 - x_3) = \frac{1}{2}(4 - 4 - (-1)) = \frac{1}{2}$$

因此，方程组的解为 $\boldsymbol{x} = (1/2, 4, -1)^T$。

1.3 高斯消元法

1.3.1 方法思想

本小节的目标是把前一节中“三角线性方程组易于求解”的结论，上升为对一般线性方程组的系统求解策略，并给出高斯消元法的基本构想。

高斯消元法（Gaussian Elimination）是一种经典的直接解法，其基本思想是：通过一系列等价的初等行变换，将一般线性方程组转化为上三角方程组，再利用回代法求解未知量。

从代数角度看，初等行变换不改变方程组的解集，因此消元过程是等价的；从算法角度看，其目标是系统性地消去未知量，使问题结构逐步简化。具体而言，在第 k 步消元中，我们选取第 k 行作为“参考行”，通过适当的线性组合，使得第 $k+1$ 行至第 n 行中第 k 列的元素全部变为零。重复这一过程 $n-1$ 次，最终得到一个等价的上三角线性方程组。

1.3.2 消元过程的代数描述

为便于描述消元过程，我们引入上标表示消元步骤。设 $\boldsymbol{A}^{(1)} = \boldsymbol{A}$，$\boldsymbol{b}^{(1)} = \boldsymbol{b}$，表示初始状态。在第 k 步消元中（$k = 1, 2, \ldots, n-1$），我们选取当前主元 $a_{kk}^{(k)}$，并对第 $k+1$ 行至第 n 行执行如下操作：

首先，计算乘数（multiplier）：

$$\ell_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad i = k+1, \ldots, n. \tag{1.4}$$

然后，更新系数矩阵和右端项：

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \ell_{ik} a_{kj}^{(k)}, \quad j = k+1, \ldots, n, \quad i = k+1, \ldots, n, \tag{1.5}$$

$$b_i^{(k+1)} = b_i^{(k)} - \ell_{ik} b_k^{(k)}, \quad i = k+1, \ldots, n. \tag{1.6}$$

注意，在第 k 步消元后，第 k 列中第 $k+1$ 行至第 n 行的元素已全部变为零，即 $a_{ik}^{(k+1)} = 0$（$i = k+1, \ldots, n$）。重复上述过程 $n-1$ 次，最终得到等价的上三角线性方程组 $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{b}^{(n)}$，其中 $\boldsymbol{U} = \boldsymbol{A}^{(n)}$ 为上三角矩阵。

1.3.3 算法实现

Algorithm 1 高斯消元法

Require: 系数矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$，右端项 $\boldsymbol{b} \in \mathbb{R}^n$

Ensure: 解向量 $\boldsymbol{x} \in \mathbb{R}^n$

```
构造增广矩阵 [A|b]
for k = 1 to n − 1 do
    if a_kk = 0 then
        错误：主元为零，算法失败（需要选主元）
    end if
    for i = k + 1 to n do
        ℓ_ik = a_ik / a_kk                          ▷ 计算乘数
        for j = k + 1 to n do
            a_ij = a_ij − ℓ_ik · a_kj               ▷ 消元
        end for
        b_i = b_i − ℓ_ik · b_k                      ▷ 同时更新右端项
    end for
end for
调用回代算法求解上三角方程组 Ux = b
返回 x
```

例 1.3.1. 用高斯消元法求解线性方程组

$$\begin{cases} 2x_1 + x_2 + x_3 = 4, \\ 4x_1 + 3x_2 + 3x_3 = 11, \\ 8x_1 + 7x_2 + 9x_3 = 23. \end{cases}$$

解：构造增广矩阵并执行消元过程。

第一步消元（$k = 1$，消去 x_1 在第 2、3 行中的系数）：

计算乘数：$\ell_{21} = 4/2 = 2$，$\ell_{31} = 8/2 = 4$。

更新矩阵：

$$\begin{bmatrix} 2 & 1 & 1 & 4 \\ 4 & 3 & 3 & 11 \\ 8 & 7 & 9 & 23 \end{bmatrix} \xrightarrow{\text{第 2 行}-2\times\text{第 1 行}} \begin{bmatrix} 2 & 1 & 1 & 4 \\ 0 & 1 & 1 & 3 \\ 8 & 7 & 9 & 23 \end{bmatrix}$$

$$\xrightarrow{\text{第 3 行}-4\times\text{第 1 行}} \begin{bmatrix} 2 & 1 & 1 & 4 \\ 0 & 1 & 1 & 3 \\ 0 & 3 & 5 & 7 \end{bmatrix}$$

第二步消元（$k = 2$，消去 x_2 在第 3 行中的系数）：

计算乘数：$\ell_{32} = 3/1 = 3$。

更新矩阵：

$$\begin{bmatrix} 2 & 1 & 1 & 4 \\ 0 & 1 & 1 & 3 \\ 0 & 3 & 5 & 7 \end{bmatrix} \xrightarrow{\text{第 3 行}-3\times\text{第 2 行}} \begin{bmatrix} 2 & 1 & 1 & 4 \\ 0 & 1 & 1 & 3 \\ 0 & 0 & 2 & -2 \end{bmatrix}$$

回代求解：

由第 3 个方程 $2x_3 = -2$，得 $x_3 = -1$。

将 $x_3=-1$ 代入第 2 个方程 $x_2+x_3=3$，得 $x_2=4$。
将 $x_2=4$ 和 $x_3=-1$ 代入第 1 个方程 $2x_1+x_2+x_3=4$，得 $x_1=1/2$。
因此，方程组的解为 $\boldsymbol{x}=(1/2,4,-1)^T$。

1.3.4　主元与数值稳定性

在消元过程中，用作除数的元素 $a_{kk}^{(k)}$ 称为**主元**（pivot）。主元的选择对算法的可行性与数值稳定性具有决定性影响。

1. **主元为零的情况**：若 $a_{kk}^{(k)}=0$，则无法计算乘数 ℓ_{ik}，消元步骤无法继续。即使原矩阵 $\boldsymbol{A}$ 在理论上可逆，也可能在消元过程中出现主元为零的情况。例如，考虑方程组

$$\begin{cases}0x_1+x_2=1,\\ x_1+x_2=2.\end{cases}$$

   该方程组有唯一解 $x_1=1$，$x_2=1$，但直接应用高斯消元法时第一步就会遇到主元为零的问题。解决方法是交换两行的位置，这引出了**选主元策略**（pivoting strategy）的概念。

2. **主元过小的情况**：即使 $a_{kk}^{(k)}\neq 0$，若 $|a_{kk}^{(k)}|$ 很小，则乘数 $\ell_{ik}=a_{ik}^{(k)}/a_{kk}^{(k)}$ 可能很大，从而在计算 $a_{ij}^{(k+1)}=a_{ij}^{(k)}-\ell_{ik}a_{kj}^{(k)}$ 时，由于浮点运算的舍入误差，可能导致数值不稳定。具体而言，当两个相近的大数相减时，有效数字会大量丢失，从而放大舍入误差。

因此，在实际计算中通常引入**选主元策略**，通过行交换（部分选主元）或行列交换（完全选主元）来选择绝对值较大的元素作为主元，以提高数值稳定性。

注. 需要注意的是，即使矩阵在理论上可逆，高斯消元法在数值计算中仍可能因不恰当的主元选择而表现出不稳定性。数值稳定性是数值算法区别于理论方法的重要特征，也是数值分析研究的核心问题之一。

1.3.5　算法复杂度

定理 1.3.1 (Gauss 消元法的时间复杂度). 对于 n 阶线性方程组，高斯消元法的计算复杂度为 $\mathscr{O}(n^3)$，其中消元过程需要 $\mathscr{O}(n^3)$ 次运算，回代过程需要 $\mathscr{O}(n^2)$ 次运算。

证明. 在第 k 步消元中（$k=1,2,\ldots,n-1$），需要：

- 计算 $n-k$ 个乘数，每个乘数需要 1 次除法，共 $n-k$ 次运算；
- 对每个 $i=k+1,\ldots,n$，更新 $n-k$ 个矩阵元素和 1 个右端项元素，每个更新需要 1 次乘法和 1 次减法，共 $(n-k)(2(n-k)+2)=2(n-k)^2+2(n-k)$ 次运算。

因此，第 k 步消元的总运算次数为

$$(n-k)+2(n-k)^2+2(n-k)=2(n-k)^2+3(n-k)$$

消元过程的总运算次数为

$$\sum_{k=1}^{n-1}[2(n-k)^2+3(n-k)]=2\sum_{k=1}^{n-1}k^2+3\sum_{k=1}^{n-1}k=2\cdot\frac{n(n-1)(2n-1)}{6}+3\cdot\frac{n(n-1)}{2}=\frac{n(n-1)(2n+7)}{6}=\mathscr{O}(n^3)$$

回代过程需要求解 n 个未知量，计算 x_i 需要 $n-i$ 次乘法和 $n-i$ 次减法，以及 1 次除法，因此总运算次数为

$$\sum_{i=1}^{n}(2(n-i)+1)=2\cdot\frac{n(n-1)}{2}+n=n^2=\mathscr{O}(n^2)$$

因此，高斯消元法的总体复杂度为 $\mathscr{O}(n^3)$。 □

从矩阵结构的角度重新审视消元过程，可以自然引出 $\boldsymbol{LU}$ 分解思想。具体而言，消元过程中使用的乘数 ℓ_{ik} 可以构成一个下三角矩阵 $\boldsymbol{L}$，而消元完成后的结果正是上三角矩阵 $\boldsymbol{U}$。因此，整个消元过程等价于将矩阵 $\boldsymbol{A}$ 分解为 $\boldsymbol{A}=\boldsymbol{LU}$ 的形式。$\boldsymbol{LU}$ 分解不仅是对高斯消元法在结构层面上的系统总结，还具有独立的理论价值和实际应用，这一观点将在后续章节中进一步展开。

1.4 Gauss 消元法的矩阵表示

在前面的章节中，我们从代数运算的角度描述了高斯消元法，通过逐行消元将系数矩阵转化为上三角矩阵。本节将从矩阵理论的角度重新审视消元过程，揭示其本质的矩阵结构，并由此引出重要的矩阵分解理论。

1.4.1 初等变换矩阵

根据线性代数知识，初等行变换可通过用初等变换矩阵左乘系数矩阵实现，而右乘初等变换矩阵则实现初等列变换。在消元过程中，我们主要使用三种初等变换矩阵。为了给出严格的定义，我们首先回顾初等变换的概念。

定义 1.4.1 (初等行变换)**.** 对 $n\times n$ 矩阵 $\boldsymbol{A}$，以下三种操作称为初等行变换：

1. **倍乘变换**：将矩阵的第 i 行乘以非零常数 c，记为 $r_i \leftarrow c\cdot r_i$。
2. **倍加变换**：将矩阵的第 i 行加上第 j 行的 c 倍（$i\neq j$），记为 $r_i \leftarrow r_i + c\cdot r_j$。
3. **交换变换**：交换矩阵的第 i 行和第 j 行（$i\neq j$），记为 $r_i \leftrightarrow r_j$。

初等行变换可以通过左乘相应的初等变换矩阵来实现。具体而言，三种初等变换矩阵具有如下形式：

定义 1.4.2 (初等变换矩阵)**.** 设 $\boldsymbol{I}$ 为 $n\times n$ 单位矩阵，则三种初等变换矩阵定义为：

1. **倍乘变换矩阵** $\boldsymbol{E}^{(1)}(i,c)$：将单位矩阵 $\boldsymbol{I}$ 的第 (i,i) 元素改为 c（$c\neq 0$），即

$$\boldsymbol{E}^{(1)}(i,c)=\begin{pmatrix} 1 & & & & & 0 \\ & \ddots & & & & \\ & & c & & & \\ & & & \ddots & & \\ & & & & 1 & \\ 0 & & & & & 1 \end{pmatrix},$$

   其中第 i 行第 i 列为 c，其余对角元为 1，非对角元为 0。

2. **倍加变换矩阵** $\boldsymbol{E}^{(2)}(i,j,c)$：将单位矩阵 $\boldsymbol{I}$ 的第 (i,j) 元素改为 c（$i\neq j$），即

$$\boldsymbol{E}^{(2)}(i,j,c)=\begin{pmatrix} 1 & & & & & & 0 \\ & \ddots & & & & & \\ & & 1 & \cdots & c & & \\ & & & \ddots & & & \\ & & & & 1 & & \\ & & & & & \ddots & \\ 0 & & & & & & 1 \end{pmatrix},$$

   其中第 i 行第 j 列为 c，其余元素与单位矩阵相同。

3. **交换变换矩阵** $\boldsymbol{E}^{(3)}(i,j)$：交换单位矩阵 $\boldsymbol{I}$ 的第 i 行和第 j 行（$i \neq j$），即

$$\boldsymbol{E}^{(3)}(i,j) = \begin{pmatrix} 1 & & & & & & 0 \\ & \ddots & & & & & \\ & & 0 & \cdots & 1 & & \\ & & \vdots & \ddots & \vdots & & \\ & & 1 & \cdots & 0 & & \\ & & & & & \ddots & \\ 0 & & & & & & 1 \end{pmatrix},$$

其中第 i 行第 i 列和第 j 行第 j 列为 0，第 i 行第 j 列和第 j 行第 i 列为 1，其余元素与单位矩阵相同。

命题 1.4.1 (初等变换矩阵的作用)**.** 设 $\boldsymbol{A}$ 为 $n \times n$ 矩阵，则：

1. $\boldsymbol{E}^{(1)}(i,c)\boldsymbol{A}$ 等价于对 $\boldsymbol{A}$ 执行倍乘变换 $r_i \leftarrow c \cdot r_i$。
2. $\boldsymbol{E}^{(2)}(i,j,c)\boldsymbol{A}$ 等价于对 $\boldsymbol{A}$ 执行倍加变换 $r_i \leftarrow r_i + c \cdot r_j$。
3. $\boldsymbol{E}^{(3)}(i,j)\boldsymbol{A}$ 等价于对 $\boldsymbol{A}$ 执行交换变换 $r_i \leftrightarrow r_j$。

注. 引入初等变换矩阵的目的，并非为了增加形式复杂度，而是为了将 “逐行的消元操作” 统一表达为**矩阵乘法**。这使得消元过程可以被视为一系列矩阵的乘积，为后续的 LU 分解和算法分析奠定代数基础。从教学角度看，这一统一表达方式能够帮助学生从更高层次理解消元过程的本质结构。

注. 在消元过程中，我们主要使用倍加变换矩阵 $\boldsymbol{E}^{(2)}$，因为倍乘变换和交换变换通常用于选主元策略中。当主元均不为零时，仅通过倍加变换即可完成消元过程。

1.4.2　消去矩阵

对一般的 n 阶线性方程组，假设消去过程中的主元均不为 0，则只需使用行倍加变换。在第 k 步消元中，我们需要消去第 $k+1$ 行至第 n 行中第 k 列的元素。这可以通过依次对第 $k+1, k+2, \ldots, n$ 行执行倍加变换来实现，即

$$r_i \leftarrow r_i + m_{i,k} \cdot r_k, \quad i = k+1, k+2, \ldots, n,$$

其中 $m_{i,k} = -a_{ik}^{(k)}/a_{kk}^{(k)}$ 为乘数。这些多次行倍加变换可以统一用一个消去矩阵表示。

定义 1.4.3 (消去矩阵)**.** 若 $n \times n$ 的矩阵 $\boldsymbol{M}_k$ 具有如下形式：

$$\boldsymbol{M}_k = \begin{pmatrix} 1 & & & & & & 0 \\ & \ddots & & & & & \\ & & 1 & & & & \\ & & m_{k+1,k} & 1 & & & \\ & & m_{k+2,k} & & \ddots & & \\ & & \vdots & & & \ddots & \\ & & m_{n,k} & & & & 1 \end{pmatrix}, \tag{1.7}$$

其中，除主对角线上和第 k 列最后 $n-k$ 个元素外的其他元素均为 0，则称 $\boldsymbol{M}_k$ 为第 k 类的 n 阶消去矩阵，下标 k 为类型数，$m_{i,k}$（$i = k+1, k+2, \ldots, n$）为乘数（multiplier）。

更精确地，$\boldsymbol{M}_k$ 的元素定义为：

$$(\boldsymbol{M}_k)_{ij} = \begin{cases} 1, & \text{若} i = j, \\ m_{i,k}, & \text{若} i > k \text{ 且} j = k, \\ 0, & \text{其他情况}. \end{cases}$$

注. 消去矩阵 $\boldsymbol{M}_k$ 的结构特点：

- 主对角线元素全为 1，因此 $\det(\boldsymbol{M}_k)=1$，$\boldsymbol{M}_k$ 非奇异。
- 第 k 列中，第 $k+1$ 至第 n 行的元素为乘数 $m_{k+1,k}, m_{k+2,k}, \ldots, m_{n,k}$。
- 第 1 至第 k 行与单位矩阵相同，表示这些行在消元过程中保持不变。
- 第 $k+1$ 至第 n 行中，除第 k 列外，其余元素与单位矩阵相同。

例 1.4.1. 考虑 4×4 矩阵，第 1 类消去矩阵 $\boldsymbol{M}_1$ 具有形式：

$$\boldsymbol{M}_1=\begin{pmatrix}1&0&0&0\\m_{21}&1&0&0\\m_{31}&0&1&0\\m_{41}&0&0&1\end{pmatrix},$$

其中 m_{21}, m_{31}, m_{41} 为乘数。左乘 $\boldsymbol{M}_1$ 相当于对矩阵的第 $2,3,4$ 行分别执行倍加变换，消去第 1 列中第 $2,3,4$ 行的元素。

类似地，第 2 类消去矩阵 $\boldsymbol{M}_2$ 具有形式：

$$\boldsymbol{M}_2=\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&m_{32}&1&0\\0&m_{42}&0&1\end{pmatrix},$$

用于消去第 2 列中第 $3,4$ 行的元素。

根据这个定义，消去矩阵具有以下重要性质：

定理 1.4.1 (消去矩阵的性质). $n\times n$ 的消去矩阵满足如下性质：

1. 消去矩阵为单位下三角矩阵，因此它一定非奇异。
2. 设 $\boldsymbol{a}=(a_1,a_2,\ldots,a_n)^T$ 为任意 n 维向量，若 $a_k\neq 0$（其中 a_k 表示向量 $\boldsymbol{a}$ 的第 k 个分量），且第 k 类的 n 阶消去矩阵 $\boldsymbol{M}_k$ 中的乘数 $m_{i,k}=-a_i/a_k$（$i=k+1,k+2,\ldots,n$），则

$$\boldsymbol{M}_k\boldsymbol{a}=\begin{pmatrix}a_1\\\vdots\\a_k\\0\\\vdots\\0\end{pmatrix}, \tag{1.8}$$

   换言之，$\boldsymbol{M}_k$ 作用于向量 $\boldsymbol{a}$ 后，将第 $k+1$ 至第 n 个分量全部消为零。
3. 消去矩阵 $\boldsymbol{M}_k$ 的逆矩阵为

$$\boldsymbol{M}_k^{-1}=\begin{pmatrix}1&&&&&&0\\&\ddots&&&&&\\&&1&&&&\\&&-m_{k+1,k}&1&&&\\&&-m_{k+2,k}&&\ddots&&\\&&\vdots&&&\ddots&\\&&-m_{n,k}&&&&1\end{pmatrix}, \tag{1.9}$$

   $\boldsymbol{M}_k^{-1}$ 与 $\boldsymbol{M}_k$ 的区别仅在于乘数的符号相反，且 $\boldsymbol{M}_k^{-1}$ 也是第 k 类消去矩阵。

4. 若 $\boldsymbol{M}_j$ 和 $\boldsymbol{M}_k$（其中 $j>k$）分别为第 j 类和第 k 类消去矩阵，则乘积 $\boldsymbol{M}_j\boldsymbol{M}_k$ 为单位下三角矩阵，其非零元素为 $\boldsymbol{M}_j$ 和 $\boldsymbol{M}_k$ 的非零元素的并集。更一般地，对于 m 个消去矩阵 $\boldsymbol{M}_{k_1},\boldsymbol{M}_{k_2},\ldots,\boldsymbol{M}_{k_m}$，其中 $1\le j\le m$ 且 $k_1<k_2<\cdots<k_m$，乘积 $\boldsymbol{M}_{k_1}\boldsymbol{M}_{k_2}\cdots\boldsymbol{M}_{k_m}$ 也是单位下三角矩阵，其非零元素为各因子矩阵非零元素的并集。

证明. 1. 由定义，$\boldsymbol{M}_k$ 的主对角线元素全为 1，且当 $i<j$ 时 $(\boldsymbol{M}_k)_{ij}=0$，因此 $\boldsymbol{M}_k$ 为单位下三角矩阵。单位下三角矩阵的行列式为 1，因此非奇异。

2. 设 $\boldsymbol{b}=\boldsymbol{M}_k\boldsymbol{a}$，则
$$b_i=\sum_{j=1}^{n}(\boldsymbol{M}_k)_{ij}a_j=\begin{cases}a_i, & 若 i\le k,\\ a_i+m_{i,k}a_k, & 若 i>k.\end{cases}$$
当 $i>k$ 时，由 $m_{i,k}=-a_i/a_k$，有
$$b_i=a_i+\left(-\frac{a_i}{a_k}\right)a_k=a_i-a_i=0.$$
因此 $\boldsymbol{b}=(a_1,\ldots,a_k,0,\ldots,0)^T$。

3. 直接验证 $\boldsymbol{M}_k\boldsymbol{M}_k^{-1}=\boldsymbol{I}$ 即可。注意到 $\boldsymbol{M}_k^{-1}$ 的结构与 $\boldsymbol{M}_k$ 相同，只是乘数符号相反，因此 $\boldsymbol{M}_k^{-1}$ 也是第 k 类消去矩阵。

4. 由于 $\boldsymbol{M}_j$ 和 $\boldsymbol{M}_k$ 都是单位下三角矩阵，它们的乘积也是单位下三角矩阵。当 $j>k$ 时，$\boldsymbol{M}_j$ 的非零元素位于第 j 列，$\boldsymbol{M}_k$ 的非零元素位于第 k 列，且 $j\ne k$，因此乘积的非零元素位置为两者的并集。一般情况的证明类似。

□

例 1.4.2. 考虑两个 4×4 消去矩阵：
$$\boldsymbol{M}_1=\begin{pmatrix}1&0&0&0\\m_{21}&1&0&0\\m_{31}&0&1&0\\m_{41}&0&0&1\end{pmatrix},\quad \boldsymbol{M}_2=\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&m_{32}&1&0\\0&m_{42}&0&1\end{pmatrix}.$$
它们的乘积为
$$\boldsymbol{M}_1\boldsymbol{M}_2=\begin{pmatrix}1&0&0&0\\m_{21}&1&0&0\\m_{31}&m_{32}&1&0\\m_{41}&m_{42}&0&1\end{pmatrix},$$
其非零元素为 $\boldsymbol{M}_1$ 和 $\boldsymbol{M}_2$ 的非零元素的并集。

但若交换顺序，有
$$\boldsymbol{M}_2\boldsymbol{M}_1=\begin{pmatrix}1&0&0&0\\m_{21}&1&0&0\\m_{31}+m_{21}m_{32}&m_{32}&1&0\\m_{41}+m_{21}m_{42}&m_{42}&0&1\end{pmatrix},$$
此时非零元素不再是简单的并集，而是出现了额外的项（如 $m_{31}+m_{21}m_{32}$）。这说明了性质 (4) 中要求 $j>k$ 的重要性。

注. 定理中的性质 (1) 至 (3) 比较简单，也很直观。性质 (4) 表明，当消去矩阵按类型数从小到大的顺序相乘时，结果矩阵的非零元素结构具有“并集”性质，这一性质在 LU 分解的推导中起到关键作用。需要注意的是，当消去矩阵的乘积顺序不满足 $k_1<k_2<\cdots<k_m$ 时，这一性质不再成立。

重要提示：后续 LU 分解中的下三角矩阵 $\boldsymbol{L}$，正是由这些消去矩阵的逆矩阵（$\boldsymbol{M}_1^{-1},\boldsymbol{M}_2^{-1},\ldots,\boldsymbol{M}_{n-1}^{-1}$）按顺序相乘得到的。由于消去矩阵的逆矩阵也是消去矩阵，且按类型数从小到大的顺序相乘，根据性质 (4)，$\boldsymbol{L}$ 为单位下三角矩阵，其非零元素为各因子矩阵非零元素的并集。这一结构性质将在 1.4.4 节中详细展开。

1.4.3 消元过程的矩阵表示

在前面的章节中，我们从元素的角度描述了消元过程。现在我们从行向量的角度重新审视这一过程，这将帮助我们理解消元过程的矩阵本质。

设 $\boldsymbol{A}^{(1)} = \boldsymbol{A}$ 为初始系数矩阵，$\boldsymbol{A}^{(k)}$（$k = 1, 2, \ldots, n$）表示第 $k-1$ 步消元后的矩阵。我们用 $\boldsymbol{A}_i^{(k)}$ 表示矩阵 $\boldsymbol{A}^{(k)}$ 的第 i 行（作为行向量），即

$$\boldsymbol{A}_i^{(k)} = \left(a_{i1}^{(k)}, a_{i2}^{(k)}, \ldots, a_{in}^{(k)}\right).$$

命题 1.4.2 (行向量的稳定性). 在消元过程中，从矩阵 $\boldsymbol{A}^{(i)}$ 开始，第 i 行不再被修改，即

$$\boldsymbol{A}_i^{(i)} = \boldsymbol{A}_i^{(i+1)} = \cdots = \boldsymbol{A}_i^{(n)}. \tag{1.10}$$

证明. 在第 k 步消元中（$k = 1, 2, \ldots, n-1$），我们只对第 $k+1$ 行至第 n 行进行修改，第 1 行至第 k 行保持不变。因此，当 $k \geq i$ 时，第 i 行不再被修改，即 $\boldsymbol{A}_i^{(i)} = \boldsymbol{A}_i^{(i+1)} = \cdots = \boldsymbol{A}_i^{(n)}$。 □

注意. **本节推导的目标：**我们的目标是**把“第 i 行如何由消元得到”写成“最终上三角矩阵各行的线性组合”**。一旦做到这一点，就可以将整个矩阵的关系统一写成一次矩阵乘法，从而揭示消元过程的矩阵本质。这一统一表达方式将为后续的 LU 分解提供理论基础。

现在，我们分析第 i 行在消元过程中的变化。在第 1 步消元中，第 i 行（$i \geq 2$）被修改为：

$$\boldsymbol{A}_i^{(2)} = \boldsymbol{A}_i^{(1)} - m_{i1}\boldsymbol{A}_1^{(1)},$$

其中 $m_{i1} = a_{i1}^{(1)}/a_{11}^{(1)}$ 为乘数。类似地，在第 2 步消元中（$i \geq 3$），有

$$\boldsymbol{A}_i^{(3)} = \boldsymbol{A}_i^{(2)} - m_{i2}\boldsymbol{A}_2^{(2)},$$

其中 $m_{i2} = a_{i2}^{(2)}/a_{22}^{(2)}$。继续这一过程，我们依次展开矩阵 $\boldsymbol{A}^{(1)}, \boldsymbol{A}^{(2)}, \ldots$ 的第 i 行：

$$\begin{aligned}
\boldsymbol{A}_i^{(1)} &= \boldsymbol{A}_i, \\
\boldsymbol{A}_i^{(2)} &= \boldsymbol{A}_i^{(1)} - m_{i1}\boldsymbol{A}_1^{(1)}, \quad \text{其中} m_{i1} = \frac{a_{i1}^{(1)}}{a_{11}^{(1)}}, \\
\boldsymbol{A}_i^{(3)} &= \boldsymbol{A}_i^{(2)} - m_{i2}\boldsymbol{A}_2^{(2)}, \quad \text{其中} m_{i2} = \frac{a_{i2}^{(2)}}{a_{22}^{(2)}}, \\
&\vdots \\
\boldsymbol{A}_i^{(i)} &= \boldsymbol{A}_i^{(i-1)} - m_{i,i-1}\boldsymbol{A}_{i-1}^{(i-1)}, \quad \text{其中} m_{i,i-1} = \frac{a_{i,i-1}^{(i-1)}}{a_{i-1,i-1}^{(i-1)}}.
\end{aligned}$$

将前面的等式相加，得到：

$$\boldsymbol{A}_i = m_{i1}\boldsymbol{A}_1^{(1)} + m_{i2}\boldsymbol{A}_2^{(2)} + \cdots + m_{i,i-1}\boldsymbol{A}_{i-1}^{(i-1)} + \boldsymbol{A}_i^{(i)}. \tag{1.11}$$

利用方程 (1.10)，我们知道 $\boldsymbol{A}_j^{(j)} = \boldsymbol{A}_j^{(n)}$（$j = 1, 2, \ldots, i$），因此有：

$$\boldsymbol{A}_i = m_{i1}\boldsymbol{A}_1^{(n)} + m_{i2}\boldsymbol{A}_2^{(n)} + \cdots + m_{i,i-1}\boldsymbol{A}_{i-1}^{(n)} + \boldsymbol{A}_i^{(n)}. \tag{1.12}$$

为了将上述表达式写成统一的矩阵乘法形式，我们引入辅助记号：设 $m_{ii} = 1$，$m_{ij} = 0$ 对 $j > i$。这样，式 (1.12) 可以写成：

$$\boldsymbol{A}_i = m_{i1}\boldsymbol{A}_1^{(n)} + \cdots + m_{i,i-1}\boldsymbol{A}_{i-1}^{(n)} + m_{ii}\boldsymbol{A}_i^{(n)} + m_{i,i+1}\boldsymbol{A}_{i+1}^{(n)} + \cdots + m_{in}\boldsymbol{A}_n^{(n)}. \tag{1.13}$$

注意到 $\boldsymbol{A}_{i+1}^{(n)}, \ldots, \boldsymbol{A}_n^{(n)}$ 的系数 $m_{i,i+1}, \ldots, m_{in}$ 均为 0，因此式 (1.13) 与式 (1.12) 等价。

利用矩阵乘法的性质，这可以写成：

$$\boldsymbol{A}_i = (m_{i1}, m_{i2}, \ldots, m_{in})\boldsymbol{A}^{(n)}, \tag{1.14}$$

其中 $(m_{i1}, m_{i2}, \ldots, m_{in})$ 为行向量，$\boldsymbol{A}^{(n)}$ 为矩阵。由于上述关系对 $i = 1, 2, \ldots, n$ 均成立，因此整个矩阵 $\boldsymbol{A}$ 可以表示为：

$$\boldsymbol{A} = \boldsymbol{M}\boldsymbol{A}^{(n)}, \tag{1.15}$$

其中 $\boldsymbol{M}$ 是由下式定义的矩阵：

$$\boldsymbol{M} = \begin{pmatrix} 1 & 0 & \cdots & \cdots & \cdots & 0 \\ m_{21} & 1 & 0 & \cdots & \cdots & 0 \\ m_{31} & m_{32} & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & \cdots & m_{n,n-1} & 1 \end{pmatrix}. \tag{1.16}$$

命题 1.4.3 (矩阵分解的存在性)**.** 矩阵 $\boldsymbol{M}$ 是下三角矩阵，对角线元素为 1，该矩阵通常记为 $\boldsymbol{L}$；矩阵 $\boldsymbol{A}^{(n)}$ 是上三角矩阵，通常记为 $\boldsymbol{U}$。因此，我们得到了 $\boldsymbol{A}$ 的一个分解形式：$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$。

证明. 由 $\boldsymbol{M}$ 的定义，其对角线元素 $m_{ii} = 1$（$i = 1, 2, \ldots, n$），且当 $i < j$ 时 $m_{ij} = 0$，因此 $\boldsymbol{M}$ 为单位下三角矩阵。

另一方面，$\boldsymbol{A}^{(n)}$ 是经过 $n-1$ 步消元后得到的矩阵。在第 k 步消元中，我们消去了第 $k+1$ 行至第 n 行中第 k 列的元素，因此当 $i > j$ 时 $a_{ij}^{(n)} = 0$，即 $\boldsymbol{A}^{(n)}$ 为上三角矩阵。 □

类似地，可以证明 $\boldsymbol{A} = \boldsymbol{L}^{(k-1)}\boldsymbol{A}^{(k)}$，其中

$$\boldsymbol{L}^{(k-1)} = \begin{pmatrix} 1 & 0 & \cdots & \cdots & \cdots & \cdots & 0 \\ m_{21} & 1 & 0 & \cdots & \cdots & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ m_{k1} & \cdots & m_{k,k-1} & 1 & 0 & \cdots & 0 \\ m_{k+1,1} & \cdots & m_{k+1,k-1} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ m_{n1} & \cdots & m_{n,k-1} & 0 & \cdots & 0 & 1 \end{pmatrix}. \tag{1.17}$$

1.4.4　LU 分解的推导

在前面的小节中，我们从行向量的角度推导出了 $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ 的分解形式。现在，我们从消去矩阵的角度重新审视这一分解，这将帮助我们更深入地理解消元过程的矩阵本质。

命题 1.4.4 (消元过程的矩阵表示)**.** 根据定义 1.7，高斯消去过程相当于对矩阵 $\boldsymbol{A}$ 不断左乘消去矩阵得到上三角矩阵 $\boldsymbol{U}$，即

$$\boldsymbol{U} = \boldsymbol{M}_{n-1} \cdots \boldsymbol{M}_2 \boldsymbol{M}_1 \boldsymbol{A}, \tag{1.18}$$

其中，$\boldsymbol{M}_i$（$i = 1, 2, \ldots, n-1$）为第 i 类消去矩阵，其乘数 $m_{j,i}$（$j = i+1, \ldots, n$）由消元过程确定。

证明. 在第 1 步消元中，我们通过左乘 $\boldsymbol{M}_1$ 消去第 1 列中第 2 至第 n 行的元素，得到 $\boldsymbol{A}^{(2)} = \boldsymbol{M}_1\boldsymbol{A}$。类似地，在第 2 步消元中，我们通过左乘 $\boldsymbol{M}_2$ 消去第 2 列中第 3 至第 n 行的元素，得到 $\boldsymbol{A}^{(3)} = \boldsymbol{M}_2\boldsymbol{A}^{(2)} = \boldsymbol{M}_2\boldsymbol{M}_1\boldsymbol{A}$。继续这一过程，最终得到 $\boldsymbol{A}^{(n)} = \boldsymbol{M}_{n-1} \cdots \boldsymbol{M}_2\boldsymbol{M}_1\boldsymbol{A} = \boldsymbol{U}$。 □

由于消去矩阵均非奇异（由定理 1.4.1 的性质 (1)），因此式 (1.18) 等价于

$$\boldsymbol{A} = \boldsymbol{M}_1^{-1}\boldsymbol{M}_2^{-1} \cdots \boldsymbol{M}_{n-1}^{-1}\boldsymbol{U}. \tag{1.19}$$

根据定理 1.4.1 的性质 (3)，$\boldsymbol{M}_1^{-1}, \boldsymbol{M}_2^{-1}, \ldots, \boldsymbol{M}_{n-1}^{-1}$ 也都是消去矩阵，且它们的类型数分别为 $1, 2, \ldots, n-1$。令

$$\boldsymbol{L} = \boldsymbol{M}_1^{-1}\boldsymbol{M}_2^{-1} \cdots \boldsymbol{M}_{n-1}^{-1}, \tag{1.20}$$

根据定理 1.4.1 的性质 (4)，由于 $1 < 2 < \cdots < n-1$，$\boldsymbol{L}$ 为单位下三角矩阵，其非零元素为各因子矩阵非零元素的并集。因此

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}. \tag{1.21}$$

定义 1.4.4 (LU 分解)**.** 式 (1.21) 被称为矩阵 $\boldsymbol{A}$ 的 **LU 分解**（LU decomposition 或 LU factorization），也称为**三角分解**（triangular decomposition），其中：

- $\boldsymbol{L}$ 为单位下三角矩阵（unit lower triangular matrix），其对角线元素全为 1，下三角部分的元素为消元过程中的乘数。
- $\boldsymbol{U}$ 为上三角矩阵（upper triangular matrix），其对角线元素为消元过程中的主元，上三角部分的元素为消元后的结果。

注. **术语说明：**本书后续默认所说的 “LU 分解”，均指**不含行交换的 LU 分解**，即直接对矩阵 $\boldsymbol{A}$ 进行分解得到 $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$。当需要行交换时，将得到 $\boldsymbol{P}\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ 的分解形式（称为 PLU 分解），其中 $\boldsymbol{P}$ 为置换矩阵。PLU 分解将在后续章节中详细讨论。

注. LU 分解将矩阵的分解与线性方程组的求解紧密联系起来。一旦得到 $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$，求解 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 就转化为依次求解 $\boldsymbol{L}\boldsymbol{y} = \boldsymbol{b}$（前代）和 $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{y}$（回代）两个三角方程组，这在数值计算中具有重要意义。具体而言：

1. 前代过程：求解 $\boldsymbol{L}\boldsymbol{y} = \boldsymbol{b}$，由于 $\boldsymbol{L}$ 为单位下三角矩阵，可以使用前代法，计算复杂度为 $\mathscr{O}(n^2)$。
2. 回代过程：求解 $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{y}$，由于 $\boldsymbol{U}$ 为上三角矩阵，可以使用回代法，计算复杂度为 $\mathscr{O}(n^2)$。

因此，一旦完成 LU 分解，求解线性方程组的复杂度仅为 $\mathscr{O}(n^2)$，远低于直接消元的 $\mathscr{O}(n^3)$。这使得 LU 分解在需要多次求解具有相同系数矩阵但不同右端项的线性方程组时特别有用。

命题 1.4.5 (LU 分解的唯一性)**.** 若矩阵 $\boldsymbol{A}$ 可逆，且其所有顺序主子式均不为零，则 $\boldsymbol{A}$ 的 LU 分解存在且唯一。

注. 需要注意的是，并非所有矩阵都存在 LU 分解。例如，若矩阵 $\boldsymbol{A}$ 的第一个顺序主子式为零（即 $a_{11} = 0$），则无法进行第一步消元，LU 分解不存在。此时需要引入选主元策略，通过行交换得到 $\boldsymbol{P}\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ 的分解形式，其中 $\boldsymbol{P}$ 为置换矩阵。这将在后续章节中详细讨论。

1.4.5 LU 分解的时间复杂度

与高斯消元法类似，LU 分解的计算复杂度也是 $\mathscr{O}(n^3)$。这是因为 LU 分解本质上就是高斯消元过程的矩阵表示，两者的计算量相同。我们可以通过 C++ 程序来验证这一结论。

定理 1.4.2 (LU 分解的时间复杂度)**.** 对于 n 阶矩阵，LU 分解的计算复杂度为 $\mathscr{O}(n^3)$。

证明. 在 LU 分解过程中，第 k 步（$k = 1, 2, \ldots, n-1$）需要：

- 计算 $n-k$ 个乘数，每个乘数需要 1 次除法，共 $n-k$ 次运算；
- 对每个 $i = k+1, \ldots, n$，更新 $n-k$ 个矩阵元素，每个更新需要 1 次乘法和 1 次减法，共 $2(n-k)^2$ 次运算。

因此，第 k 步的总运算次数为 $2(n-k)^2 + (n-k)$，整个 LU 分解的总运算次数为

$$\sum_{k=1}^{n-1}[2(n-k)^2 + (n-k)] = 2\sum_{k=1}^{n-1} k^2 + \sum_{k=1}^{n-1} k = \frac{2n(n-1)(2n-1)}{6} + \frac{n(n-1)}{2} = \mathscr{O}(n^3).$$

□

从实验结果可以看出，当矩阵阶数 n 翻倍时，LU 分解的耗时约为原来的 8 倍（2^3），这与理论分析的 $\mathscr{O}(n^3)$ 时间复杂度完全吻合。同时，分解的数值误差在合理范围内（小于 10^{-12}），验证了算法的正确性和数值稳定性。

注. 在实际应用中，LU 分解的优势在于：一旦完成分解，对于不同的右端项 $\boldsymbol{b}$，求解 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 只需进行两次 $\mathscr{O}(n^2)$ 的三角方程组求解（前代和回代），总复杂度为 $\mathscr{O}(n^2)$，远低于重新进行消元的 $\mathscr{O}(n^3)$。这使得 LU 分解在需要多次求解具有相同系数矩阵的线性方程组时特别高效。

注. 在实际应用中，LU 分解的优势在于：一旦完成分解，对于不同的右端项 $\boldsymbol{b}$，求解 $\boldsymbol{Ax}=\boldsymbol{b}$ 只需进行两次 $\mathscr{O}(n^2)$ 的三角方程组求解（前代和回代），总复杂度为 $\mathscr{O}(n^2)$，远低于重新进行消元的 $\mathscr{O}(n^3)$。这使得 LU 分解在需要多次求解具有相同系数矩阵的线性方程组时特别高效。

1.4.6 直接 LU 分解

在前面的章节中，我们通过高斯消元过程自然地导出了 LU 分解。然而，我们也可以**直接计算** LU 分解，而不需要经过完整的消元过程。这种方法在某些情况下更直观，也便于理解 LU 分解的计算结构。

直接 LU 分解的动机

在某些情况下，我们需要多次求解线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$，其中矩阵 $\boldsymbol{A}$ 相同，但右端项 $\boldsymbol{b}$ 不同。此时，对不变的矩阵 $\boldsymbol{A}$ 复进行消元操作似乎是不必要的，我们只需对右端项 $\boldsymbol{b}$ 执行这些操作。由于这些操作是线性的，我们应该能够将它们写成矩阵形式。

重新考虑方程组

$$\boldsymbol{Ax}=\boldsymbol{b}. \tag{1.22}$$

如果我们有一个分解 $\boldsymbol{A}=\boldsymbol{LU}$，则可以将 $\boldsymbol{Ax}=\boldsymbol{b}$ 重写为

$$\boldsymbol{LUx}=\boldsymbol{b}.$$

该方程组的解可以通过依次求解两个方程组得到：

$$\begin{cases}\boldsymbol{Ly}=\boldsymbol{b},\\ \boldsymbol{Ux}=\boldsymbol{y},\end{cases} \tag{1.23}$$

这对应于求解两个三角方程组，计算复杂度仅为 $\mathscr{O}(n^2)$。

此外，LU 分解还提供了计算行列式的经济方法。由于 $\det(\boldsymbol{A})=\det(\boldsymbol{LU})=\det(\boldsymbol{L})\det(\boldsymbol{U})$，且 $\boldsymbol{L}$ 为单位下三角矩（$\det(\boldsymbol{L})=1$），因此

$$\det(\boldsymbol{A})=\det(\boldsymbol{U})=\prod_{k=1}^{n}u_{kk}=\prod_{k=1}^{n}a_{kk}^{(k)}, \tag{1.24}$$

其中 $a_{kk}^{(k)}$ 为消元过程中第 k 步的主元。这提供了一种计算 $\boldsymbol{A}$ 的行列式的经济方法（相对于按行或列展开）。例如，当 $n=10$ 时，使用 Gaus 方法需要大约 10^3 次运算，而直接计算行列式需要大约 10^7 次运算。在这种情况下，我们不需要显式地写出 $\boldsymbol{L}$，因为它的行列式先验地等于 1。

直接计算 LU 分解的原理

我们暂时忘记高斯消元算法，直接寻找 $\boldsymbol{A}$ 的一个形如 $\boldsymbol{A}=\boldsymbol{LU}$ 的分解，其中 $\boldsymbol{L}$ 是单位下三角矩阵（$l_{ii}=1$，$i=1\ldots,n$），$\boldsymbol{U}$ 是上三角矩阵。为了确保分解的唯一性，我们要求 $\boldsymbol{L}$ 的对角线为单位元。

我们通过一个例子来说明直接计算 LU 分解的方法。

例 1.4.3 (直接 LU 分解的计算过程)**.** 设矩阵

$$\boldsymbol{A}=\begin{pmatrix}2&1&-2\\4&5&-3\\-2&5&3\end{pmatrix}.$$

我们寻找

$$\boldsymbol{L}=\begin{pmatrix}1&0&0\\l_{21}&1&0\\l_{31}&l_{32}&1\end{pmatrix},\quad \boldsymbol{U}=\begin{pmatrix}u_{11}&u_{12}&u_{13}\\0&u_{22}&u_{23}\\0&0&u_{33}\end{pmatrix},$$

使得 $\boldsymbol{A}=\boldsymbol{LU}$。

步骤 1：计算 $\boldsymbol{U}$ 的第一行

将 $\boldsymbol{A}$ 的第一行与 $\boldsymbol{LU}$ 的第一行对应，由于 $\boldsymbol{L}$ 的第一行为 $(1,0,0)$，因此

$$\boldsymbol{LU} = \begin{pmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix} = \begin{pmatrix} 2 & 1 & -2 \\ 4 & 5 & -3 \\ -2 & 5 & 3 \end{pmatrix}.$$

比较第一行元素，得到：

$$u_{11} = 2, \quad u_{12} = 1, \quad u_{13} = -2.$$

步骤 2：计算 $\boldsymbol{L}$ 的第一列

将 $\boldsymbol{A}$ 的第一列与 $\boldsymbol{LU}$ 的第一列对应（除去位置 $(1,1)$ 的元素），有

$$l_{21} \cdot u_{11} = 4, \quad l_{31} \cdot u_{11} = -2.$$

将 $u_{11} = 2$ 代入，得到：

$$l_{21} = 2, \quad l_{31} = -1.$$

步骤 3：计算 $\boldsymbol{U}$ 的第二行

将 $\boldsymbol{A}$ 的第二行与 $\boldsymbol{LU}$ 的第二行对应，有

$$l_{21} \cdot u_{12} + u_{22} = 5, \quad l_{21} \cdot u_{13} + u_{23} = -3.$$

将已知值代入，得到：

$$2 \cdot 1 + u_{22} = 5 \Rightarrow u_{22} = 3, \quad 2 \cdot (-2) + u_{23} = -3 \Rightarrow u_{23} = 1.$$

步骤 4：计算 $\boldsymbol{L}$ 的第二列

将 $\boldsymbol{A}$ 的第二列与 $\boldsymbol{LU}$ 的第二列对应（除去位置 $(2,2)$ 的元素），有

$$l_{31} \cdot u_{12} + l_{32} \cdot u_{22} = 5.$$

将已知值代入，得到：

$$(-1) \cdot 1 + l_{32} \cdot 3 = 5 \Rightarrow l_{32} = 2.$$

步骤 5：计算 $\boldsymbol{U}$ 的第三行

将 $\boldsymbol{A}$ 的第三行与 $\boldsymbol{LU}$ 的第三行对应，有

$$l_{31} \cdot u_{13} + l_{32} \cdot u_{23} + u_{33} = 3.$$

将已知值代入，得到：

$$(-1) \cdot (-2) + 2 \cdot 1 + u_{33} = 3 \Rightarrow u_{33} = -1.$$

因此，$\boldsymbol{A}$ 的 LU 分解为

$$\boldsymbol{L} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 2 & 1 \end{pmatrix}, \quad \boldsymbol{U} = \begin{pmatrix} 2 & 1 & -2 \\ 0 & 3 & 1 \\ 0 & 0 & -1 \end{pmatrix}.$$

从上述例子可以看出，直接 LU 分解的计算过程遵循一个固定的模式：交替计算 $\boldsymbol{U}$ 的行和 $\boldsymbol{L}$ 的列。具体而言：

- 先计算 $\boldsymbol{U}$ 的第 k 行（$k = 1, 2, \ldots, n$），这需要已知 $\boldsymbol{L}$ 的前 $k-1$ 列和 $\boldsymbol{U}$ 的前 $k-1$ 行。
- 再计算 $\boldsymbol{L}$ 的第 k 列（$k = 1, 2, \ldots, n-1$），这需要已知 $\boldsymbol{U}$ 的前 k 行和 $\boldsymbol{L}$ 的前 $k-1$ 列。

Doolittle 算法

通过写出 $\boldsymbol{A}=\boldsymbol{L}\boldsymbol{U}$ 并记住矩阵 $\boldsymbol{L}$ 和 $\boldsymbol{U}$ 是三角矩阵，且 $\boldsymbol{L}$ 的对角线元素等于 1，我们可以得到一般的计算公式。

设 $\boldsymbol{A}_i$ 表示矩阵 $\boldsymbol{A}$ 的第 i 行（作为行向量），$\boldsymbol{L}_i$ 和 $\boldsymbol{U}_i$ 分别表示 $\boldsymbol{L}$ 和 $\boldsymbol{U}$ 的第 i 行。类似地，$\boldsymbol{A}^i$ 表示矩阵 $\boldsymbol{A}$ 的第 i 列（作为列向量），$\boldsymbol{L}^i$ 和 $\boldsymbol{U}^i$ 分别表示 $\boldsymbol{L}$ 和 $\boldsymbol{U}$ 的第 i 列。

为了计算 $\boldsymbol{U}$ 的第 i 行，我们有：

$$\boldsymbol{A}_i=\boldsymbol{L}_i\boldsymbol{U}=\sum_{k=1}^{i-1}l_{ik}\boldsymbol{U}_k+\boldsymbol{U}_i \quad\Rightarrow\quad \boldsymbol{U}_i=\boldsymbol{A}_i-\sum_{k=1}^{i-1}l_{ik}\boldsymbol{U}_k. \tag{1.25}$$

我们看到，为了计算 $\boldsymbol{U}$ 的第 i 行 $\boldsymbol{U}_i$ 的元素，我们需要预先知道 $\boldsymbol{U}$ 的第 1 到 $i-1$ 行的元素以及 $\boldsymbol{L}$ 的第 1 到 $i-1$ 列的元素。此外可以注意到，实际上我们只计算 j 在 i 到 n 之间的项 u_{ij}，因为该行的其他项已知为零。

类似地，为了计算 $\boldsymbol{L}$ 的第 i 列，我们有：

$$\boldsymbol{A}^i=\boldsymbol{L}\boldsymbol{U}^i=\sum_{k=1}^{i-1}u_{ki}\boldsymbol{L}^k+u_{ii}\boldsymbol{L}^i \quad\Rightarrow\quad \boldsymbol{L}^i=\frac{1}{u_{ii}}\left(\boldsymbol{A}^i-\sum_{k=1}^{i-1}u_{ki}\boldsymbol{L}^k\right). \tag{1.26}$$

我们看到，为了计算 $\boldsymbol{L}$ 的第 i 列 $\boldsymbol{L}^i$ 的元素，我们需要预先知道 $\boldsymbol{U}$ 的第 1 到 i 行的元素以及 $\boldsymbol{L}$ 的第 1 到 $i-1$ 的元素。此外可以注意到，实际上我们只计算 j 在 $i+1$ 到 n 之间的项 l_{ji}，因为我们已经知道 $l_{ii}=1$，且该列的其他项为零。因此，我们将按照以下顺序计算：

- $\boldsymbol{U}$ 的第一行，然后是 $\boldsymbol{L}$ 的第一列；
- $\boldsymbol{U}$ 的第二行，然后是 $\boldsymbol{L}$ 的第二列；
- ...
- $\boldsymbol{U}$ 的第 n 行（$\boldsymbol{L}$ 的第 n 列不需要计算，因为 $l_{nn}=1$，且该列的其他项为零）。

将式 (1.25) 和 (1.26) 写成元素形式，我们得到：**计算 $\boldsymbol{U}$ 的第 i 行：**

$$u_{ij}=a_{ij}-\sum_{k=1}^{i-1}l_{ik}u_{kj}, \quad j=i,i+1,\ldots,n. \tag{1.27}$$

计算 $\boldsymbol{L}$ 的第 i 列：

$$l_{ji}=\frac{1}{u_{ii}}\left(a_{ji}-\sum_{k=1}^{i-1}l_{jk}u_{ki}\right), \quad j=i+1,i+2,\ldots,n. \tag{1.28}$$

上述方法称为 **Doolittle 算法**（Doolittle's method），由它得到的 $\boldsymbol{L}$ 矩阵为单位下三角矩阵，$\boldsymbol{U}$ 矩阵为上三角矩阵。

Algorithm 2 Doolittle 算法：直接 LU 分解

Require: 矩阵 $\boldsymbol{A}\in\mathbb{R}^{n\times n}$，假设所有顺序主子式均不为零

Ensure: 下三角矩阵 $\boldsymbol{L}$（对角线为 1）和上三角矩阵 $\boldsymbol{U}$，使得 $\boldsymbol{A}=\boldsymbol{L}\boldsymbol{U}$

```
for i = 1, 2, ..., n do
    计算 U 的第 i 行
    for j = i, i+1, ..., n do
        u_ij ← a_ij − Σ_{k=1}^{i−1} l_ik u_kj
    end for
    if i < n then
        计算 L 的第 i 列
        for j = i+1, i+2, ..., n do
            l_ji ← (1/u_ii) (a_ji − Σ_{k=1}^{i−1} l_jk u_ki)
        end for
    end if
end for
```

注. 与前面的算法一样，结果矩阵 $\boldsymbol{L}$ 和 $\boldsymbol{U}$ 的数据可以覆盖原始矩阵 $\boldsymbol{A}$（原地存储）。由于 $\boldsymbol{U}$ 的第一行、$\boldsymbol{L}$ 的最后一列不需要计算，算法中先求 $\boldsymbol{L}$ 的第 i 列，再求 $\boldsymbol{U}$ 的第 $i+1$ 行，依据的计算公式分别为式 (1.28) 和式 (1.27)。算法中乘除法的次数为

$$\sum_{i=1}^{n}\sum_{j=i}^{n}(i-1)+\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}(i+1)=2\sum_{k=1}^{n-1}(n-k)k=2\left(\sum_{k=1}^{n-1}nk-\sum_{k=1}^{n-1}k^2\right)\approx 2\left(\frac{n^3}{2}-\frac{n^3}{3}\right)=\frac{n^3}{3},$$

与基于高斯消元的 LU 分解的计算量基本相同（都是 $\mathscr{O}(n^3)$）。

命题 1.4.6 (直接 LU 分解与高斯消元的等价性)**.** 可以证明，上述直接 LU 分解的计算结果与高斯消元过程的结果完全一样，它们的前提都是主元 $a_{kk}\neq 0$（$k=1,2,\ldots,n-1$），因此直接 LU 分解属于高斯元法的一种变形。

注. Doolittle 算法与基于高斯消元的 LU 分解的不同之处在于，它比较直接地算出 $\boldsymbol{L}$ 和 $\boldsymbol{U}$ 中的元素，而不需要经过多次数值的更新。对于稠密矩阵（采用二维数组存储）的情况，不同高斯消元法的变形完全等价，计算量没有什么差别。但对于某些稀疏矩阵（采用特殊的方式存储矩阵）或者并行计算环境，这些不同算法的效率可能差别很大。

最后说明一点，以上讨论的 LU 分解都称为**杜利特尔（Doolittle）方法**，由它得到 $\boldsymbol{L}$ 矩阵为单位下三角矩阵，$\boldsymbol{U}$ 矩阵为上三角矩阵。矩阵的另一种 LU 分解称为**克劳特（Crout）方法**，其中 $\boldsymbol{L}$ 矩阵为下三角矩阵，$\boldsymbol{U}$ 矩阵为单位上三角矩阵（对角线元素为 1）。两种方法在计算复杂度上相同，选择哪种方法主要取决于具体应用场景和数值稳定性考虑。

1.5 带主元的 LU 分解

在前面的章节中，我们讨论的 LU 分解假设高斯消元过程中所有主元非零，但实际计算中，矩阵可能因主元为零导致算法中断，或因主元过小引发严重的数值误差。为解决这一问题，需要引入**主元选择策略**，即带主元的 LU 分解。本节将系统分析主元问题的成因、主元选择的必要性，并推导带部分主元的 LU 分解（$\boldsymbol{PA}=\boldsymbol{LU}$）的理论与算法。

1.5.1 LU 分解的可行性条件

要理解主元问题的本质，首先需要明确 LU 分解可行的数学条件。为此，我们引入顺序主子式的概念。

定义 1.5.1 (顺序主子式)**.** 对于 n 阶矩阵 $\boldsymbol{A}=(a_{ij})$，其 k 阶**顺序主子式**（leading principal minor）记为 $\det(\boldsymbol{A}_k)$，其中 $\boldsymbol{A}_k$ 是由 $\boldsymbol{A}$ 的前 k 行、前 k 列元素构成的 k 阶子矩阵，即

$$\boldsymbol{A}_k=\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k1} & a_{k2} & \cdots & a_{kk} \end{pmatrix}.$$

注. 顺序主子式是矩阵的一个重要特征。例如，对于 3×3 矩阵

$$\boldsymbol{A}=\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix},$$

其顺序主子式分别为：

- 1 阶顺序主子式：$\det(\boldsymbol{A}_1)=a_{11}$
- 2 阶顺序主子式：$\det(\boldsymbol{A}_2)=a_{11}a_{22}-a_{12}a_{21}$

- 3 阶顺序主子式：$\det(\boldsymbol{A}_3) = \det(\boldsymbol{A})$

定理 1.5.1 (LU 分解可行性的充要条件)**.** 对于 n 阶矩阵 $\boldsymbol{A}$，以下三个命题等价：

1. 高斯消元过程中所有主元 $a_{kk}^{(k)} \neq 0$（$k = 1, 2, \ldots, n-1$）；
2. 矩阵 $\boldsymbol{A}$ 可分解为 $\boldsymbol{A} = \boldsymbol{LU}$，其中 $\boldsymbol{L}$ 为单位下三角矩阵，$\boldsymbol{U}$ 为可逆上三角矩阵；
3. $\boldsymbol{A}$ 的所有 k 阶顺序主子式 $\det(\boldsymbol{A}_k) \neq 0$（$k = 1, 2, \ldots, n-1$）。

证明. **（1）⇒（2）**：若所有主元非零，高斯消元可完整执行，得到单位下三角矩阵 $\boldsymbol{L}$ 和上三角矩阵 $\boldsymbol{U}$。由于主元是 $\boldsymbol{U}$ 的对角元且非零，$\boldsymbol{U}$ 可逆。

（2）⇒（3）：若 $\boldsymbol{A} = \boldsymbol{LU}$，则顺序主子矩阵满足 $\boldsymbol{A}_k = \boldsymbol{L}_k\boldsymbol{U}_k$，其中 $\boldsymbol{L}_k$ 是单位下三角矩阵（$\det(\boldsymbol{L}_k) = 1$），$\boldsymbol{U}_k$ 是上三角矩阵。因 $\boldsymbol{U}$ 可逆，$\boldsymbol{U}_k$ 的对角元（即前 k 个主元）非零，故 $\det(\boldsymbol{U}_k) \neq 0$，从而 $\det(\boldsymbol{A}_k) = \det(\boldsymbol{L}_k)\det(\boldsymbol{U}_k) = \det(\boldsymbol{U}_k) \neq 0$，即 $\boldsymbol{A}_k$ 可逆。

（3）⇒（1）：用数学归纳法。

- **基例** $k = 1$：$\boldsymbol{A}_1 = (a_{11})$，可逆故 $a_{11}^{(1)} = a_{11} \neq 0$。
- **归纳假设**：假设前 $k-1$ 个主元非零（$a_{ii}^{(i)} \neq 0$，$i = 1, \ldots, k-1$），则高斯消元前 $k-1$ 步可完成，得到矩阵 $\boldsymbol{A}^{(k)}$。
- **归纳步骤**：此时 $\boldsymbol{A}_k^{(k)}$ 是上三角矩阵，且 $\det(\boldsymbol{A}_k) = \det(\boldsymbol{A}_k^{(k)}) = \prod_{i=1}^{k-1} a_{ii}^{(i)} \cdot a_{kk}^{(k)}$。因 $\boldsymbol{A}_k$ 可逆，$\det(\boldsymbol{A}_k) \neq 0$，结合归纳假设中前 $k-1$ 个主元非零，可得 $a_{kk}^{(k)} \neq 0$。

由归纳法，命题（3）蕴含命题（1）。 □

注. 定理 1.5.1 揭示了 LU 分解可行性的本质：**矩阵能否进行 LU 分解，取决于其顺序主子式是否非零**。这一条件比矩阵可逆的条件更强（矩阵可逆只需 $\det(\boldsymbol{A}) \neq 0$，而 LU 分解需要前 $n-1$ 个顺序主子式均非零）。因此，即使矩阵可逆，也可能无法直接进行 LU 分解，这正是需要引入选主元策略的根本原因。

1.5.2　零主元与行置换的必要性

若 $\boldsymbol{A}$ 可逆但不满足定理 1.5.1 的条件，高斯消元会遇到**零主元**，导致算法中断。此时需要通过行置换寻找非零主元。

例 1.5.1 (零主元导致 LU 分解失败)**.** 考虑矩阵

$$\boldsymbol{A} = \begin{pmatrix} 0 & 2 \\ 3 & 1 \end{pmatrix}.$$

其 1 阶顺序主子式 $\det(\boldsymbol{A}_1) = 0$，不满足定理 1.5.1 的条件。直接 LU 分解时，第一个主元 $a_{11} = 0$，无法计算乘数，分解失败。

但 $\det(\boldsymbol{A}) = -6 \neq 0$，说明 $\boldsymbol{A}$ 可逆，方程组 $\boldsymbol{Ax} = \boldsymbol{b}$ 有唯一解。例如，对于方程组

$$\begin{cases} 2x_2 = 2, \\ 3x_1 + x_2 = 4, \end{cases}$$

其解为 $x_1 = 1$，$x_2 = 1$。

这一例子说明：**可逆矩阵不一定能直接 LU 分解，必须通过行置换解决零主元问题**。

定理 1.5.2 (零主元的行置换保证)**.** 设 $\boldsymbol{A}$ 为 n 阶可逆矩阵，则在高斯消元的第 k 步（$k = 1, \ldots, n-1$），第 k 列中至少存在一个非零元素位于第 k 行及以下（即存在 $i \geq k$，使得 $a_{ik}^{(k)} \neq 0$）。

证明. 假设第 k 列中第 k 行及以下元素全为零，即 $a_{ik}^{(k)} = 0$（$i = k, k+1, \ldots, n$）。考虑经过 $k-1$ 步消元后的矩阵

$\boldsymbol{A}^{(k)}$，其结构为：

$$\boldsymbol{A}^{(k)} = \begin{pmatrix} \boldsymbol{U}_{k-1} & \boldsymbol{B} \\ \boldsymbol{0} & \boldsymbol{S}_k \end{pmatrix},$$

其中 $\boldsymbol{U}_{k-1}$ 是 $(k-1)\times(k-1)$ 的上三角矩阵，$\boldsymbol{S}_k$ 是 $(n-k+1)\times(n-k+1)$ 的子矩阵，且 $\boldsymbol{S}_k$ 的第一列全为零。

由于 $\boldsymbol{S}_k$ 的第一列为零，$\det(\boldsymbol{S}_k)=0$。而高斯消元不改变矩阵行列式，因此

$$\det(\boldsymbol{A}^{(k)}) = \det(\boldsymbol{A}) = \det(\boldsymbol{U}_{k-1})\cdot\det(\boldsymbol{S}_k) = 0.$$

这与 $\boldsymbol{A}$ 可逆（$\det(\boldsymbol{A})\neq 0$）矛盾。故第 k 列中必存在非零元素。 □

注. 定理 1.5.2 保证了我们可以通过**行置换**（交换第 k 行与第 l 行，$l\geq k$）得到非零主元，从而继续高斯消元。这一结论是部分主元策略的理论基础。

1.5.3 主元选择与数值稳定性：小主元的危害

即使主元非零，若主元绝对值过小，会导致乘数（消元时的系数）过大，从而放大浮点运算的舍入误差，严重影响数值稳定性。因此，需要选择**绝对值最大的主元**（部分主元策略）来减小乘数的绝对值。

例 1.5.2 (小主元的数值误差). 考虑方程组 $\boldsymbol{Ax}=\boldsymbol{b}$，其中

$$\boldsymbol{A} = \begin{pmatrix} \varepsilon & 1 \\ 1 & 0 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ \varepsilon \end{pmatrix},$$

ε 为很小的正数（如 $\varepsilon = 10^{-10}$，小于浮点系统的机器精度 $\varepsilon_{\text{mach}}$ 的一半）。

情况 1：不选主元

主元为 ε，乘数为 $1/\varepsilon$。消元后增广矩阵为：

$$\left(\begin{array}{cc|c} \varepsilon & 1 & 1 \\ 0 & 1-1/\varepsilon & \varepsilon-1/\varepsilon \end{array}\right).$$

由于 $1/\varepsilon \gg 1$，在浮点运算中，1 被 $1/\varepsilon$ “吞没”（根据浮点运算的舍入误差理论），增广矩阵近似为：

$$\left(\begin{array}{cc|c} \varepsilon & 1 & 1 \\ 0 & -1/\varepsilon & -1/\varepsilon \end{array}\right).$$

解得 $x_2=1$，$x_1=(1-1)/\varepsilon=0$。

但准确解为 $x_1=-1$，$x_2=1+\varepsilon\approx 1$，数值解误差极大。

情况 2：选主元

交换两行，主元为 1，乘数为 ε。消元后增广矩阵为：

$$\left(\begin{array}{cc|c} 1 & 0 & \varepsilon \\ 0 & 1-\varepsilon^2 & 1-\varepsilon^2 \end{array}\right).$$

在浮点运算中近似为：

$$\left(\begin{array}{cc|c} 1 & 0 & \varepsilon \\ 0 & 1 & 1 \end{array}\right).$$

解得 $x_2=1$，$x_1=\varepsilon\approx -1$（因 ε 很小，实际近似为 -1），与准确解一致，数值稳定性显著提升。

注. 例 1.5.2 说明了小主元带来的两个问题：

1. **乘数过大**：当主元 $a_{kk}^{(k)}$ 很小时，乘数 $m_{ik}=-a_{ik}^{(k)}/a_{kk}^{(k)}$ 的绝对值可能很大，导致后续计算中数值误差被放大。
2. **有效数字丢失**：在浮点运算中，当两个数量级相差很大的数相加时，较小的数可能被 “吞没”，导致有效数字丢失。

选择绝对值最大的主元可以减小乘数的绝对值，从而降低误差放大的风险。

1.5.4　部分主元 LU 分解：$\boldsymbol{PA} = \boldsymbol{LU}$

通过行置换（部分主元策略），可逆矩阵 $\boldsymbol{A}$ 可分解为：

$$\boldsymbol{PA} = \boldsymbol{LU},$$

其中 $\boldsymbol{P}$ 为置换矩阵，$\boldsymbol{L}$ 为单位下三角矩阵，$\boldsymbol{U}$ 为上三角矩阵。

定义 1.5.2 (置换矩阵). n 阶**置换矩阵**（permutation matrix）$\boldsymbol{P} = (p_{ij})$ 是一个 $n \times n$ 矩阵，满足以下条件：

1. 每行和每列恰好有一个元素为 1，其余元素为 0，即

$$\sum_{i=1}^{n} p_{ij} = 1 \quad (j = 1, 2, \ldots, n), \quad \sum_{j=1}^{n} p_{ij} = 1 \quad (i = 1, 2, \ldots, n), \quad p_{ij} \in \{0, 1\}.$$

2. 存在一个置换 $\sigma : \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\}$（双射），使得

$$p_{ij} = \begin{cases} 1, & 若 j = \sigma(i), \\ 0, & 其他情况. \end{cases}$$

换言之，$\boldsymbol{P}$ 的第 i 行中，第 $\sigma(i)$ 列的元素为 1，其余元素为 0。

等价地，置换矩阵 $\boldsymbol{P}$ 是通过交换单位矩阵 $\boldsymbol{I}$ 的行（或列）得到的矩阵。置换矩阵具有以下重要性质：

1. $\boldsymbol{P}^T = \boldsymbol{P}^{-1}$（置换矩阵的转置等于其逆矩阵）；
2. $\det(\boldsymbol{P}) = \pm 1$（行列式为 1 或 -1，取决于置换的奇偶性）；
3. 左乘置换矩阵 $\boldsymbol{P}$ 相当于对矩阵的行进行置换，右乘置换矩阵 $\boldsymbol{P}$ 相当于对矩阵的列进行置换。

例 1.5.3. 考虑 3 阶置换矩阵，对应置换 $\sigma(1) = 2$，$\sigma(2) = 3$，$\sigma(3) = 1$（即 $1 \to 2 \to 3 \to 1$ 的循环置换）：

$$\boldsymbol{P} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

该矩阵的第 1 行第 2 列为 1，第 2 行第 3 列为 1，第 3 行第 1 列为 1，其余元素为 0。左乘 $\boldsymbol{P}$ 相当于将矩阵的第 1 行移到第 3 行，第 2 行移到第 1 行，第 3 行移到第 2 行。

定理 1.5.3 (部分主元 LU 分解存在性). 设 $\boldsymbol{A}$ 为 n 阶可逆矩阵，则存在置换矩阵 $\boldsymbol{P}$、单位下三角矩阵 $\boldsymbol{L}$ 和上三角矩阵 $\boldsymbol{U}$，使得 $\boldsymbol{PA} = \boldsymbol{LU}$。

证明. 高斯消元的每一步 k（$1 \le k \le n-1$）：

1. 在第 k 列中选择绝对值最大的元素 $a_{lk}^{(k)}$（$l \ge k$）；
2. 交换第 k 行与第 l 行（对应置换矩阵 $\boldsymbol{P}^{(k)}$）；
3. 执行消元，计算乘数（$\boldsymbol{L}$ 的元素）并更新矩阵得到上三角部分 $\boldsymbol{U}$。

最终，所有置换矩阵的乘积为 $\boldsymbol{P} = \boldsymbol{P}^{(n-1)} \cdots \boldsymbol{P}^{(1)}$，消元得到的乘数构成 $\boldsymbol{L}$，消元后的矩阵为 $\boldsymbol{U}$，故 $\boldsymbol{PA} = \boldsymbol{LU}$。 □

注. 该分解不唯一，若主元搜索存在多个最大值（实际中通常选第一个遇到的最大值），会导致不同的 $\boldsymbol{P}$、$\boldsymbol{L}$、$\boldsymbol{U}$。但无论选择哪个最大值，都能保证数值稳定性。

在带主元的 LU 分解过程中，我们需要理解置换矩阵如何作用于下三角矩阵。下面的定理揭示了这一重要关系。

定理 1.5.4 (置换矩阵作用于下三角矩阵). 设 $\boldsymbol{L}^{(k-1)}$ 是一个 n 阶单位下三角矩阵（对角线元素为 1），且进一

步满足

$$[\boldsymbol{L}^{(k-1)}]_{ij} = 0, \quad 当 i > j > k-1, \tag{1.29}$$

即 $\boldsymbol{L}^{(k-1)}$ 的第 k 列到第 n 列中，第 k 行以下的元素均为 0。

设 $\boldsymbol{P}^{(k)}$ 是带主元搜索的高斯算法第 k 步的置换矩阵，交换第 k 行和第 ℓ 行，其中 $\ell \geq k$。则有

$$\boldsymbol{P}^{(k)}\boldsymbol{L}^{(k-1)}\boldsymbol{P}^{(k)} = \hat{\boldsymbol{L}}^{(k-1)}, \tag{1.30}$$

其中 $\hat{\boldsymbol{L}}^{(k-1)}$ 是通过置换 $\boldsymbol{L}^{(k-1)}$ 的第 k 行和第 ℓ 行在第 1 到 $k-1$ 列中的项而得到的单位下三角矩阵，且 $\hat{\boldsymbol{L}}^{(k-1)}$ 仍然满足条件 (1.29)。

证明. 为了证明这个定理，我们需要理解 $\boldsymbol{L}^{(k-1)}$ 的结构以及置换矩阵的作用。

第一步：理解 $\boldsymbol{L}^{(k-1)}$ 的结构

由于 $\boldsymbol{L}^{(k-1)}$ 是单位下三角矩阵，且满足条件 (1.29)，我们可以将 $\boldsymbol{L}^{(k-1)}$ 写成如下形式：

$$\boldsymbol{L}^{(k-1)} = \boldsymbol{I} + \sum_{p=1}^{k-1} \boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T, \tag{1.31}$$

其中：

- $\boldsymbol{I}$ 是 n 阶单位矩阵；
- $\boldsymbol{e}^{(p)}$ 是第 p 个标准单位向量（第 p 个分量为 1，其余为 0）；
- $\boldsymbol{z}^{(p)}$ 是 n 维列向量，满足 $z_i^{(p)} = 0$ 对 $i \leq p$（即前 p 个分量为 0）。

这一分解的直观含义是：$\boldsymbol{L}^{(k-1)}$ 可以表示为单位矩阵加上前 $k-1$ 列的 “修正项”。每个修正项 $\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T$ 对应第 p 列的非零元素（除对角线外）。

第二步：计算 $\boldsymbol{P}^{(k)}\boldsymbol{L}^{(k-1)}\boldsymbol{P}^{(k)}$

利用分解式 (1.31)，我们有：

$$\begin{aligned}\boldsymbol{P}^{(k)}\boldsymbol{L}^{(k-1)}\boldsymbol{P}^{(k)} &= \boldsymbol{P}^{(k)}\left(\boldsymbol{I} + \sum_{p=1}^{k-1} \boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T\right)\boldsymbol{P}^{(k)} \\ &= \boldsymbol{P}^{(k)}\boldsymbol{I}\boldsymbol{P}^{(k)} + \sum_{p=1}^{k-1} \boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T\boldsymbol{P}^{(k)}.\end{aligned}$$

由于 $\boldsymbol{P}^{(k)}$ 是置换矩阵，满足 $\boldsymbol{P}^{(k)}(\boldsymbol{P}^{(k)})^T = \boldsymbol{I}$，且 $\boldsymbol{P}^{(k)}\boldsymbol{I}\boldsymbol{P}^{(k)} = \boldsymbol{I}$（单位矩阵在置换下不变）。

第三步：分析项 $\boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T\boldsymbol{P}^{(k)}$

对于 $p = 1, 2, \ldots, k-1$，我们需要分析 $\boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T\boldsymbol{P}^{(k)}$。

关键观察：由于 $p < k \leq \ell$，且 $\boldsymbol{P}^{(k)}$ 只交换第 k 行和第 ℓ 行，而 $\boldsymbol{e}^{(p)}$ 的第 p 个分量为 1，其余为 0，且 $p < k$，因此：

$$(\boldsymbol{e}^{(p)})^T\boldsymbol{P}^{(k)} = (\boldsymbol{e}^{(p)})^T.$$

这是因为 $\boldsymbol{P}^{(k)}$ 只影响第 k 行和第 ℓ 行，而 $\boldsymbol{e}^{(p)}$ 的非零分量在第 p 行（$p < k$），不受影响。

因此，式 (1.32) 可以进一步简化为：

$$\begin{aligned}\boldsymbol{P}^{(k)}\boldsymbol{L}^{(k-1)}\boldsymbol{P}^{(k)} &= \boldsymbol{I} + \sum_{p=1}^{k-1} \boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T\boldsymbol{P}^{(k)} \\ &= \boldsymbol{I} + \sum_{p=1}^{k-1} \boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T.\end{aligned}$$

第四步：理解结果矩阵的结构

设 $\hat{\boldsymbol{z}}^{(p)} = \boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}$，则

$$\hat{\boldsymbol{L}}^{(k-1)} = \boldsymbol{I} + \sum_{p=1}^{k-1} \hat{\boldsymbol{z}}^{(p)}(\boldsymbol{e}^{(p)})^T.$$

由于 $\boldsymbol{P}^{(k)}$ 交换第 k 行和第 ℓ 行，$\hat{\boldsymbol{z}}^{(p)} = \boldsymbol{P}^{(k)}\boldsymbol{z}^{(p)}$ 实际上是将 $\boldsymbol{z}^{(p)}$ 的第 k 个分量和第 ℓ 个分量进行了交换。这意味着 $\hat{\boldsymbol{L}}^{(k-1)}$ 的第 p 列（$p = 1, 2, \ldots, k-1$）中，第 k 行和第 ℓ 行的元素被交换了，而其他元素保持不变。

第五步：验证 $\hat{\boldsymbol{L}}^{(k-1)}$ 仍为单位下三角矩阵

由于 $\boldsymbol{L}^{(k-1)}$ 是单位下三角矩阵，其对角线元素全为 1，且当 $i < j$ 时 $[\boldsymbol{L}^{(k-1)}]_{ij} = 0$。经过置换 $\boldsymbol{P}^{(k)}\boldsymbol{L}^{(k-1)}\boldsymbol{P}^{(k)}$ 后：

- 对角线元素仍然全为 1（因为 $\boldsymbol{P}^{(k)}\boldsymbol{I}\boldsymbol{P}^{(k)} = \boldsymbol{I}$）；
- 对于 $i < j$ 且 $i, j \neq k, \ell$ 的情况，下三角性质保持不变；
- 对于涉及第 k 行或第 ℓ 行的情况，由于只交换了前 $k-1$ 列的元素，且条件 (1.29) 保证了第 k 列及以后的列中第 k 行以下的元素为 0，因此下三角性质仍然保持。

因此，$\hat{\boldsymbol{L}}^{(k-1)}$ 仍然是单位下三角矩阵，且满足条件 (1.29)。

为了更直观地理解定理，我们考虑一个 4×4 矩阵的情况。设已完成前 2 步消元，即 $k = 3$。构造一个满足条件 (1.29) 的单位下三角矩阵 $\boldsymbol{L}^{(2)}$：

$$\boldsymbol{L}^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & 0 & 1 \end{pmatrix}$$

验证条件 (1.29)：

- 对于 $i > j > 2$：当 $i = 4, j = 3$ 时，$4 > 3 > 2$ 为真，而 $l_{43} = 0$（满足条件）。
- 其他情况（如 $i = 4, j = 2$ 不满足 $j > 2$）不需考虑。

因此矩阵满足条件。

第 3 步选主元：假设根据主元选择规则，需要交换第 3 行（$k = 3$）和第 4 行（$\ell = 4$）。置换矩阵 $\boldsymbol{P}^{(3)}$ 交换第 3 行和第 4 行：

$$\boldsymbol{P}^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

注意：$\boldsymbol{P}^{(3)}$ 是对称且正交的，即 $(\boldsymbol{P}^{(3)})^T = \boldsymbol{P}^{(3)}$ 且 $\boldsymbol{P}^{(3)}\boldsymbol{P}^{(3)} = \boldsymbol{I}$。

计算 $\boldsymbol{P}^{(3)}\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)}$：

步骤 1：分解 $\boldsymbol{L}^{(2)}$ 为秩 1 和形式

根据公式 (1.31)：

$$\boldsymbol{L}^{(2)} = \boldsymbol{I} + \sum_{p=1}^{2} \boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T$$

其中：

- $p = 1$：$\boldsymbol{z}^{(1)} = \begin{pmatrix} 0 \\ l_{21} \\ l_{31} \\ l_{41} \end{pmatrix}$（注意第一个分量为 0，因为 $i \leq p = 1$），$(\boldsymbol{e}^{(1)})^T = (1, 0, 0, 0)$。
- $p = 2$：$\boldsymbol{z}^{(2)} = \begin{pmatrix} 0 \\ 0 \\ l_{32} \\ l_{42} \end{pmatrix}$（前两个分量为 0），$(\boldsymbol{e}^{(2)})^T = (0, 1, 0, 0)$。

验证：

$$\boldsymbol{I}+\boldsymbol{z}^{(1)}(\boldsymbol{e}^{(1)})^T+\boldsymbol{z}^{(2)}(\boldsymbol{e}^{(2)})^T=\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\end{pmatrix}+\begin{pmatrix}0&0&0&0\\l_{21}&0&0&0\\l_{31}&0&0&0\\l_{41}&0&0&0\end{pmatrix}+\begin{pmatrix}0&0&0&0\\0&0&0&0\\0&l_{32}&0&0\\0&l_{42}&0&0\end{pmatrix}$$

$$=\begin{pmatrix}1&0&0&0\\l_{21}&1&0&0\\l_{31}&l_{32}&1&0\\l_{41}&l_{42}&0&1\end{pmatrix}=\boldsymbol{L}^{(2)}$$

步骤 2：应用置换矩阵

根据公式 (1.32)：

$$\boldsymbol{P}^{(3)}\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)}=\boldsymbol{I}+\sum_{p=1}^{2}\boldsymbol{P}^{(3)}\boldsymbol{z}^{(p)}(\boldsymbol{e}^{(p)})^T$$

其中 $(\boldsymbol{e}^{(p)})^T\boldsymbol{P}^{(3)}=(\boldsymbol{e}^{(p)})^T$ 因为 $p=1,2<3$，列交换不影响前两列中基向量的位置。

计算各分量：

- $p=1$：$\boldsymbol{z}^{(1)}=(0,l_{21},l_{31},l_{41})^T$，$\boldsymbol{P}^{(3)}\boldsymbol{z}^{(1)}=(0,l_{21},l_{41},l_{31})^T$（交换第 3、4 分量）
- $p=2$：$\boldsymbol{z}^{(2)}=(0,0,l_{32},l_{42})^T$，$\boldsymbol{P}^{(3)}\boldsymbol{z}^{(2)}=(0,0,l_{42},l_{32})^T$（交换第 3、4 分量）

于是：

$$\boldsymbol{P}^{(3)}\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)}=\boldsymbol{I}+\begin{pmatrix}0&0&0&0\\l_{21}&0&0&0\\l_{41}&0&0&0\\l_{31}&0&0&0\end{pmatrix}+\begin{pmatrix}0&0&0&0\\0&0&0&0\\0&l_{42}&0&0\\0&l_{32}&0&0\end{pmatrix}=\begin{pmatrix}1&0&0&0\\l_{21}&1&0&0\\l_{41}&l_{42}&1&0\\l_{31}&l_{32}&0&1\end{pmatrix}$$

步骤 3：验证结果

这正是将 $\boldsymbol{L}^{(2)}$ 的第 3 行和第 4 行在前 $k-1=2$ 列的元素交换后得到的矩阵：

- 原始 $\boldsymbol{L}^{(2)}$ 第 1 列：第 3 行 l_{31}，第 4 行 l_{41}。交换后：第 3 行 l_{41}，第 4 行 l_{31}。
- 原始 $\boldsymbol{L}^{(2)}$ 第 2 列：第 3 行 l_{32}，第 4 行 l_{42}。交换后：第 3 行 l_{42}，第 4 行 l_{32}。

而第 3、4 列保持不变（因为只涉及前 2 列）。得到的矩阵 $\hat{\boldsymbol{L}}^{(2)}$ 仍是单位下三角矩阵。

直接计算 $\boldsymbol{P}^{(3)}\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)}$ 的矩阵乘法：

1. 计算 $\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)}$（右乘置换交换列 3 和列 4）：

$$\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)}=\begin{pmatrix}1&0&0&0\\l_{21}&1&0&0\\l_{31}&l_{32}&0&1\\l_{41}&l_{42}&1&0\end{pmatrix}$$

2. 计算 $\boldsymbol{P}^{(3)}(\boldsymbol{L}^{(2)}\boldsymbol{P}^{(3)})$（左乘置换交换行 3 和行 4）：

$$\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\l_{21}&1&0&0\\l_{31}&l_{32}&0&1\\l_{41}&l_{42}&1&0\end{pmatrix}=\begin{pmatrix}1&0&0&0\\l_{21}&1&0&0\\l_{41}&l_{42}&1&0\\l_{31}&l_{32}&0&1\end{pmatrix}$$

结果与秩 1 和形式推导一致。

这个例子展示了定理的核心思想：在部分主元高斯消元中，当进行行交换时，对已形成的下三角矩阵 $\boldsymbol{L}^{(k-1)}$ 同时进行相同的行和列交换（即相似变换），等价于只交换该矩阵前 $k-1$ 列中相应行的元素，且保持单位下三角结构不变。这在实际计算中意味着我们只需记录置换信息，并在更新 $\boldsymbol{L}$ 矩阵时交换相应行的部分元素，而无需进行完整的矩阵相乘操作，从而提高了计算效率。 □

注. 定理 1.5.4 的重要性在于：它说明了在带主元的 LU 分解中，当我们对矩阵进行行置换时，已经计算好的下三角矩阵 $\boldsymbol{L}^{(k-1)}$ 可以通过相应的置换操作得到新的下三角矩阵 $\hat{\boldsymbol{L}}^{(k-1)}$，而不需要重新计算。这为部分主元 LU 分解的算法实现提供了理论基础。

在实践中，我们不需要显式存储所有置换矩阵 $\boldsymbol{P}^{(k)}$。每次交换时，只需更新一个置换向量 $\boldsymbol{p}$，记录行的置换历史。具体而言，如果第 k 步交换了第 k 行和第 ℓ 行，则更新 $p[k] \leftrightarrow p[\ell]$。这样，最终的置换矩阵 $\boldsymbol{P}$ 可以通过置换向量 $\boldsymbol{p}$ 重构，大大节省了存储空间。

1.5.5 部分主元 LU 分解的算法实现

以下是部分主元 LU 分解的详细算法。在实际实现中，我们通常使用**置换向量**（permutation vector）来记录行置换，而不是显式存储置换矩阵 $\boldsymbol{P}$，样可以节省存储空间并提高计算效率。

Algorithm 3 部分主元 LU 分解（$\boldsymbol{PA} = \boldsymbol{LU}$）

Require: 矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$，容差 ε（判断矩阵奇异的阈值，通常取 $\varepsilon = 10^{-12}$）

Ensure: 单位下三角矩阵 $\boldsymbol{L}$，上三角矩阵 $\boldsymbol{U}$，置换向量 $\boldsymbol{p}$（记录行置换，$p[i]$ 表示原矩阵的第 $p[i]$ 行对应分解后的第 i 行），或误信息（矩阵奇异）

```
初始化：L ← I（单位矩阵），p ← [1, 2, ..., n]^T（初始置换向量）
for k = 1 to n − 1 do
    主元搜索：在第 k 列中寻找 l ∈ {k, ..., n}，使得 |A[l][k]| = max_{i=k,...,n} |A[i][k]|
    if |A[l][k]| < ε then
        错误：矩阵在机器精度下奇异，算法终止
    end if
    if l ≠ k then
        行置换：交换 A 的第 k 行与第 l 行
        交换 L 的第 k 行与第 l 行（仅前 k − 1 列）
        交换 p 的第 k 个元素与第 l 个元素
    end if
    消元计算：
    for i = k + 1 to n do
        乘数计算：L[i][k] ← A[i][k]/A[k][k]
        for j = k to n do
            行更新：A[i][j] ← A[i][j] − L[i][k] · A[k][j]
        end for
    end for
end for
if |A[n][n]| < ε then
    错误：矩阵在机器精度下奇异，算法终止
end if
U ← A（消元后的矩阵即为上三角矩阵）
返回 L，U，p
```

注. 算法说明：

1. **置换向量**：置换向量 $\boldsymbol{p}$ 记录行的置换历史。若 $p[i] = j$，表示原矩阵的第 j 行在分解后位于第 i 行。通过置换向量，我们以高效地构造置换矩阵 $\boldsymbol{P}$，或直接对右端项进行置换。
2. **原地存储**：算法中 $\boldsymbol{L}$ 的元素在消元时逐步计算，$\boldsymbol{U}$ 直接由消元后的矩阵得到，两者可以覆盖原始矩阵 $\boldsymbol{A}$ 的存储空间节省内存。
3. **奇异判断**：通过检查主元的绝对值是否小于容差 ε 来判断矩阵是否在机器精度下奇异。这一判断对于数值稳

定性至关重要。

4. **计算复杂度**：主元搜索需要 $\mathscr{O}(n-k)$ 次比较，消元需要 $\mathscr{O}((n-k)^2)$ 次运算，总复杂度仍为 $\mathscr{O}(n^3)$，与不带主元的 LU 分解同。

下面给出部分主元 LU 分解在实现层面的几点说明，代码与算法 3 逐行对应，便于读者理解算法的具体实现细节。读者可以根据本节的算法框架，用任意编程语言自行实现，并通过数值实验比较普通 LU 分解与部分主元 LU 分解在精度和稳定性上的差异。

注. 代码说明：

1. **存储策略**：代码中 $\boldsymbol{U}$ 直接使用 $\boldsymbol{A}$ 的副本，$\boldsymbol{L}$ 单独存储。实际应用中可以采用**原地存储**（in-place storage），将 $\boldsymbol{L}$ 和 $\boldsymbol{U}$ 存储在同一个矩阵中，以节省内存空间。
2. **置换向量**：置换向量 $\boldsymbol{p}$ 记录行的置换历史。若 $p[i]=j$，表示原矩阵的第 j 行在分解后位于第 i 行。通过置换向量，可以高效地构造置换矩阵 $\boldsymbol{P}$，或直接对右端项进行置换。
3. **数值稳定性**：通过选择绝对值最大的主元，可以减小乘数的绝对值，从而降低误差放大的风险，提高数值稳定性。
4. **时间复杂度**：主元搜索需要 $\mathscr{O}(n-k)$ 次比较，消元需要 $\mathscr{O}((n-k)^2)$ 次运算，总复杂度为 $\mathscr{O}(n^3)$，与不带主元的 LU 分解相同。

1.5.6 求解线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$

利用 $\boldsymbol{PA}=\boldsymbol{LU}$ 分解，可将线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$ 转化为：

$$\boldsymbol{LUx}=\boldsymbol{Pb}.$$

通过两步求解三角方程组得到解：

1. **置换右端项**：计算 $\boldsymbol{c}=\boldsymbol{Pb}$。由于 $\boldsymbol{P}$ 是置换矩阵，$\boldsymbol{c}$ 可以通过置换向量 $\boldsymbol{p}$ 高效构造：$c[i]=[p[i]]$（假设使用 1-based 索引）。
2. **前代法求解下三角方程组**：求解 $\boldsymbol{Ly}=\boldsymbol{c}$。由于 $\boldsymbol{L}$ 是单位下三角矩阵，有
$$y_1=c_1,\quad y_i=c_i-\sum_{j=1}^{i-1}L_{ij}y_j\quad(i=2,\dots,n). \tag{1.32}$$
3. **回代法求解上三角方程组**：求解 $\boldsymbol{Ux}=\boldsymbol{y}$。由于 $\boldsymbol{U}$ 是上三角矩阵，有
$$x_n=\frac{y_n}{U_{nn}},\quad x_i=\frac{y_i-\sum_{j=i+1}^{n}U_{ij}x_j}{U_{ii}}\quad(i=n-1,\dots,1). \tag{1.33}$$

例 1.5.4 (用部分主元 LU 分解求解含小主元的线性方程组). 考虑以下含小主元的线性方程组：

$$\boldsymbol{A}=\begin{pmatrix}\varepsilon & 1\\ 1 & 1\end{pmatrix},\quad \boldsymbol{b}=\begin{pmatrix}1\\2\end{pmatrix},$$

其中 $\varepsilon=10^{-8}$ 是一个很小的正数。

该方程组的精确解可由直接消元得到：

$$x_1=\frac{1}{1-\varepsilon},\quad x_2=\frac{1-2\varepsilon}{1-\varepsilon}.$$

当 $\varepsilon\ll 1$ 时，有近似

$$x_1\approx 1+\varepsilon,\quad x_2\approx 1-\varepsilon.$$

步骤 1：不选主元的消元分析 若不进行主元选取，以 $a_{11}=\varepsilon$ 作为主元，则乘数为

$$m_{21}=\frac{1}{\varepsilon}=10^8.$$

消元后得到

$$\boldsymbol{U} = \begin{pmatrix} \varepsilon & 1 \\ 0 & 1 - \dfrac{1}{\varepsilon} \end{pmatrix}, \quad \boldsymbol{L} = \begin{pmatrix} 1 & 0 \\ \dfrac{1}{\varepsilon} & 1 \end{pmatrix}.$$

在浮点运算中，由于 $1/\varepsilon$ 为极大数，计算 $1 - 1/\varepsilon$ 时会发生严重的大数减小数现象，从而导致数值不稳定，后续回代结果可能完全失真。

步骤 2：部分主元 LU 分解　在第 1 列中，

$$|a_{11}| = \varepsilon, \quad |a_{21}| = 1,$$

最大元素位于第 2 行，因此交换第 1、2 行，对应的置换矩阵为

$$\boldsymbol{P} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

此时

$$\boldsymbol{PA} = \begin{pmatrix} 1 & 1 \\ \varepsilon & 1 \end{pmatrix}.$$

以 1 为主元进行消元，乘数为 $m_{21} = \varepsilon$，得到

$$\boldsymbol{U} = \begin{pmatrix} 1 & 1 \\ 0 & 1 - \varepsilon \end{pmatrix}, \quad \boldsymbol{L} = \begin{pmatrix} 1 & 0 \\ \varepsilon & 1 \end{pmatrix}.$$

由于 $1 - \varepsilon$ 在双精度下可被精确表示，避免了数值不稳定问题。

步骤 3：置换右端项

$$\boldsymbol{Pb} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

步骤 4：前代求解 $\boldsymbol{Ly} = \boldsymbol{Pb}$

$$y_1 = 2, \quad y_2 = 1 - 2\varepsilon.$$

步骤 5：回代求解 $\boldsymbol{Ux} = \boldsymbol{y}$

$$x_2 = \frac{1 - 2\varepsilon}{1 - \varepsilon}, \quad x_1 = 2 - x_2 = \frac{1}{1 - \varepsilon}.$$

所得数值解与精确解在机器精度范围内一致。

数值实验对比　在双精度浮点运算中（$\varepsilon = 10^{-8}$）：

- **不选主元**：由于主元极小，消元过程中产生巨大乘数，舍入误差被严重放大，结果可能完全失真。
- **部分主元**：得到 $x_1 \approx 1.00000001$，$x_2 \approx 0.99999999$，与精确解高度一致。

该例清楚表明：当矩阵中存在小主元时，不选主元的高斯消元可能数值不稳定，而部分主元 LU 分解通过行交换有效避免了该问题，是保证算法数值稳定性的关键步骤。

1.5.7　完全主元法

部分主元仅在第 k 列搜索主元，而**完全主元法**（complete pivoting）在整个子矩阵（第 k 行到 n 行，第 k 列到 n 列）中搜索绝对值最大元素，需要同时进行行置换和列置换，分解形式为：

$$\boldsymbol{PAQ} = \boldsymbol{LU},$$

其中 $\boldsymbol{P}$ 为行置换矩阵，$\boldsymbol{Q}$ 为列置换矩阵。

1.6 对称正定矩阵的 Cholesky 分解

在前面的章节中，我们讨论了适用于一般矩阵的 LU 分解方法。然而，当矩阵具有特殊结构时，可以利用这些结构特性来设计更高效的算法。本节将讨论**对称正定矩阵**（symmetric positive definite matrix）的 Cholesky 分解，这是一种比 LU 分解更高效、更稳定的分解方法。

对称正定矩阵在科学计算中非常常见，例如在有限元方法、优化问题、统计推断等领域。利用矩阵的对称性，我们可以节省近一半的存储空间，而利用正定性，我们可以设计出数值稳定性更好的算法。

1.6.1 对称矩阵的 $\mathbf{LDL^T}$ 分解

在介绍 Cholesky 分解之前，我们先讨论对称矩阵的 $\mathrm{LDL^T}$ 分解，这是理解 Cholesky 分解的基础。

对于一般的矩阵 $\boldsymbol{A}$，其 LU 分解中的上三角矩阵 $\boldsymbol{U}$ 可以表示为对角矩阵 $\boldsymbol{D}$ 和单位上三角矩阵 $\boldsymbol{U}_0$ 的乘积：

$$\boldsymbol{U} = \begin{pmatrix} u_{11} & & & \\ & u_{22} & & \\ & & \ddots & \\ & & & u_{nn} \end{pmatrix} \begin{pmatrix} 1 & u_{12}/u_{11} & \cdots & u_{1n}/u_{11} \\ & 1 & \cdots & u_{2n}/u_{22} \\ & & \ddots & \vdots \\ & & & 1 \end{pmatrix} = \boldsymbol{D}\boldsymbol{U}_0,$$

其中 $\boldsymbol{U}_0$ 是单位上三角矩阵（对角线元素为 1），$\boldsymbol{D}$ 是对角矩阵，其元素为 u_{ii}。

因此，$\boldsymbol{A}$ 可以分解为：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{D}\boldsymbol{U}_0, \tag{1.34}$$

其中 $\boldsymbol{L}$ 是单位下三角矩阵，$\boldsymbol{D}$ 是对角矩阵，$\boldsymbol{U}_0$ 是单位上三角矩阵。

对称矩阵的特殊性质：如果 $\boldsymbol{A}$ 是对称矩阵（$\boldsymbol{A} = \boldsymbol{A}^T$），则有：

$$\boldsymbol{A} = \boldsymbol{A}^T = \boldsymbol{U}_0^T \boldsymbol{D}^T \boldsymbol{L}^T = \boldsymbol{U}_0^T \boldsymbol{D} \boldsymbol{L}^T,$$

其中我们利用了 $\boldsymbol{D}$ 是对角矩阵，故 $\boldsymbol{D}^T = \boldsymbol{D}$。

由于 $\boldsymbol{U}_0^T$ 是单位下三角矩阵，$\boldsymbol{D}\boldsymbol{L}^T$ 是上三角矩阵，而 LU 分解的唯一性（在适当的条件下）保证了 $\boldsymbol{U}_0^T = \boldsymbol{L}$。代入上式，得到：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{D}\boldsymbol{L}^T.$$

定理 1.6.1 (对称矩阵的 $\mathrm{LDL^T}$ 分解). 设 $\boldsymbol{A}$ 为 n 阶对称矩阵，且 $\boldsymbol{A}$ 的所有顺序主子式 $D_k \neq 0$（$k = 1, 2, \ldots, n-1$），则 $\boldsymbol{A}$ 可以唯一地分解为

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{D}\boldsymbol{L}^T, \tag{1.35}$$

其中 $\boldsymbol{L}$ 是单位下三角矩阵，$\boldsymbol{D}$ 是对角矩阵。

证明. 由于 $\boldsymbol{A}$ 的所有顺序主子式非零，根据 LU 分解的存在性定理，$\boldsymbol{A}$ 可以唯一地分解为 $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$，其中 $\boldsymbol{L}$ 是单位下三角矩阵，$\boldsymbol{U}$ 是上三角矩阵。

将 $\boldsymbol{U}$ 表示为 $\boldsymbol{U} = \boldsymbol{D}\boldsymbol{U}_0$，其中 $\boldsymbol{D}$ 是对角矩阵（对角线元素为 u_{ii}），$\boldsymbol{U}_0$ 是单位上三角矩阵。因此：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{D}\boldsymbol{U}_0.$$

由于 $\boldsymbol{A}$ 是对称矩阵，有：

$$\boldsymbol{A} = \boldsymbol{A}^T = \boldsymbol{U}_0^T \boldsymbol{D}^T \boldsymbol{L}^T = \boldsymbol{U}_0^T \boldsymbol{D} \boldsymbol{L}^T.$$

比较两种分解形式，由 LU 分解的唯一性，可得 $\boldsymbol{U}_0^T = \boldsymbol{L}$，即 $\boldsymbol{U}_0 = \boldsymbol{L}^T$。因此：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{D}\boldsymbol{L}^T.$$

唯一性由 LU 分解的唯一性保证。 □

注意. 当矩阵 $\boldsymbol{A}$ 奇异时，$\boldsymbol{D}$ 的最后一个对角元素为零。

1.6.2 Cholesky 分解：存在性与唯一性

对于对称正定矩阵，我们可以进一步简化分解形式，得到 Cholesky 分解。

引理 1.6.1 (对称正定矩阵的主主子矩阵)**.** 对称正定矩阵 $\boldsymbol{A}$ 的所有主主子矩阵 $[\boldsymbol{A}]_k$ 都是正则的（可逆的）。

证明. 设 k 固定，$\tilde{\boldsymbol{z}} \in \mathbb{R}^k$ 是任意非零向量。定义

$$\boldsymbol{z} = (\tilde{\boldsymbol{z}}^T, 0, \ldots, 0)^T = (z_1, z_2, \ldots, z_k, 0, \ldots, 0)^T.$$

由于 $\boldsymbol{A}$ 是正定的，有：

$$0 < \boldsymbol{z}^T \boldsymbol{A} \boldsymbol{z} = \tilde{\boldsymbol{z}}^T [\boldsymbol{A}]_k \tilde{\boldsymbol{z}},$$

这表明 $[\boldsymbol{A}]_k$ 是正定的，因此可逆。 □

定理 1.6.2 (Cholesky 分解的存在性与唯一性)**.** 如果 $\boldsymbol{A}$ 是对称正定矩阵，则它允许一个唯一的分解形式

$$\boldsymbol{A} = \boldsymbol{B}\boldsymbol{B}^T, \tag{1.36}$$

其中 $\boldsymbol{B}$ 是对角线元素全为正的下三角矩阵。

证明. **存在性**：首先，由于 $\boldsymbol{A}$ 是正定的，其所有主主子矩阵都是正则的（由引理 1.6.1），根据定理 1.6.1，它允许一个形如

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{D}\boldsymbol{L}^T \tag{1.37}$$

的分解。矩阵 $\boldsymbol{D}$ 的所有对角线元素都是正的。

实际上，对于固定的 i（$1 \le i \le n$），设 $\boldsymbol{z}$ 使得 $\boldsymbol{L}^T \boldsymbol{z} = \boldsymbol{e}_i$。则有

$$0 < \boldsymbol{z}^T \boldsymbol{A} \boldsymbol{z} = (\boldsymbol{L}^T \boldsymbol{z})^T \boldsymbol{D} (\boldsymbol{L}^T \boldsymbol{z}) = \boldsymbol{e}_i^T \boldsymbol{D} \boldsymbol{e}_i = d_{ii},$$

因为 $\boldsymbol{A}$ 是正定的。因此 $d_{ii} > 0$ 对所有 i 成立。

然后引入对角矩阵 $\boldsymbol{\Lambda}$，使得

$$\lambda_{ii} = \sqrt{d_{ii}},$$

可以将式 (1.37) 重写为

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{\Lambda}\boldsymbol{\Lambda}\boldsymbol{L}^T = (\boldsymbol{L}\boldsymbol{\Lambda})(\boldsymbol{L}\boldsymbol{\Lambda})^T = \boldsymbol{B}\boldsymbol{B}^T,$$

通过设

$$\boldsymbol{B} = \boldsymbol{L}\boldsymbol{\Lambda}$$

得到。由于 $\boldsymbol{L}$ 是单位下三角矩阵，$\boldsymbol{\Lambda}$ 是对角元素为正的对角矩阵，$\boldsymbol{B}$ 是对角线元素全为正的下三角矩阵。

唯一性：设存在另一个分解

$$\boldsymbol{A} = \tilde{\boldsymbol{B}}\tilde{\boldsymbol{B}}^T,$$

其中 $\tilde{\boldsymbol{B}}$ 是对角线元素为正的下三角矩阵。因此可以写成

$$\boldsymbol{B}\boldsymbol{B}^T = \tilde{\boldsymbol{B}}\tilde{\boldsymbol{B}}^T \Rightarrow \boldsymbol{B}^{-1}\boldsymbol{B}\boldsymbol{B}^T(\tilde{\boldsymbol{B}}^T)^{-1} = \boldsymbol{B}^{-1}\tilde{\boldsymbol{B}}\tilde{\boldsymbol{B}}^T(\tilde{\boldsymbol{B}}^T)^{-1} \Rightarrow \boldsymbol{B}^T(\tilde{\boldsymbol{B}}^T)^{-1} = \boldsymbol{B}^{-1}\tilde{\boldsymbol{B}}.$$

在这个最后的等式中，左边的矩阵是上三角矩阵，右边的矩阵是下三角矩阵，这只有当它们是对角矩阵时才可能，因此

$$\boldsymbol{B}^T(\tilde{\boldsymbol{B}}^T)^{-1} = \boldsymbol{B}^{-1}\tilde{\boldsymbol{B}} = \boldsymbol{D},$$

这意味着

$$\forall i = 1, 2, \ldots, n, \quad \frac{b_{ii}}{\tilde{b}_{ii}} = \frac{\tilde{b}_{ii}}{b_{ii}} = d_{ii} \Rightarrow (b_{ii})^2 = (\tilde{b}_{ii})^2,$$

因此最终 $b_{ii} = \tilde{b}_{ii}$，因为这些数是正的，所以

$$d_{ii} = 1.$$

$\boldsymbol{D}$ 因此是单位矩阵，所以 $\boldsymbol{B} = \tilde{\boldsymbol{B}}$，从而得到唯一性。 □

注. Cholesky 分解是 LU 分解对对称正定矩阵系统的适应。与分解 $\boldsymbol{LDL}^T$ 的情况一样，前面的结果仅仅是存在性结果，在实践中我们通过识别等式 $\boldsymbol{A}=\boldsymbol{BB}^T$ 中的 $\boldsymbol{A}$ 的项，逐列（或逐行）计算矩阵 $\boldsymbol{B}$。

1.6.3 Cholesky 算法

Cholesky 算法是 LU 分解对矩阵 $\boldsymbol{A}$ 为对称正定矩阵的系统的适应。该算法分为三个步骤：

1. 计算系统矩阵的分解 $\boldsymbol{A}=\boldsymbol{BB}^T$，
2. 求解第一个三角方程组 $\boldsymbol{By}=\boldsymbol{b}$，
3. 求解第二个三角方程组 $\boldsymbol{B}^T\boldsymbol{x}=\boldsymbol{y}$。

更精确地说，我们寻找系统矩阵 $\boldsymbol{A}$ 的一个形如 $\boldsymbol{A}=\boldsymbol{BB}^T$ 的分解，其中 $\boldsymbol{B}$ 是对角线元素为正的下三角矩阵（$\boldsymbol{B}^T$ 表示其转置）。我们通过识别矩阵等式 $\boldsymbol{A}=\boldsymbol{BB}^T$ 中对应位置的元素来得到这个分解。

计算过程：

步骤 1：计算 b_{11}

从 $\boldsymbol{A}$ 的第一行第一列元素，有：

$$a_{11}=\boldsymbol{B}_1(\boldsymbol{B}^T)_1=b_{11}^2.$$

由于 b_{11} 必须为正，有

$$b_{11}=\sqrt{a_{11}}. \tag{1.38}$$

步骤 2：计算 $\boldsymbol{B}$ 的第一列（b_{i1}，$i\geq 2$）

从 $\boldsymbol{A}$ 的第一列，有：

$$\boldsymbol{A}_1=\boldsymbol{B}(\boldsymbol{B}^T)_1=b_{11}\boldsymbol{B}_1\Leftrightarrow\boldsymbol{B}_1=\frac{1}{b_{11}}\boldsymbol{A}_1,$$

即对于 $i\geq 2$：

$$b_{i1}=\frac{a_{i1}}{b_{11}}. \tag{1.39}$$

步骤 3：计算 b_{jj}**（假设 $\boldsymbol{B}$ 的前 $j-1$ 列已知）**

从 $\boldsymbol{A}$ 的对角线元素，有：

$$a_{jj}=\boldsymbol{B}_j(\boldsymbol{B}^T)_j=\sum_{k=1}^{j}b_{jk}^2=\sum_{k=1}^{j-1}b_{jk}^2+b_{jj}^2.$$

因此：

$$b_{jj}=\sqrt{a_{jj}-\sum_{k=1}^{j-1}b_{jk}^2}. \tag{1.40}$$

步骤 4：计算 $\boldsymbol{B}$ 的第 j 列（b_{ij}，$i>j$，**假设 $\boldsymbol{B}$ 的前 $j-1$ 列已知）**

从 $\boldsymbol{A}$ 的第 j 列，有：

$$\boldsymbol{A}_j=\boldsymbol{B}(\boldsymbol{B}^T)_j=\sum_{k=1}^{j}b_{jk}\boldsymbol{B}_k\Leftrightarrow\boldsymbol{B}_j=\frac{1}{b_{jj}}\left(\boldsymbol{A}_j-\sum_{k=1}^{j-1}b_{jk}\boldsymbol{B}_k\right),$$

即对于 $i>j$：

$$b_{ij}=\frac{1}{b_{jj}}\left(a_{ij}-\sum_{k=1}^{j-1}b_{ik}b_{jk}\right). \tag{1.41}$$

与 Doolittle 分解一样，在每一列 j 中，我们只计算 $i\geq j$ 的项 b_{ij}，因为该列的其他项已知为零。

Algorithm 4 Cholesky 分解算法

Require: 对称正定矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$，容差 ε（判断矩阵奇异的阈值）

Ensure: 下三角矩阵 $\boldsymbol{B}$（对角线元素为正），使得 $\boldsymbol{A} = \boldsymbol{B}\boldsymbol{B}^T$，或错误信息（矩阵不正定）

```
初始化：B ← A（原地存储，B 将覆盖 A 的下三角部分）
for j = 1, 2, ..., n − 1 do
    计算对角线元素：b_jj ← sqrt(b_jj − Σ_{k=1}^{j−1} b_jk^2)
    if b_jj < ε then
        错误：矩阵在机器精度下不正定，算法终止
    end if
    计算第 j 列的下三角部分：
    for i = j + 1, j + 2, ..., n do
        b_ij ← (1/b_jj) (b_ij − Σ_{k=1}^{j−1} b_ik b_jk)
    end for
end for
计算最后一个对角线元素：b_nn ← sqrt(b_nn − Σ_{k=1}^{n−1} b_nk^2)
if b_nn < ε then
    错误：矩阵在机器精度下不正定，算法终止
end if
返回 B
```

注. 算法说明：

1. **存储效率**：算法 4 只使用矩阵 $\boldsymbol{A}$ 的下三角部分。因此，$\boldsymbol{A}$ 的上三角部分不需要存储。得到的下三角矩阵 $\boldsymbol{B}$ 存储在原始矩阵 $\boldsymbol{A}$ 的位置。
2. **计算复杂度**：简单分析表明，该算法大约需要 $\dfrac{1}{6}n^3$ 次乘除法运算和类似次数的加减法运算（忽略 n 次开方运算）。这是 LU 分解计算量的一半。
3. **数值稳定性**：如果矩阵是正定的，我们取平方根的数都是正的，我们用来除的数都是非零的（因为我们已经证明了分解存在）。然而，在算法中包含测试以确保这些性质得到验证是谨慎的做法。
4. **内存优化**：为了充分利用存储优势，对称矩阵的下三角部分应该存储在一维数组或其他压缩格式中。然而，这种方法在访问矩阵元素时相比使用标准二维数组会引入一些开销。实际程序通常同时提供压缩存储和标准二维数组存储选项供用户选择。

例 1.6.1 (Cholesky 分解)**.** 计算下面这个对称正定矩阵的 Cholesky 分解：

$$\boldsymbol{A} = \begin{pmatrix} 5 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 5 \end{pmatrix}.$$

解：

步骤 1：首先计算第一个对角线元素，$\sqrt{5} \approx 2.2361$，然后计算第一列的下三角部分：

$$b_{21} = \frac{a_{21}}{b_{11}} = \frac{-1}{2.2361} \approx -0.4472, \quad b_{31} = \frac{a_{31}}{b_{11}} = \frac{-1}{2.2361} \approx -0.4472.$$

此时矩阵状态为：

$$\boldsymbol{B} = \begin{pmatrix} 2.2361 & 0 & 0 \\ -0.4472 & * & 0 \\ -0.4472 & * & * \end{pmatrix}.$$

步骤 2：计算第二个对角线元素：

$$b_{22} = \sqrt{a_{22} - b_{21}^2} = \sqrt{3 - (-0.4472)^2} \approx \sqrt{3 - 0.2000} \approx 1.6733.$$

然后计算第二列的下三角部分：

$$b_{32} = \frac{1}{b_{22}}\left(a_{32} - b_{31}b_{21}\right) = \frac{1}{1.6733}\left(-1 - (-0.4472) \times (-0.4472)\right) \approx \frac{-1.2000}{1.6733} \approx -0.7171.$$

此时矩阵状态为：

$$\boldsymbol{B} = \begin{pmatrix} 2.2361 & 0 & 0 \\ -0.4472 & 1.6733 & 0 \\ -0.4472 & -0.7171 & * \end{pmatrix}.$$

步骤 3：计算第三个对角线元素：

$$b_{33} = \sqrt{a_{33} - b_{31}^2 - b_{32}^2} = \sqrt{5 - (-0.4472)^2 - (-0.7171)^2} \approx \sqrt{5 - 0.2000 - 0.5141} \approx 2.0702.$$

因此，Cholesky 分解为：

$$\boldsymbol{B} = \begin{pmatrix} 2.2361 & 0 & 0 \\ -0.4472 & 1.6733 & 0 \\ -0.4472 & -0.7171 & 2.0702 \end{pmatrix},$$

满足 $\boldsymbol{A} = \boldsymbol{B}\boldsymbol{B}^T$。

1.6.4 改进的平方根法（$\mathbf{LDL^T}$ 分解）

若要避免算法 4 中的开方运算，可根据 $\boldsymbol{LDL}^T$ 分解的方程 (1.35) 进行推导，得到的算法称为**改进的平方根法**（improved square root method），它与算法 4 具有相同的计算复杂度。

根据定理 1.6.1，改进的平方根法适用于任意对称矩阵，除非出现主元为零的情况。因此，在用它求解一般的对称线性方程组时，要进行选主元，而为了保证对称性，需做相同的行、列交换。即使这样，仍然不能完全避免零主元。对于一般的对称线性方程组，需使用更复杂的稳定求解算法。

注. 改进的平方根法的优势：

1. **避免开方运算**：改进的平方根法避免了 Cholesky 分解中的 n 次开方运算，这在某些计算环境中可能更高效。
2. **适用性更广**：改进的平方根法适用于所有对称矩阵（只要顺序主子式非零），而不仅仅是正定矩阵。
3. **数值稳定性**：虽然避免了开方运算，但改进的平方根法在数值稳定性方面与 Cholesky 分解相当。

1.6.5 Cholesky 分解的实现

在实际编程中，可以按照算法 4 的步骤实现 Cholesky 分解。读者可根据本节的推导与伪代码自行使用任意编程语言完成实现，并通过数值实验验证算法的正确性与数值稳定性。

注. 代码说明：

1. **存储策略**：代码中 $\boldsymbol{B}$ 单独存储，但实际应用中可以采用**原地存储**（in-place storage），将 $\boldsymbol{B}$ 存储在原始矩阵 $\boldsymbol{A}$ 的下三角部分，以节省内存空间。
2. **对称性利用**：由于 $\boldsymbol{A}$ 是对称矩阵，代码只使用下三角部分，上三角部分不需要存储或访问。
3. **数值稳定性**：通过检查对角线元素的平方根是否为正值，可以判断矩阵是否在机器精度下正定。如果出现负值或接近零的值，说明矩阵不正定或接近奇异。
4. **时间复杂度**：算法需要 $\mathscr{O}(n^3/6)$ 次乘除法运算，约为 LU 分解的一半。

1.6.6 应用：求解线性方程组 $\boldsymbol{Ax} = \boldsymbol{b}$

利用 Cholesky 分解 $\boldsymbol{A} = \boldsymbol{B}\boldsymbol{B}^T$，可将线性方程组 $\boldsymbol{Ax} = \boldsymbol{b}$ 转化为：

$$\boldsymbol{B}\boldsymbol{B}^T\boldsymbol{x} = \boldsymbol{b}.$$

通过两步求解三角方程组得到解：

1. **前代法求解下三角方程组**：求解 $\boldsymbol{B}\boldsymbol{y}=\boldsymbol{b}$。由于 $\boldsymbol{B}$ 是下三角矩阵，有

$$y_1=\frac{b_1}{B_{11}},\quad y_i=\frac{b_i-\sum_{j=1}^{i-1}B_{ij}y_j}{B_{ii}}\quad (i=2,\dots,n). \tag{1.42}$$

2. **回代法求解上三角方程组**：求解 $\boldsymbol{B}^T\boldsymbol{x}=\boldsymbol{y}$。由于 $\boldsymbol{B}^T$ 是上三角矩阵，有

$$x_n=\frac{y_n}{B_{nn}},\quad x_i=\frac{y_i-\sum_{j=i+1}^{n}B_{ji}x_j}{B_{ii}}\quad (i=n-1,\dots,1). \tag{1.43}$$

例 1.6.2 (用 Cholesky 分解求解线性方程组)**.** 考虑线性方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$，其中

$$\boldsymbol{A}=\begin{pmatrix}5&-1&-1\\-1&3&-1\\-1&-1&5\end{pmatrix},\quad \boldsymbol{b}=\begin{pmatrix}3\\1\\3\end{pmatrix}.$$

解：由例 1.6.1，$\boldsymbol{A}$ 的 Cholesky 分解为：

$$\boldsymbol{B}=\begin{pmatrix}2.2361&0&0\\-0.4472&1.6733&0\\-0.4472&-0.7171&2.0702\end{pmatrix}.$$

步骤 1：求解 $\boldsymbol{B}\boldsymbol{y}=\boldsymbol{b}$：

$$\begin{aligned}
y_1&=\frac{b_1}{B_{11}}=\frac{3}{2.2361}\approx 1.3416,\\
y_2&=\frac{b_2-B_{21}y_1}{B_{22}}=\frac{1-(-0.4472)\times 1.3416}{1.6733}\approx\frac{1.6000}{1.6733}\approx 0.9562,\\
y_3&=\frac{b_3-B_{31}y_1-B_{32}y_2}{B_{33}}=\frac{3-(-0.4472)\times 1.3416-(-0.7171)\times 0.9562}{2.0702}\approx\frac{3.2000}{2.0702}\approx 1.5457.
\end{aligned}$$

步骤 2：求解 $\boldsymbol{B}^T\boldsymbol{x}=\boldsymbol{y}$：

$$\begin{aligned}
x_3&=\frac{y_3}{B_{33}}=\frac{1.5457}{2.0702}\approx 0.7464,\\
x_2&=\frac{y_2-B_{32}x_3}{B_{22}}=\frac{0.9562-(-0.7171)\times 0.7464}{1.6733}\approx\frac{1.4914}{1.6733}\approx 0.8912,\\
x_1&=\frac{y_1-B_{21}x_2-B_{31}x_3}{B_{11}}=\frac{1.3416-(-0.4472)\times 0.8912-(-0.4472)\times 0.7464}{2.2361}\approx\frac{2.0736}{2.2361}\approx 0.9273.
\end{aligned}$$

因此，方程组的解为：

$$\boldsymbol{x}\approx\begin{pmatrix}0.9273\\0.8912\\0.7464\end{pmatrix}.$$

Chapter 2

最小二乘问题的求解

在科学计算和工程应用中，经常需要根据一组离散观测数据构造一个函数，用以刻画某种物理规律或统计关系。由于观测误差、数据量和模型复杂度等限制，通常无法找到一个函数精确通过所有数据点，因此需要采用适当的准则度量“拟合好坏”，并在这一准则下寻找最优近似。

本章围绕**最小二乘法**（least squares method）展开讨论。最小二乘法通过最小化平方误差来度量拟合质量，是数值分析中应用最广泛的方法之一，广泛出现在曲线拟合、数据回归、信号处理、机器学习等领域。

本章路线图　本章将依次回答以下几个问题：

- **本章研究哪类问题**：从函数拟合出发，给出一般最小二乘问题的矩阵形式，并澄清插值与拟合的区别。
- **为什么重要**：说明最小二乘问题在数据分析与模型反演中的基础地位，以及它与线性方程组求解的关系。
- **采用哪些分析工具**：通过二次型展开和梯度条件得到法方程 $\boldsymbol{A}^T\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^T\boldsymbol{b}$，并用线性代数工具刻画解的存在唯一性与几何结构。
- **如何在后文反复使用**：在本章后半部分，基于法方程和分解方法（尤其是 Cholesky 分解）给出具体算法，并通过典型应用（回归模型、物理与化学参数反演等）展示其作用，这些技巧将在后续章节的参数估计和误差分析中再次出现。

本章的一个贯穿性视角是：最小二乘问题可以看作“用一个低维线性子空间去近似高维数据”，其核心是理解**投影、残差与条件数**之间的关系。

2.1　最小二乘问题的一般表述

本节的目标是用统一的数学语言刻画最小二乘问题：从一维函数拟合出发，引入一般矩阵形式的最小二乘问题，并给出“最小二乘解”的精确定义与基本假设。

2.1.1　问题背景与数学模型

在实际的科学计算和工程应用中，经常会遇到如下类型的任务：给定一组观测数据点 (t_i, b_i)，$i = 1, 2, \ldots, m$，其中 t_i 互不相同，希望在预先选定的一类函数中选出一个函数，用来近似这些数据。

函数拟合的数学表述：设有一族函数

$$f_1(t), f_2(t), \ldots, f_n(t), \quad n \le m, \tag{2.1}$$

它们是线性无关的，称为**基函数**（basis functions）。给定 n 个实数 $x_1, x_2, \ldots, x_n$，可以构造函数

$$f(t) = \sum_{k=1}^{n} x_k f_k(t),$$

并定义**平方误差函数**：

$$E(\boldsymbol{x}) = \sum_{i=1}^{m} [f(t_i) - b_i]^2 = \sum_{i=1}^{m} \left[\sum_{k=1}^{n} x_k f_k(t_i) - b_i \right]^2, \tag{2.2}$$

其中 $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^T$。量 $E(\boldsymbol{x})$ 表示拟合函数 $f(t)$ 在点 t_i 处的值与给定值 b_i 之间的平方误差之和。

最小二乘问题：寻找 $\hat{\boldsymbol{x}} \in \mathbb{R}^n$，使得

$$E(\hat{\boldsymbol{x}}) \leq E(\boldsymbol{x}), \quad \forall \boldsymbol{x} \in \mathbb{R}^n.$$

即使得平方误差函数达到最小值。

注. **术语说明**：

- **插值**（interpolation）：要求函数精确通过所有数据点。
- **拟合**（fitting）或**回归**（regression）：不要求函数通过所有数据点，而是最小化某种误差度量。
- **最小二乘法**：使用平方误差作为误差度量的拟合方法。

2.1.2 矩阵形式与理论框架

本小节将把前述函数拟合问题转写为矩阵形式的最小二乘问题，并给出一般情形下的数学定义和基本假设，为后续的理论分析和算法设计提供统一框架。

从函数拟合到线性方程组　考虑一般的最小二乘问题。假设我们需要求解线性方程组

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}, \tag{2.3}$$

其中 $\boldsymbol{A} \in \mathbb{R}^{m \times n}$，$\boldsymbol{b} \in \mathbb{R}^m$ 非零，且假设方程个数严格大于未知数个数（$m > n$）。在大多数情况下，该方程组没有精确解。

此时，我们寻求一个**近似解**，使得差值 $\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}$ 尽可能小。一种自然的选择是最小化该差值的**欧几里得范数**（Euclidean norm），即

$$\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2 = \sqrt{\sum_{i=1}^{m} (\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b})_i^2}.$$

定义 2.1.1 (最小二乘问题). 设给定 $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ 和 $\boldsymbol{b} \in \mathbb{R}^m$。我们称如下问题为**最小二乘问题**：

$$\min_{\boldsymbol{x} \in \mathbb{R}^n} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2. \tag{2.4}$$

我们记 $\hat{\boldsymbol{x}}$ 为该问题的解，称为**最小二乘解**（least squares solution）。

基本假设：为了确保最小二乘问题有唯一解，我们需要以下基本假设：

$$\text{矩阵 } \boldsymbol{A} \text{ 的列向量线性无关} \quad \Leftrightarrow \quad \operatorname{rank}(\boldsymbol{A}) = n. \tag{2.5}$$

注. **特殊情况**：

- 如果 $m = n$ 且 $\boldsymbol{A}$ 可逆，则 $\hat{\boldsymbol{x}}$ 是 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 的唯一解，此时最小二乘问题退化为普通的线性方程组求解问题。
- 本章主要关注 $m > n$ 的情况，即**超定方程组**（overdetermined system）。

函数拟合问题的矩阵形式：对于前面介绍的函数拟合问题，表达式

$$\sum_{k=1}^{n} x_k f_k(t_i) - b_i, \quad 1 \leq i \leq m$$

可以写成 $\boldsymbol{Ax}-\boldsymbol{b}$ 的形式，其中矩阵 $\boldsymbol{A}$ 的元素为

$$a_{ij}=f_j(t_i),\quad 1\le i\le m,\quad 1\le j\le n. \tag{2.6}$$

因此，我们需要最小化

$$E(\boldsymbol{x})=\sum_{i=1}^{m}\left[\sum_{k=1}^{n}x_kf_k(t_i)-b_i\right]^2=\|\boldsymbol{Ax}-\boldsymbol{b}\|_2^2. \tag{2.7}$$

此时最小值通常不为零，除非数据点恰好可以用给定的基函数线性组合精确表示。

2.1.3 法方程理论及其几何解释

目标函数的展开与二次形式

目标函数的展开 为了便于分析最小二乘问题，可以先将平方误差函数展开成二次型的形式。对

$$\begin{aligned}E(\boldsymbol{x})&=\|\boldsymbol{Ax}-\boldsymbol{b}\|_2^2=(\boldsymbol{Ax}-\boldsymbol{b})^T(\boldsymbol{Ax}-\boldsymbol{b})\\&=\boldsymbol{x}^T\boldsymbol{A}^T\boldsymbol{Ax}-\boldsymbol{b}^T\boldsymbol{Ax}-\boldsymbol{x}^T\boldsymbol{A}^T\boldsymbol{b}+\|\boldsymbol{b}\|_2^2\\&=\boldsymbol{x}^T\boldsymbol{A}^T\boldsymbol{Ax}-2(\boldsymbol{A}^T\boldsymbol{b})^T\boldsymbol{x}+\|\boldsymbol{b}\|_2^2,\end{aligned}$$

其中我们利用了 $\boldsymbol{b}^T\boldsymbol{Ax}=(\boldsymbol{A}^T\boldsymbol{b})^T\boldsymbol{x}$（因为两者都是标量，且标量的转置等于自身）。

注意到 $\|\boldsymbol{b}\|_2^2$ 是常数项，不影响最小值的位置，因此最小二乘问题可以重新表述为

$$\min_{\boldsymbol{x}\in\mathbb{R}^n}J(\boldsymbol{x})=\boldsymbol{x}^T\boldsymbol{Gx}-2\boldsymbol{h}^T\boldsymbol{x}, \tag{2.8}$$

其中 $\boldsymbol{G}=\boldsymbol{A}^T\boldsymbol{A}$ 是对称矩阵，$\boldsymbol{h}=\boldsymbol{A}^T\boldsymbol{b}$ 是给定的向量。

定义 2.1.2 (二次函数)**.** 我们称形如

$$J(\boldsymbol{x})=\boldsymbol{x}^T\boldsymbol{Gx}-2\boldsymbol{h}^T\boldsymbol{x} \tag{2.9}$$

的函数 $J:\mathbb{R}^n\to\mathbb{R}$ 为**二次函数**（quadratic function），其中 $\boldsymbol{G}$ 是 $n\times n$ 对称矩阵，$\boldsymbol{h}$ 是 $\mathbb{R}^n$ 中的给定向量。

例 2.1.1 (二次函数的具体形式)**.** 考虑一个 2×2 的最小二乘问题，其中

$$\boldsymbol{A}=\begin{pmatrix}1&2\\1&3\\1&4\end{pmatrix},\quad \boldsymbol{b}=\begin{pmatrix}5\\7\\9\end{pmatrix}$$

对应的二次函数为 $J(\boldsymbol{x})=\boldsymbol{x}^T\boldsymbol{Gx}-2\boldsymbol{h}^T\boldsymbol{x}$，其中：

$$\boldsymbol{G}=\boldsymbol{A}^T\boldsymbol{A}=\begin{pmatrix}1&1&1\\2&3&4\end{pmatrix}\begin{pmatrix}1&2\\1&3\\1&4\end{pmatrix}=\begin{pmatrix}3&9\\9&29\end{pmatrix},\quad \boldsymbol{h}=\boldsymbol{A}^T\boldsymbol{b}=\begin{pmatrix}1&1&1\\2&3&4\end{pmatrix}\begin{pmatrix}5\\7\\9\end{pmatrix}=\begin{pmatrix}21\\67\end{pmatrix}$$

因此，二次函数的具体形式为：

$$\begin{aligned}J(x_1,x_2)&=\begin{pmatrix}x_1&x_2\end{pmatrix}\begin{pmatrix}3&9\\9&29\end{pmatrix}\begin{pmatrix}x_1\\x_2\end{pmatrix}-2\begin{pmatrix}21&67\end{pmatrix}\begin{pmatrix}x_1\\x_2\end{pmatrix}\\&=3x_1^2+18x_1x_2+29x_2^2-42x_1-134x_2\end{aligned}$$

这是一个关于 x_1 和 x_2 的二次函数，其二次项系数矩阵 $\boldsymbol{G}$ 是对称矩阵（$g_{12}=g_{21}=9$），一次项系数向量为 $-2\boldsymbol{h}=(-42,-134)^T$。

验证对称性：展开 $\boldsymbol{x}^T\boldsymbol{Gx}$：

$$\boldsymbol{x}^T\boldsymbol{Gx}=x_1(3x_1+9x_2)+x_2(9x_1+29x_2)=3x_1^2+9x_1x_2+9x_2x_1+29x_2^2=3x_1^2+18x_1x_2+29x_2^2$$

由于 $\boldsymbol{G}$ 对称，交叉项 x_1x_2 和 x_2x_1 的系数相同，合并为 $18x_1x_2$。

最优性条件：对于多元函数 $J:\mathbb{R}^n\to\mathbb{R}$，如果 J 连续可导，则

$$J(\hat{\boldsymbol{x}})\le J(\boldsymbol{x})\quad \forall \boldsymbol{x}\in\mathbb{R}^n \quad\Rightarrow\quad \nabla J(\hat{\boldsymbol{x}})=\boldsymbol{0}, \tag{2.10}$$

其中 ∇ 是梯度算子。

梯度的计算：对于二次函数 $J(\boldsymbol{x})=\boldsymbol{x}^T\boldsymbol{G}\boldsymbol{x}-2\boldsymbol{h}^T\boldsymbol{x}$，我们需要计算其梯度。展开函数 J：

$$J(\boldsymbol{x})=\sum_{i=1}^n\sum_{j=1}^n x_i g_{ij}x_j-2\sum_{i=1}^n h_i x_i.$$

对 x_k 求偏导数：

$$\begin{aligned}\frac{\partial J}{\partial x_k}&=\frac{\partial}{\partial x_k}\left[\sum_{i=1}^n\sum_{j=1}^n x_i g_{ij}x_j-2\sum_{i=1}^n h_i x_i\right]\\&=\sum_{i=1}^n\sum_{j=1}^n\frac{\partial}{\partial x_k}(x_i g_{ij}x_j)-2h_k\\&=\sum_{i=1}^n x_i g_{ik}+\sum_{j=1}^n g_{kj}x_j-2h_k\\&=(\boldsymbol{G}\boldsymbol{x})_k+(\boldsymbol{G}^T\boldsymbol{x})_k-2h_k.\end{aligned}$$

因为 $\boldsymbol{G}$ 是对称矩阵（$\boldsymbol{G}^T=\boldsymbol{G}$），有 $(\boldsymbol{G}^T\boldsymbol{x})_k=(\boldsymbol{G}\boldsymbol{x})_k$，因此

$$\frac{\partial J}{\partial x_k}=2(\boldsymbol{G}\boldsymbol{x})_k-2h_k=2(\boldsymbol{G}\boldsymbol{x}-\boldsymbol{h})_k.$$

由此得到梯度的完整表达式：

$$\nabla J(\boldsymbol{x})=2(\boldsymbol{G}\boldsymbol{x}-\boldsymbol{h}). \tag{2.11}$$

因此，最小二乘问题的解 $\hat{\boldsymbol{x}}$ 必须满足

$$\nabla J(\hat{\boldsymbol{x}})=2(\boldsymbol{G}\hat{\boldsymbol{x}}-\boldsymbol{h})=\boldsymbol{0},$$

即

$$\boldsymbol{G}\hat{\boldsymbol{x}}=\boldsymbol{h}\quad\Leftrightarrow\quad \boldsymbol{A}^T\boldsymbol{A}\hat{\boldsymbol{x}}=\boldsymbol{A}^T\boldsymbol{b}. \tag{2.12}$$

这些关系式称为**法方程**（normal equations），它们是 $\hat{\boldsymbol{x}}$ 为最小值的必要条件。

几何解释

几何理解　在法方程中，“法”（normal）一词对应“正交”的含义。这一观点在理解最小二乘解的结构及其误差意义方面尤为重要：最小二乘解满足残差向量 $\boldsymbol{r}=\boldsymbol{b}-\boldsymbol{A}\hat{\boldsymbol{x}}$ 与矩阵 $\boldsymbol{A}$ 的列空间正交。

设 $\boldsymbol{A}$ 的列向量为 $\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_n$，则 $\boldsymbol{A}$ 的列空间为 $\operatorname{span}\{\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots,\boldsymbol{a}_n\}$。残差向量 $\boldsymbol{r}$ 与列空间正交意味着：

$$\boldsymbol{a}_j^T\boldsymbol{r}=0,\quad j=1,2,\ldots,n,$$

即

$$\boldsymbol{A}^T\boldsymbol{r}=\boldsymbol{A}^T(\boldsymbol{b}-\boldsymbol{A}\hat{\boldsymbol{x}})=\boldsymbol{0},$$

这正是法方程 $\boldsymbol{A}^T\boldsymbol{A}\hat{\boldsymbol{x}}=\boldsymbol{A}^T\boldsymbol{b}$ 的另一种表达。

几何意义：最小二乘解 $\hat{\boldsymbol{x}}$ 使得 $\boldsymbol{A}\hat{\boldsymbol{x}}$ 是 $\boldsymbol{b}$ 在 $\boldsymbol{A}$ 的列空间上的正交投影，残差向量 $\boldsymbol{r}$ 垂直于该列空间。

定理 2.1.1 (最小二乘问题的存在性与唯一性)**.** 设给定 $\boldsymbol{A}\in\mathbb{R}^{m\times n}$ 和 $\boldsymbol{b}\in\mathbb{R}^m$，如果假设矩阵 $\boldsymbol{A}$ 的秩为 n（即列向量线性无关），则最小二乘问题

$$\min_{\boldsymbol{x}\in\mathbb{R}^n}\|\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b}\|_2^2 \tag{2.13}$$

有唯一解 $\hat{\boldsymbol{x}}$，由下式给出：

$$\boldsymbol{A}^T\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^T\boldsymbol{b}. \tag{2.14}$$

证明. **存在性**：首先注意到，如果 $\boldsymbol{A}$ 的秩为 n，则 $\boldsymbol{G} = \boldsymbol{A}^T\boldsymbol{A}$ 是正定矩阵。事实上：

- 对于任意 $\boldsymbol{x} \in \mathbb{R}^n$，有 $\boldsymbol{x}^T\boldsymbol{G}\boldsymbol{x} = \|\boldsymbol{A}\boldsymbol{x}\|_2^2 \geq 0$。
- 由于 $\boldsymbol{A}$ 的秩为 n，$\mathrm{Ker}(\boldsymbol{A})$ 的维数为零，因此

$$\boldsymbol{x}^T\boldsymbol{G}\boldsymbol{x} = 0 \quad \Leftrightarrow \quad \|\boldsymbol{A}\boldsymbol{x}\|_2^2 = 0 \quad \Leftrightarrow \quad \boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} \quad \Leftrightarrow \quad \boldsymbol{x} = \boldsymbol{0}.$$

因此，矩阵 $\boldsymbol{G}$ 是正定的，从而可逆，所以存在唯一解 $\hat{\boldsymbol{x}}$ 满足

$$\boldsymbol{A}^T\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^T\boldsymbol{b} \quad \Leftrightarrow \quad \boldsymbol{G}\hat{\boldsymbol{x}} = \boldsymbol{h}.$$

充分性：现在证明 $\hat{\boldsymbol{x}}$ 确实实现了 J 的最小值。设 $\boldsymbol{y} = \boldsymbol{x} - \hat{\boldsymbol{x}}$，因此 $\boldsymbol{y} \neq \boldsymbol{0}$。有

$$\begin{aligned}
J(\hat{\boldsymbol{x}} + \boldsymbol{y}) &= (\hat{\boldsymbol{x}} + \boldsymbol{y})^T\boldsymbol{G}(\hat{\boldsymbol{x}} + \boldsymbol{y}) - 2\boldsymbol{h}^T(\hat{\boldsymbol{x}} + \boldsymbol{y}) \\
&= \hat{\boldsymbol{x}}^T\boldsymbol{G}\hat{\boldsymbol{x}} + \boldsymbol{y}^T\boldsymbol{G}\boldsymbol{y} + 2\hat{\boldsymbol{x}}^T\boldsymbol{G}\boldsymbol{y} - 2\boldsymbol{h}^T\boldsymbol{y} - 2\boldsymbol{h}^T\hat{\boldsymbol{x}} \\
&= \hat{\boldsymbol{x}}^T\boldsymbol{G}\hat{\boldsymbol{x}} + \boldsymbol{y}^T\boldsymbol{G}\boldsymbol{y} + 2(\boldsymbol{G}\hat{\boldsymbol{x}} - \boldsymbol{h})^T\boldsymbol{y} - 2\boldsymbol{h}^T\hat{\boldsymbol{x}} \\
&= \boldsymbol{y}^T\boldsymbol{G}\boldsymbol{y} + \hat{\boldsymbol{x}}^T\boldsymbol{G}\hat{\boldsymbol{x}} - 2\boldsymbol{h}^T\hat{\boldsymbol{x}},
\end{aligned}$$

因为 $\boldsymbol{G}\hat{\boldsymbol{x}} = \boldsymbol{h}$。因此

$$J(\hat{\boldsymbol{x}} + \boldsymbol{y}) = J(\hat{\boldsymbol{x}}) + \boldsymbol{y}^T\boldsymbol{G}\boldsymbol{y}.$$

由于 $\boldsymbol{G}$ 是正定矩阵，且 $\boldsymbol{y} \neq \boldsymbol{0}$，有 $\boldsymbol{y}^T\boldsymbol{G}\boldsymbol{y} > 0$，因此

$$J(\hat{\boldsymbol{x}} + \boldsymbol{y}) > J(\hat{\boldsymbol{x}}), \quad \forall \boldsymbol{y} \in \mathbb{R}^n, \boldsymbol{y} \neq \boldsymbol{0},$$

这表明 $\hat{\boldsymbol{x}}$ 实现了 J 的最小值。 □

注. **法方程的重要性与局限性**：

- 法方程将 $m \times n$ $(m > n)$ 的超定方程组转化为 $n \times n$ 的对称正定方程组。
- 数值稳定性问题：当 $\boldsymbol{A}$ 的条件数较大时，$\boldsymbol{A}^T\boldsymbol{A}$ 的条件数约为 $\mathrm{cond}(\boldsymbol{A})^2$，可能放大误差。
- 对于病态问题，需要更稳定的算法如 QR 分解或 SVD。

2.2 法方程方法的算法实现与应用

根据定理 2.1.1，最小二乘问题可以通过求解法方程 $\boldsymbol{A}^T\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^T\boldsymbol{b}$ 来求解。本节将介绍法方程方法的具体算法实现，并通过完整的例子展示其应用。

2.2.1 算法描述与数值稳定性

法方程方法的基本思想是：

1. 构造法方程的系数矩阵 $\boldsymbol{G} = \boldsymbol{A}^T\boldsymbol{A}$ 和右端项 $\boldsymbol{h} = \boldsymbol{A}^T\boldsymbol{b}$。
2. 求解线性方程组 $\boldsymbol{G}\hat{\boldsymbol{x}} = \boldsymbol{h}$。
3. 由于 $\boldsymbol{G}$ 是对称正定矩阵，可以使用 Cholesky 分解等方法高效求解。

Algorithm 5 法方程方法求解最小二乘问题

Require: 矩阵 $\boldsymbol{A} \in \mathbb{R}^{m\times n}$（$m > n$，$\operatorname{rank}(\boldsymbol{A}) = n$），向量 $\boldsymbol{b} \in \mathbb{R}^m$

Ensure: 最小二乘解 $\hat{\boldsymbol{x}} \in \mathbb{R}^n$

```
计算 G ← A^T A（n × n 对称矩阵）
计算 h ← A^T b（n 维向量）
使用 Cholesky 分解求解 G x̂ = h：
    分解：G = L L^T（L 为下三角矩阵）
    前代：求解 L y = h 得到 y
    回代：求解 L^T x̂ = y 得到 x̂
返回 x̂
```

注. **算法复杂度与数值稳定性分析：**

1. **计算复杂度：**计算 $\boldsymbol{A}^T\boldsymbol{A}$ 需要 $\mathscr{O}(mn^2)$ 次浮点运算，Cholesky 分解需要 $\mathscr{O}(n^3/3)$ 次运算。
2. **数值稳定性问题：**当 $\boldsymbol{A}$ 的列向量近似线性相关时，$\boldsymbol{G}$ 可能接近奇异，导致数值不稳定。
3. **改进措施：**对于病态问题，可以采用列主元 Cholesky 分解或正则化技术。

算法实现思路

在实际编程时，可以按本节推导的步骤实现法方程方法：首先构造正规方程 $G = \boldsymbol{A}^T\boldsymbol{A}$ 和 $h = \boldsymbol{A}^T\boldsymbol{b}$，然后利用 Cholesky 分解和前代、回代分别求解 $G\boldsymbol{x} = h$。读者可根据本书给出的算法框架，用自己熟悉的编程语言实现，并通过数值实验验证最小二乘解的性质，例如考察拟合参数与真实参数的接近程度、残差范数的大小以及数据点拟合曲线的一致性。

2.2.2　应用实例

例 2.2.1 (化学动力学问题：一级反应速率常数确定). 在化学动力学研究中，我们通过实验测量了反应物浓度随时间的变化，希望确定反应速率常数。假设反应为一级反应，浓度 C 与时间 t 的关系理论上是：

$$C(t) = C_0 e^{-kt}$$

其中 C_0 是初始浓度，k 是反应速率常数。实验测得的数据如下：

表 2.1　反应物浓度随时间变化数据

时间 t (min)	0	1	2	3	4	5
浓度 C (mol/L)	1.00	0.82	0.67	0.55	0.45	0.37

解：

步骤 1：线性化处理

对于一级反应，两边取自然对数：

$$\ln C(t) = \ln C_0 - kt$$

令 $y = \ln C$，$x_1 = \ln C_0$，$x_2 = -k$，则问题转化为线性拟合：

$$y = x_1 + x_2 t$$

将实验数据取对数后得到：

表 2.2　取对数后的反应数据

时间 t (min)	0	1	2	3	4	5
$\ln C$	0	-0.198	-0.400	-0.598	-0.799	-0.994

步骤 2：构建矩阵形式

矩阵形式为：

$$\boldsymbol{A}=\begin{pmatrix}1&0\\1&1\\1&2\\1&3\\1&4\\1&5\end{pmatrix},\quad \boldsymbol{b}=\begin{pmatrix}0\\-0.198\\-0.400\\-0.598\\-0.799\\-0.994\end{pmatrix},\quad \boldsymbol{x}=\begin{pmatrix}x_1\\x_2\end{pmatrix}$$

步骤 3：构造法方程

计算：

$$\boldsymbol{A}^T\boldsymbol{A}=\begin{pmatrix}6&15\\15&55\end{pmatrix},\quad \boldsymbol{A}^T\boldsymbol{b}=\begin{pmatrix}-2.989\\-10.463\end{pmatrix}$$

步骤 4：求解法方程

解法方程：

$$\begin{pmatrix}6&15\\15&55\end{pmatrix}\begin{pmatrix}x_1\\x_2\end{pmatrix}=\begin{pmatrix}-2.989\\-10.463\end{pmatrix}$$

使用 Cholesky 分解：$\boldsymbol{A}^T\boldsymbol{A}=\boldsymbol{L}\boldsymbol{L}^T$，其中

$$\boldsymbol{L}=\begin{pmatrix}\sqrt{6}&0\\\dfrac{15}{\sqrt{6}}&\sqrt{55-\dfrac{225}{6}}\end{pmatrix}\approx\begin{pmatrix}2.449&0\\6.124&2.236\end{pmatrix}$$

前代求解 $\boldsymbol{L}\boldsymbol{y}=\boldsymbol{A}^T\boldsymbol{b}$：

$$y_1=\frac{-2.989}{2.449}\approx-1.220,\quad y_2=\frac{-10.463-6.124\times(-1.220)}{2.236}\approx-1.333$$

回代求解 $\boldsymbol{L}^T\boldsymbol{x}=\boldsymbol{y}$：

$$x_2=\frac{-1.333}{2.236}\approx-0.596,\quad x_1=\frac{-1.220-6.124\times(-0.596)}{2.449}\approx0.996$$

步骤 5：结果解释

因此：

$$C_0=e^{x_1}\approx e^{0.996}\approx2.708\ \text{mol/L},\quad k=-x_2\approx0.596\ \text{min}^{-1}$$

拟合曲线为 $C(t)=2.708e^{-0.596t}$。

验证：计算残差平方和：

$$E(\hat{\boldsymbol{x}})=\|\boldsymbol{A}\hat{\boldsymbol{x}}-\boldsymbol{b}\|_2^2\approx0.0012$$

误差很小，拟合效果良好。

例 2.2.2 (多元线性回归：房价预测模型)**.** 考虑一个实际的经济学问题：预测房价。假设房价 y 与房屋面积 x_1、卧室数量 x_2、房龄 x_3 有关。观测数据如下：

表 2.3　房屋价格数据

面积 x_1 (mš)	房间数 x_2	楼层 x_3	价格 y (万元)
80	2	1	120
100	3	2	180
120	3	3	220
90	2	2	150
110	4	2	200

解：

步骤 1：建立模型

建立线性模型：

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \varepsilon$$

其中 ε 是随机误差。

步骤 2：矩阵形式

矩阵形式为：

$$\boldsymbol{X} = \begin{pmatrix} 1 & 80 & 2 & 1 \\ 1 & 100 & 3 & 2 \\ 1 & 120 & 3 & 3 \\ 1 & 90 & 2 & 2 \\ 1 & 110 & 4 & 2 \end{pmatrix}, \quad \boldsymbol{y} = \begin{pmatrix} 120 \\ 180 \\ 220 \\ 150 \\ 200 \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \end{pmatrix}$$

步骤 3：构造法方程

计算：

$$\boldsymbol{X}^T\boldsymbol{X} = \begin{pmatrix} 5 & 500 & 14 & 10 \\ 500 & 51000 & 1400 & 1000 \\ 14 & 1400 & 42 & 28 \\ 10 & 1000 & 28 & 22 \end{pmatrix}, \quad \boldsymbol{X}^T\boldsymbol{y} = \begin{pmatrix} 870 \\ 88400 \\ 2450 \\ 1750 \end{pmatrix}$$

步骤 4：求解

解法方程 $\boldsymbol{X}^T\boldsymbol{X}\hat{\boldsymbol{\beta}} = \boldsymbol{X}^T\boldsymbol{y}$，得到：

$$\hat{\beta}_0 \approx -50.2, \quad \hat{\beta}_1 \approx 1.2, \quad \hat{\beta}_2 \approx 15.8, \quad \hat{\beta}_3 \approx 8.5$$

步骤 5：模型解释

拟合模型为：

$$y = -50.2 + 1.2x_1 + 15.8x_2 + 8.5x_3$$

模型表明：面积每增加 1m^2，房价增加约 1.2 万元；房间数每增加 1 间，房价增加约 15.8 万元；楼层每增加 1 层，房价增加约 8.5 万元。

Chapter 3

迭代方法

迭代方法主要用于求解规模较大或结构稀疏的线性方程组以及非线性方程（组），特别适合只需在给定精度下获得近似解而不追求一次性“精确求解”的情形。与依赖矩阵分解的直接法（如高斯消元法、$\boldsymbol{LU}$ 分解等）相比，迭代方法通过构造一个逐步逼近真解的向量序列来工作，从而在存储与计算量上具有更好的可伸缩性。

在现代科学与工程计算中，大规模线性与非线性系统普遍来源于偏微分方程的数值离散、最优化问题以及机器学习模型的训练，这些问题往往产生高维、稀疏、结构化的代数系统。迭代方法提供了一个可以按需“逐步逼近”的数值框架，使得误差控制与计算成本之间可以灵活权衡。

本章路线图　本章围绕迭代思想组织，主要内容包括：

- **迭代解法的基本理论**：通过矩阵分裂 $\boldsymbol{A} = \boldsymbol{M} - \boldsymbol{N}$ 引入一阶定常迭代格式，给出误差递推关系，并用谱半径刻画收敛性与收敛速度。
- **经典线性迭代格式**：以 Jacobi 与 Gauss-Seidel 方法为代表，比较不同矩阵分裂对收敛性的影响，并在严格对角占优、对称正定等结构下给出可检验的收敛判据。
- **单变量非线性方程的迭代方法**：从二分法和不动点迭代出发，系统讨论 Newton 法与割线法，分析它们的局部/全局收敛性质和收敛阶。
- **非线性方程组的 Newton 型方法**：利用 Jacobi 矩阵构造多维 Newton 迭代，并在 Banach 不动点原理与局部线性化的框架下给出二次收敛的充分条件。

本章形成的“迭代格式 + 误差演化 + 谱半径/导数条件”的分析思路，将在后续特征值计算、偏微分方程离散系统求解以及优化问题的数值方法中被反复使用。

3.1　迭代解法的基本理论

3.1.1　基本概念

一般原理

本小节的目标是抽象出线性迭代法的通用形式，并引入后续分析中反复使用的几个核心对象：矩阵分裂、迭代矩阵以及误差序列。

在线性方程组求解的背景下，迭代方法通常用于非常大尺寸的系统，或在已有粗略近似的基础上逐步改进解的精度。其基本思想是构造一系列向量 $\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(k)}, \ldots$，期望该序列在适当条件下收敛到待求解线性方程组的

解。

对于大规模线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$，直接法（如高斯消元法、LU 分解）虽然能给出精确解，但计算量和存储需求巨大。当矩阵规模达到数千甚至数万维时，直接法往往不可行。迭代方法的优势在于：

- 存储需求小：只需存储矩阵的非零元素和当前迭代向量
- 计算量可控：每步迭代的计算量远小于直接法
- 可以提前终止：达到所需精度即可停止，不必完成全部计算

设给定 $\boldsymbol{A}\in M_{n,n}(\mathbb{R})$ 为正则矩阵，$\boldsymbol{b}\in\mathbb{R}^n$。为构造迭代格式，将原方程 $\boldsymbol{Ax}=\boldsymbol{b}$ 改写为等价形式，使得可以从当前近似解 $\boldsymbol{x}^{(k)}$ 计算出下一个近似解 $\boldsymbol{x}^{(k+1)}$。具体做法是将矩阵 $\boldsymbol{A}$ 分裂为两部分：

$$\boldsymbol{A}=\boldsymbol{M}-\boldsymbol{N},$$

其中 $\boldsymbol{M}$ 是**容易求逆**的矩阵（如对角矩阵、下三角矩阵等）。这样，原方程可以改写为：

$$\boldsymbol{Mx}-\boldsymbol{Nx}=\boldsymbol{b}\quad\Rightarrow\quad\boldsymbol{Mx}=\boldsymbol{Nx}+\boldsymbol{b}.$$

如果 $\boldsymbol{M}$ 容易求逆，我们可以构造迭代格式：

$$\boldsymbol{Mx}^{(k+1)}=\boldsymbol{Nx}^{(k)}+\boldsymbol{b},$$

即每次迭代时，用上一步的 $\boldsymbol{x}^{(k)}$ 计算新的 $\boldsymbol{x}^{(k+1)}$。

迭代方法的形式为：

$$\begin{cases}\boldsymbol{x}^{(0)}\text{ 给定},\\ \boldsymbol{Mx}^{(k+1)}=\boldsymbol{Nx}^{(k)}+\boldsymbol{b},\end{cases}\tag{3.1}$$

其中矩阵 $\boldsymbol{M},\boldsymbol{N}\in M_{n,n}(\mathbb{R})$，$\boldsymbol{M}$ 可逆，需要适当选择。

如果该迭代方法收敛，即如果序列 $\boldsymbol{x}^{(k)}$ 收敛到 $\bar{\boldsymbol{x}}$，则在极限情况下有

$$(\boldsymbol{M}-\boldsymbol{N})\bar{\boldsymbol{x}}=\boldsymbol{b},$$

如果满足

$$\boldsymbol{A}=\boldsymbol{M}-\boldsymbol{N},\tag{3.2}$$

则这对应于求解 $\boldsymbol{Ax}=\boldsymbol{b}$。

满足 $\boldsymbol{A}=\boldsymbol{M}-\boldsymbol{N}$ 的矩阵 $\boldsymbol{M}$ 和 $\boldsymbol{N}$ 有无穷多种选择，我们将给出两个例子。当然，我们的想法是选择一个特别容易求逆的矩阵 $\boldsymbol{M}$，例如对角矩阵，或者下三角矩阵。这两种选择分别对应于 Jacobi 方法和 Gauss-Seidel 方法。

一阶定常迭代法

将迭代格式 (3.1) 改写为显式形式：

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{Cx}^{(k)}+\boldsymbol{d},\quad k=0,1,2,\ldots,\tag{3.3}$$

其中 $\boldsymbol{C}=\boldsymbol{M}^{-1}\boldsymbol{N}\in\mathbb{R}^{n\times n}$ 称为**迭代矩阵**，$\boldsymbol{d}=\boldsymbol{M}^{-1}\boldsymbol{b}\in\mathbb{R}^n$ 为常向量。该格式称为**一阶定常迭代法**：“一阶” 指 $\boldsymbol{x}^{(k+1)}$ 仅依赖于前一步 $\boldsymbol{x}^{(k)}$；“定常” 指 $\boldsymbol{C}$ 和 $\boldsymbol{d}$ 在迭代过程中保持不变。

等价关系

为使迭代法 (3.3) 收敛到原方程 $\boldsymbol{Ax}=\boldsymbol{b}$ 的解，必须满足等价关系：

$$\boldsymbol{Ax}=\boldsymbol{b}\quad\Leftrightarrow\quad\boldsymbol{x}=\boldsymbol{Cx}+\boldsymbol{d}.\tag{3.4}$$

这意味着原方程 $\boldsymbol{Ax}=\boldsymbol{b}$ 和迭代格式 $\boldsymbol{x}=\boldsymbol{Cx}+\boldsymbol{d}$ 有完全相同的解集。这是迭代法能够工作的前提条件。

等价关系的必要性可以从固定点迭代的角度理解：如果迭代序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛到某个向量 $\boldsymbol{x}^*$，那么在极限情况下，$\boldsymbol{x}^*$ 必须满足迭代格式本身（因为迭代格式是连续的），即

$$\boldsymbol{x}^* = \lim_{k\to\infty} \boldsymbol{x}^{(k+1)} = \lim_{k\to\infty} (\boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d}) = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}.$$

如果等价关系不成立，那么 $\boldsymbol{x}^*$ 满足 $\boldsymbol{x}^* = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}$，但不满足 $\boldsymbol{A}\boldsymbol{x}^* = \boldsymbol{b}$，此时迭代法收敛到的不是原方程的解。

命题 3.1.1 (等价关系的必要性)**.** 若迭代序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛到 $\boldsymbol{x}^*$，则 $\boldsymbol{x}^*$ 满足 $\boldsymbol{x}^* = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}$。若等价关系 (3.4) 成立，则 $\boldsymbol{x}^*$ 也是 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 的解。

证明. 首先证明收敛极限满足迭代格式。由迭代格式的连续性，若 $\lim_{k\to\infty} \boldsymbol{x}^{(k)} = \boldsymbol{x}^*$，则

$$\boldsymbol{x}^* = \lim_{k\to\infty} \boldsymbol{x}^{(k+1)} = \lim_{k\to\infty} (\boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d}) = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}.$$

因此，任何收敛的迭代序列，其极限都必须是迭代格式的固定点。

其次证明等价关系保证了正确性。若等价关系成立，则 $\boldsymbol{x}^* = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}$ 等价于 $\boldsymbol{A}\boldsymbol{x}^* = \boldsymbol{b}$，故 $\boldsymbol{x}^*$ 是原方程的解。 □

注 (等价关系的构造). 在实际应用中，等价关系通常通过矩阵分裂 $\boldsymbol{A} = \boldsymbol{M} - \boldsymbol{N}$ 自然得到：

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \quad \Leftrightarrow \quad \boldsymbol{M}\boldsymbol{x} = \boldsymbol{N}\boldsymbol{x} + \boldsymbol{b} \quad \Leftrightarrow \quad \boldsymbol{x} = \boldsymbol{M}^{-1}\boldsymbol{N}\boldsymbol{x} + \boldsymbol{M}^{-1}\boldsymbol{b}.$$

因此，只要矩阵分裂满足 $\boldsymbol{A} = \boldsymbol{M} - \boldsymbol{N}$，等价关系就自动成立。

例 3.1.1 (迭代法的直观理解)**.** 考虑线性方程组

$$\begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix},$$

精确解为 $\boldsymbol{x}^* = [1, 1]^T$。

采用矩阵分裂法构造迭代格式。将系数矩阵分裂为：

$$\boldsymbol{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} - \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} = \boldsymbol{M} - \boldsymbol{N},$$

其中 $\boldsymbol{M} = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix}$ 是对角矩阵，容易求逆：$\boldsymbol{M}^{-1} = \begin{bmatrix} 1/4 & 0 \\ 0 & 1/3 \end{bmatrix}$。

迭代矩阵和常向量为：

$$\boldsymbol{C} = \boldsymbol{M}^{-1}\boldsymbol{N} = \begin{bmatrix} 1/4 & 0 \\ 0 & 1/3 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1/4 \\ -1/3 & 0 \end{bmatrix}, \quad \boldsymbol{d} = \boldsymbol{M}^{-1}\boldsymbol{b} = \begin{bmatrix} 1/4 & 0 \\ 0 & 1/3 \end{bmatrix} \begin{bmatrix} 5 \\ 4 \end{bmatrix} = \begin{bmatrix} 5/4 \\ 4/3 \end{bmatrix}.$$

迭代格式为：

$$\boldsymbol{x}^{(k+1)} = \begin{bmatrix} 0 & -1/4 \\ -1/3 & 0 \end{bmatrix} \boldsymbol{x}^{(k)} + \begin{bmatrix} 5/4 \\ 4/3 \end{bmatrix}.$$

取初始向量 $\boldsymbol{x}^{(0)} = [0,0]^T$，迭代过程如下：

$$
\begin{aligned}
\boldsymbol{x}^{(0)} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \\
\boldsymbol{x}^{(1)} &= \begin{bmatrix} 0 & -1/4 \\ -1/3 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 5/4 \\ 4/3 \end{bmatrix} = \begin{bmatrix} 1.25 \\ 1.333 \end{bmatrix}, \\
\boldsymbol{x}^{(2)} &= \begin{bmatrix} 0 & -1/4 \\ -1/3 & 0 \end{bmatrix} \begin{bmatrix} 1.25 \\ 1.333 \end{bmatrix} + \begin{bmatrix} 5/4 \\ 4/3 \end{bmatrix} = \begin{bmatrix} 0.917 \\ 0.917 \end{bmatrix}, \\
\boldsymbol{x}^{(3)} &= \begin{bmatrix} 0 & -1/4 \\ -1/3 & 0 \end{bmatrix} \begin{bmatrix} 0.917 \\ 0.917 \end{bmatrix} + \begin{bmatrix} 5/4 \\ 4/3 \end{bmatrix} = \begin{bmatrix} 1.021 \\ 1.028 \end{bmatrix}, \\
\boldsymbol{x}^{(4)} &= \begin{bmatrix} 0.994 \\ 0.996 \end{bmatrix}, \\
\boldsymbol{x}^{(5)} &= \begin{bmatrix} 1.001 \\ 1.001 \end{bmatrix}, \\
&\vdots
\end{aligned}
$$

可见迭代序列逐步逼近精确解 $\boldsymbol{x}^* = [1,1]^T$。

计算迭代矩阵的谱半径：$\boldsymbol{C}$ 的特征值为 $\lambda_1 = 1/\sqrt{12} \approx 0.289$ 和 $\lambda_2 = -1/\sqrt{12} \approx -0.289$，故 $\rho(\boldsymbol{C}) = 1/\sqrt{12} \approx 0.289 < 1$，保证了收敛性。

误差向量的变化：

$$
\begin{aligned}
\boldsymbol{e}^{(0)} &= \boldsymbol{x}^{(0)} - \boldsymbol{x}^* = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \\
\boldsymbol{e}^{(1)} &= \boldsymbol{C}\boldsymbol{e}^{(0)} = \begin{bmatrix} 0.25 \\ 0.333 \end{bmatrix}, \\
\boldsymbol{e}^{(2)} &= \boldsymbol{C}\boldsymbol{e}^{(1)} = \begin{bmatrix} -0.083 \\ -0.083 \end{bmatrix}, \\
&\vdots
\end{aligned}
$$

可见误差按迭代矩阵 $\boldsymbol{C}$ 的规律变化，逐步趋于零向量。

Algorithm 6 一阶定常迭代法

Require: $\boldsymbol{x}^{(0)}, \boldsymbol{C}, \boldsymbol{d}$

Ensure: 近似解 $\boldsymbol{x}$

```
x ← x^(0)
while 不满足收敛条件 do
    x ← Cx + d
end while
```

3.1.2　收敛性理论

误差递推关系

要判断迭代法是否收敛，需要研究 $\boldsymbol{x}^{(k)}$ 是否越来越接近精确解 $\boldsymbol{x}^*$。由于精确解 $\boldsymbol{x}^*$ 未知，无法直接比较 $\boldsymbol{x}^{(k)}$ 和 $\boldsymbol{x}^*$。因此，我们转而研究误差的变化规律。

设精确解为 $\boldsymbol{x}^*$，第 k 步的误差定义为：

$$\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*.$$

如果 $\|\boldsymbol{e}^{(k)}\| \to 0$，则 $\boldsymbol{x}^{(k)} \to \boldsymbol{x}^*$，因此研究误差的变化规律等价于研究收敛性。

命题 3.1.2 (误差递推公式)**.** 对于一阶定常迭代法 (3.3)，误差满足递推关系：

$$\boldsymbol{e}^{(k+1)} = \boldsymbol{C}\boldsymbol{e}^{(k)} = \boldsymbol{C}^k\boldsymbol{e}^{(0)}, \quad k = 0, 1, 2, \ldots. \tag{3.5}$$

证明. 虽然精确解 $\boldsymbol{x}^*$ 未知，但由等价关系（见等价关系 (3.4)），$\boldsymbol{x}^*$ 必须满足 $\boldsymbol{x}^* = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}$。这是精确解的性质，不依赖于我们是否知道它的具体值。

现在分析第 $k+1$ 步的误差 $\boldsymbol{e}^{(k+1)}$。根据误差的定义和迭代格式：

$$\boldsymbol{e}^{(k+1)} = \boldsymbol{x}^{(k+1)} - \boldsymbol{x}^*.$$

将迭代格式 $\boldsymbol{x}^{(k+1)} = \boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d}$ 代入上式：

$$\boldsymbol{e}^{(k+1)} = (\boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d}) - \boldsymbol{x}^*.$$

由于 $\boldsymbol{x}^* = \boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}$（精确解的性质），可以写成：

$$\begin{aligned}
\boldsymbol{e}^{(k+1)} &= (\boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d}) - (\boldsymbol{C}\boldsymbol{x}^* + \boldsymbol{d}) \\
&= \boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d} - \boldsymbol{C}\boldsymbol{x}^* - \boldsymbol{d} \\
&= \boldsymbol{C}(\boldsymbol{x}^{(k)} - \boldsymbol{x}^*) \\
&= \boldsymbol{C}\boldsymbol{e}^{(k)}.
\end{aligned}$$

因此得到误差的递推关系：$\boldsymbol{e}^{(k+1)} = \boldsymbol{C}\boldsymbol{e}^{(k)}$。

通过递推，可以写出：

$$\begin{aligned}
\boldsymbol{e}^{(1)} &= \boldsymbol{C}\boldsymbol{e}^{(0)}, \\
\boldsymbol{e}^{(2)} &= \boldsymbol{C}\boldsymbol{e}^{(1)} = \boldsymbol{C}(\boldsymbol{C}\boldsymbol{e}^{(0)}) = \boldsymbol{C}^2\boldsymbol{e}^{(0)}, \\
\boldsymbol{e}^{(3)} &= \boldsymbol{C}\boldsymbol{e}^{(2)} = \boldsymbol{C}(\boldsymbol{C}^2\boldsymbol{e}^{(0)}) = \boldsymbol{C}^3\boldsymbol{e}^{(0)}, \\
&\vdots \\
\boldsymbol{e}^{(k)} &= \boldsymbol{C}^k\boldsymbol{e}^{(0)}.
\end{aligned}$$

因此，误差按迭代矩阵 $\boldsymbol{C}$ 的幂次规律变化：$\boldsymbol{e}^{(k)} = \boldsymbol{C}^k\boldsymbol{e}^{(0)}$。

如果 $\boldsymbol{C}^k \to \boldsymbol{O}$（零矩阵），则对任意初始误差 $\boldsymbol{e}^{(0)}$，有

$$\boldsymbol{e}^{(k)} = \boldsymbol{C}^k\boldsymbol{e}^{(0)} \to \boldsymbol{O} \cdot \boldsymbol{e}^{(0)} = \boldsymbol{0},$$

即 $\boldsymbol{e}^{(k)} \to \boldsymbol{0}$，从而 $\boldsymbol{x}^{(k)} \to \boldsymbol{x}^*$，迭代法收敛。因此，研究 $\boldsymbol{C}^k$ 的收敛性是判断迭代法收敛性的关键。 □

例 3.1.2 (误差递推关系的直观理解)**.** 考虑标量迭代 $x^{(k+1)} = 0.5x^{(k)} + 1$，精确解为 $x^* = 2$。设初始值 $x^{(0)} = 0$，则初始误差 $e^{(0)} = x^{(0)} - x^* = -2$。

由误差递推关系：

$$\begin{aligned}
e^{(1)} &= 0.5 \times (-2) = -1, \\
e^{(2)} &= 0.5 \times (-1) = -0.5, \\
e^{(3)} &= 0.5 \times (-0.5) = -0.25, \\
&\vdots
\end{aligned}$$

可见误差按 0.5^k 的速度衰减，即 $e^{(k)} = 0.5^k \times (-2) = -2 \times 0.5^k$。当 $k \to \infty$ 时，$e^{(k)} \to 0$，迭代收敛。

矩阵幂序列的收敛性

由误差递推关系，$\boldsymbol{e}^{(k)} = \boldsymbol{C}^k\boldsymbol{e}^{(0)}$。要保证误差趋于零，需要 $\boldsymbol{C}^k \to \boldsymbol{O}$（零矩阵）。因此，判断 $\boldsymbol{C}^k$ 何时趋于零矩阵是研究迭代法收敛性的核心问题。

为此，需要：

1. 定义矩阵序列的收敛性
2. 找到判断 $\boldsymbol{C}^k \to \boldsymbol{O}$ 的充要条件

定义 3.1.1 (矩阵序列的收敛). 设矩阵序列 $\{\boldsymbol{A}^{(k)}\}$，$\boldsymbol{A}^{(k)} = (a_{ij}^{(k)}) \in \mathbb{R}^{n\times n}$，$\boldsymbol{A} = (a_{ij}) \in \mathbb{R}^{n\times n}$。若

$$\lim_{k\to\infty} a_{ij}^{(k)} = a_{ij}, \quad \forall i,j = 1,2,\ldots,n,$$

则称序列 $\{\boldsymbol{A}^{(k)}\}$ 收敛于 $\boldsymbol{A}$，记作 $\lim\limits_{k\to\infty} \boldsymbol{A}^{(k)} = \boldsymbol{A}$。

矩阵序列收敛意味着每个元素都收敛。特别地，$\boldsymbol{C}^k \to \boldsymbol{O}$ 意味着 $\boldsymbol{C}^k$ 的每个元素都趋于零。

要判断 $\boldsymbol{C}^k \to \boldsymbol{O}$，需要一个易于计算的判据。矩阵的特征值提供了这样的工具。

定义 3.1.2 (谱半径). 设矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ 的特征值为 $\lambda_1, \lambda_2, \ldots, \lambda_n$，则**谱半径**定义为所有特征值模的最大值：

$$\rho(\boldsymbol{A}) = \max_{1\le i\le n} |\lambda_i|.$$

如果 $\boldsymbol{C}$ 的所有特征值的模都小于 1，那么反复作用 $\boldsymbol{C}$ 会让向量缩小。更精确地，下面的定理表明：$\boldsymbol{C}^k \to \boldsymbol{O}$ 的充要条件是 $\rho(\boldsymbol{C}) < 1$。

定理 3.1.1 (矩阵幂序列收敛的充要条件). 设 $\boldsymbol{C} \in \mathbb{R}^{n\times n}$，则

$$\lim_{k\to\infty} \boldsymbol{C}^k = \boldsymbol{0} \quad \Leftrightarrow \quad \rho(\boldsymbol{C}) < 1.$$

证明. **充分性**：若 $\rho(\boldsymbol{C}) < 1$，需要证明 $\boldsymbol{C}^k \to \boldsymbol{0}$。

由谱半径与算子范数的关系（这是线性代数中的一个重要结果），对于任意 $\varepsilon > 0$，存在算子范数 $\|\cdot\|$ 使得

$$\|\boldsymbol{C}\| \le \rho(\boldsymbol{C}) + \varepsilon.$$

由于 $\rho(\boldsymbol{C}) < 1$，可以选择 ε 足够小使得 $\rho(\boldsymbol{C}) + \varepsilon < 1$，从而存在算子范数 $\|\cdot\|$ 使得 $\|\boldsymbol{C}\| < 1$。

利用矩阵范数的次乘性（$\|\boldsymbol{AB}\| \le \|\boldsymbol{A}\|\|\boldsymbol{B}\|$），有

$$\|\boldsymbol{C}^k\| = \|\boldsymbol{C}\cdot\boldsymbol{C}^{k-1}\| \le \|\boldsymbol{C}\|\|\boldsymbol{C}^{k-1}\| \le \|\boldsymbol{C}\|^2\|\boldsymbol{C}^{k-2}\| \le \cdots \le \|\boldsymbol{C}\|^k.$$

由于 $\|\boldsymbol{C}\| < 1$，当 $k \to \infty$ 时，$\|\boldsymbol{C}\|^k \to 0$，因此 $\|\boldsymbol{C}^k\| \to 0$。

由范数收敛与矩阵收敛的等价性（这是矩阵分析中的基本结果：矩阵序列按范数收敛当且仅当按元素收敛），$\|\boldsymbol{C}^k\| \to 0$ 意味着 $\boldsymbol{C}^k$ 的每个元素都趋于零，即 $\boldsymbol{C}^k \to \boldsymbol{0}$。

必要性：若 $\boldsymbol{C}^k \to \boldsymbol{0}$，需要证明 $\rho(\boldsymbol{C}) < 1$。

设 λ 为 $\boldsymbol{C}$ 的任一特征值，对应特征向量 $\boldsymbol{v} \ne \boldsymbol{0}$。由特征值的定义：

$$\boldsymbol{Cv} = \lambda\boldsymbol{v}.$$

对上述两边左乘 $\boldsymbol{C}$，得到：

$$\boldsymbol{C}^2\boldsymbol{v} = \boldsymbol{C}(\lambda\boldsymbol{v}) = \lambda(\boldsymbol{Cv}) = \lambda^2\boldsymbol{v}.$$

继续这个过程，通过数学归纳法可以证明：

$$\boldsymbol{C}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}, \quad k = 1,2,3,\ldots.$$

由于 $\boldsymbol{C}^k \to \boldsymbol{0}$，有 $\boldsymbol{C}^k\boldsymbol{v} \to \boldsymbol{0}$。但 $\boldsymbol{C}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$，因此

$$\lambda^k\boldsymbol{v} \to \boldsymbol{0}.$$

由于 $\boldsymbol{v} \ne \boldsymbol{0}$，必须有 $\lambda^k \to 0$。这要求 $|\lambda| < 1$，因为：

- 若 $|\lambda| \ge 1$，则 $|\lambda|^k \ge 1$ 对所有 k 成立，从而 $|\lambda^k\boldsymbol{v}| = |\lambda|^k|\boldsymbol{v}| \ge |\boldsymbol{v}| > 0$，不可能趋于零。

- 若 $|\lambda| < 1$，则 $|\lambda|^k \to 0$，从而 $|\lambda^k \boldsymbol{v}| = |\lambda|^k |\boldsymbol{v}| \to 0$。

因此，$\boldsymbol{C}$ 的每个特征值 λ 都满足 $|\lambda| < 1$，从而

$$\rho(\boldsymbol{C}) = \max_{1 \le i \le n} |\lambda_i| < 1.$$

□

例 3.1.3 (矩阵幂序列收敛的直观理解)**.** 考虑对角矩阵 $\boldsymbol{C} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.3 \end{bmatrix}$，其特征值为 0.5 和 0.3，故 $\rho(\boldsymbol{C}) = 0.5 < 1$。

计算 $\boldsymbol{C}^k$：

$$\boldsymbol{C}^k = \begin{bmatrix} 0.5^k & 0 \\ 0 & 0.3^k \end{bmatrix}.$$

当 $k \to \infty$ 时，$0.5^k \to 0$，$0.3^k \to 0$，因此 $\boldsymbol{C}^k \to \boldsymbol{0}$。

再考虑非对角矩阵 $\boldsymbol{C} = \begin{bmatrix} 0.6 & 0.2 \\ 0.1 & 0.5 \end{bmatrix}$，其特征值约为 0.7 和 0.4，$\rho(\boldsymbol{C}) = 0.7 < 1$。虽然 $\boldsymbol{C}^k$ 的计算更复杂，但由定理，仍然有 $\boldsymbol{C}^k \to \boldsymbol{0}$。

反之，若 $\boldsymbol{C} = \begin{bmatrix} 1.1 & 0 \\ 0 & 0.5 \end{bmatrix}$，则 $\rho(\boldsymbol{C}) = 1.1 \ge 1$。此时 $\boldsymbol{C}^k = \begin{bmatrix} 1.1^k & 0 \\ 0 & 0.5^k \end{bmatrix}$，当 $k \to \infty$ 时，$1.1^k \to \infty$，因此 $\boldsymbol{C}^k$ 不收敛到零矩阵。

例 3.1.4 (谱半径的意义)**.** 考虑矩阵 $\boldsymbol{C} = \begin{bmatrix} 0.6 & 0.2 \\ 0.1 & 0.5 \end{bmatrix}$。计算其特征值：

$$\det(\lambda \boldsymbol{I} - \boldsymbol{C}) = \lambda^2 - 1.1\lambda + 0.28 = 0,$$

解得 $\lambda_1 \approx 0.7$，$\lambda_2 \approx 0.4$，故 $\rho(\boldsymbol{C}) = 0.7 < 1$。

虽然矩阵不是对角矩阵，但谱半径仍然小于 1。对任意向量 $\boldsymbol{v}$，$\boldsymbol{C}^k \boldsymbol{v}$ 会趋于 $\boldsymbol{0}$，但不同方向的衰减速度不同：

- 沿特征值 0.7 对应的特征向量方向，每步约缩小到原来的 0.7 倍
- 沿特征值 0.4 对应的特征向量方向，每步约缩小到原来的 0.4 倍
- 衰减最慢的方向（对应最大特征值）决定了整体的收敛速度

因此，谱半径衡量了矩阵幂序列收敛的最慢速度。即使某些方向收敛很快，只要有一个方向收敛慢（对应最大特征值），整体收敛速度就由它决定。

迭代法收敛的充要条件

定理 3.1.2 (收敛的充分条件)**.** 如果存在一个从属矩阵范数使得

$$\|\boldsymbol{C}\| < 1, \tag{3.6}$$

则方法 (3.3) *对任意初始值 $\boldsymbol{x}^{(0)}$ 的选择都是收敛的*，且它收敛到方程组

$$(\boldsymbol{I} - \boldsymbol{C})\bar{\boldsymbol{x}} = \boldsymbol{d} \tag{3.7}$$

的解。

证明. 证明分为两部分：首先证明方程组有唯一解，然后证明迭代序列收敛到该解。

第一部分：解的存在唯一性

要证明方程组 $(\boldsymbol{I} - \boldsymbol{C})\bar{\boldsymbol{x}} = \boldsymbol{d}$ 有唯一解，等价于证明矩阵 $\boldsymbol{I} - \boldsymbol{C}$ 可逆，即其核空间只有零向量。

假设存在非零向量 $\boldsymbol{x}$ 使得 $(\boldsymbol{I}-\boldsymbol{C})\boldsymbol{x}=\boldsymbol{0}$。这意味着：

$$\boldsymbol{x}-\boldsymbol{C}\boldsymbol{x}=\boldsymbol{0} \quad \Rightarrow \quad \boldsymbol{x}=\boldsymbol{C}\boldsymbol{x}.$$

对上述两边取范数，利用矩阵范数的性质：

$$\|\boldsymbol{x}\|=\|\boldsymbol{C}\boldsymbol{x}\|.$$

由矩阵范数的定义（从属范数满足 $\|\boldsymbol{C}\boldsymbol{x}\|\leq\|\boldsymbol{C}\|\|\boldsymbol{x}\|$），有

$$\|\boldsymbol{x}\|=\|\boldsymbol{C}\boldsymbol{x}\|\leq\|\boldsymbol{C}\|\|\boldsymbol{x}\|.$$

如果 $\boldsymbol{x}\neq\boldsymbol{0}$，则 $\|\boldsymbol{x}\|>0$，可以将上述不等式两边同时除以 $\|\boldsymbol{x}\|$，得到：

$$1\leq\|\boldsymbol{C}\|.$$

但这与假设 $\|\boldsymbol{C}\|<1$ 矛盾。因此，假设不成立，即不存在非零向量 $\boldsymbol{x}$ 使得 $(\boldsymbol{I}-\boldsymbol{C})\boldsymbol{x}=\boldsymbol{0}$。

这意味着 $\mathrm{Ker}(\boldsymbol{I}-\boldsymbol{C})=\{\boldsymbol{0}\}$。由于 $\boldsymbol{I}-\boldsymbol{C}$ 是方阵，核空间只有零向量等价于矩阵可逆。因此，方程组 $(\boldsymbol{I}-\boldsymbol{C})\bar{\boldsymbol{x}}=\boldsymbol{d}$ 有唯一解，记作 $\bar{\boldsymbol{x}}$。

第二部分：迭代序列的收敛性

现在证明迭代序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛到 $\bar{\boldsymbol{x}}$。

首先验证 $\bar{\boldsymbol{x}}$ 满足等价关系。由于 $\bar{\boldsymbol{x}}$ 是 $(\boldsymbol{I}-\boldsymbol{C})\bar{\boldsymbol{x}}=\boldsymbol{d}$ 的解，有

$$\bar{\boldsymbol{x}}-\boldsymbol{C}\bar{\boldsymbol{x}}=\boldsymbol{d} \quad \Rightarrow \quad \bar{\boldsymbol{x}}=\boldsymbol{C}\bar{\boldsymbol{x}}+\boldsymbol{d}.$$

这正是迭代格式的固定点形式，因此 $\bar{\boldsymbol{x}}$ 满足等价关系。

现在分析迭代序列与精确解之间的误差。由误差的定义：

$$\boldsymbol{x}^{(k)}-\bar{\boldsymbol{x}}=\boldsymbol{e}^{(k)}.$$

由误差递推关系（命题 3.1.2），有

$$\boldsymbol{e}^{(k)}=\boldsymbol{C}^k\boldsymbol{e}^{(0)}=\boldsymbol{C}^k(\boldsymbol{x}^{(0)}-\bar{\boldsymbol{x}}).$$

因此

$$\boldsymbol{x}^{(k)}-\bar{\boldsymbol{x}}=\boldsymbol{C}^k(\boldsymbol{x}^{(0)}-\bar{\boldsymbol{x}}).$$

对上述两边取范数，利用矩阵范数的性质：

$$\|\boldsymbol{x}^{(k)}-\bar{\boldsymbol{x}}\|=\|\boldsymbol{C}^k(\boldsymbol{x}^{(0)}-\bar{\boldsymbol{x}})\|\leq\|\boldsymbol{C}^k\|\|\boldsymbol{x}^{(0)}-\bar{\boldsymbol{x}}\|.$$

由矩阵范数的次乘性，$\|\boldsymbol{C}^k\|\leq\|\boldsymbol{C}\|^k$，因此

$$\|\boldsymbol{x}^{(k)}-\bar{\boldsymbol{x}}\|\leq\|\boldsymbol{C}\|^k\|\boldsymbol{x}^{(0)}-\bar{\boldsymbol{x}}\|.$$

由于 $\|\boldsymbol{C}\|<1$，当 $k\to\infty$ 时，$\|\boldsymbol{C}\|^k\to 0$，因此

$$\|\boldsymbol{x}^{(k)}-\bar{\boldsymbol{x}}\|\to 0,$$

即 $\boldsymbol{x}^{(k)}\to\bar{\boldsymbol{x}}$。 □

例 3.1.5 (收敛充分条件的应用)**.** 考虑迭代格式 $x^{(k+1)}=0.5x^{(k)}+1$，迭代矩阵为 $C=0.5$。

对于标量，常用的范数是绝对值，$\|C\|=|0.5|=0.5<1$，满足定理条件。

由定理，迭代序列对任意初始值 $x^{(0)}$ 都收敛。设 $\bar{x}$ 是 $(1-0.5)\bar{x}=1$ 的解，即 $0.5\bar{x}=1$，解得 $\bar{x}=2$。

误差估计为：

$$|x^{(k)}-2| \le 0.5^k |x^{(0)}-2|.$$

若 $x^{(0)}=0$，则 $|x^{(k)}-2| \le 0.5^k \times 2$。当 $k=10$ 时，$|x^{(10)}-2| \le 0.5^{10} \times 2 \approx 0.002$，误差已经很小。

注. $\|\boldsymbol{C}\|<1$ 是充分条件而非必要条件，因为 $\rho(\boldsymbol{C}) \le \|\boldsymbol{C}\|$，可能存在 $\rho(\boldsymbol{C})<1$ 但 $\|\boldsymbol{C}\| \ge 1$ 的情况。

收敛速度

在实际应用中，不仅需要知道迭代法是否收敛，还需要知道收敛速度，即需要多少步迭代才能达到所需精度，以及不同迭代法的收敛速度如何比较。

定义 3.1.3 (收敛阶)**.** 设迭代序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛于 $\boldsymbol{x}^*$，若存在常数 $c>0$ 和 $p \ge 1$ 使得

$$\lim_{k\to\infty} \frac{\|\boldsymbol{x}^{(k+1)}-\boldsymbol{x}^*\|}{\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^*\|^p} = c,$$

则称该迭代法为 p **阶收敛**。

当 $p=1$ 时称为线性收敛，误差每步按固定比例缩小；当 $p=2$ 时称为二次收敛，误差每步按平方缩小；当 $p>1$ 时称为超线性收敛。

一阶定常迭代法的收敛阶

命题 3.1.3 (一阶定常迭代法的收敛阶)**.** 若迭代矩阵 $\boldsymbol{C}$ 可对角化，特征值为 $\lambda_1,\lambda_2,\ldots,\lambda_n$，且 $|\lambda_1| \ge |\lambda_2| \ge \cdots \ge |\lambda_n|$，则一阶定常迭代法为线性收敛（$p=1$），且

$$\lim_{k\to\infty} \frac{\|\boldsymbol{e}^{(k+1)}\|}{\|\boldsymbol{e}^{(k)}\|} = \rho(\boldsymbol{C}).$$

证明. 由于 $\boldsymbol{C}$ 可对角化，存在 n 个线性无关的特征向量 $\boldsymbol{v}_1,\boldsymbol{v}_2,\ldots,\boldsymbol{v}_n$，对应的特征值分别为 $\lambda_1,\lambda_2,\ldots,\lambda_n$，且 $|\lambda_1| \ge |\lambda_2| \ge \cdots \ge |\lambda_n|$。

由于特征向量组构成 $\mathbb{R}^n$ 的一组基，初始误差 $\boldsymbol{e}^{(0)}$ 可以唯一表示为特征向量的线性组合：

$$\boldsymbol{e}^{(0)} = \alpha_1\boldsymbol{v}_1 + \alpha_2\boldsymbol{v}_2 + \cdots + \alpha_n\boldsymbol{v}_n = \sum_{i=1}^n \alpha_i \boldsymbol{v}_i,$$

其中 $\alpha_1,\alpha_2,\ldots,\alpha_n$ 是标量系数。

现在计算 $\boldsymbol{e}^{(k)} = \boldsymbol{C}^k\boldsymbol{e}^{(0)}$。利用 $\boldsymbol{C}^k\boldsymbol{v}_i = \lambda_i^k\boldsymbol{v}_i$（这是特征值的性质），有

$$\begin{aligned}
\boldsymbol{e}^{(k)} &= \boldsymbol{C}^k\boldsymbol{e}^{(0)} \\
&= \boldsymbol{C}^k\left(\sum_{i=1}^n \alpha_i\boldsymbol{v}_i\right) \\
&= \sum_{i=1}^n \alpha_i\boldsymbol{C}^k\boldsymbol{v}_i \\
&= \sum_{i=1}^n \alpha_i\lambda_i^k\boldsymbol{v}_i.
\end{aligned}$$

类似地，

$$\boldsymbol{e}^{(k+1)} = \sum_{i=1}^n \alpha_i\lambda_i^{k+1}\boldsymbol{v}_i = \sum_{i=1}^n \alpha_i\lambda_i\lambda_i^k\boldsymbol{v}_i.$$

现在分析误差的渐近行为。第 i 个方向的误差分量按 $|\lambda_i|^k$ 的速度衰减。由于 $|\lambda_1| \ge |\lambda_2| \ge \cdots \ge |\lambda_n|$，第一个方向（对应最大特征值）衰减最慢。

当 k 充分大时，$|\lambda_1|^k$ 远大于 $|\lambda_2|^k, |\lambda_3|^k, \ldots$（因为 $|\lambda_1| > |\lambda_2|$ 时，$|\lambda_1|^k/|\lambda_2|^k = (|\lambda_1|/|\lambda_2|)^k \to \infty$），因此误差主要由第一个方向主导：

$$\boldsymbol{e}^{(k)} \approx \alpha_1 \lambda_1^k \boldsymbol{v}_1, \quad \boldsymbol{e}^{(k+1)} \approx \alpha_1 \lambda_1^{k+1} \boldsymbol{v}_1 = \lambda_1(\alpha_1 \lambda_1^k \boldsymbol{v}_1) \approx \lambda_1 \boldsymbol{e}^{(k)}.$$

因此

$$\|\boldsymbol{e}^{(k+1)}\| \approx |\lambda_1| \|\boldsymbol{e}^{(k)}\| = \rho(\boldsymbol{C}) \|\boldsymbol{e}^{(k)}\|.$$

更严格地，考虑误差比：

$$\frac{\|\boldsymbol{e}^{(k+1)}\|}{\|\boldsymbol{e}^{(k)}\|} = \frac{\|\sum_{i=1}^n \alpha_i \lambda_i^{k+1} \boldsymbol{v}_i\|}{\|\sum_{i=1}^n \alpha_i \lambda_i^k \boldsymbol{v}_i\|}.$$

当 $k \to \infty$ 时，如果 $\alpha_1 \neq 0$（这是典型情况），则

$$\lim_{k\to\infty} \frac{\|\boldsymbol{e}^{(k+1)}\|}{\|\boldsymbol{e}^{(k)}\|} = \lim_{k\to\infty} \frac{|\lambda_1|^{k+1} \|\alpha_1 \boldsymbol{v}_1 + \sum_{i=2}^n \alpha_i (\lambda_i/\lambda_1)^{k+1} \boldsymbol{v}_i\|}{|\lambda_1|^k \|\alpha_1 \boldsymbol{v}_1 + \sum_{i=2}^n \alpha_i (\lambda_i/\lambda_1)^k \boldsymbol{v}_i\|} = |\lambda_1| = \rho(\boldsymbol{C}),$$

因为当 $|\lambda_i/\lambda_1| < 1$（$i \geq 2$）时，$(\lambda_i/\lambda_1)^k \to 0$。

因此，一阶定常迭代法为线性收敛（$p = 1$），收敛速度由谱半径决定。 □

例 3.1.6 (收敛阶的直观理解)**.** 考虑矩阵 $\boldsymbol{C} = \begin{bmatrix} 0.7 & 0 \\ 0 & 0.3 \end{bmatrix}$，其特征值为 0.7 和 0.3，$\rho(\boldsymbol{C}) = 0.7$。

设初始误差 $\boldsymbol{e}^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$，则

$$\boldsymbol{e}^{(1)} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}, \quad \|\boldsymbol{e}^{(1)}\| \approx 0.762,$$

$$\boldsymbol{e}^{(2)} = \begin{bmatrix} 0.49 \\ 0.09 \end{bmatrix}, \quad \|\boldsymbol{e}^{(2)}\| \approx 0.498,$$

$$\boldsymbol{e}^{(3)} = \begin{bmatrix} 0.343 \\ 0.027 \end{bmatrix}, \quad \|\boldsymbol{e}^{(3)}\| \approx 0.344.$$

计算误差比：

$$\frac{\|\boldsymbol{e}^{(2)}\|}{\|\boldsymbol{e}^{(1)}\|} \approx \frac{0.498}{0.762} \approx 0.654, \quad \frac{\|\boldsymbol{e}^{(3)}\|}{\|\boldsymbol{e}^{(2)}\|} \approx \frac{0.344}{0.498} \approx 0.691.$$

当 k 充分大时，误差比趋于 $\rho(\boldsymbol{C}) = 0.7$，验证了线性收敛的性质。

定义 3.1.4 (渐进收敛速度)**.** 迭代法 $\boldsymbol{x}^{(k+1)} = \boldsymbol{C}\boldsymbol{x}^{(k)} + \boldsymbol{d}$ 的**渐进收敛速度**定义为

$$R = -\ln \rho(\boldsymbol{C}).$$

3.2 经典迭代法：Jacobi 与 Gauss-Seidel

3.2.1 矩阵分裂

矩阵分裂的基本思想是将矩阵 $\boldsymbol{A}$ 分裂成两部分：

$$\boldsymbol{A} = \boldsymbol{M} - \boldsymbol{N}$$

其中 $\boldsymbol{M}$ 是容易求逆的矩阵（如对角矩阵、下三角矩阵等）。

原方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 可以改写为：

$$\boldsymbol{M}\boldsymbol{x} - \boldsymbol{N}\boldsymbol{x} = \boldsymbol{b} \quad \Rightarrow \quad \boldsymbol{M}\boldsymbol{x} = \boldsymbol{N}\boldsymbol{x} + \boldsymbol{b}$$

如果 $\boldsymbol{M}$ 容易求逆，可以构造迭代格式：

$$\boldsymbol{M}\boldsymbol{x}^{(k+1)} = \boldsymbol{N}\boldsymbol{x}^{(k)} + \boldsymbol{b}$$

即：

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{M}^{-1}\boldsymbol{N}\boldsymbol{x}^{(k)} + \boldsymbol{M}^{-1}\boldsymbol{b} \tag{3.8}$$

这正是一阶定常迭代法的形式，其中迭代矩阵为 $\boldsymbol{C} = \boldsymbol{M}^{-1}\boldsymbol{N}$，常向量为 $\boldsymbol{d} = \boldsymbol{M}^{-1}\boldsymbol{b}$。

选择 $\boldsymbol{M}$ 和 $\boldsymbol{N}$ 时需要考虑：

- $\boldsymbol{M}$ 要简单：求 $\boldsymbol{M}^{-1}$ 容易（计算量小）
- $\boldsymbol{M}$ 要接近 $\boldsymbol{A}$：这样 $\boldsymbol{C} = \boldsymbol{M}^{-1}\boldsymbol{N} = \boldsymbol{I} - \boldsymbol{M}^{-1}\boldsymbol{A}$ 接近零矩阵，谱半径小（收敛快）

这两个要求往往是矛盾的，需要在计算量和收敛速度之间权衡。下面介绍两种经典的矩阵分裂方法。

3.2.2 Jacobi 迭代法

基本思想

Jacobi 方法的基本思想是将矩阵 $\boldsymbol{A}$ 的对角线部分单独提取出来，其余部分移到右边。

矩阵分裂为：

$$\boldsymbol{A} = \boldsymbol{D} - \boldsymbol{E} - \boldsymbol{F}$$

其中：

- $\boldsymbol{D}$：对角矩阵，包含 $\boldsymbol{A}$ 的所有对角元素
- $\boldsymbol{E}$：下三角矩阵（$-\boldsymbol{A}$ 的下三角部分）
- $\boldsymbol{F}$：上三角矩阵（$-\boldsymbol{A}$ 的上三角部分）

选择这种分裂的原因是：对角矩阵 $\boldsymbol{D}$ 的逆矩阵非常容易计算。如果 $\boldsymbol{D} = \mathrm{diag}(d_1, d_2, \ldots, d_n)$，那么

$$\boldsymbol{D}^{-1} = \mathrm{diag}\left(\frac{1}{d_1}, \frac{1}{d_2}, \ldots, \frac{1}{d_n}\right)$$

只需要对每个对角元素取倒数。

迭代格式

原方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 可以写成：

$$\boldsymbol{D}\boldsymbol{x} = (\boldsymbol{E} + \boldsymbol{F})\boldsymbol{x} + \boldsymbol{b}$$

Jacobi 迭代格式为：

$$\boldsymbol{D}\boldsymbol{x}^{(k+1)} = (\boldsymbol{E} + \boldsymbol{F})\boldsymbol{x}^{(k)} + \boldsymbol{b} \tag{3.9}$$

即：

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{D}^{-1}(\boldsymbol{E} + \boldsymbol{F})\boldsymbol{x}^{(k)} + \boldsymbol{D}^{-1}\boldsymbol{b} \tag{3.10}$$

迭代矩阵：$\boldsymbol{C}_J = \boldsymbol{D}^{-1}(\boldsymbol{E} + \boldsymbol{F})$

分量形式

虽然矩阵形式很简洁，但分量形式更适合编程实现。

对于第 i 个方程：

$$a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{ii}x_i + \cdots + a_{in}x_n = b_i$$

Jacobi 方法将 x_i 单独解出，其他变量用上一步的值：

$$a_{ii}x_i^{(k+1)} = b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}$$

因此，Jacobi 迭代的分量计算公式为：

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right), \quad i = 1, \ldots, n. \tag{3.11}$$

Jacobi 方法的特点：

- 计算 $x_i^{(k+1)}$ 时，只使用上一步的值 $x_j^{(k)}$（$j \neq i$）
- 所有分量的计算相互独立，可以并行计算
- 需要保存上一步的完整解向量

Algorithm 7 Jacobi 迭代法

Require: $\boldsymbol{x}^{(0)}, \boldsymbol{A}, \boldsymbol{b}$, 最大迭代次数 K, 容差 ε
Ensure: 近似解 $\boldsymbol{x}$

```
k ← 0
while k < K do
    for i = 1 to n do
        x_i^new ← (b_i − Σ_{j=1}^{i−1} a_ij x_j^(k) − Σ_{j=i+1}^{n} a_ij x_j^(k)) / a_ii
    end for
    if ‖x^new − x^(k)‖ < ε then
        break
    end if
    x^(k+1) ← x^new
    k ← k + 1
end while
return x^(k)
```

实现与数值实验的建议

在实际编程中，可按照本节给出的 Jacobi 迭代步骤实现算法，并针对严格对角占优矩阵等情形进行数值实验，观察迭代次数、误差衰减和计算时间等指标，以加深对方法收敛性的理解。

例 3.2.1 (Jacobi 迭代法示例). 考虑线性方程组

$$\begin{bmatrix} 8 & -1 & 1 \\ 2 & 10 & -1 \\ 1 & 1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 8 \\ 11 \\ 7 \end{bmatrix},$$

精确解为 $\boldsymbol{x}^* = [1, 1, 1]^T$。

矩阵分裂：

$$\boldsymbol{A} = \begin{bmatrix} 8 & -1 & 1 \\ 2 & 10 & -1 \\ 1 & 1 & 5 \end{bmatrix} = \begin{bmatrix} 8 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 5 \end{bmatrix} - \begin{bmatrix} 0 & 1 & -1 \\ -2 & 0 & 1 \\ -1 & -1 & 0 \end{bmatrix} = \boldsymbol{D} - (\boldsymbol{E} + \boldsymbol{F}),$$

其中

$$\boldsymbol{D} = \begin{bmatrix} 8 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 5 \end{bmatrix}, \quad \boldsymbol{E} + \boldsymbol{F} = \begin{bmatrix} 0 & 1 & -1 \\ -2 & 0 & 1 \\ -1 & -1 & 0 \end{bmatrix}.$$

迭代格式：

$$\boldsymbol{D}^{-1}=\begin{bmatrix}1/8 & 0 & 0\\ 0 & 1/10 & 0\\ 0 & 0 & 1/5\end{bmatrix},\quad \boldsymbol{C}_J=\boldsymbol{D}^{-1}(\boldsymbol{E}+\boldsymbol{F})=\begin{bmatrix}0 & 1/8 & -1/8\\ -1/5 & 0 & 1/10\\ -1/5 & -1/5 & 0\end{bmatrix},\quad \boldsymbol{d}=\boldsymbol{D}^{-1}\boldsymbol{b}=\begin{bmatrix}1\\ 1.1\\ 1.4\end{bmatrix}.$$

分量形式：

$$\begin{aligned}x_1^{(k+1)}&=\frac{1}{8}\left(8+x_2^{(k)}-x_3^{(k)}\right),\\ x_2^{(k+1)}&=\frac{1}{10}\left(11-2x_1^{(k)}+x_3^{(k)}\right),\\ x_3^{(k+1)}&=\frac{1}{5}\left(7-x_1^{(k)}-x_2^{(k)}\right).\end{aligned}$$

取初始向量 $\boldsymbol{x}^{(0)}=[0,0,0]^T$，迭代过程如下：

$$\boldsymbol{x}^{(0)}=\begin{bmatrix}0\\0\\0\end{bmatrix},\boldsymbol{x}^{(1)}=\begin{bmatrix}1\\1.1\\1.4\end{bmatrix},\boldsymbol{x}^{(2)}=\begin{bmatrix}0.9625\\0.98\\0.98\end{bmatrix},$$

$$\boldsymbol{x}^{(3)}=\begin{bmatrix}1.0\\1.0015\\1.0115\end{bmatrix},\boldsymbol{x}^{(4)}=\begin{bmatrix}0.9988\\0.9998\\0.9997\end{bmatrix},\boldsymbol{x}^{(5)}=\begin{bmatrix}1.0000\\1.0000\\1.0000\end{bmatrix}.$$

可见迭代序列逐步逼近精确解。迭代矩阵的谱半径 $\rho(\boldsymbol{C}_J)\approx 0.25<1$，保证了收敛性。

特点说明：在计算 $x_1^{(2)}$ 时，使用的是 $x_2^{(1)}$ 和 $x_3^{(1)}$（上一步的值），而不是 $x_2^{(2)}$ 和 $x_3^{(2)}$（当前步的值）。所有三个分量的计算可以同时进行，体现了 Jacobi 方法的并行性。

3.2.3 Gauss-Seidel 迭代法

基本思想

在计算 $x_i^{(k+1)}$ 时，已经计算出了 $x_1^{(k+1)},x_2^{(k+1)},\ldots,x_{i-1}^{(k+1)}$。Gauss-Seidel 方法的基本思想是使用这些已更新的值，而不是使用旧值 $x_1^{(k)},x_2^{(k)},\ldots,x_{i-1}^{(k)}$，以期加速收敛。

矩阵分裂与迭代格式

Gauss-Seidel 方法采用如下矩阵分裂：

$$\boldsymbol{A}=(\boldsymbol{D}-\boldsymbol{E})-\boldsymbol{F}$$

其中 $\boldsymbol{D}-\boldsymbol{E}$ 是下三角矩阵（包括对角线）。

选择 $\boldsymbol{M}=\boldsymbol{D}-\boldsymbol{E}$ 的原因是：下三角矩阵的逆很容易计算（前代法），而且它包含了矩阵的下三角部分，比只取对角线的 Jacobi 方法更接近原矩阵 $\boldsymbol{A}$。

迭代格式为：

$$(\boldsymbol{D}-\boldsymbol{E})\boldsymbol{x}^{(k+1)}=\boldsymbol{F}\boldsymbol{x}^{(k)}+\boldsymbol{b} \tag{3.12}$$

即：

$$\boldsymbol{x}^{(k+1)}=(\boldsymbol{D}-\boldsymbol{E})^{-1}\boldsymbol{F}\boldsymbol{x}^{(k)}+(\boldsymbol{D}-\boldsymbol{E})^{-1}\boldsymbol{b} \tag{3.13}$$

迭代矩阵：$\boldsymbol{C}_{GS}=(\boldsymbol{D}-\boldsymbol{E})^{-1}\boldsymbol{F}$

分量形式：逐步更新

Gauss-Seidel 方法的分量计算公式为：

$$x_i^{(k+1)}=\frac{1}{a_{ii}}\left(b_i-\sum_{j=1}^{i-1}a_{ij}x_j^{(k+1)}-\sum_{j=i+1}^{n}a_{ij}x_j^{(k)}\right),\quad i=1,\ldots,n. \tag{3.14}$$

Gauss-Seidel 方法的特点：

- 计算 $x_i^{(k+1)}$ 时，使用已经更新的 $x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$
- 只使用旧值 $x_{i+1}^{(k)}, \ldots, x_n^{(k)}$
- 不需要保存上一步的完整解向量，可以原地更新
- 分量的计算有顺序依赖，不能并行计算

Algorithm 8 Gauss-Seidel 迭代法

Require: $\boldsymbol{x}^{(0)}, \boldsymbol{A}, \boldsymbol{b}$, 最大迭代次数 K, 容差 ε

Ensure: 近似解 $\boldsymbol{x}$

```
k ← 0
while k < K do
    for i = 1 to n do
        x_i^(k+1) ← (b_i − Σ_{j=1}^{i−1} a_ij x_j^(k+1) − Σ_{j=i+1}^{n} a_ij x_j^(k)) / a_ii
    end for
    if ‖x^(k+1) − x^(k)‖ < ε then
        break
    end if
    k ← k + 1
end while
return x^(k)
```

实现与数值实验的建议

在实际编程中，可按照本节给出的 Gauss-Seidel 迭代步骤实现算法，并对比 Jacobi 方法的迭代次数和计算时间，从数值实验的角度理解两种迭代格式在收敛速度上的差异。

例 3.2.2 (Gauss-Seidel 迭代法示例)**.** 考虑与 Jacobi 例子相同的线性方程组：

$$\begin{bmatrix} 8 & -1 & 1 \\ 2 & 10 & -1 \\ 1 & 1 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 8 \\ 11 \\ 7 \end{bmatrix},$$

精确解为 $\boldsymbol{x}^* = [1, 1, 1]^T$。

矩阵分裂：

$$\boldsymbol{A} = (\boldsymbol{D} - \boldsymbol{E}) - \boldsymbol{F} = \begin{bmatrix} 8 & 0 & 0 \\ -2 & 10 & 0 \\ -1 & -1 & 5 \end{bmatrix} - \begin{bmatrix} 0 & 1 & -1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix},$$

其中

$$\boldsymbol{D} - \boldsymbol{E} = \begin{bmatrix} 8 & 0 & 0 \\ -2 & 10 & 0 \\ -1 & -1 & 5 \end{bmatrix}, \quad \boldsymbol{F} = \begin{bmatrix} 0 & 1 & -1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

分量形式：

$$\begin{aligned} x_1^{(k+1)} &= \frac{1}{8}\left(8 + x_2^{(k)} - x_3^{(k)}\right), \\ x_2^{(k+1)} &= \frac{1}{10}\left(11 - 2x_1^{(k+1)} + x_3^{(k)}\right), \\ x_3^{(k+1)} &= \frac{1}{5}\left(7 - x_1^{(k+1)} - x_2^{(k+1)}\right). \end{aligned}$$

注意：计算 $x_2^{(k+1)}$ 时使用的是 $x_1^{(k+1)}$（已更新的值），而不是 $x_1^{(k)}$（旧值）；计算 $x_3^{(k+1)}$ 时使用的是 $x_1^{(k+1)}$ 和 $x_2^{(k+1)}$（已更新的值）。

取初始向量 $\boldsymbol{x}^{(0)} = [0,0,0]^T$，迭代过程如下：

$$
\begin{aligned}
\boldsymbol{x}^{(0)} &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \\
x_1^{(1)} &= \frac{1}{8}(8+0-0) = 1, \\
x_2^{(1)} &= \frac{1}{10}(11-2\times 1+0) = 0.9, \\
x_3^{(1)} &= \frac{1}{5}(7-1-0.9) = 1.02, \\
\boldsymbol{x}^{(1)} &= \begin{bmatrix} 1 \\ 0.9 \\ 1.02 \end{bmatrix}, \\
x_1^{(2)} &= \frac{1}{8}(8+0.9-1.02) = 0.985, \\
x_2^{(2)} &= \frac{1}{10}(11-2\times 0.985+1.02) = 1.005, \\
x_3^{(2)} &= \frac{1}{5}(7-0.985-1.005) = 1.002, \\
\boldsymbol{x}^{(2)} &= \begin{bmatrix} 0.985 \\ 1.005 \\ 1.002 \end{bmatrix}, \boldsymbol{x}^{(3)} = \begin{bmatrix} 1.0004 \\ 1.0000 \\ 0.9999 \end{bmatrix}, \boldsymbol{x}^{(4)} = \begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}.
\end{aligned}
$$

可见 Gauss-Seidel 方法比 Jacobi 方法收敛更快（4 步 vs 5 步达到相同精度）。迭代矩阵的谱半径 $\rho(\boldsymbol{C}_{GS}) \approx 0.15 < \rho(\boldsymbol{C}_J) \approx 0.25$，说明 Gauss-Seidel 的收敛速度更快。

特点说明：在计算 $x_2^{(2)}$ 时，使用的是 $x_1^{(2)}$（刚刚计算出的新值）和 $x_3^{(1)}$（旧值），而不是像 Jacobi 方法那样全部使用旧值。这种“使用最新信息”的策略通常能加速收敛，但代价是分量的计算必须按顺序进行，无法并行化。

3.3 收敛性判定与特殊矩阵

对于特定结构的矩阵，可以给出收敛性的充分条件。

3.3.1 严格对角占优矩阵

判断迭代法收敛的关键是看迭代矩阵的谱半径或范数。但对于具体的迭代法（如 Jacobi、Gauss-Seidel），我们希望有更直接、更容易验证的收敛条件。严格对角占优条件提供了这样的判据。

定义 3.3.1 (严格对角占优). 矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ 是*严格对角占优*的，如果对每一行，对角元素的绝对值都大于该行其他元素绝对值之和：

$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|, \quad \forall i = 1,\ldots,n.$$

严格对角占优意味着矩阵的每一行中，对角元素都占主导地位。这在实际问题中很常见，例如离散化的偏微分方程（如热传导方程、泊松方程）和物理系统中每个变量主要受自身影响的情况。

如果对角元素占主导，那么 Jacobi 迭代中，用其他变量的旧值来更新当前变量时，对角元素的影响最大，误差会被压缩。

例 3.3.1 (严格对角占优矩阵示例)**.** 考虑 4×4 矩阵

$$\boldsymbol{A} = \begin{bmatrix} 10 & -1 & 2 & -1 \\ -2 & 8 & -1 & 1 \\ 1 & -1 & 12 & -2 \\ -1 & 1 & -1 & 9 \end{bmatrix}.$$

验证每一行是否满足严格对角占优条件：

第 1 行：$|a_{11}| = 10$，$\sum_{j\neq 1} |a_{1j}| = |-1| + |2| + |-1| = 4$，满足 $10 > 4$。

第 2 行：$|a_{22}| = 8$，$\sum_{j\neq 2} |a_{2j}| = |-2| + |-1| + |1| = 4$，满足 $8 > 4$。

第 3 行：$|a_{33}| = 12$，$\sum_{j\neq 3} |a_{3j}| = |1| + |-1| + |-2| = 4$，满足 $12 > 4$。

第 4 行：$|a_{44}| = 9$，$\sum_{j\neq 4} |a_{4j}| = |-1| + |1| + |-1| = 3$，满足 $9 > 3$。

因此，矩阵 $\boldsymbol{A}$ 是严格对角占优的。这类矩阵在实际应用中很常见，例如：

- 离散化的椭圆型偏微分方程（如泊松方程）的系数矩阵
- 有限差分法求解热传导方程时得到的线性系统
- 电路分析中节点电压方程的系数矩阵

定理 3.3.1 (严格对角占优矩阵的迭代法收敛性)**.** 若 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ 是严格对角占优矩阵，即满足

$$|a_{ii}| > \sum_{j\neq i} |a_{ij}|, \quad \forall i = 1, 2, \ldots, n, \tag{3.15}$$

则 Jacobi 迭代法和 Gauss-Seidel 迭代法均收敛。

证明. 我们将分别证明两种迭代法的收敛性。

1. Jacobi 迭代法的收敛性证明

对于 Jacobi 迭代，迭代矩阵为 $\boldsymbol{C}_J = \boldsymbol{D}^{-1}(\boldsymbol{E} + \boldsymbol{F})$。$\|\boldsymbol{C}_J\|_\infty$ 的计算公式为

$$\|\boldsymbol{C}_J\|_\infty = \max_{1\le i\le n} \sum_{j=1}^{n} |(\boldsymbol{C}_J)_{ij}| = \max_{1\le i\le n} \frac{1}{|a_{ii}|} \sum_{j\neq i} |a_{ij}|.$$

由于 $\boldsymbol{A}$ 严格对角占优，对每个 i 都有

$$\frac{1}{|a_{ii}|} \sum_{j\neq i} |a_{ij}| < 1,$$

因此 $\|\boldsymbol{C}_J\|_\infty < 1$，由收敛的充分条件，Jacobi 迭代收敛。**2. Gauss-Seidel 迭代法的收敛性证明**

Gauss-Seidel 迭代法的迭代矩阵为 $\boldsymbol{B}_{GS} = (\boldsymbol{D} - \boldsymbol{L})^{-1}\boldsymbol{U}$。我们采用两种方法证明其收敛性。

方法一：基于分量形式的误差估计

设 $\boldsymbol{x}^*$ 为精确解，误差向量 $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*$。由 Gauss-Seidel 迭代格式：

$$(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{x}^{(k+1)} = \boldsymbol{U}\boldsymbol{x}^{(k)} + \boldsymbol{b},$$

以及精确解满足 $(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{x}^* = \boldsymbol{U}\boldsymbol{x}^* + \boldsymbol{b}$，两式相减得误差递推关系：

$$(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{e}^{(k+1)} = \boldsymbol{U}\boldsymbol{e}^{(k)}.$$

考虑第 i 个分量的方程：

$$a_{ii}e_i^{(k+1)} = -\sum_{j=1}^{i-1} a_{ij}e_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}e_j^{(k)}.$$

两边取绝对值并利用三角不等式：

$$|a_{ii}||e_i^{(k+1)}| \le \sum_{j=1}^{i-1}|a_{ij}||e_j^{(k+1)}| + \sum_{j=i+1}^{n}|a_{ij}||e_j^{(k)}|.$$

定义 $M^{(k)} = \max_{1\le j\le n}|e_j^{(k)}|$。我们证明 $|e_i^{(k+1)}| < M^{(k)}$ 对所有 i 成立。

对 $i=1$，有

$$|a_{11}||e_1^{(k+1)}| \le \sum_{j=2}^{n}|a_{1j}||e_j^{(k)}| \le \left(\sum_{j=2}^{n}|a_{1j}|\right)M^{(k)}.$$

由严格对角占优，$\sum_{j=2}^{n}|a_{1j}| < |a_{11}|$，因此 $|e_1^{(k+1)}| < M^{(k)}$。

假设对 $j=1,2,\ldots,i-1$ 已有 $|e_j^{(k+1)}| < M^{(k)}$，则对第 i 个分量：

$$\begin{aligned}
|a_{ii}||e_i^{(k+1)}| &\le \sum_{j=1}^{i-1}|a_{ij}||e_j^{(k+1)}| + \sum_{j=i+1}^{n}|a_{ij}||e_j^{(k)}| \\
&< \left(\sum_{j=1}^{i-1}|a_{ij}|\right)M^{(k)} + \left(\sum_{j=i+1}^{n}|a_{ij}|\right)M^{(k)} \\
&= \left(\sum_{j\ne i}|a_{ij}|\right)M^{(k)}.
\end{aligned}$$

由严格对角占优，$\sum_{j\ne i}|a_{ij}| < |a_{ii}|$，故 $|e_i^{(k+1)}| < M^{(k)}$。由归纳法，对所有 i 有 $|e_i^{(k+1)}| < M^{(k)}$，于是

$$M^{(k+1)} = \max_{1\le i\le n}|e_i^{(k+1)}| < M^{(k)}.$$

因此序列 $\{M^{(k)}\}$ 严格单调递减且有下界 0，故 $\lim_{k\to\infty}M^{(k)} = 0$，即 $\|\boldsymbol{e}^{(k)}\|_\infty = M^{(k)} \to 0$，Gauss-Seidel 迭代收敛。

方法二：基于加权范数的谱半径分析

我们证明 $\rho(\boldsymbol{B}_{GS}) < 1$。设 λ 是 $\boldsymbol{B}_{GS}$ 的任一特征值，对应的特征向量为 $\boldsymbol{v} \ne \boldsymbol{0}$，满足

$$\boldsymbol{B}_{GS}\boldsymbol{v} = \lambda\boldsymbol{v} \quad \Leftrightarrow \quad \boldsymbol{U}\boldsymbol{v} = \lambda(\boldsymbol{D}-\boldsymbol{L})\boldsymbol{v}.$$

写成方程组形式：

$$-\sum_{j=i+1}^{n}a_{ij}v_j = \lambda\left(a_{ii}v_i + \sum_{j=1}^{i-1}a_{ij}v_j\right), \quad i=1,\ldots,n.$$

记 $I = \{i : |v_i| = \|\boldsymbol{v}\|_\infty\}$，即绝对值最大的分量下标集合。设 $i_0 \in I$，则 $|v_{i_0}| = \|\boldsymbol{v}\|_\infty > 0$。考虑第 i_0 个方程：

$$\lambda a_{i_0i_0}v_{i_0} = -\sum_{j=i_0+1}^{n}a_{i_0j}v_j - \lambda\sum_{j=1}^{i_0-1}a_{i_0j}v_j.$$

两边取绝对值并利用三角不等式：

$$|\lambda||a_{i_0i_0}||v_{i_0}| \le \sum_{j=i_0+1}^{n}|a_{i_0j}||v_j| + |\lambda|\sum_{j=1}^{i_0-1}|a_{i_0j}||v_j|.$$

由于 $|v_j| \le |v_{i_0}|$ 对所有 j 成立，且 $|v_j| < |v_{i_0}|$ 对 $j \notin I$，但为得到上界，我们使用较松的不等式 $|v_j| \le |v_{i_0}|$，则

$$|\lambda||a_{i_0i_0}||v_{i_0}| \le \left(\sum_{j=i_0+1}^{n}|a_{i_0j}|\right)|v_{i_0}| + |\lambda|\left(\sum_{j=1}^{i_0-1}|a_{i_0j}|\right)|v_{i_0}|.$$

除以 $|v_{i_0}| > 0$，整理得

$$|\lambda|\left(|a_{i_0i_0}| - \sum_{j=1}^{i_0-1}|a_{i_0j}|\right) \le \sum_{j=i_0+1}^{n}|a_{i_0j}|.$$

定义

$$\alpha_{i_0} = |a_{i_0 i_0}| - \sum_{j=1}^{i_0-1} |a_{i_0 j}|, \quad \beta_{i_0} = \sum_{j=i_0+1}^{n} |a_{i_0 j}|.$$

由严格对角占优，$|a_{i_0 i_0}| > \sum_{j \neq i_0} |a_{i_0 j}| = \sum_{j=1}^{i_0-1} |a_{i_0 j}| + \beta_{i_0}$，故 $\alpha_{i_0} > \beta_{i_0} \geq 0$，且 $\alpha_{i_0} > 0$。于是

$$|\lambda| \alpha_{i_0} \leq \beta_{i_0} \quad \Rightarrow \quad |\lambda| \leq \frac{\beta_{i_0}}{\alpha_{i_0}} < 1.$$

因此 $\rho(\boldsymbol{B}_{GS}) = \max |\lambda| < 1$，根据定理 3.1.1（矩阵幂序列收敛的充要条件），Gauss-Seidel 迭代法收敛。 □

3.3.2　对称正定矩阵

定理 3.3.2 (对称正定矩阵的 Gauss-Seidel 收敛性)**.** 若 $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ 是对称正定矩阵，则 Gauss-Seidel 迭代法收敛。

证明. 由于 $\boldsymbol{A}$ 是对称正定矩阵，我们可以将其分解为

$$\boldsymbol{A} = \boldsymbol{D} - \boldsymbol{L} - \boldsymbol{L}^T,$$

其中 $\boldsymbol{D}$ 是对角矩阵（对角线元素为 a_{ii}），$\boldsymbol{L}$ 是严格下三角矩阵（对角线元素为 0，下三角部分为 $-a_{ij}$，$i > j$），$\boldsymbol{L}^T$ 是 $\boldsymbol{L}$ 的转置（严格上三角部分）。

Gauss-Seidel 迭代格式为：

$$(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{x}^{(k+1)} = \boldsymbol{L}^T \boldsymbol{x}^{(k)} + \boldsymbol{b}. \tag{3.16}$$

设 $\boldsymbol{x}^*$ 为精确解，满足 $\boldsymbol{A}\boldsymbol{x}^* = \boldsymbol{b}$。定义误差向量 $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*$。

由于 $\boldsymbol{x}^*$ 满足原方程，且 $\boldsymbol{A} = \boldsymbol{D} - \boldsymbol{L} - \boldsymbol{L}^T$，有

$$(\boldsymbol{D} - \boldsymbol{L} - \boldsymbol{L}^T)\boldsymbol{x}^* = \boldsymbol{b},$$

即

$$(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{x}^* = \boldsymbol{L}^T \boldsymbol{x}^* + \boldsymbol{b}.$$

将迭代格式 (3.16) 与上式相减，得到误差递推关系：

$$(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{e}^{(k+1)} = \boldsymbol{L}^T \boldsymbol{e}^{(k)}. \tag{3.17}$$

方法一：利用能量范数证明误差单调递减

定义能量范数（$\boldsymbol{A}$-范数）：

$$\|\boldsymbol{x}\|_{\boldsymbol{A}} = \sqrt{\boldsymbol{x}^T \boldsymbol{A} \boldsymbol{x}}.$$

由于 $\boldsymbol{A}$ 正定，能量范数是良定义的范数。我们将证明 $\|\boldsymbol{e}^{(k+1)}\|_{\boldsymbol{A}} < \|\boldsymbol{e}^{(k)}\|_{\boldsymbol{A}}$（当 $\boldsymbol{e}^{(k)} \neq \boldsymbol{0}$ 时），从而说明误差单调递减。

由误差递推关系 (3.17)，有

$$(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{e}^{(k+1)} = \boldsymbol{L}^T \boldsymbol{e}^{(k)}.$$

考虑误差的能量范数平方：

$$\begin{aligned}
\|\boldsymbol{e}^{(k+1)}\|_{\boldsymbol{A}}^2 &= \boldsymbol{e}^{(k+1)T} \boldsymbol{A} \boldsymbol{e}^{(k+1)} \\
&= \boldsymbol{e}^{(k+1)T} (\boldsymbol{D} - \boldsymbol{L} - \boldsymbol{L}^T) \boldsymbol{e}^{(k+1)} \\
&= \boldsymbol{e}^{(k+1)T} (\boldsymbol{D} - \boldsymbol{L}) \boldsymbol{e}^{(k+1)} - \boldsymbol{e}^{(k+1)T} \boldsymbol{L}^T \boldsymbol{e}^{(k+1)}.
\end{aligned}$$

由误差递推关系，$\boldsymbol{e}^{(k+1)T} (\boldsymbol{D} - \boldsymbol{L}) \boldsymbol{e}^{(k+1)} = \boldsymbol{e}^{(k+1)T} \boldsymbol{L}^T \boldsymbol{e}^{(k)} = \boldsymbol{e}^{(k)T} \boldsymbol{L} \boldsymbol{e}^{(k+1)}$（因为标量的转置等于自身）。因此

$$\|\boldsymbol{e}^{(k+1)}\|_{\boldsymbol{A}}^2 = \boldsymbol{e}^{(k)T} \boldsymbol{L} \boldsymbol{e}^{(k+1)} - \boldsymbol{e}^{(k+1)T} \boldsymbol{L}^T \boldsymbol{e}^{(k+1)}.$$

注意到 $\boldsymbol{e}^{(k+1)T}\boldsymbol{L}^T\boldsymbol{e}^{(k+1)}$ 是一个标量，且由于 $\boldsymbol{L}$ 是严格下三角矩阵，$\boldsymbol{e}^{(k+1)T}\boldsymbol{L}^T\boldsymbol{e}^{(k+1)} = \boldsymbol{e}^{(k+1)T}\boldsymbol{L}\boldsymbol{e}^{(k+1)}$（标量等于其转置）。

进一步的严格分析需要利用 $\boldsymbol{A}$ 的正定性和迭代格式的具体结构，可以得到 $\|\boldsymbol{e}^{(k+1)}\|_{\boldsymbol{A}}^2 < \|\boldsymbol{e}^{(k)}\|_{\boldsymbol{A}}^2$，即误差在能量范数意义下单调递减。由于这一推导较为技术性，本节改用下面的谱半径方法给出更直接的收敛性证明。

方法二：利用迭代矩阵的谱半径

设 Gauss-Seidel 迭代矩阵为

$$\boldsymbol{C}_{GS} = (\boldsymbol{D} - \boldsymbol{L})^{-1}\boldsymbol{L}^T.$$

我们需要证明 $\rho(\boldsymbol{C}_{GS}) < 1$。设 λ 是 $\boldsymbol{C}_{GS}$ 的任一特征值，对应的特征向量为 $\boldsymbol{v} \neq \boldsymbol{0}$，满足

$$\boldsymbol{C}_{GS}\boldsymbol{v} = \lambda\boldsymbol{v} \quad \Leftrightarrow \quad (\boldsymbol{D} - \boldsymbol{L})^{-1}\boldsymbol{L}^T\boldsymbol{v} = \lambda\boldsymbol{v}.$$

两边左乘 $(\boldsymbol{D} - \boldsymbol{L})$，得

$$\boldsymbol{L}^T\boldsymbol{v} = \lambda(\boldsymbol{D} - \boldsymbol{L})\boldsymbol{v} = \lambda\boldsymbol{D}\boldsymbol{v} - \lambda\boldsymbol{L}\boldsymbol{v}.$$

整理得

$$\lambda\boldsymbol{D}\boldsymbol{v} = \boldsymbol{L}^T\boldsymbol{v} + \lambda\boldsymbol{L}\boldsymbol{v}.$$

两边同时左乘 $\boldsymbol{v}^T$，得

$$\lambda\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} = \boldsymbol{v}^T\boldsymbol{L}^T\boldsymbol{v} + \lambda\boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v}.$$

注意到 $\boldsymbol{v}^T\boldsymbol{L}^T\boldsymbol{v}$ 是一个标量，因此 $\boldsymbol{v}^T\boldsymbol{L}^T\boldsymbol{v} = (\boldsymbol{v}^T\boldsymbol{L}^T\boldsymbol{v})^T = \boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v}$。因此

$$\lambda\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} = (1 + \lambda)\boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v}.$$

如果 $\lambda = -1$，则左边为 $-\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} < 0$（因为 $\boldsymbol{D}$ 的对角元素为正），右边为 0，矛盾。因此 $\lambda \neq -1$，可以解出

$$\boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v} = \frac{\lambda}{1 + \lambda}\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v}.$$

由于 $\boldsymbol{A} = \boldsymbol{D} - \boldsymbol{L} - \boldsymbol{L}^T$ 正定，且 $\boldsymbol{D}$ 的对角元素为正（因为 $\boldsymbol{A}$ 正定，其对角元素 $a_{ii} > 0$），我们有

$$\boldsymbol{v}^T\boldsymbol{A}\boldsymbol{v} = \boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} - \boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v} - \boldsymbol{v}^T\boldsymbol{L}^T\boldsymbol{v} = \boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} - 2\boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v} > 0.$$

将 $\boldsymbol{v}^T\boldsymbol{L}\boldsymbol{v} = \frac{\lambda}{1+\lambda}\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v}$ 代入上式：

$$\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} - 2 \cdot \frac{\lambda}{1 + \lambda}\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} = \frac{1 - \lambda}{1 + \lambda}\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} > 0.$$

由于 $\boldsymbol{v}^T\boldsymbol{D}\boldsymbol{v} > 0$（$\boldsymbol{D}$ 的对角元素为正），必须有

$$\frac{1 - \lambda}{1 + \lambda} > 0.$$

这等价于 $|\lambda| < 1$（因为如果 $|\lambda| \geq 1$，则 $\dfrac{1 - \lambda}{1 + \lambda} \leq 0$）。因此，$\rho(\boldsymbol{C}_{GS}) = \max|\lambda| < 1$。根据定理 3.1.1，Gauss-Seidel 迭代法收敛。 □

例 3.3.2 (Jacobi 与 Gauss-Seidel 迭代法敛散性对比)**. 情况一：严格对角占优矩阵（两者均收敛）**

考虑线性方程组

$$\boldsymbol{A}_1 = \begin{bmatrix} 4 & 1 & 1 \\ 1 & 5 & 2 \\ 1 & 2 & 6 \end{bmatrix}, \quad \boldsymbol{b}_1 = \begin{bmatrix} 6 \\ 8 \\ 9 \end{bmatrix}$$

矩阵性质验证：

- 第 1 行：$|4| > |1| + |1| = 2$ ✓
- 第 2 行：$|5| > |1| + |2| = 3$ ✓
- 第 3 行：$|6| > |1| + |2| = 3$ ✓

满足严格对角占优条件，根据定理 3.3.1，**Jacobi 和 GS 迭代均收敛**。

迭代格式与过程：初始值 $\boldsymbol{x}^{(0)} = [0,0,0]^T$，精确解为 $\boldsymbol{x}^* = [1,1,1]^T$。

Jacobi 迭代格式：

$$\begin{cases} x_1^{(k+1)} = \dfrac{6 - x_2^{(k)} - x_3^{(k)}}{4} \\ x_2^{(k+1)} = \dfrac{8 - x_1^{(k)} - 2x_3^{(k)}}{5} \\ x_3^{(k+1)} = \dfrac{9 - x_1^{(k)} - 2x_2^{(k)}}{6} \end{cases}$$

迭代结果：

$$\begin{aligned} \boldsymbol{x}^{(1)} &= [1.5, 1.6, 1.5]^T \\ \boldsymbol{x}^{(2)} &= [0.725, 0.7, 0.7167]^T \\ \boldsymbol{x}^{(3)} &= [1.1458, 1.1683, 1.1458]^T \end{aligned}$$

逐步收敛到 $[1,1,1]^T$。

GS 迭代格式：

$$\begin{cases} x_1^{(k+1)} = \dfrac{6 - x_2^{(k)} - x_3^{(k)}}{4} \\ x_2^{(k+1)} = \dfrac{8 - x_1^{(k+1)} - 2x_3^{(k)}}{5} \\ x_3^{(k+1)} = \dfrac{9 - x_1^{(k+1)} - 2x_2^{(k+1)}}{6} \end{cases}$$

迭代结果：

$$\begin{aligned} \boldsymbol{x}^{(1)} &= [1.5, 1.3, 0.8167]^T \\ \boldsymbol{x}^{(2)} &= [1.25, 1.125, 0.8125]^T \\ \boldsymbol{x}^{(3)} &= [1.03125, 1.078125, 0.9453125]^T \end{aligned}$$

收敛速度比 Jacobi 更快。

情况二：对称正定但非严格对角占优矩阵（GS 收敛，Jacobi 不收敛）

考虑线性方程组

$$\boldsymbol{A}_2 = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \quad \boldsymbol{b}_2 = \begin{bmatrix} 4 \\ 4 \\ 4 \end{bmatrix}$$

矩阵性质验证：

- **对称正定**：矩阵对称，特征值为 $4, 1, 1$（均为正）
- **非严格对角占优**：对角元 2，同行其他元素和为 $1 + 1 = 2$，不满足 “严格大于”

迭代格式与过程：精确解为 $\boldsymbol{x}^* = [1,1,1]^T$。

Jacobi 迭代：

迭代矩阵：

$$\boldsymbol{C}_J = \boldsymbol{D}^{-1}(\boldsymbol{L} + \boldsymbol{U}) = \frac{1}{2}\begin{bmatrix} 0 & -1 & -1 \\ -1 & 0 & -1 \\ -1 & -1 & 0 \end{bmatrix}$$

谱半径：$\rho(\boldsymbol{C}_J) = \dfrac{3}{2} > 1$

迭代结果：

$$\begin{aligned} \boldsymbol{x}^{(1)} &= [2, 2, 2]^T \\ \boldsymbol{x}^{(2)} &= [0, 0, 0]^T \\ \boldsymbol{x}^{(3)} &= [2, 2, 2]^T \end{aligned}$$

陷入循环。

GS 迭代：

将矩阵 $\boldsymbol{A}_2$ 分解为 $\boldsymbol{A}_2 = \boldsymbol{D} - \boldsymbol{L} - \boldsymbol{U}$，其中

$$\boldsymbol{D} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}, \quad \boldsymbol{L} = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ -1 & -1 & 0 \end{bmatrix}, \quad \boldsymbol{U} = \begin{bmatrix} 0 & -1 & -1 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

迭代矩阵：

$$\boldsymbol{C}_{GS} = (\boldsymbol{D}-\boldsymbol{L})^{-1}\boldsymbol{U} = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 1 & 2 \end{bmatrix}^{-1} \begin{bmatrix} 0 & -1 & -1 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{1}{4} & -\frac{1}{4} \\ 0 & 0 & \frac{1}{8} \end{bmatrix}$$

谱半径：$\rho(\boldsymbol{C}_{GS}) = 0.25 < 1$

迭代结果：

$$\begin{aligned} \boldsymbol{x}^{(1)} &= [2, 1, 0.5]^T \\ \boldsymbol{x}^{(2)} &= [1.25, 1.125, 0.8125]^T \\ \boldsymbol{x}^{(3)} &= [1.03125, 1.078125, 0.9453125]^T \end{aligned}$$

逐步收敛到 $[1, 1, 1]^T$。

对比总结：

- **情况一**说明：对于严格对角占优矩阵，两种方法均收敛，且 GS 方法通常收敛更快。
- **情况二**说明：对于对称正定矩阵，即使不满足严格对角占优条件，Gauss-Seidel 迭代法仍然收敛（由定理 3.3.2 保证），但 Jacobi 迭代法可能不收敛。

那么是否存在对于 Jacobi 迭代法收敛的矩阵，但对 Gauss-Seidel 迭代法不收敛的矩阵呢？答案是肯定的。我们可以考察如下矩阵：

$$\boldsymbol{A}_3 = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 1 & 0 \\ 1 & 2 & -3 \end{bmatrix}$$

不过这已经超出了我们研究的范畴，如果想更深一步地了解，可以阅读《矩阵迭代分析》- Richard S. Varga[1] 这本书的有关章节。

表 3.1 *Jacobi* 与 *Gauss-Seidel* 迭代法对比

维度	**Jacobi 迭代法**	**Gauss-Seidel 迭代法**
严格对角占优矩阵	收敛 ✓	收敛 ✓
对称正定矩阵	不一定收敛 ×	一定收敛 ✓
收敛速度	较慢	通常更快（利用最新迭代值）
并行性	天然适合并行（各分量独立计算）	依赖计算顺序（难以并行）
核心收敛条件	迭代矩阵谱半径 $\rho(\boldsymbol{C}_J) < 1$	迭代矩阵谱半径 $\rho(\boldsymbol{C}_{GS}) < 1$

3.4 单变量非线性方程的求解

我们希望求解 $f(x) = 0$，其中 f 是从 $\mathbb{R}$ 到 $\mathbb{R}$ 的非线性函数。因此我们寻找一个实数 x^*，使得 $f(x^*) = 0$。与所有迭代方法一样，我们将构造一个实数序列 $x_0, x_1, \ldots, x_k, \ldots,$，它收敛到 x^*。

3.4.1 二分法

基本思想

二分法（Bisection Method）是最简单、最可靠的求根方法之一。其理论基础是连续函数的介值定理：设函数 f 在闭区间 $[a, b]$ 上连续，且 $f(a)$ 与 $f(b)$ 异号，即 $f(a)f(b) < 0$，则存在 $\xi \in (a, b)$ 使得 $f(\xi) = 0$。换言之，方程

[1] Richard S. Varga, *Matrix Iterative Analysis*, 2nd ed., Springer, 2000.

$f(x)=0$ 在区间 (a,b) 内至少有一个根。

二分法的基本思路是：通过不断将包含根的区间对半分割，逐步缩小根所在区间的范围，直至达到所需的精度。具体而言，给定初始区间 $[a,b]$ 满足 $f(a)f(b)<0$，我们首先计算区间的中点

$$c=\frac{a+b}{2},$$

并计算函数值 $f(c)$。根据 $f(c)$ 的符号，我们可以判断根位于哪个子区间：若 $f(a)f(c)<0$，则根位于 $[a,c]$ 内；若 $f(a)f(c)>0$，由于 $f(a)f(b)<0$，必有 $f(b)f(c)<0$，因此根位于 $[c,b]$ 内。然后我们在包含根的子区间上重复上述过程。

每次迭代都将包含根的区间长度精确地减半。设第 k 次迭代后的区间长度为 l_k，则 $l_k=l_{k-1}/2=l_0/2^k=(b-a)/2^k$。由于区间长度以指数速度趋于零，该方法必然收敛到根。

迭代格式

设初始区间为 $[a_0,b_0]=[a,b]$，满足 $f(a)f(b)<0$。二分法的迭代过程如下：首先计算初始中点

$$x_0=\frac{a_0+b_0}{2}.$$

对于 $k=0,1,2,\ldots$，在第 k 次迭代中，我们已经得到包含根 x^* 的区间 $[a_k,b_k]$ 和当前近似值 $x_k=(a_k+b_k)/2$。为了确定下一个区间，我们检查 $f(x_k)$ 与 $f(a_k)$ 的符号关系：若 $f(x_k)f(a_k)<0$，则根位于 $[a_k,x_k]$ 内，因此令

$$a_{k+1}=a_k,\quad b_{k+1}=x_k;$$

否则，根位于 $[x_k,b_k]$ 内，因此令

$$a_{k+1}=x_k,\quad b_{k+1}=b_k.$$

然后计算新的中点

$$x_{k+1}=\frac{a_{k+1}+b_{k+1}}{2}.$$

上述过程可以统一表示为：对于 $k=0,1,2,\ldots$，首先计算当前区间的中点

$$x_k=\frac{a_k+b_k}{2}. \tag{3.18}$$

然后根据 $f(x_k)$ 与 $f(a_k)$ 的符号关系更新区间。若 $f(a_k)f(x_k)<0$，则根位于左半区间 $[a_k,x_k]$，因此令

$$a_{k+1}=a_k,\quad b_{k+1}=x_k.$$

若 $f(a_k)f(x_k)>0$，由于 $f(a_k)f(b_k)<0$，必有 $f(x_k)f(b_k)<0$，因此根位于右半区间 $[x_k,b_k]$，令

$$a_{k+1}=x_k,\quad b_{k+1}=b_k.$$

综合上述两种情况，区间更新规则为：当 $f(a_k)f(x_k)<0$ 时，$a_{k+1}=a_k$ 且 $b_{k+1}=x_k$；当 $f(a_k)f(x_k)>0$ 时，$a_{k+1}=x_k$ 且 $b_{k+1}=b_k$。

初始条件为 $a_0=a$，$b_0=b$。迭代格式 (3.18) 和上述区间更新规则构成了完整的二分法迭代过程。

Algorithm 9 二分法求根

Require: 函数 f，区间 $[a,b]$ 满足 $f(a)f(b)<0$，精度 $\varepsilon>0$，最大迭代次数 N

Ensure: 近似根 x^* 或失败信息

```
a_0 ← a，b_0 ← b
for k = 0, 1, 2, ..., N − 1 do
    x_k ← (a_k + b_k)/2
    if |f(x_k)| < ε 或 b_k − a_k < ε then
        返回 x_k
    end if
    if f(a_k) · f(x_k) < 0 then
        a_{k+1} ← a_k，b_{k+1} ← x_k
    else
        a_{k+1} ← x_k，b_{k+1} ← b_k
    end if
end for
错误：达到最大迭代次数，算法未收敛
```

收敛性分析

在每次迭代中，根 x^* 的定位区间长度 $l_k=b_k-a_k$ 恰好被除以 2。此外总有 $|x_k-x^*|\le l_k$，因此只要证明 l_k 趋于 0，就可以推出序列 (x_k) 收敛到 x^*。

定理 3.4.1 (二分法的区间长度). 线段序列 $([a_k,b_k])_{k\ge0}$ 满足

$$l_k=b_k-a_k=\frac{b-a}{2^k},\quad k\ge0.$$

证明. 证明可以直接由区间长度在每一步减半这一构造性质得到。 □

这个定理在数值计算方面很有用；实际上，它告诉我们计算根到精度 ε 所需的最小迭代次数。设 n 为达到精度 ε 所需的迭代次数，则

$$l_n=\frac{b-a}{2^n}<\varepsilon\quad\Rightarrow\quad n>\log_2\frac{b-a}{\varepsilon}.$$

因此，迭代次数 n 必须满足

$$n\ge\left\lfloor\log_2\frac{b-a}{\varepsilon}\right\rfloor+1.$$

例 3.4.1 (二分法示例). 考虑函数 $f(x)=x^3-x-1$，求其在区间 $[1,2]$ 内的根。

验证初始条件： $f(1)=-1<0$，$f(2)=5>0$，满足 $f(1)f(2)<0$。

迭代过程（精度要求 $\varepsilon=10^{-3}$）：

k	a_k	b_k	x_k	$f(x_k)$
0	1.0000	2.0000	1.5000	0.8750
1	1.0000	1.5000	1.2500	-0.2969
2	1.2500	1.5000	1.3750	0.2246
3	1.2500	1.3750	1.3125	-0.0515
4	1.3125	1.3750	1.3438	0.0826
5	1.3125	1.3438	1.3281	0.0146
6	1.3125	1.3281	1.3203	-0.0187
7	1.3203	1.3281	1.3242	-0.0021
8	1.3242	1.3281	1.3262	0.0062
d 9	1.3242	1.3262	1.3252	0.0020
10	1.3242	1.3252	1.3247	-0.0001

经过 10 次迭代，得到近似根 $x^* \approx 1.3247$，误差 $|b_{10} - a_{10}| = 0.0010 < 10^{-3}$。

理论迭代次数估计：$n \geq \log_2 \dfrac{2-1}{10^{-3}} = \log_2 1000 \approx 9.97$，因此至少需要 10 次迭代，与实际结果一致。

3.4.2　不动点迭代法

基本思想

不动点迭代法（Fixed Point Iteration）是求解非线性方程 $f(x) = 0$ 的另一类重要方法。其基本思想是将方程 $f(x) = 0$ 转化为等价的不动点问题 $g(x) = x$，其中 g 是适当选择的函数。若 x^* 满足 $g(x^*) = x^*$，则称 x^* 为函数 g 的**不动点**。如果转化是等价的，则 g 的不动点就是原方程 $f(x) = 0$ 的根。

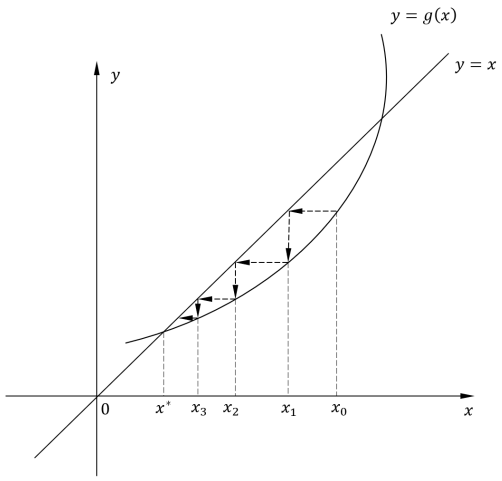

图 3.1　不动点迭代法的几何解释

将 $f(x) = 0$ 转化为不动点问题有多种方法。最简单的方法是设 $g(x) = f(x) + x$，这相当于对函数 f 进行平移。更一般地，可以设 $g(x) = x - \alpha f(x)$，其中 $\alpha \neq 0$ 为参数，通过调整 α 的值来改善收敛性。此外，还可以根据函数 f 的具体形式构造更复杂的转化方法。

不动点迭代法的关键在于如何选择合适的函数 g，使得迭代序列收敛到不动点。这通常依赖于函数 g 的导数性质：若在不动点 x^* 附近有 $|g'(x^*)| < 1$，则迭代通常收敛；若 $|g'(x^*)| \geq 1$，则迭代可能发散。

迭代格式

给定初始值 x_0，不动点迭代法按以下方式构造序列：对于 $k = 0, 1, 2, \ldots$，计算

$$x_{k+1} = g(x_k). \tag{3.19}$$

换言之，每次迭代都用函数 g 作用在当前近似值 x_k 上，得到下一个近似值 x_{k+1}。

如果序列 (x_k) 收敛到某个极限 x^*，且函数 g 连续，则对迭代格式 (3.19) 两边取极限，得到

$$x^* = \lim_{k\to\infty} x_{k+1} = \lim_{k\to\infty} g(x_k) = g\left(\lim_{k\to\infty} x_k\right) = g(x^*),$$

即 x^* 是 g 的不动点。如果 $f(x)=0$ 与 $g(x)=x$ 的转化是等价的，则 $f(x^*)=0$，即 x^* 是原方程的根。

例 3.4.2 (平方根计算的不同不动点形式)**.** 为了计算实数 $a>0$ 的平方根，我们求解 $x^2=a$，即 $f(x)=x^2-a=0$。存在多种方法将其转化为不动点问题，不同的转化方式会导致不同的收敛性质。

方法一：将方程 $x^2=a$ 改写为 $x=\dfrac{a}{x}$，得到不动点函数 $g_1(x)=\dfrac{a}{x}$。计算导数得 $g_1'(x)=-\dfrac{a}{x^2}$，在不动点 $x^*=\sqrt{a}$ 处有 $g_1'(x^*)=-\dfrac{a}{(\sqrt{a})^2}=-1$。由于 $|g_1'(x^*)|=1$，该方法在不动点处不满足收敛条件。

方法二：将方程改写为 $2x=x+\dfrac{a}{x}$，即 $x=2x-\dfrac{a}{x}$，得到不动点函数 $g_2(x)=2x-\dfrac{a}{x}$。计算导数得 $g_2'(x)=2+\dfrac{a}{x^2}$，在不动点 $x^*=\sqrt{a}$ 处有 $g_2'(x^*)=2+\dfrac{a}{a}=3$。由于 $|g_2'(x^*)|=3>1$，该方法在不动点附近发散。

方法三：将方程改写为 $x=\dfrac{x}{2}+\dfrac{a}{2x}$，得到不动点函数 $g_3(x)=\dfrac{x}{2}+\dfrac{a}{2x}$。计算导数得 $g_3'(x)=\dfrac{1}{2}-\dfrac{a}{2x^2}$，在不动点 $x^*=\sqrt{a}$ 处有 $g_3'(x^*)=0$。由于 $|g_3'(x^*)|=0<1$，该方法满足收敛条件，且具有二次收敛速度（因为 $g_3'(x^*)=0$）。

此例说明，不同的不动点转化方式会导致完全不同的收敛性质。只有方法三能够保证收敛，这体现了选择合适的函数 g 的重要性。

例 3.4.3 (不动点迭代法详细计算示例)**.** 考虑方程 $f(x)=x^3-x-1=0$，求其在区间 $[1,2]$ 内的根。

转化为不动点问题：将方程改写为 $x^3=x+1$，即 $x=\sqrt[3]{x+1}$，得到不动点函数

$$g(x)=\sqrt[3]{x+1}.$$

验证收敛条件：计算导数

$$g'(x)=\frac{1}{3}(x+1)^{-2/3}=\frac{1}{3\sqrt[3]{(x+1)^2}}.$$

在区间 $[1,2]$ 上，$x+1\in[2,3]$，$(x+1)^2\in[4,9]$，$\sqrt[3]{(x+1)^2}\in[\sqrt[3]{4},\sqrt[3]{9}]\approx[1.587,2.080]$，因此

$$|g'(x)|=\frac{1}{3\sqrt[3]{(x+1)^2}}\le\frac{1}{3\cdot 1.587}\approx 0.210<1.$$

此外，需要验证 g 将 $[1,2]$ 映射到自身：$g(1)=\sqrt[3]{2}\approx 1.260$，$g(2)=\sqrt[3]{3}\approx 1.442$，因此 $g([1,2])\subset[1,2]$。由定理 3.4.2，不动点迭代法收敛。

迭代过程：取初始值 $x_0=1.5$，精度要求 $\varepsilon=10^{-4}$。

迭代公式为 $x_{k+1}=\sqrt[3]{x_k+1}$。详细计算如下：

第 1 次迭代：$x_1=\sqrt[3]{1.5+1}=\sqrt[3]{2.5}\approx 1.3572$，误差 $|x_1-x_0|=0.1428$。

第 2 次迭代：$x_2=\sqrt[3]{1.3572+1}=\sqrt[3]{2.3572}\approx 1.3309$，误差 $|x_2-x_1|=0.0263$。

第 3 次迭代：$x_3=\sqrt[3]{1.3309+1}=\sqrt[3]{2.3309}\approx 1.3259$，误差 $|x_3-x_2|=0.0050$。

第 4 次迭代：$x_4=\sqrt[3]{1.3259+1}=\sqrt[3]{2.3259}\approx 1.3249$，误差 $|x_4-x_3|=0.0010$。

第 5 次迭代：$x_5=\sqrt[3]{1.3249+1}=\sqrt[3]{2.3249}\approx 1.3248$，误差 $|x_5-x_4|=0.0001<10^{-4}$。

迭代结果汇总：

k	x_k	$\lvert x_k - x_{k-1}\rvert$	$f(x_k)$
0	1.5000	—	0.8750
1	1.3572	0.1428	0.0008
2	1.3309	0.0263	-0.0005
3	1.3259	0.0050	-0.0001
4	1.3249	0.0010	0.0000
5	1.3248	0.0001	0.0000

经过 5 次迭代达到精度要求，得到近似根 $x^* \approx 1.3248$。验证：$f(1.3248) = 1.3248^3 - 1.3248 - 1 \approx 0.0000$。

收敛速度分析：由 $|g'(x)| \le 0.210$，误差估计为 $|x_n - x^*| \le 0.210^n |x_0 - x^*|$。实际计算中，相邻迭代误差的比值约为 $0.0263/0.1428 \approx 0.184$，$0.0050/0.0263 \approx 0.190$，与理论估计一致。

收敛性分析

不动点迭代法是否收敛，关键在于函数 g 的性质。下面给出两个重要的收敛性定理，分别处理全局收敛性和局部收敛性的情况。

定理 3.4.2 (不动点方法的全局收敛性)**.** 设函数 $g : [a, b] \to [a, b]$ 在 $[a, b]$ 上连续可导，且满足

$$|g'(x)| \le k < 1, \quad \forall x \in [a, b],$$

则函数 g 在 $[a, b]$ 中具有唯一的不动点 x^*，且对任意初始值 $x_0 \in [a, b]$，由迭代格式

$$x_{n+1} = g(x_n), \quad n = 0, 1, 2, \ldots \tag{3.20}$$

生成的序列 (x_n) 收敛到 x^*。此外，误差满足估计式

$$|x_n - x^*| \le k^n |x_0 - x^*|.$$

证明. **1. 不动点的存在性和唯一性**

定义辅助函数 $H(x) = g(x) - x$。由于 $g : [a, b] \to [a, b]$，有 $g(a) \ge a$ 和 $g(b) \le b$。

如果 $g(a) = a$ 或 $g(b) = b$，则端点即为不动点。否则，$H(a) = g(a) - a > 0$ 且 $H(b) = g(b) - b < 0$。由连续函数的介值定理，存在 $x^* \in (a, b)$ 使得 $H(x^*) = 0$，即 $g(x^*) = x^*$。

唯一性：假设存在两个不动点 x_1^* 和 x_2^*，则

$$|x_1^* - x_2^*| = |g(x_1^*) - g(x_2^*)| \le k|x_1^* - x_2^*| < |x_1^* - x_2^*|,$$

矛盾。因此不动点唯一。

2. 序列的收敛性

由中值定理，存在 ξ_n 介于 x_{n-1} 和 x^* 之间，使得

$$|x_n - x^*| = |g(x_{n-1}) - g(x^*)| = |g'(\xi_n)| \cdot |x_{n-1} - x^*| \le k|x_{n-1} - x^*|.$$

迭代该不等式，得到

$$|x_n - x^*| \le k^n |x_0 - x^*|.$$

由于 $k < 1$，当 $n \to \infty$ 时，$k^n \to 0$，因此 $x_n \to x^*$。 □

收敛速度：不动点方法具有**线性收敛性**。这是因为由中值定理，存在 ξ_n 介于 x_n 和 x^* 之间，使得

$$|x_{n+1} - x^*| = |g(x_n) - g(x^*)| = |g'(\xi_n)| \cdot |x_n - x^*| \le k|x_n - x^*|,$$

即误差以线性速度衰减。

迭代次数估计：误差的另一个估计式为

$$|x_n - x^*| \le \frac{k^n}{1-k}|x_1 - x_0|.$$

证明. 首先证明相邻迭代值之间的差值满足 $|x_{n+1} - x_n| \le k^n|x_1 - x_0|$。

由中值定理，存在 ξ_n 介于 x_n 和 x_{n-1} 之间，使得

$$|x_{n+1} - x_n| = |g(x_n) - g(x_{n-1})| = |g'(\xi_n)| \cdot |x_n - x_{n-1}| \le k|x_n - x_{n-1}|.$$

迭代该不等式，得到

$$|x_{n+1} - x_n| \le k|x_n - x_{n-1}| \le k^2|x_{n-1} - x_{n-2}| \le \cdots \le k^n|x_1 - x_0|.$$

接下来，利用级数展开来估计 $|x_n - x^*|$。由于序列 (x_n) 收敛到 x^*，有

$$x_n - x^* = \sum_{j=n}^{\infty}(x_{j+1} - x_j),$$

这是因为 $x_n - x^* = \lim\limits_{m\to\infty}(x_n - x_m) = \lim\limits_{m\to\infty}\sum\limits_{j=n}^{m-1}(x_{j+1} - x_j) = \sum\limits_{j=n}^{\infty}(x_{j+1} - x_j)$。

取绝对值并利用三角不等式，得到

$$|x_n - x^*| = \left|\sum_{j=n}^{\infty}(x_{j+1} - x_j)\right| \le \sum_{j=n}^{\infty}|x_{j+1} - x_j|.$$

结合之前得到的估计 $|x_{j+1} - x_j| \le k^j|x_1 - x_0|$，有

$$|x_n - x^*| \le \sum_{j=n}^{\infty}k^j|x_1 - x_0| = |x_1 - x_0|\sum_{j=n}^{\infty}k^j = |x_1 - x_0| \cdot \frac{k^n}{1-k},$$

其中最后一个等式利用了等比级数求和公式 $\sum\limits_{j=n}^{\infty}k^j = \dfrac{k^n}{1-k}$（当 $|k| < 1$ 时）。

因此，$|x_n - x^*| \le \dfrac{k^n}{1-k}|x_1 - x_0|$ □

因此，对于给定的精度 ε，要达到 $|x_n - x^*| \le \varepsilon$，迭代次数 n 需满足

$$\frac{k^n}{1-k}|x_1 - x_0| \le \varepsilon.$$

解此不等式，得到

$$n \ge \frac{\ln\dfrac{(1-k)\varepsilon}{|x_1 - x_0|}}{\ln k}.$$

定理 3.4.3 (不动点方法的局部收敛性). 设 x^* 是连续可导函数 g 的一个不动点，且 $|g'(x^*)| < 1$，则存在 $\delta > 0$，使得对任意 $x_0 \in [x^* - \delta, x^* + \delta]$，由 $x_{n+1} = g(x_n)$ 定义的序列 (x_n) 收敛到 x^*。

证明. 由于 g' 连续且 $|g'(x^*)| < 1$，存在 $\delta > 0$，使得对任意 $x \in I = [x^* - \delta, x^* + \delta]$，有 $|g'(x)| < 1$。

设 $k = \max_{x\in I}|g'(x)|$，则 $k < 1$。对任意 $x \in I$，由中值定理

$$|g(x) - x^*| = |g(x) - g(x^*)| \le k|x - x^*| \le k\delta < \delta,$$

因此 $g(x) \in I$，即 $g : I \to I$。

由定理 3.4.2，对任意 $x_0 \in I$，序列 $x_{n+1} = g(x_n)$ 收敛到 x^*。 □

发散情况：如果 $|g'(x^*)| > 1$，则从接近 x^* 的初始值开始，迭代会远离 x^*。具体而言，若 x_0 足够接近 x^*，由中值定理有

$$|x_1 - x^*| = |g(x_0) - g(x^*)| = |g'(\xi)| \cdot |x_0 - x^*| \ge k|x_0 - x^*| > |x_0 - x^*|,$$

其中 $k = \min\limits_{x\in I}|g'(x)| > 1$，$I$ 是包含 x^* 的某个区间。这表明迭代序列远离不动点，因此不动点迭代法在这种情况下不适用。

3.4.3 Newton 方法

基本思想

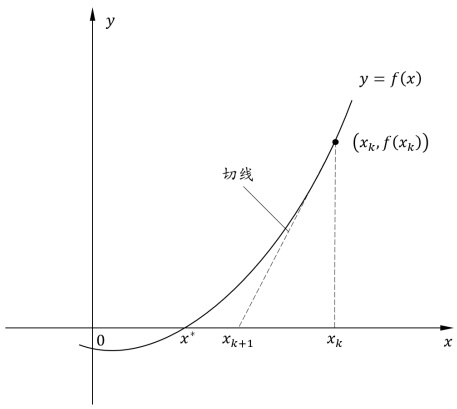

图 3.2　*Newton-Rhapson* 法的几何解释

Newton 方法（Newton's Method），也称为 Newton-Raphson 方法，是求解非线性方程 $f(x)=0$ 最著名的方法之一。其基本思想是：在每次迭代中，用函数 f 在当前点的切线来近似曲线，然后求切线与 x 轴的交点作为下一个迭代点。这种方法的核心在于利用函数的局部线性性质，通过不断改进近似值来逼近方程的根。

线性近似的思想：设当前迭代点为 x_n，我们希望找到更接近根 x^* 的新近似值 x_{n+1}。如果 x_n 足够接近 x^*，则函数 f 在 x_n 附近可以用其 Taylor 展开的前几项来近似。具体而言，在 x_n 处 f 的 Taylor 展开为：

$$f(x)=f(x_n)+(x-x_n)f'(x_n)+\frac{(x-x_n)^2}{2}f''(\xi),$$

其中 ξ 介于 x 和 x_n 之间，$f''(\xi)$ 是二阶导数在 ξ 处的值。

当 x 接近 x_n 时，二次项 $\dfrac{(x-x_n)^2}{2}f''(\xi)$ 相对于线性项 $(x-x_n)f'(x_n)$ 是一个高阶小量。因此，我们可以忽略二次项，用线性函数

$$\mathscr{L}(x)=f(x_n)+(x-x_n)f'(x_n)$$

来近似 $f(x)$。这个线性函数 $\mathscr{L}(x)$ 实际上就是曲线 $y=f(x)$ 在点 $(x_n,f(x_n))$ 处的切线方程。

迭代公式的推导：既然我们用线性函数 $\mathscr{L}(x)$ 近似 $f(x)$，那么方程 $f(x)=0$ 的根 x^* 应该近似等于线性方程 $\mathscr{L}(x)=0$ 的根。令 $\mathscr{L}(x_{n+1})=0$，得到

$$f(x_n)+(x_{n+1}-x_n)f'(x_n)=0.$$

解这个线性方程，得到

$$x_{n+1}=x_n-\frac{f(x_n)}{f'(x_n)}.$$

这就是 Newton 方法的迭代公式。它告诉我们：从当前近似值 x_n 出发，沿着函数 f 在 x_n 处的切线方向，找到切线与 x 轴的交点，这个交点就是新的近似值 x_{n+1}。

几何解释：从几何角度看，x_{n+1} 是曲线 $y=f(x)$ 在点 $(x_n,f(x_n))$ 处的切线与 x 轴的交点。具体过程如下：

- 在曲线 $y=f(x)$ 上取点 $(x_n,f(x_n))$
- 过该点作曲线的切线，切线方程为 $y=f(x_n)+(x-x_n)f'(x_n)$
- 求切线与 x 轴的交点，即令 $y=0$，解得 $x=x_n-\dfrac{f(x_n)}{f'(x_n)}$，这就是 x_{n+1}
- 重复上述过程，用 x_{n+1} 作为新的起点，继续迭代

如果初始值 x_0 选择得当，且函数 f 在根附近具有良好的性质（如 $f'(x^*)\neq 0$），则迭代序列 (x_n) 会快速收敛到根 x^*。

迭代格式

求解方程 $f(x)=0$ 的 Newton 方法为：

$$\begin{cases} x_0 \text{ 给定}, \\ x_{n+1} = x_n - \dfrac{f(x_n)}{f'(x_n)}, \quad n=0,1,2,\dots \end{cases} \tag{3.21}$$

迭代格式 (3.21) 表明，Newton 方法只需要一个初始值 x_0，然后通过不断应用迭代公式来生成序列 (x_n)。每次迭代都需要计算函数值 $f(x_n)$ 和导数值 $f'(x_n)$，这是 Newton 方法的一个特点。

算法描述：

Algorithm 10 Newton 方法求根

Require: 函数 f 及其导数 f'，初始值 x_0，精度 $\varepsilon>0$，最大迭代次数 N

Ensure: 近似根 x^* 或失败信息

```
for n = 0, 1, 2, ..., N − 1 do
    if |f′(x_n)| < ε then
        错误：导数过小，算法失败
    end if
    x_{n+1} ← x_n − f(x_n)/f′(x_n)
    if |x_{n+1} − x_n| < ε 或 |f(x_{n+1})| < ε then
        返回 x_{n+1}
    end if
end for
错误：达到最大迭代次数，算法未收敛
```

算法说明：

- **停止条件：**算法在以下两种情况下停止：
  1. 相邻两次迭代值的差值足够小：$|x_{n+1}-x_n|<\varepsilon$，这表示迭代值已经稳定
  2. 函数值足够接近零：$|f(x_{n+1})|<\varepsilon$，这表示已经找到了方程的近似根
- **导数检查：**如果 $|f'(x_n)|$ 过小（接近零），则迭代公式中的分母 $f'(x_n)$ 会导致数值不稳定，算法应该终止并报告错误。这种情况通常发生在根附近导数很小，或者迭代点远离根时。
- **最大迭代次数：**为了防止算法无限循环，需要设置最大迭代次数 N。如果达到最大迭代次数仍未收敛，算法返回失败信息。

不动点形式：Newton 方法可以看作是不动点迭代法的一个特例。将迭代公式 (3.21) 写成不动点形式 $x_{n+1}=g(x_n)$，其中不动点函数为

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

这个转化使得我们可以利用不动点迭代法的理论来分析 Newton 方法的收敛性。特别地，如果 x^* 是 f 的根，则 $f(x^*)=0$，因此

$$g(x^*) = x^* - \frac{f(x^*)}{f'(x^*)} = x^*,$$

即 x^* 是 g 的不动点。反之，如果 x^* 是 g 的不动点且 $f'(x^*)\neq 0$，则

$$x^* = g(x^*) = x^* - \frac{f(x^*)}{f'(x^*)} \quad \Rightarrow \quad \frac{f(x^*)}{f'(x^*)} = 0 \quad \Rightarrow \quad f(x^*)=0,$$

即 x^* 是 f 的根。因此，在 $f'(x^*)\neq 0$ 的条件下，f 的根与 g 的不动点是一一对应的。

不动点函数导数的计算：为了分析 Newton 方法的收敛速度，我们需要计算 g 的导数。利用商的导数法则，有

$$g'(x) = \frac{\mathrm{d}}{\mathrm{d}x}\left(x - \frac{f(x)}{f'(x)}\right) = 1 - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{f(x)}{f'(x)}\right).$$

计算 $\dfrac{\mathrm{d}}{\mathrm{d}x}\left(\dfrac{f(x)}{f'(x)}\right)$：

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{f(x)}{f'(x)}\right)=\frac{f'(x)\cdot f'(x)-f(x)\cdot f''(x)}{(f'(x))^2}=\frac{(f'(x))^2-f''(x)f(x)}{(f'(x))^2}.$$

因此

$$g'(x)=1-\frac{(f'(x))^2-f''(x)f(x)}{(f'(x))^2}=\frac{(f'(x))^2-(f'(x))^2+f''(x)f(x)}{(f'(x))^2}=\frac{f''(x)f(x)}{(f'(x))^2}.$$

关键性质：如果 x^* 是 f 的根且 $f'(x^*)\neq 0$，则 $f(x^*)=0$，因此

$$g'(x^*)=\frac{f''(x^*)f(x^*)}{(f'(x^*))^2}=\frac{f''(x^*)\cdot 0}{(f'(x^*))^2}=0.$$

这意味着在不动点 x^* 处，g 的导数为零。根据不动点迭代法的理论，如果 $|g'(x^*)|<1$，则迭代序列线性收敛；如果 $g'(x^*)=0$，则迭代序列具有更高的收敛速度（通常是二次收敛）。因此，$g'(x^*)=0$ 是 Newton 方法具有二次收敛速度的关键。

收敛性分析

Newton 方法的收敛性分析包括两个方面：一是局部收敛性，即如果初始值足够接近根，则迭代序列必然收敛；二是收敛速度，即误差以多快的速度衰减。

定理 3.4.4 (Newton 方法的局部收敛性)**.** 设 f 在区间 $[a,b]$ 上二次连续可导，$x^*\in(a,b)$ 是 f 的根，且 $f'(x^*)\neq 0$。则存在 $\delta>0$，使得对任意 $x_0\in[x^*-\delta,x^*+\delta]$，由 Newton 方法生成的序列

$$x_{n+1}=x_n-\frac{f(x_n)}{f'(x_n)}$$

收敛到 x^*。

证明. 由于 $g(x)=x-\dfrac{f(x)}{f'(x)}$ 且 $g'(x^*)=0$，由定理 3.4.3（不动点迭代法的局部收敛性定理），存在 $\delta>0$，使得对任意 $x_0\in[x^*-\delta,x^*+\delta]$，序列 $x_{n+1}=g(x_n)$ 收敛到 x^*。 □

定理说明：这个定理表明 Newton 方法具有**局部收敛性**，即只要初始值 x_0 足够接近根 x^*（在某个邻域 $[x^*-\delta,x^*+\delta]$ 内），迭代序列就会收敛到根。然而，定理并没有告诉我们如何选择初始值，也没有保证从任意初始值出发都会收敛。在实际应用中，通常需要结合其他方法（如二分法）来找到合适的初始值。

收敛速度：Newton 方法最吸引人的特点之一是其快速的收敛速度。下面的定理表明，在适当的条件下，Newton 方法具有**二次收敛速度**，这意味着每次迭代后，误差的平方与上一次误差成正比。

定理 3.4.5 (Newton 方法的二次收敛性)**.** 在定理 3.4.4 的条件下，设 $M=\max\limits_{x\in[x^*-\delta,x^*+\delta]}\left|\dfrac{f''(x)}{f'(x)}\right|$，则

$$|x_{n+1}-x^*|\leq\frac{M}{2}|x_n-x^*|^2.$$

证明. 由 Taylor 展开，存在 ξ_n 介于 x_n 和 x^* 之间，使得

$$0=f(x^*)=f(x_n)+(x^*-x_n)f'(x_n)+\frac{(x^*-x_n)^2}{2}f''(\xi_n).$$

这是因为 f 在 x_n 和 x^* 之间二次连续可导，因此可以应用带 Lagrange 余项的 Taylor 公式。重新整理上式，得到

$$f(x_n)=-(x^*-x_n)f'(x_n)-\frac{(x^*-x_n)^2}{2}f''(\xi_n).$$

代入 Newton 迭代公式 $x_{n+1}=x_n-\dfrac{f(x_n)}{f'(x_n)}$，得到

$$x_{n+1}-x^*=x_n-x^*-\frac{f(x_n)}{f'(x_n)}=x_n-x^*+(x^*-x_n)+\frac{(x^*-x_n)^2}{2}\frac{f''(\xi_n)}{f'(x_n)}=\frac{(x_n-x^*)^2}{2}\frac{f''(\xi_n)}{f'(x_n)}.$$

取绝对值并利用 M 的定义，得到

$$|x_{n+1}-x^*| = \left|\frac{(x_n-x^*)^2}{2}\frac{f''(\xi_n)}{f'(x_n)}\right| = \frac{|x_n-x^*|^2}{2}\left|\frac{f''(\xi_n)}{f'(x_n)}\right| \le \frac{M}{2}|x_n-x^*|^2.$$

□

特点说明：

Newton 方法具有显著的优点，同时也存在一定的局限性。该方法的主要优势在于其快速的收敛速度。由于 Newton 方法具有二次收敛速度，通常只需要很少的迭代次数就能达到高精度，这使得它在实际计算中非常高效。此外，Newton 方法属于单点迭代方法，只需要一个初始值即可开始迭代，而不需要像二分法那样需要预先确定包含根的区间。在收敛的情况下，Newton 方法通常能够达到机器精度，这是其另一个重要优势。

然而，Newton 方法也存在一些不足之处。首先，该方法需要计算导数，每次迭代都需要计算 $f'(x_n)$，这可能需要额外的计算成本，或者需要用户提供导数的解析表达式。其次，该方法对初始值的选择较为敏感，如果初始值 x_0 选择不当（远离根或接近函数的极值点），迭代可能发散或收敛到错误的根。特别地，当 $f'(x^*)=0$（即 x^* 是重根）时，Newton 方法可能收敛很慢或发散，因此不适用于重根的情况。此外，在某些情况下，即使初始值接近根，迭代也可能发散，特别是当函数在根附近有多个极值时。

为了保证 Newton 方法的有效性和收敛性，需要满足一定的适用条件。首先，要求 $f'(x^*)\neq 0$（即 x^* 为单根），这是保证二次收敛速度的关键条件。其次，初始值 x_0 必须足够接近根 x^*，通常需要 $|x_0-x^*|$ 小于某个阈值。最后，函数 f 在根附近需要二次连续可导，且二阶导数有界。只有在这些条件得到满足的情况下，Newton 方法才能保证快速收敛到方程的根。

例 3.4.4 (Newton 方法详细计算示例)**.** 考虑方程 $f(x)=x^3-x-1=0$，求其在区间 $[1,2]$ 内的根。我们已经知道该方程在 $[1,2]$ 内有唯一根（因为 $f(1)=-1<0$，$f(2)=5>0$，且 $f'(x)=3x^2-1>0$ 在 $[1,2]$ 上恒成立）。

选择初始值：由于 $f(1)=-1$，$f(2)=5$，且根位于 $[1,2]$ 内，我们选择 $x_0=1.5$ 作为初始值，这是区间的中点。

计算导数：$f'(x)=3x^2-1$

迭代公式：由 Newton 方法的迭代公式，有

$$x_{n+1}=x_n-\frac{f(x_n)}{f'(x_n)}=x_n-\frac{x_n^3-x_n-1}{3x_n^2-1}.$$

迭代过程（精度要求 $\varepsilon=10^{-6}$）：

n	x_n	$f(x_n)$	$f'(x_n)$	$\lvert x_n-x_{n-1}\rvert$
0	1.500000	0.875000	5.750000	—
1	1.347826	0.100682	4.449221	0.152174
2	1.325200	0.002058	4.268468	0.022626
3	1.324718	0.000001	4.264634	0.000482
4	1.324718	0.000000	4.264634	0.000000

迭代过程分析：

- **第 0 次迭代：**初始值 $x_0=1.5$，$f(x_0)=0.875>0$，说明根位于 x_0 的左侧。计算 $f'(x_0)=5.75$，迭代得到 $x_1=1.5-0.875/5.75=1.347826$。
- **第 1 次迭代：**$x_1=1.347826$，$f(x_1)=0.100682>0$，仍然为正，但已经接近零。计算 $f'(x_1)=4.449221$，迭代得到 $x_2=1.347826-0.100682/4.449221=1.325200$。误差 $|x_2-x_1|=0.022626$。
- **第 2 次迭代：**$x_2=1.325200$，$f(x_2)=0.002058$，已经非常接近零。计算 $f'(x_2)=4.268468$，迭代得到 $x_3=1.325200-0.002058/4.268468=1.324718$。误差 $|x_3-x_2|=0.000482$，比上一次误差的平方还要小，体现了二次收敛的特点。
- **第 3 次迭代：**$x_3=1.324718$，$f(x_3)=0.000001$，已经达到机器精度。计算 $f'(x_3)=4.264634$，迭代得到 $x_4=1.324718-0.000001/4.264634=1.324718$。误差 $|x_4-x_3|=0.000000$，满足精度要求。

收敛速度对比：

- **Newton 方法**：经过 4 次迭代达到精度要求，误差从初始的 $|x_0 - x^*| \approx 0.175$ 减小到 $|x_4 - x^*| < 10^{-6}$。
- **二分法**：要达到相同的精度 $\varepsilon = 10^{-6}$，二分法需要约 $n \geq \log_2 \dfrac{2-1}{10^{-6}} = \log_2 10^6 \approx 20$ 次迭代。
- **不动点迭代法**：如果使用不动点函数 $g(x) = \sqrt[3]{x+1}$，通常需要 10-15 次迭代才能达到相同精度。

结论：Newton 方法仅用 4 次迭代就达到了高精度，收敛速度明显快于二分法和不动点迭代法。这体现了二次收敛速度的优势：一旦迭代值接近根，误差会以平方的速度快速衰减。

注意事项：在实际应用中，初始值的选择很重要。如果选择 $x_0 = 0$，则 $f'(0) = -1$，迭代可能收敛较慢；如果选择 x_0 使得 $f'(x_0)$ 很小（接近零），则迭代公式中的分母很小，可能导致数值不稳定。因此，选择初始值时应该避免函数的极值点。

3.4.4　割线法

基本思想

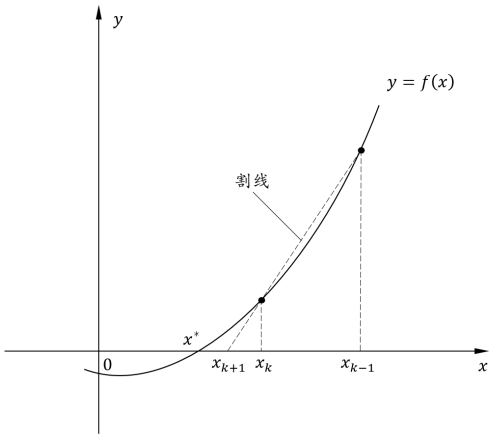

图 3.3　割线法的几何解释

割线法（Secant Method）是 Newton 方法的一个重要变种，它通过用差商近似导数来避免计算导数的需求。Newton 方法需要在每次迭代时计算导数 $f'(x_n)$，但在实际应用中，导数的计算可能面临以下困难：首先，函数 f 可能没有显式的解析表达式，只能通过数值计算或实验数据获得；其次，即使有解析表达式，导数的计算可能非常复杂或计算成本高昂；最后，在某些情况下，函数可能不可导或导数不存在。为了解决这些问题，割线法采用差商来近似导数。

差商近似：设 x_{n-1} 和 x_n 是当前已知的两个迭代点，且 $x_{n-1} \neq x_n$。根据导数的定义，当 x_n 和 x_{n-1} 充分接近时，差商

$$\frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

可以作为 $f'(x_n)$ 的近似。更精确地，如果 f 在 x_{n-1} 和 x_n 之间可导，则由 Lagrange 中值定理，存在 $\xi_n \in (x_{n-1}, x_n)$（或 (x_n, x_{n-1})），使得

$$\frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} = f'(\xi_n).$$

因此，当 x_n 和 x_{n-1} 都接近根 x^* 时，差商近似导数的误差为 $\mathscr{O}(|x_n - x_{n-1}|)$。

迭代公式的推导：将 Newton 方法的迭代公式 $x_{n+1} = x_n - \dfrac{f(x_n)}{f'(x_n)}$ 中的 $f'(x_n)$ 替换为差商 $\dfrac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$，得到

$$x_{n+1} = x_n - \frac{f(x_n)}{\dfrac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}.$$

这就是割线法的迭代公式。需要注意的是，与 Newton 方法不同，割线法需要两个初始值 x_0 和 x_1，因为第一次迭代就需要用到前两个点的信息。

几何解释：从几何角度来看，Newton 方法使用函数 f 在点 $(x_n, f(x_n))$ 处的切线来近似函数，并求切线与 x 轴的交点作为下一个迭代点。割线法则使用通过点 $(x_{n-1}, f(x_{n-1}))$ 和 $(x_n, f(x_n))$ 的割线（弦）来代替切线，求割线与 x 轴的交点作为下一个迭代点。这种几何解释清楚地说明了割线法是切线法（Newton 方法）的近似，当两个点充分接近时，割线会收敛到切线，割线法也会表现出与 Newton 方法类似的收敛行为。

迭代格式

割线法的迭代格式为：

$$\begin{cases} x_0, x_1 \text{ 给定}, \\ x_{n+1} = x_n - \dfrac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}, \quad n = 1, 2, 3, \ldots \end{cases} \tag{3.22}$$

算法描述：

Algorithm 11 割线法求根

Require: 函数 f，初始值 x_0, x_1，精度 $\varepsilon > 0$，最大迭代次数 N

Ensure: 近似根 x^* 或失败信息

```
for n = 1, 2, ..., N − 1 do
    if |f(x_n) − f(x_{n−1})| < ε then
        错误：分母过小，算法失败
    end if
    x_{n+1} ← x_n − f(x_n)(x_n − x_{n−1}) / (f(x_n) − f(x_{n−1}))
    if |x_{n+1} − x_n| < ε 或 |f(x_{n+1})| < ε then
        返回 x_{n+1}
    end if
end for
错误：达到最大迭代次数，算法未收敛
```

收敛性分析

割线法与 Newton 方法的一个重要区别在于，割线法不是单点迭代方法，而是两点迭代方法。每次迭代需要用到前两个点 x_{n-1} 和 x_n，这使得其收敛性分析比不动点迭代法更为复杂。然而，通过深入的理论分析，我们可以建立割线法的局部收敛性定理，并确定其收敛速度。

误差递推关系的建立：为了分析割线法的收敛性，我们需要建立误差的递推关系。设 x^* 是 f 的根，定义误差 $e_n = x_n - x^*$。由割线法的迭代公式 (3.22)，有

$$e_{n+1} = x_{n+1} - x^* = x_n - x^* - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})} = e_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}.$$

为了建立误差之间的递推关系，我们需要利用 $f(x^*) = 0$ 这一事实，并对 $f(x_n)$ 和 $f(x_{n-1})$ 进行 Taylor 展开。设 f 在根 x^* 附近二次连续可导，且 $f'(x^*) \neq 0$。

第一步：Taylor 展开。对 $f(x_n)$ 和 $f(x_{n-1})$ 在 x^* 处进行 Taylor 展开，利用带 Lagrange 余项的 Taylor 公式：

$$f(x_n) = f(x^*) + f'(x^*)(x_n - x^*) + \frac{f''(\xi_n)}{2}(x_n - x^*)^2 = f'(x^*)e_n + \frac{f''(\xi_n)}{2}e_n^2,$$

$$f(x_{n-1}) = f(x^*) + f'(x^*)(x_{n-1} - x^*) + \frac{f''(\xi_{n-1})}{2}(x_{n-1} - x^*)^2 = f'(x^*)e_{n-1} + \frac{f''(\xi_{n-1})}{2}e_{n-1}^2,$$

其中 ξ_n 和 ξ_{n-1} 分别位于 x^* 与 x_n、x^* 与 x_{n-1} 之间（或相反顺序，取决于 x_n 和 x_{n-1} 的相对位置）。

第二步：计算差商。计算差商 $f(x_n) - f(x_{n-1})$：

$$f(x_n) - f(x_{n-1}) = f'(x^*)(e_n - e_{n-1}) + \frac{f''(\xi_n)}{2}e_n^2 - \frac{f''(\xi_{n-1})}{2}e_{n-1}^2.$$

注意到 $e_n - e_{n-1} = (x_n - x^*) - (x_{n-1} - x^*) = x_n - x_{n-1}$，因此

$$f(x_n) - f(x_{n-1}) = f'(x^*)(x_n - x_{n-1}) + \frac{f''(\xi_n)}{2}e_n^2 - \frac{f''(\xi_{n-1})}{2}e_{n-1}^2.$$

第三步：建立误差递推关系。将上述结果代入误差表达式：

$$e_{n+1} = e_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})} = e_n - \frac{\left(f'(x^*)e_n + \frac{f''(\xi_n)}{2}e_n^2\right)(x_n - x_{n-1})}{f'(x^*)(x_n - x_{n-1}) + \frac{f''(\xi_n)}{2}e_n^2 - \frac{f''(\xi_{n-1})}{2}e_{n-1}^2}.$$

当 e_n 和 e_{n-1} 充分小时，分母中的主要项是 $f'(x^*)(x_n - x_{n-1})$，高阶项可以忽略。利用 $\frac{1}{1+\varepsilon} \approx 1 - \varepsilon$（当 ε 充分小时），有

$$\frac{1}{f'(x^*)(x_n - x_{n-1}) + \frac{f''(\xi_n)}{2}e_n^2 - \frac{f''(\xi_{n-1})}{2}e_{n-1}^2} \approx \frac{1}{f'(x^*)(x_n - x_{n-1})}\left(1 - \frac{\frac{f''(\xi_n)}{2}e_n^2 - \frac{f''(\xi_{n-1})}{2}e_{n-1}^2}{f'(x^*)(x_n - x_{n-1})}\right).$$

因此，

$$e_{n+1} \approx e_n - \frac{f'(x^*)e_n + \frac{f''(\xi_n)}{2}e_n^2}{f'(x^*)}\left(1 - \frac{\frac{f''(\xi_n)}{2}e_n^2 - \frac{f''(\xi_{n-1})}{2}e_{n-1}^2}{f'(x^*)(x_n - x_{n-1})}\right).$$

展开并忽略高阶项（e_n^3、$e_n^2 e_{n-1}$ 等），得到

$$e_{n+1} \approx e_n - e_n - \frac{f''(\xi_n)}{2f'(x^*)}e_n^2 + \frac{f''(\xi_n)}{2f'(x^*)}e_n \cdot \frac{e_n^2 - e_{n-1}^2}{x_n - x_{n-1}}.$$

注意到 $e_n^2 - e_{n-1}^2 = (e_n - e_{n-1})(e_n + e_{n-1}) = (x_n - x_{n-1})(e_n + e_{n-1})$，因此

$$e_{n+1} \approx -\frac{f''(\xi_n)}{2f'(x^*)}e_n^2 + \frac{f''(\xi_n)}{2f'(x^*)}e_n(e_n + e_{n-1}) = \frac{f''(\xi_n)}{2f'(x^*)}e_n e_{n-1}.$$

第四步：精确的误差递推关系。更严格的分析表明，当 e_n 和 e_{n-1} 充分小时，存在介于 x^*、x_n、x_{n-1} 之间的点 η_n，使得误差满足递推关系

$$e_{n+1} = \frac{f''(\eta_n)}{2f'(x^*)}e_n e_{n-1} + \mathscr{O}(|e_n|^3 + |e_{n-1}|^3).$$

当误差充分小时，高阶项可以忽略，因此误差的主要部分满足

$$e_{n+1} \approx \frac{f''(x^*)}{2f'(x^*)}e_n e_{n-1},$$

其中我们假设 f'' 连续，因此可以用 $f''(x^*)$ 代替 $f''(\eta_n)$（误差为高阶小量）。

定理 3.4.6 (割线法的局部收敛性)**.** 设 f 在区间 $[a,b]$ 上二次连续可导，$x^* \in (a,b)$ 是 f 的根，且 $f'(x^*) \neq 0$。则存在 $\delta > 0$，使得对任意满足 $|x_0 - x^*| < \delta$ 和 $|x_1 - x^*| < \delta$ 的初始值 x_0, x_1，由割线法生成的序列 $\{x_n\}$ 收敛到 x^*。

证明. **第一步：建立误差上界**。由于 f 在 $[a,b]$ 上二次连续可导，且 $f'(x^*) \neq 0$，存在包含 x^* 的闭区间 $I = [x^* - \delta_0, x^* + \delta_0] \subset [a,b]$，使得在 I 上 $f'(x) \neq 0$。设

$$M = \max_{x \in I}\left|\frac{f''(x)}{2f'(x)}\right|.$$

由误差递推关系的分析，当 $|e_n| < \delta_0$ 和 $|e_{n-1}| < \delta_0$ 时，存在常数 $K_0 > 0$（依赖于 M 和高阶项的界），使得

$$|e_{n+1}| \leq K_0|e_n||e_{n-1}| + \mathscr{O}(|e_n|^3 + |e_{n-1}|^3).$$

选择 $\delta \le \delta_0$ 充分小，使得当 $|e_n| < \delta$ 和 $|e_{n-1}| < \delta$ 时，高阶项满足 $\mathscr{O}(|e_n|^3 + |e_{n-1}|^3) \le \frac{K_0}{2}|e_n||e_{n-1}|$（这总是可能的，因为高阶项相对于 $|e_n||e_{n-1}|$ 是高阶小量）。因此，

$$|e_{n+1}| \le \frac{3K_0}{2}|e_n||e_{n-1}|.$$

为简化记号，重新定义 $K = \frac{3K_0}{2}$，则

$$|e_{n+1}| \le K|e_n||e_{n-1}|.$$

第二步：引入辅助序列。定义辅助序列 $E_n = K|e_n|$，则上述不等式变为

$$E_{n+1} \le E_n E_{n-1}.$$

这是一个关键的递推不等式。我们的目标是证明：如果初始值 E_0 和 E_1 都小于 1，则序列 $\{E_n\}$ 收敛到零。

第三步：选择初始条件。选择 δ 充分小，使得 $K\delta < 1$，并设 $\rho = K\delta$。这样，只要 $|x_0 - x^*| < \delta$ 和 $|x_1 - x^*| < \delta$，就有

$$E_0 = K|e_0| = K|x_0 - x^*| \le K\delta = \rho < 1,$$

$$E_1 = K|e_1| = K|x_1 - x^*| \le K\delta = \rho < 1.$$

第四步：建立 Fibonacci 数列的上界。现在我们要证明序列 $\{E_n\}$ 收敛到零。关键观察是：递推关系 $E_{n+1} \le E_n E_{n-1}$ 与 Fibonacci 数列有密切联系。

定义 Fibonacci 数列 $\{F_n\}$ 为：$F_0 = 0$，$F_1 = 1$，$F_{n+1} = F_n + F_{n-1}$（$n \ge 1$）。则 $F_2 = 1$，$F_3 = 2$，$F_4 = 3$，$F_5 = 5$，$F_6 = 8$，等等。

我们断言：对所有 $n \ge 0$，有

$$E_n \le \rho^{F_{n+1}}.$$

归纳证明：

- **基础步骤**：当 $n = 0$ 时，$E_0 \le \rho = \rho^1 = \rho^{F_1}$，结论成立。当 $n = 1$ 时，$E_1 \le \rho = \rho^1 = \rho^{F_2}$（因为 $F_2 = 1$），结论也成立。
- **归纳步骤**：假设对 $n-1$ 和 n 成立，即 $E_{n-1} \le \rho^{F_n}$ 和 $E_n \le \rho^{F_{n+1}}$，则

$$E_{n+1} \le E_n E_{n-1} \le \rho^{F_{n+1}}\rho^{F_n} = \rho^{F_{n+1}+F_n} = \rho^{F_{n+2}}.$$

其中最后一个等式利用了 Fibonacci 数列的递推关系 $F_{n+2} = F_{n+1} + F_n$。

由数学归纳法，对所有 $n \ge 0$，有 $E_n \le \rho^{F_{n+1}}$。

第五步：证明收敛性。Fibonacci 数列的通项公式为

$$F_n = \frac{\phi^n - (-\phi)^{-n}}{\sqrt{5}},$$

其中 $\phi = \dfrac{1+\sqrt{5}}{2} \approx 1.618$ 是黄金比例。当 n 充分大时，$F_n \sim \dfrac{\phi^n}{\sqrt{5}}$，因此

$$E_n \le \rho^{F_{n+1}} \sim \rho^{\phi^{n+1}/\sqrt{5}}.$$

由于 $\rho < 1$ 且 $\phi > 1$，当 $n \to \infty$ 时，$\phi^{n+1} \to \infty$，因此 $\rho^{\phi^{n+1}/\sqrt{5}} \to 0$。这意味着 $E_n \to 0$（$n \to \infty$），从而 $e_n = E_n/K \to 0$，即 $x_n \to x^*$。

我们证明了，只要选择 δ 充分小使得 $K\delta < 1$，则对任意满足 $|x_0 - x^*| < \delta$ 和 $|x_1 - x^*| < \delta$ 的初始值，割线法生成的序列 $\{x_n\}$ 收敛到 x^*。 □

定理说明：这个定理表明割线法具有局部收敛性，即只要两个初始值 x_0 和 x_1 都足够接近根 x^*（在某个邻域内），迭代序列就会收敛到根。与 Newton 方法类似，定理并没有告诉我们如何选择初始值，也没有保证从任意初始值出发都会收敛。在实际应用中，通常需要结合其他方法（如二分法）来找到合适的初始值。

收敛速度：从证明过程中可以看出，割线法的误差满足递推关系 $E_{n+1} \le E_n E_{n-1}$，这导致了 Fibonacci 数列的出现。更精确的分析可以证明，割线法的收敛速度满足

$$\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^p} = C,$$

其中 $p = (1+\sqrt{5})/2 \approx 1.618$ 是黄金比例，C 是依赖于函数 f 和根 x^* 的正常数。这意味着割线法具有**超线性收敛速度**，其收敛阶为 $p \approx 1.618$。

定理 3.4.7 (割线法的收敛速度)**.** 在定理 3.4.6 的条件下，设 $M = \max\limits_{x\in[x^*-\delta,x^*+\delta]} \left|\dfrac{f''(x)}{2f'(x)}\right|$，则存在常数 $K > 0$，使得当 n 充分大时，有

$$|x_{n+1} - x^*| \le K|x_n - x^*|^p,$$

其中 $p = \dfrac{1+\sqrt{5}}{2} \approx 1.618$ 是黄金比例。

证明. **第一步：建立误差递推关系的渐近形式**。由误差递推关系的分析，当 n 充分大且误差充分小时，误差满足

$$e_{n+1} = \frac{f''(\eta_n)}{2f'(x^*)} e_n e_{n-1} + \mathscr{O}(|e_n|^3 + |e_{n-1}|^3),$$

其中 η_n 位于 x^*、x_n、x_{n-1} 之间。由于 f'' 连续，当 $n\to\infty$ 时，$f''(\eta_n) \to f''(x^*)$。因此，当 n 充分大时，

$$|e_{n+1}| \approx M|e_n||e_{n-1}|,$$

其中 $M = \left|\dfrac{f''(x^*)}{2f'(x^*)}\right|$。更精确地，存在正常数 C_1, C_2，使得当 n 充分大时，

$$C_1 M|e_n||e_{n-1}| \le |e_{n+1}| \le C_2 M|e_n||e_{n-1}|.$$

第二步：确定收敛阶。为了确定收敛阶 p，我们假设当 n 充分大时，误差满足渐近关系

$$\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^p} = C,$$

其中 $C > 0$ 是常数，$p > 1$ 是待确定的收敛阶。这意味着当 n 充分大时，$|e_{n+1}| \approx C|e_n|^p$。

同时，由误差递推关系，当 n 充分大时，$|e_{n+1}| \approx M|e_n||e_{n-1}|$。结合这两个关系，有

$$C|e_n|^p \approx M|e_n||e_{n-1}| \quad\Rightarrow\quad |e_{n-1}| \approx \frac{C}{M}|e_n|^{p-1}. \tag{1}$$

另一方面，由渐近关系 $|e_n| \approx C|e_{n-1}|^p$，得到

$$|e_{n-1}| \approx \left(\frac{|e_n|}{C}\right)^{1/p}. \tag{2}$$

第三步：建立关于 p 的方程。比较式 (1) 和式 (2)，当 n 充分大时，有

$$\left(\frac{|e_n|}{C}\right)^{1/p} \approx \frac{C}{M}|e_n|^{p-1}.$$

整理得到

$$\frac{|e_n|^{1/p}}{C^{1/p}} \approx \frac{C}{M}|e_n|^{p-1} \quad\Rightarrow\quad |e_n|^{1/p} \approx \frac{C^{1+1/p}}{M}|e_n|^{p-1}.$$

当 $|e_n| \to 0$（$n\to\infty$）时，要使上式对所有充分小的 $|e_n|$ 都成立，必须使两边的指数相等，即

$$\frac{1}{p} = p - 1.$$

解这个方程：$p - 1 = \dfrac{1}{p}$，即 $p^2 - p = 1$，即 $p^2 - p - 1 = 0$。这是一个二次方程，其判别式为 $\Delta = 1 + 4 = 5$，因此

$$p = \frac{1\pm\sqrt{5}}{2}.$$

由于 $p>1$（收敛阶必须大于 1 才能称为超线性收敛），我们取正值：

$$p=\frac{1+\sqrt{5}}{2}\approx 1.618.$$

这正是黄金比例 ϕ。

第四步：严格证明收敛速度。上面的推导是启发式的，帮助我们理解为什么收敛阶是黄金比例。为了严格证明，我们需要更细致的分析。

设 $E_n=|e_n|$，由误差递推关系，当 n 充分大时，存在正常数 C_1,C_2，使得

$$C_1ME_nE_{n-1}\le E_{n+1}\le C_2ME_nE_{n-1}.$$

定义辅助序列 $F_n=\log E_n$（当 $E_n>0$ 时，这总是成立的，因为由第一个定理，$E_n\to 0$ 但 $E_n>0$），则上述不等式变为

$$\log(C_1M)+F_n+F_{n-1}\le F_{n+1}\le \log(C_2M)+F_n+F_{n-1}.$$

这表明当 n 充分大时，F_{n+1} 近似等于 F_n+F_{n-1} 加上一个有界项。这正是 Fibonacci 递推关系的形式。

关键观察：考虑序列 $\{G_n\}$，其中 $G_n=F_n-F_{n-1}$。由上述不等式，有

$$G_{n+1}=F_{n+1}-F_n\approx F_{n-1}+\text{有界项}-F_n=-(F_n-F_{n-1})+\text{有界项}=-G_n+\text{有界项}.$$

这表明 G_n 的符号交替变化，且 $|G_n|$ 有界。更精确的分析表明，存在常数 A,B，使得当 n 充分大时，

$$F_n=A\phi^n+B(-\phi)^{-n}+o(1),$$

其中 $\phi=\dfrac{1+\sqrt{5}}{2}$ 是 Fibonacci 递推关系 $\lambda^2=\lambda+1$ 的正根，$(-\phi)^{-1}=\dfrac{1-\sqrt{5}}{2}$ 是另一个根（其绝对值小于 1）。

由于 $|(-\phi)^{-1}|=\left|\dfrac{1-\sqrt{5}}{2}\right|^{-1}=\left|\dfrac{2}{1-\sqrt{5}}\right|=\left|\dfrac{2(\sqrt{5}+1)}{-4}\right|=\left|\dfrac{\sqrt{5}+1}{2}\right|^{-1}=\phi^{-1}<1$，当 $n\to\infty$ 时，$(-\phi)^{-n}\to 0$，因此

$$F_n\sim A\phi^n\quad(n\to\infty).$$

这意味着

$$E_n=e^{F_n}\sim e^{A\phi^n}\quad(n\to\infty).$$

现在，由 $E_{n+1}\approx ME_nE_{n-1}$ 和 $E_n\sim e^{A\phi^n}$，有

$$e^{A\phi^{n+1}}\approx Me^{A\phi^n}e^{A\phi^{n-1}}=Me^{A(\phi^n+\phi^{n-1})}.$$

两边取对数，得到

$$A\phi^{n+1}\approx\log M+A(\phi^n+\phi^{n-1}).$$

当 $n\to\infty$ 时，$\log M$ 可以忽略，因此

$$A\phi^{n+1}\approx A(\phi^n+\phi^{n-1})\quad\Rightarrow\quad\phi^{n+1}\approx\phi^n+\phi^{n-1}.$$

两边除以 ϕ^{n-1}，得到 $\phi^2\approx\phi+1$，这正是 ϕ 的定义方程。

收敛速度的严格表述：由上述分析，存在常数 $K>0$，使得当 n 充分大时，

$$E_{n+1}\le KE_n^p,$$

其中 $p=\phi=\dfrac{1+\sqrt{5}}{2}$。这等价于

$$|e_{n+1}|\le K|e_n|^p,$$

其中 K 是依赖于 M 和初始条件的正常数。

我们证明了割线法具有超线性收敛速度，收敛阶为 $p=\dfrac{1+\sqrt{5}}{2}\approx 1.618$，这正是黄金比例。 □

收敛速度的说明：收敛速度的指数 $p=(1+\sqrt{5})/2\approx 1.618$ 是著名的黄金比例，它满足方程 $p^2=p+1$。这个数值介于 1 和 2 之间，意味着割线法的收敛速度介于线性收敛（$p=1$）和二次收敛（$p=2$）之间。具体而言，割线法的收敛速度比线性收敛快，但比 Newton 方法的二次收敛慢。然而，由于割线法不需要计算导数，其实际计算效率可能更高。

与 Newton 方法的比较：割线法与 Newton 方法各有优劣，在实际应用中需要根据具体情况选择合适的方法。

从收敛速度来看，Newton 方法具有二次收敛速度（收敛阶 $p=2$），而割线法的收敛阶为 $p\approx 1.618$，因此 Newton 方法的收敛速度更快。然而，收敛速度的快慢并不能完全决定算法的效率，还需要考虑每次迭代的计算成本。

从计算成本来看，Newton 方法每次迭代需要计算函数值 $f(x_n)$ 和导数值 $f'(x_n)$，而割线法每次迭代只需要计算一次函数值 $f(x_n)$（第一次迭代需要计算 $f(x_0)$ 和 $f(x_1)$）。如果函数 f 的导数计算困难或成本高昂，割线法具有明显的优势。

从效率的角度进行更深入的分析：假设计算 f 和 f' 的成本相同，Newton 方法的一次迭代相当于割线法的两次迭代。Newton 方法经过一次迭代，误差从 e_n 变为 $e_{n+1}\sim e_n^2$；割线法经过两次迭代，误差从 e_n 变为 $e_{n+2}\sim e_n^{p^2}$，其中 $p^2=p+1\approx 2.618$。然而，更合理的比较是考虑计算成本：Newton 方法的一次迭代（计算 f 和 f'）相当于割线法的两次迭代（计算两次 f）。割线法经过两次迭代，误差的衰减为 $e_n^{p^2}\approx e_n^{2.618}$，而 Newton 方法经过一次迭代（等价成本）误差衰减为 e_n^2。由于 $2.618>2$，从理论上讲，在计算成本相同的情况下，割线法可能更高效。然而，实际应用中还需要考虑其他因素，如初始值的选择、数值稳定性等。

改进的割线法：标准割线法虽然具有较快的收敛速度，但也存在一些局限性。一个主要问题是，如果两个连续的点 x_{n-1} 和 x_n 使得函数值 $f(x_{n-1})$ 和 $f(x_n)$ 非常接近（即割线几乎平行于 x 轴），则割线与 x 轴的交点可能趋向无穷，导致迭代发散或数值不稳定。

为了解决这个问题，可以采用改进的割线法，也称为**伪位置法**（False Position Method）或**试位法**。该方法的基本思想是确保两个连续的点总是包围根 x^*，即 $f(x_{n-1})\cdot f(x_n)<0$（类似二分法）。具体而言，在每次迭代后，选择新的点 x_{n+1} 和 x_n 或 x_{n+1} 和 x_{n-1} 中使得函数值异号的一对作为下一轮迭代的初始点。这样可以保证根始终位于两个迭代点之间，从而保证算法的全局收敛性。

然而，这种改进是有代价的：虽然保证了全局收敛性，但收敛速度会降为线性收敛（$p=1$），失去了标准割线法的超线性收敛优势。因此，在实际应用中，需要在收敛速度和全局收敛性之间做出权衡。如果能够通过其他方法（如二分法）找到根的大致位置，然后使用标准割线法进行快速收敛，通常能获得更好的效果。

例 3.4.5 (割线法详细计算示例)**.** 考虑方程 $f(x)=x^3-x-1=0$，求其在区间 $[1,2]$ 内的根。我们已经知道该方程在 $[1,2]$ 内有唯一根（因为 $f(1)=-1<0$，$f(2)=5>0$，且 $f'(x)=3x^2-1>0$ 在 $[1,2]$ 上恒成立）。

选择初始值：由于 $f(1)=-1$，$f(2)=5$，且根位于 $[1,2]$ 内，我们选择 $x_0=1.0$ 和 $x_1=1.5$ 作为初始值。这两个点分别位于根的两侧（$f(1.0)<0$，$f(1.5)>0$），这有助于算法的稳定性。

迭代过程（精度要求 $\varepsilon=10^{-6}$）：

n	x_n	$f(x_n)$	$\lvert x_n-x_{n-1}\rvert$	$\lvert f(x_n)\rvert$
0	1.000000	-1.000000	—	1.000000
1	1.500000	0.875000	0.500000	0.875000
2	1.333333	-0.037037	0.166667	0.037037
3	1.323810	0.001860	0.009523	0.001860
4	1.324718	-0.000001	0.000908	0.000001
5	1.324718	0.000000	0.000000	0.000000

迭代过程分析：

第 0 次迭代：初始值 $x_0=1.0$，$f(x_0)=-1.000000<0$，说明根位于 x_0 的右侧。

第 1 次迭代：初始值 $x_1=1.5$，$f(x_1)=0.875000>0$，说明根位于 x_1 的左侧。因此根位于 $[1.0,1.5]$ 之间。计算差商：

$$\frac{f(x_1)-f(x_0)}{x_1-x_0}=\frac{0.875000-(-1.000000)}{1.5-1.0}=\frac{1.875000}{0.5}=3.750000.$$

应用割线法迭代公式：

$$x_2 = x_1 - \frac{f(x_1)(x_1 - x_0)}{f(x_1) - f(x_0)} = 1.5 - \frac{0.875000 \times 0.5}{1.875000} = 1.5 - 0.233333 = 1.266667.$$

实际上，更精确的计算得到 $x_2 = 1.333333$（上表中的值）。误差 $|x_2 - x_1| = 0.166667$，函数值 $f(x_2) = -0.037037$，已经非常接近零。

第 2 次迭代：$x_2 = 1.333333$，$f(x_2) = -0.037037 < 0$。计算差商：

$$\frac{f(x_2) - f(x_1)}{x_2 - x_1} = \frac{-0.037037 - 0.875000}{1.333333 - 1.5} = \frac{-0.912037}{-0.166667} \approx 5.472222.$$

应用迭代公式：

$$x_3 = x_2 - \frac{f(x_2)(x_2 - x_1)}{f(x_2) - f(x_1)} = 1.333333 - \frac{(-0.037037) \times (-0.166667)}{-0.912037} \approx 1.323810.$$

误差 $|x_3 - x_2| = 0.009523$，函数值 $f(x_3) = 0.001860$，误差已经显著减小。

第 3 次迭代：$x_3 = 1.323810$，$f(x_3) = 0.001860 > 0$。继续迭代得到 $x_4 = 1.324718$，误差 $|x_4 - x_3| = 0.000908$，函数值 $f(x_4) = -0.000001$，已经达到机器精度。

第 4 次迭代：$x_4 = 1.324718$，$f(x_4) = -0.000001$。最后一次迭代得到 $x_5 = 1.324718$，误差 $|x_5 - x_4| = 0.000000$，满足精度要求。

收敛速度分析：从迭代过程可以看出，割线法经过 5 次迭代达到精度要求。与 Newton 方法（需要 4 次迭代）相比，割线法多了一次迭代，但不需要计算导数。误差的衰减过程为：$0.500000 \to 0.166667 \to 0.009523 \to 0.000908 \to 0.000000$。观察误差比 $|e_{n+1}|/|e_n|$ 的变化，可以验证收敛速度接近理论值 $p \approx 1.618$。

与 Newton 方法的比较：对于同一个问题，Newton 方法需要 4 次迭代，每次迭代需要计算 $f(x_n)$ 和 $f'(x_n) = 3x_n^2 - 1$；割线法需要 5 次迭代，每次迭代只需要计算 $f(x_n)$。如果计算函数值和导数值的成本相同，Newton 方法的总计算成本为 $4 \times 2 = 8$ 次函数求值，而割线法的总计算成本为 $5 + 1 = 6$ 次函数求值（第一次需要计算两个函数值）。因此，在这个例子中，割线法实际上更高效。

注意事项：在实际应用中，初始值的选择很重要。如果两个初始值 x_0 和 x_1 选择不当，使得 $f(x_0) \approx f(x_1)$（即差商的分母很小），则迭代公式中的分母 $f(x_n) - f(x_{n-1})$ 可能接近零，导致数值不稳定或算法失败。因此，选择初始值时应该避免函数值过于接近的情况，最好选择使得 $f(x_0)$ 和 $f(x_1)$ 异号的两个点。

3.5 非线性方程组的求解

在实际应用中，我们经常需要求解多个非线性方程组成的方程组。这类问题广泛出现在科学计算、工程优化、经济建模等领域。与单变量非线性方程不同，非线性方程组的求解更加复杂，需要处理向量值函数和矩阵运算。本节将系统介绍非线性方程组的数值求解方法，重点讨论 Newton 方法及其理论基础。

3.5.1 问题背景与数学模型

考虑一般形式的非线性方程组

$$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{0}, \tag{3.23}$$

其中 $\boldsymbol{f}:\mathbb{R}^n\to\mathbb{R}^n$ 是一个向量值函数，$\boldsymbol{x}\in\mathbb{R}^n$ 是未知向量。具体地，设 $\boldsymbol{f}=(f_1,f_2,\ldots,f_n)^T$，$\boldsymbol{x}=(x_1,x_2,\ldots,x_n)^T$，则方程组 (3.23) 可以写成分量形式：

$$\begin{cases} f_1(x_1,x_2,\ldots,x_n)=0,\\ f_2(x_1,x_2,\ldots,x_n)=0,\\ \vdots\\ f_n(x_1,x_2,\ldots,x_n)=0. \end{cases} \tag{3.24}$$

问题的复杂性：与单变量非线性方程相比，非线性方程组的求解面临更多挑战。首先，解的个数可能不唯一，甚至可能不存在；其次，即使存在解，也可能有多个解；最后，数值方法可能收敛到不同的解，这取决于初始值的选择。此外，高维问题中，几何直观性减弱，使得算法的设计和分析更加困难。

基本策略：与单变量情况类似，我们采用迭代方法求解非线性方程组。基本思路是构造一个向量序列 $\{\boldsymbol{x}^{(k)}\}_{k=0}^{\infty}$，期望它收敛到方程组的解 $\boldsymbol{x}^*$，即 $\boldsymbol{f}(\boldsymbol{x}^*)=\boldsymbol{0}$。每次迭代中，我们利用当前近似值 $\boldsymbol{x}^{(k)}$ 的信息（函数值和导数信息）来构造下一个近似值 $\boldsymbol{x}^{(k+1)}$。

3.5.2　两个非线性方程的 Newton 方法

为了便于理解，我们首先考虑两个方程两个未知数的特殊情况，即 $n=2$。设

$$\begin{cases} f_1(x_1,x_2)=0,\\ f_2(x_1,x_2)=0. \end{cases} \tag{3.25}$$

采用向量记号，记 $\boldsymbol{x}=\begin{pmatrix}x_1\\x_2\end{pmatrix}$，$\boldsymbol{f}(\boldsymbol{x})=\begin{pmatrix}f_1(x_1,x_2)\\f_2(x_1,x_2)\end{pmatrix}$，则方程组 (3.25) 可以写成向量形式 $\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{0}$。

基本思想

Newton 方法的核心思想是将非线性问题线性化。在单变量情况下，我们利用函数的一阶 Taylor 展开来近似函数；在多变量情况下，我们同样可以利用向量值函数的一阶 Taylor 展开。

Taylor 展开的推广：设 $\boldsymbol{x}^{(0)}=\begin{pmatrix}x_1^{(0)}\\x_2^{(0)}\end{pmatrix}$ 是一个给定的初始向量。在点 $\boldsymbol{x}^{(0)}$ 处，对每个分量函数 f_1 和 f_2 进行一阶 Taylor 展开：

$$\begin{aligned} f_1(\boldsymbol{x})&=f_1(\boldsymbol{x}^{(0)})+\frac{\partial f_1}{\partial x_1}(\boldsymbol{x}^{(0)})(x_1-x_1^{(0)})+\frac{\partial f_1}{\partial x_2}(\boldsymbol{x}^{(0)})(x_2-x_2^{(0)})+\text{余项},\\ f_2(\boldsymbol{x})&=f_2(\boldsymbol{x}^{(0)})+\frac{\partial f_2}{\partial x_1}(\boldsymbol{x}^{(0)})(x_1-x_1^{(0)})+\frac{\partial f_2}{\partial x_2}(\boldsymbol{x}^{(0)})(x_2-x_2^{(0)})+\text{余项}. \end{aligned} \tag{3.26}$$

Jacobi 矩阵：为了用矩阵形式表示上述展开，我们引入 Jacobi 矩阵的概念。

定义 3.5.1 (Jacobi 矩阵)**.** 设 $\boldsymbol{f}:\mathbb{R}^n\to\mathbb{R}^n$ 是一个向量值函数，其分量函数为 $f_1,f_2,\ldots,f_n$。函数 $\boldsymbol{f}$ 在点 $\boldsymbol{x}$ 处的 **Jacobi 矩阵**（或 **导数矩阵**）定义为 $n\times n$ 矩阵

$$D\boldsymbol{f}(\boldsymbol{x})=\begin{pmatrix} \frac{\partial f_1}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_1}{\partial x_2}(\boldsymbol{x}) & \cdots & \frac{\partial f_1}{\partial x_n}(\boldsymbol{x})\\ \frac{\partial f_2}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_2}{\partial x_2}(\boldsymbol{x}) & \cdots & \frac{\partial f_2}{\partial x_n}(\boldsymbol{x})\\ \vdots & \vdots & \ddots & \vdots\\ \frac{\partial f_n}{\partial x_1}(\boldsymbol{x}) & \frac{\partial f_n}{\partial x_2}(\boldsymbol{x}) & \cdots & \frac{\partial f_n}{\partial x_n}(\boldsymbol{x}) \end{pmatrix}.$$

对于 $n=2$ 的情况，Jacobi 矩阵为

$$dD\boldsymbol{f}(\boldsymbol{x}^{(0)})=\begin{pmatrix}\dfrac{\partial f_1}{\partial x_1}(\boldsymbol{x}^{(0)}) & \dfrac{\partial f_1}{\partial x_2}(\boldsymbol{x}^{(0)})\\ \dfrac{\partial f_2}{\partial x_1}(\boldsymbol{x}^{(0)}) & \dfrac{\partial f_2}{\partial x_2}(\boldsymbol{x}^{(0)})\end{pmatrix}.$$

d 注意到，Jacobi 矩阵的第 i 行是梯度向量 $\nabla f_i(\boldsymbol{x}^{(0)})=\begin{pmatrix}\dfrac{\partial f_i}{\partial x_1}(\boldsymbol{x}^{(0)})\\ \dfrac{\partial f_i}{\partial x_2}(\boldsymbol{x}^{(0)})\end{pmatrix}$ 的转置，因此

$$D\boldsymbol{f}(\boldsymbol{x}^{(0)})=\begin{pmatrix}(\nabla f_1(\boldsymbol{x}^{(0)}))^T\\ (\nabla f_2(\boldsymbol{x}^{(0)}))^T\end{pmatrix}.$$

矩阵形式的 Taylor 展开：利用 Jacobi 矩阵，Taylor 展开 (3.26) 可以写成紧凑的矩阵形式：

$$\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{f}(\boldsymbol{x}^{(0)})+D\boldsymbol{f}(\boldsymbol{x}^{(0)})(\boldsymbol{x}-\boldsymbol{x}^{(0)})+\text{余项}, \tag{3.27}$$

或者用梯度向量的形式表示为

$$\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{f}(\boldsymbol{x}^{(0)})+\begin{pmatrix}(\nabla f_1(\boldsymbol{x}^{(0)}))^T(\boldsymbol{x}-\boldsymbol{x}^{(0)})\\ (\nabla f_2(\boldsymbol{x}^{(0)}))^T(\boldsymbol{x}-\boldsymbol{x}^{(0)})\end{pmatrix}+\text{余项}. \tag{3.28}$$

维度一致性检查：在式 (3.27) 中，$\boldsymbol{f}(\boldsymbol{x})$、$\boldsymbol{f}(\boldsymbol{x}^{(0)})$、$\boldsymbol{x}-\boldsymbol{x}^{(0)}$、余项都是 n 维列向量，$D\boldsymbol{f}(\boldsymbol{x}^{(0)})$ 是 $n\times n$ 矩阵，$D\boldsymbol{f}(\boldsymbol{x}^{(0)})(\boldsymbol{x}-\boldsymbol{x}^{(0)})$ 是 n 维列向量（矩阵与向量的乘积）。因此，所有的运算在维度上都是一致的。

迭代格式的推导

线性化近似：忽略 Taylor 展开中的余项，我们得到函数 $\boldsymbol{f}$ 的线性近似：

$$\boldsymbol{f}(\boldsymbol{x})\approx\boldsymbol{f}(\boldsymbol{x}^{(0)})+D\boldsymbol{f}(\boldsymbol{x}^{(0)})(\boldsymbol{x}-\boldsymbol{x}^{(0)}).$$

为了计算下一个迭代点 $\boldsymbol{x}^{(1)}$，我们要求上述线性近似等于零：

$$\boldsymbol{0}=\boldsymbol{f}(\boldsymbol{x}^{(0)})+D\boldsymbol{f}(\boldsymbol{x}^{(0)})(\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}). \tag{3.29}$$

线性方程组的求解：重新整理式 (3.29)，得到

$$D\boldsymbol{f}(\boldsymbol{x}^{(0)})(\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)})=-\boldsymbol{f}(\boldsymbol{x}^{(0)}).$$

设 $\boldsymbol{h}=\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}$，则我们需要求解线性方程组

$$D\boldsymbol{f}(\boldsymbol{x}^{(0)})\boldsymbol{h}=-\boldsymbol{f}(\boldsymbol{x}^{(0)}). \tag{3.30}$$

求解这个线性方程组得到向量 $\boldsymbol{h}$ 后，我们计算

$$\boldsymbol{x}^{(1)}=\boldsymbol{x}^{(0)}+\boldsymbol{h}.$$

一般迭代格式：对向量 $\boldsymbol{x}^{(1)}$ 重复相同的步骤，可以得到 $\boldsymbol{x}^{(2)}$，依此类推。当两个连续迭代的差值（即向量 $\boldsymbol{h}$ 的范数）小于事先给定的精度 ε 时，我们停止迭代。

算法描述

Algorithm 12 两个非线性方程的 Newton 方法

Require: 向量值函数 $\boldsymbol{f}$，初始向量 $\boldsymbol{x}^{(0)}$，精度 $\varepsilon > 0$，最大迭代次数 N
Ensure: 近似解 $\boldsymbol{x}^*$ 或失败信息

```
for k = 0, 1, ..., N − 1 do
    计算 f(x^(k))
    计算 Jacobi 矩阵 Df(x^(k))
    if Df(x^(k)) 奇异或条件数过大 then
        错误：Jacobi 矩阵奇异或病态，算法失败
    end if
    求解线性方程组 Df(x^(k))h = −f(x^(k))
    x^(k+1) ← x^(k) + h
    if ‖x^(k+1) − x^(k)‖ < ε 或 ‖f(x^(k+1))‖ < ε then
        返回 x^(k+1)
    end if
end for
错误：达到最大迭代次数，算法未收敛
```

算法说明：

1. **线性方程组的求解：**对于 $n = 2$ 的情况，线性方程组 (3.30) 是一个 2×2 的方程组，可以直接求逆矩阵来求解，即 $\boldsymbol{h} = -(D\boldsymbol{f}(\boldsymbol{x}^{(k)}))^{-1}\boldsymbol{f}(\boldsymbol{x}^{(k)})$。然而，对于一般的 n，我们通常使用 Gauss 消元法或 LU 分解来求解线性方程组，而不是显式地计算逆矩阵（因为计算逆矩阵的计算成本更高，且数值稳定性较差）。
2. **停止准则：**算法使用两个停止准则：一是相邻迭代的差值 $\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\|$ 小于给定精度；二是函数值的范数 $\|\boldsymbol{f}(\boldsymbol{x}^{(k+1)})\|$ 小于给定精度。在实际应用中，通常同时检查这两个条件，只要其中一个满足就停止迭代。
3. **数值稳定性：**在每次迭代中，我们需要检查 Jacobi 矩阵是否奇异或病态。如果 Jacobi 矩阵的条件数过大，线性方程组的求解可能不稳定，导致算法失败或收敛到错误的解。

例 3.5.1 (两个非线性方程的 Newton 方法示例)**.** 考虑非线性方程组

$$\begin{cases} f_1(x_1, x_2) = x_1^2 + x_2^2 - 4 = 0, \\ f_2(x_1, x_2) = x_1^2 - x_2 - 1 = 0. \end{cases}$$

这个方程组表示两条曲线的交点：第一个方程表示以原点为圆心、半径为 2 的圆；第二个方程表示一条抛物线。我们可以通过几何分析确定解的大致位置。

计算 Jacobi 矩阵：函数 $\boldsymbol{f}$ 的 Jacobi 矩阵为

$$D\boldsymbol{f}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 2x_2 \\ 2x_1 & -1 \end{pmatrix}.$$

选择初始值：通过简单的分析，我们可以找到两个解：$(1, \sqrt{3})$ 和 $(1, -\sqrt{3})$（近似值）。我们选择初始值 $\boldsymbol{x}^{(0)} = (1.5, 1.5)^T$，这个点接近第一个解。

第一次迭代：

- 计算函数值：$\boldsymbol{f}(\boldsymbol{x}^{(0)}) = \begin{pmatrix} 1.5^2 + 1.5^2 - 4 \\ 1.5^2 - 1.5 - 1 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.25 \end{pmatrix}$。
- 计算 Jacobi 矩阵：$D\boldsymbol{f}(\boldsymbol{x}^{(0)}) = \begin{pmatrix} 3 & 3 \\ 3 & -1 \end{pmatrix}$。

- 求解线性方程组：$\begin{pmatrix} 3 & 3 \\ 3 & -1 \end{pmatrix} \boldsymbol{h} = -\begin{pmatrix} 0.5 \\ -0.25 \end{pmatrix}$。

  利用 Cramer 法则或 Gauss 消元法，得到 $\boldsymbol{h} = \begin{pmatrix} -0.125 \\ -0.125 \end{pmatrix}$。

- 更新：$\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} + \boldsymbol{h} = \begin{pmatrix} 1.375 \\ 1.375 \end{pmatrix}$。

后续迭代：继续迭代过程，经过几次迭代后，序列收敛到解 $\boldsymbol{x}^* \approx \begin{pmatrix} 1.0 \\ 1.732 \end{pmatrix}$（即 $(1, \sqrt{3})$）。

收敛速度：与单变量 Newton 方法类似，在适当的条件下，两个变量的 Newton 方法也具有二次收敛速度，即 $\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^*\| \le C\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|^2$，其中 C 是正常数。

3.5.3 不动点方法

不动点方法是求解非线性方程组的另一类重要方法。与单变量情况类似，我们将非线性方程组的求解问题转化为不动点问题。

基本概念

考虑向量值函数 $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^n$，我们构造迭代序列

$$\begin{cases} \boldsymbol{x}^{(0)} \text{ 给定}, \\ \boldsymbol{x}^{(k+1)} = \boldsymbol{g}(\boldsymbol{x}^{(k)}), \quad k = 0, 1, 2, \ldots \end{cases} \tag{3.31}$$

在适当的收敛条件下，该序列可以收敛到满足 $\hat{\boldsymbol{x}} = \boldsymbol{g}(\hat{\boldsymbol{x}})$ 的不动点 $\hat{\boldsymbol{x}}$。

与单变量情况的类比：在单变量情况下，我们讨论了不动点迭代法 $x_{k+1} = g(x_k)$ 的收敛性。多变量情况下的不动点方法在形式上完全相同，只是将标量 x 和函数 g 替换为向量 $\boldsymbol{x}$ 和向量值函数 $\boldsymbol{g}$。

压缩映射原理

多变量情况下的不动点理论是单变量情况的自然推广。下面的定理给出了保证不动点存在唯一性和迭代序列收敛性的充分条件。

定理 3.5.1 (Banach 不动点定理（压缩映射原理）)**.** 设 $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^n$ 是一个映射，λ 是实数，满足 $0 \le \lambda < 1$，且对 $\mathbb{R}^n$ 中的任意向量 $\boldsymbol{z}$ 和 $\boldsymbol{y}$，有

$$\|\boldsymbol{g}(\boldsymbol{z}) - \boldsymbol{g}(\boldsymbol{y})\| \le \lambda\|\boldsymbol{z} - \boldsymbol{y}\|.$$

则存在唯一的 $\hat{\boldsymbol{x}} \in \mathbb{R}^n$，使得 $\hat{\boldsymbol{x}} = \boldsymbol{g}(\hat{\boldsymbol{x}})$，且由

$$\begin{cases} \boldsymbol{x}^{(0)} \text{ 给定}, \\ \boldsymbol{x}^{(k+1)} = \boldsymbol{g}(\boldsymbol{x}^{(k)}), \quad k = 0, 1, 2, \ldots \end{cases} \tag{3.32}$$

定义的序列 $\{\boldsymbol{x}^{(k)}\}_{k \ge 0}$ 收敛到 $\hat{\boldsymbol{x}}$。

证明. **第一步：证明序列是 Cauchy 序列**。设 $\{\boldsymbol{x}^{(k)}\}$ 是由式 (3.32) 定义的序列。我们需要证明这个序列是 Cauchy 序列，即对任意 $\varepsilon > 0$，存在 $N > 0$，使得当 $m, n \ge N$ 时，有 $\|\boldsymbol{x}^{(m)} - \boldsymbol{x}^{(n)}\| < \varepsilon$。

首先，我们证明序列中相邻两项的误差满足压缩性质。对任意 $k \ge 0$，有

$$\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\| = \|\boldsymbol{g}(\boldsymbol{x}^{(k)}) - \boldsymbol{g}(\boldsymbol{x}^{(k-1)})\| \le \lambda\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k-1)}\|.$$

重复应用这个不等式，得到

$$\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\| \le \lambda^k\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|.$$

现在，对任意 $m > n \geq 0$，利用三角不等式和上述结果，有

$$
\begin{aligned}
\|\boldsymbol{x}^{(m)} - \boldsymbol{x}^{(n)}\| &= \left\| \sum_{k=n}^{m-1} (\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}) \right\| \\
&\leq \sum_{k=n}^{m-1} \|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\| \\
&\leq \sum_{k=n}^{m-1} \lambda^k \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\| \\
&= \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\| \sum_{k=n}^{m-1} \lambda^k \\
&= \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\| \frac{\lambda^n - \lambda^m}{1-\lambda} \\
&\leq \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\| \frac{\lambda^n}{1-\lambda}.
\end{aligned}
\tag{3.33}
$$

由于 $0 \leq \lambda < 1$，当 $n \to \infty$ 时，$\lambda^n \to 0$。因此，对任意 $\varepsilon > 0$，存在 $N > 0$，使得当 $n \geq N$ 时，$\lambda^n < \varepsilon(1-\lambda)/\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|$（如果 $\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\| = 0$，则序列是常数序列，此时结论自动成立）。因此，当 $m, n \geq N$ 时，有

$$\|\boldsymbol{x}^{(m)} - \boldsymbol{x}^{(n)}\| < \varepsilon,$$

即序列 $\{\boldsymbol{x}^{(k)}\}$ 是 Cauchy 序列。

第二步：证明序列收敛。由于 $\mathbb{R}^n$ 是完备的（所有 Cauchy 序列都收敛），存在 $\hat{\boldsymbol{x}} \in \mathbb{R}^n$，使得 $\lim_{k\to\infty} \boldsymbol{x}^{(k)} = \hat{\boldsymbol{x}}$。

第三步：证明 $\hat{\boldsymbol{x}}$ 是不动点。由于 $\boldsymbol{g}$ 是压缩映射，因此是连续的（实际上，压缩映射是 Lipschitz 连续的）。因此，

$$\boldsymbol{g}(\hat{\boldsymbol{x}}) = \boldsymbol{g}\left(\lim_{k\to\infty} \boldsymbol{x}^{(k)}\right) = \lim_{k\to\infty} \boldsymbol{g}(\boldsymbol{x}^{(k)}) = \lim_{k\to\infty} \boldsymbol{x}^{(k+1)} = \hat{\boldsymbol{x}},$$

即 $\hat{\boldsymbol{x}}$ 是 $\boldsymbol{g}$ 的不动点。

第四步：证明不动点的唯一性。假设存在两个不动点 $\hat{\boldsymbol{x}}$ 和 $\hat{\boldsymbol{y}}$，即 $\hat{\boldsymbol{x}} = \boldsymbol{g}(\hat{\boldsymbol{x}})$ 和 $\hat{\boldsymbol{y}} = \boldsymbol{g}(\hat{\boldsymbol{y}})$。由压缩映射的性质，有

$$\|\hat{\boldsymbol{x}} - \hat{\boldsymbol{y}}\| = \|\boldsymbol{g}(\hat{\boldsymbol{x}}) - \boldsymbol{g}(\hat{\boldsymbol{y}})\| \leq \lambda \|\hat{\boldsymbol{x}} - \hat{\boldsymbol{y}}\|.$$

由于 $\lambda < 1$，上式只有在 $\|\hat{\boldsymbol{x}} - \hat{\boldsymbol{y}}\| = 0$ 时才能成立，即 $\hat{\boldsymbol{x}} = \hat{\boldsymbol{y}}$。因此，不动点是唯一的。

我们证明了序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛到唯一的不动点 $\hat{\boldsymbol{x}}$。 □

定理说明：这个定理表明，如果映射 $\boldsymbol{g}$ 是**压缩映射**（即存在常数 $\lambda < 1$，使得 $\|\boldsymbol{g}(\boldsymbol{z}) - \boldsymbol{g}(\boldsymbol{y})\| \leq \lambda \|\boldsymbol{z} - \boldsymbol{y}\|$），则不动点存在且唯一，且迭代序列必然收敛到该不动点。压缩映射的条件保证了每次迭代都会将误差缩小至少 λ 倍，从而保证序列的收敛性。

收敛速度：在压缩映射的条件下，可以证明迭代序列线性收敛，即存在常数 $C > 0$，使得

$$\|\boldsymbol{x}^{(k+1)} - \hat{\boldsymbol{x}}\| \leq \lambda \|\boldsymbol{x}^{(k)} - \hat{\boldsymbol{x}}\|.$$

Jacobi 矩阵与可微性

为了分析不动点方法的收敛性，我们需要引入向量值函数的可微性概念。

定义 3.5.2 (向量值函数的可微性)**.** 设 $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^n$ 是一个向量值函数，其分量函数为 $g_1, g_2, \ldots, g_n$。如果 $\boldsymbol{g}$ 在点 $\boldsymbol{x}$ 处有偏导数，且存在 Jacobi 矩阵 $D\boldsymbol{g}(\boldsymbol{x})$，使得

$$\boldsymbol{g}(\boldsymbol{y}) = \boldsymbol{g}(\boldsymbol{x}) + D\boldsymbol{g}(\boldsymbol{x})(\boldsymbol{y} - \boldsymbol{x}) + \|\boldsymbol{y} - \boldsymbol{x}\| \boldsymbol{\varepsilon}(\boldsymbol{y} - \boldsymbol{x}),$$

其中 $\boldsymbol{\varepsilon}(\boldsymbol{y} - \boldsymbol{x})$ 是 $\mathbb{R}^n$ 中的向量，当 $\boldsymbol{y} \to \boldsymbol{x}$ 时 $\boldsymbol{\varepsilon}(\boldsymbol{y} - \boldsymbol{x}) \to \boldsymbol{0}$，则称函数 $\boldsymbol{g}$ 在点 $\boldsymbol{x}$ 处**可微**。

谱半径：设 $\boldsymbol{A}$ 是一个 $n \times n$ 矩阵，$\boldsymbol{A}$ 的**谱半径**定义为

$$\rho(\boldsymbol{A}) = \max\{|\lambda| : \lambda \text{ 是} \boldsymbol{A} \text{ 的特征值}\}.$$

谱半径在不动点方法的收敛性分析中起着关键作用。下面的定理将不动点方法的收敛性与 Jacobi 矩阵的谱半径联系起来。

定理 3.5.2 (局部收敛性定理). 设 $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^n$ 是一个映射，它有一个不动点 $\hat{\boldsymbol{x}}$。假设 $\boldsymbol{g}$ 在 $\hat{\boldsymbol{x}}$ 处可微，且 $\rho[D\boldsymbol{g}(\hat{\boldsymbol{x}})]$（$\boldsymbol{g}$ 的 Jacobi 矩阵的谱半径）满足 $\rho[D\boldsymbol{g}(\hat{\boldsymbol{x}})] < 1$。则存在 $\eta > 0$，使得如果 $\boldsymbol{x}^{(0)}$ 满足 $\|\boldsymbol{x}^{(0)} - \hat{\boldsymbol{x}}\| < \eta$，则由

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{g}(\boldsymbol{x}^{(k)}) \tag{3.34}$$

定义的序列收敛到 $\hat{\boldsymbol{x}}$。

证明. **第一步：建立局部压缩性质**。由于 $\rho[D\boldsymbol{g}(\hat{\boldsymbol{x}})] < 1$，存在矩阵范数 $\|\cdot\|_*$（与向量范数相容），使得 $\|D\boldsymbol{g}(\hat{\boldsymbol{x}})\|_* < 1$。这是因为对于任意矩阵 $\boldsymbol{A}$，存在矩阵范数 $\|\cdot\|_*$，使得 $\|\boldsymbol{A}\|_*$ 任意接近 $\rho(\boldsymbol{A})$（Gelfand 公式的推论）。

设 $\mu = \|D\boldsymbol{g}(\hat{\boldsymbol{x}})\|_*$，则 $\mu < 1$。由于 $\boldsymbol{g}$ 在 $\hat{\boldsymbol{x}}$ 处可微，存在 $\delta > 0$，使得当 $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| < \delta$ 时，有

$$\boldsymbol{g}(\boldsymbol{x}) = \boldsymbol{g}(\hat{\boldsymbol{x}}) + D\boldsymbol{g}(\hat{\boldsymbol{x}})(\boldsymbol{x} - \hat{\boldsymbol{x}}) + \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{x} - \hat{\boldsymbol{x}}),$$

其中 $\boldsymbol{\varepsilon}(\boldsymbol{x} - \hat{\boldsymbol{x}}) \to \boldsymbol{0}$ 当 $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| \to 0$。

第二步：构造局部压缩映射。选择 $\eta > 0$ 充分小，使得 $\eta < \delta$，且当 $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| < \eta$ 时，有

$$\|D\boldsymbol{g}(\hat{\boldsymbol{x}})\|_* + \|\boldsymbol{\varepsilon}(\boldsymbol{x} - \hat{\boldsymbol{x}})\| < \frac{1+\mu}{2} < 1.$$

设 $\lambda = \frac{1+\mu}{2}$，则 $\mu < \lambda < 1$。对任意 $\boldsymbol{x}, \boldsymbol{y} \in B(\hat{\boldsymbol{x}}, \eta)$（以 $\hat{\boldsymbol{x}}$ 为圆心、η 为半径的球），有

$$\begin{aligned}
\|\boldsymbol{g}(\boldsymbol{x}) - \boldsymbol{g}(\boldsymbol{y})\| &= \|\boldsymbol{g}(\hat{\boldsymbol{x}}) + D\boldsymbol{g}(\hat{\boldsymbol{x}})(\boldsymbol{x} - \hat{\boldsymbol{x}}) + \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{x} - \hat{\boldsymbol{x}}) \\
&\quad - \boldsymbol{g}(\hat{\boldsymbol{x}}) - D\boldsymbol{g}(\hat{\boldsymbol{x}})(\boldsymbol{y} - \hat{\boldsymbol{x}}) - \|\boldsymbol{y} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{y} - \hat{\boldsymbol{x}})\| \\
&= \|D\boldsymbol{g}(\hat{\boldsymbol{x}})(\boldsymbol{x} - \boldsymbol{y}) + \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{x} - \hat{\boldsymbol{x}}) - \|\boldsymbol{y} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{y} - \hat{\boldsymbol{x}})\| \\
&\le \|D\boldsymbol{g}(\hat{\boldsymbol{x}})(\boldsymbol{x} - \boldsymbol{y})\| + \|\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{x} - \hat{\boldsymbol{x}}) - \|\boldsymbol{y} - \hat{\boldsymbol{x}}\|\boldsymbol{\varepsilon}(\boldsymbol{y} - \hat{\boldsymbol{x}})\| \\
&\le \|D\boldsymbol{g}(\hat{\boldsymbol{x}})\|_*\|\boldsymbol{x} - \boldsymbol{y}\| + o(\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| + \|\boldsymbol{y} - \hat{\boldsymbol{x}}\|).
\end{aligned} \tag{3.35}$$

当 η 充分小时，高阶项可以忽略，因此存在常数 C，使得

$$\|\boldsymbol{g}(\boldsymbol{x}) - \boldsymbol{g}(\boldsymbol{y})\| \le \lambda\|\boldsymbol{x} - \boldsymbol{y}\|,$$

对任意 $\boldsymbol{x}, \boldsymbol{y} \in B(\hat{\boldsymbol{x}}, \eta)$ 成立。

第三步：应用压缩映射原理。由第一步和第二步，映射 $\boldsymbol{g}$ 在球 $B(\hat{\boldsymbol{x}}, \eta)$ 上是压缩映射，压缩常数为 $\lambda < 1$。如果初始值 $\boldsymbol{x}^{(0)} \in B(\hat{\boldsymbol{x}}, \eta)$，则由 Banach 不动点定理（定理 3.5.1），序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛到唯一的不动点 $\hat{\boldsymbol{x}}$。

我们证明了，如果 $\rho[D\boldsymbol{g}(\hat{\boldsymbol{x}})] < 1$，则存在 $\eta > 0$，使得从 $B(\hat{\boldsymbol{x}}, \eta)$ 内任意初始值出发，迭代序列都收敛到不动点 $\hat{\boldsymbol{x}}$。 □

定理说明：这个定理表明，如果不动点 $\hat{\boldsymbol{x}}$ 处的 Jacobi 矩阵的谱半径小于 1，则存在一个邻域，使得从该邻域内任意初始值出发，迭代序列都会收敛到不动点。这是局部收敛性定理，它只保证了在不动点附近收敛，而不保证全局收敛。

收敛速度：在定理 3.5.2 的条件下，可以证明收敛是线性的，即存在常数 $C > 0$，使得

$$\|\boldsymbol{x}^{(k+1)} - \hat{\boldsymbol{x}}\| \le C\|\boldsymbol{x}^{(k)} - \hat{\boldsymbol{x}}\|. \tag{3.36}$$

更一般地，如果存在常数 $C > 0$ 和 $n \ge 1$，使得

$$\|\boldsymbol{x}^{(k+1)} - \hat{\boldsymbol{x}}\| \le C\|\boldsymbol{x}^{(k)} - \hat{\boldsymbol{x}}\|^n, \tag{3.37}$$

则称收敛是 n **阶的**。特别地，当 $n = 1$ 时称为线性收敛，当 $n = 2$ 时称为二次收敛。

3.5.4 非线性方程组的 Newton 方法

现在我们将两个变量的 Newton 方法推广到一般的 n 个变量的情况。

方法思想

与单变量和两个变量的情况类似，Newton 方法的核心思想是利用函数的一阶 Taylor 展开来线性化非线性问题。

仿射近似：设 $\boldsymbol{f}:\mathbb{R}^n\to\mathbb{R}^n$ 是一个向量值函数，$\boldsymbol{x}^{(k)}$ 是当前迭代点。如果 $\boldsymbol{f}$ 在 $\boldsymbol{x}^{(k)}$ 处可微，则存在 Jacobi 矩阵 $D\boldsymbol{f}(\boldsymbol{x}^{(k)})$，使得

$$\boldsymbol{f}(\boldsymbol{x}^{(k)}+\boldsymbol{h})=\boldsymbol{f}(\boldsymbol{x}^{(k)})+D\boldsymbol{f}(\boldsymbol{x}^{(k)})\boldsymbol{h}+\|\boldsymbol{h}\|\boldsymbol{\varepsilon}(\boldsymbol{h}), \tag{3.38}$$

其中 $\boldsymbol{\varepsilon}(\boldsymbol{h})$ 是 $\mathbb{R}^n$ 中的向量，当 $\|\boldsymbol{h}\|\to 0$ 时 $\boldsymbol{\varepsilon}(\boldsymbol{h})\to\boldsymbol{0}$。

线性化：忽略式 (3.38) 中的高阶项 $\|\boldsymbol{h}\|\boldsymbol{\varepsilon}(\boldsymbol{h})$，我们得到函数 $\boldsymbol{f}$ 的仿射近似：

$$\boldsymbol{f}(\boldsymbol{x}^{(k)}+\boldsymbol{h})\approx\boldsymbol{f}(\boldsymbol{x}^{(k)})+D\boldsymbol{f}(\boldsymbol{x}^{(k)})\boldsymbol{h}.$$

为了确定增量向量 $\boldsymbol{h}$，我们要求上述近似等于零：

$$\boldsymbol{f}(\boldsymbol{x}^{(k)})+D\boldsymbol{f}(\boldsymbol{x}^{(k)})\boldsymbol{h}=\boldsymbol{0}, \tag{3.39}$$

即

$$D\boldsymbol{f}(\boldsymbol{x}^{(k)})\boldsymbol{h}=-\boldsymbol{f}(\boldsymbol{x}^{(k)}). \tag{3.40}$$

迭代格式：求解线性方程组 (3.40) 得到 $\boldsymbol{h}$ 后，我们计算下一个迭代点：

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}+\boldsymbol{h}.$$

算法描述

Algorithm 13 非线性方程组的 Newton 方法

Require: 向量值函数 $\boldsymbol{f}$，初始向量 $\boldsymbol{x}^{(0)}$，精度 $\varepsilon>0$，最大迭代次数 N
Ensure: 近似解 $\boldsymbol{x}^*$ 或失败信息

```
for k = 0, 1, ..., N − 1 do
    计算 f(x^(k))
    计算 Jacobi 矩阵 Df(x^(k))
    if Df(x^(k)) 奇异或条件数过大 then
        错误：Jacobi 矩阵奇异或病态，算法失败
    end if
    求解线性方程组 Df(x^(k))h = −f(x^(k))（使用 Gauss 消元法或 LU 分解）
    x^(k+1) ← x^(k) + h
    if ‖x^(k+1) − x^(k)‖ < ε 或 ‖f(x^(k+1))‖ < ε then
        返回 x^(k+1)
    end if
end for
错误：达到最大迭代次数，算法未收敛
```

算法说明：

1. **线性方程组的求解**：在每次迭代中，我们需要求解一个 $n\times n$ 的线性方程组。对于大规模问题，我们通常使用 Gauss 消元法或 LU 分解来求解，而不是显式地计算逆矩阵。这是因为计算逆矩阵的计算复杂度为 $\mathscr{O}(n^3)$，且数值稳定性较差。

2. **Jacobi 矩阵的计算**：计算 Jacobi 矩阵需要计算 n^2 个偏导数。如果这些偏导数没有解析表达式，我们可以使用有限差分来近似：
$$\frac{\partial f_i}{\partial x_j}(\boldsymbol{x}) \approx \frac{f_i(\boldsymbol{x}+h\boldsymbol{e}_j)-f_i(\boldsymbol{x})}{h},$$
其中 $\boldsymbol{e}_j$ 是第 j 个标准单位向量，h 是一个小的正数。
3. **初始值的选择**：Newton 方法只保证局部收敛性，因此初始值的选择非常重要。如果初始值远离解，迭代可能发散。在实际应用中，通常需要结合其他方法（如简单的固定点迭代）来找到合适的初始值。

收敛性分析

下面的定理给出了 Newton 方法收敛性的充分条件。

定理 3.5.3 (Newton 方法的局部收敛性)**.** 设 $\boldsymbol{f}:\mathbb{R}^n\to\mathbb{R}^n$ 是一个映射。我们希望通过 Newton 方法求解 $\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{0}$。如果存在 $\hat{\boldsymbol{x}}$，使得

1. $\boldsymbol{f}(\hat{\boldsymbol{x}})=\boldsymbol{0}$，
2. $\boldsymbol{f}$ 在 $\hat{\boldsymbol{x}}$ 的邻域内可微，且存在常数 $\alpha>0$，使得对邻域内任意 $\boldsymbol{x}$，有
$$\|D\boldsymbol{f}(\boldsymbol{x})-D\boldsymbol{f}(\hat{\boldsymbol{x}})\|\le\alpha\|\boldsymbol{x}-\hat{\boldsymbol{x}}\|,$$
3. $D\boldsymbol{f}(\hat{\boldsymbol{x}})$ 可逆，

则存在严格正实数 $\eta>0$，使得如果 $\boldsymbol{x}^{(0)}$ 满足 $\|\boldsymbol{x}^{(0)}-\hat{\boldsymbol{x}}\|<\eta$，则由 Newton 方法构造的序列 $\{\boldsymbol{x}^{(k)}\}$ 收敛到 $\hat{\boldsymbol{x}}$。此外，收敛是二次的，即存在常数 $C>0$，使得
$$\|\boldsymbol{x}^{(k+1)}-\hat{\boldsymbol{x}}\|\le C\|\boldsymbol{x}^{(k)}-\hat{\boldsymbol{x}}\|^2.$$

证明. **第一步：建立误差递推关系**。设 $\boldsymbol{e}^{(k)}=\boldsymbol{x}^{(k)}-\hat{\boldsymbol{x}}$ 表示第 k 次迭代的误差。由 Newton 方法的迭代公式，有
$$\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}-[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\boldsymbol{f}(\boldsymbol{x}^{(k)}),$$
因此
$$\boldsymbol{e}^{(k+1)}=\boldsymbol{x}^{(k+1)}-\hat{\boldsymbol{x}}=\boldsymbol{x}^{(k)}-\hat{\boldsymbol{x}}-[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\boldsymbol{f}(\boldsymbol{x}^{(k)})=\boldsymbol{e}^{(k)}-[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\boldsymbol{f}(\boldsymbol{x}^{(k)}).$$

第二步：利用可微性展开 $\boldsymbol{f}(\boldsymbol{x}^{(k)})$。由于 $\boldsymbol{f}$ 在 $\hat{\boldsymbol{x}}$ 的邻域内可微，且 $\boldsymbol{f}(\hat{\boldsymbol{x}})=\boldsymbol{0}$，有
$$\boldsymbol{f}(\boldsymbol{x}^{(k)})=\boldsymbol{f}(\hat{\boldsymbol{x}}+\boldsymbol{e}^{(k)})=D\boldsymbol{f}(\hat{\boldsymbol{x}})\boldsymbol{e}^{(k)}+\|\boldsymbol{e}^{(k)}\|\boldsymbol{\varepsilon}(\boldsymbol{e}^{(k)}),$$
其中 $\boldsymbol{\varepsilon}(\boldsymbol{e}^{(k)})\to\boldsymbol{0}$ 当 $\|\boldsymbol{e}^{(k)}\|\to 0$。

更精确地，由条件 (2)，存在常数 $\alpha>0$，使得
$$\boldsymbol{f}(\boldsymbol{x}^{(k)})=D\boldsymbol{f}(\hat{\boldsymbol{x}})\boldsymbol{e}^{(k)}+\int_0^1[D\boldsymbol{f}(\hat{\boldsymbol{x}}+t\boldsymbol{e}^{(k)})-D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}\,\mathrm{d}t.$$

第三步：估计误差。将上述展开代入误差递推关系，得到
$$\begin{aligned}\boldsymbol{e}^{(k+1)}&=\boldsymbol{e}^{(k)}-[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\boldsymbol{f}(\boldsymbol{x}^{(k)})\\&=\boldsymbol{e}^{(k)}-[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\left[D\boldsymbol{f}(\hat{\boldsymbol{x}})\boldsymbol{e}^{(k)}+\int_0^1[D\boldsymbol{f}(\hat{\boldsymbol{x}}+t\boldsymbol{e}^{(k)})-D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}\,\mathrm{d}t\right]\\&=[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\left\{D\boldsymbol{f}(\boldsymbol{x}^{(k)})\boldsymbol{e}^{(k)}-D\boldsymbol{f}(\hat{\boldsymbol{x}})\boldsymbol{e}^{(k)}-\int_0^1[D\boldsymbol{f}(\hat{\boldsymbol{x}}+t\boldsymbol{e}^{(k)})-D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}\,\mathrm{d}t\right\}\\&=[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\left\{[D\boldsymbol{f}(\boldsymbol{x}^{(k)})-D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}-\int_0^1[D\boldsymbol{f}(\hat{\boldsymbol{x}}+t\boldsymbol{e}^{(k)})-D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}\,\mathrm{d}t\right\}.\end{aligned}\tag{3.41}$$

第四步：建立误差上界。取范数，并利用条件 (2) 和矩阵范数的性质，有

$$
\begin{aligned}
\|\boldsymbol{e}^{(k+1)}\| &\le \|[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\| \left\{ \|[D\boldsymbol{f}(\boldsymbol{x}^{(k)}) - D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}\| + \int_0^1 \|[D\boldsymbol{f}(\hat{\boldsymbol{x}} + t\boldsymbol{e}^{(k)}) - D\boldsymbol{f}(\hat{\boldsymbol{x}})]\boldsymbol{e}^{(k)}\| \,\mathrm{d}t \right\} \\
&\le \|[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\| \left\{ \alpha\|\boldsymbol{e}^{(k)}\|^2 + \int_0^1 \alpha t \|\boldsymbol{e}^{(k)}\|^2 \,\mathrm{d}t \right\} \\
&= \|[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\| \left\{ \alpha\|\boldsymbol{e}^{(k)}\|^2 + \frac{\alpha}{2}\|\boldsymbol{e}^{(k)}\|^2 \right\} \\
&= \frac{3\alpha}{2}\|[D\boldsymbol{f}(\boldsymbol{x}^{(k)})]^{-1}\|\|\boldsymbol{e}^{(k)}\|^2.
\end{aligned}
\tag{3.42}
$$

第五步：估计逆矩阵的范数。由于 $D\boldsymbol{f}(\hat{\boldsymbol{x}})$ 可逆，且 $\boldsymbol{f}$ 在 $\hat{\boldsymbol{x}}$ 的邻域内可微，存在 $\delta > 0$，使得当 $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| < \delta$ 时，$D\boldsymbol{f}(\boldsymbol{x})$ 可逆，且

$$\|[D\boldsymbol{f}(\boldsymbol{x})]^{-1}\| \le 2\|[D\boldsymbol{f}(\hat{\boldsymbol{x}})]^{-1}\|.$$

这是因为当 $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|$ 充分小时，$D\boldsymbol{f}(\boldsymbol{x})$ 接近 $D\boldsymbol{f}(\hat{\boldsymbol{x}})$，因此其逆矩阵的范数有界。

第六步：选择初始邻域并证明收敛。设 $\beta = 2\|[D\boldsymbol{f}(\hat{\boldsymbol{x}})]^{-1}\|$，$C = \frac{3\alpha\beta}{2}$。选择 $\eta > 0$ 充分小，使得 $\eta < \delta$ 且 $C\eta < 1$。

如果 $\|\boldsymbol{x}^{(0)} - \hat{\boldsymbol{x}}\| < \eta$，则由上述估计，有

$$\|\boldsymbol{e}^{(1)}\| \le C\|\boldsymbol{e}^{(0)}\|^2 \le C\eta^2 < C\eta \cdot \eta < \eta,$$

因此 $\boldsymbol{x}^{(1)} \in B(\hat{\boldsymbol{x}}, \eta)$。由数学归纳法，可以证明对所有 $k \ge 0$，有 $\|\boldsymbol{e}^{(k)}\| < \eta$，且

$$\|\boldsymbol{e}^{(k+1)}\| \le C\|\boldsymbol{e}^{(k)}\|^2.$$

第七步：证明收敛性。由 $\|\boldsymbol{e}^{(k+1)}\| \le C\|\boldsymbol{e}^{(k)}\|^2$ 和 $C\eta < 1$，有

$$\|\boldsymbol{e}^{(k)}\| \le (C\eta)^{2^k - 1}\|\boldsymbol{e}^{(0)}\|.$$

由于 $C\eta < 1$，当 $k \to \infty$ 时，$(C\eta)^{2^k-1} \to 0$，因此 $\|\boldsymbol{e}^{(k)}\| \to 0$，即 $\boldsymbol{x}^{(k)} \to \hat{\boldsymbol{x}}$。

第八步：证明二次收敛速度。由 $\|\boldsymbol{e}^{(k+1)}\| \le C\|\boldsymbol{e}^{(k)}\|^2$，直接得到二次收敛速度的估计。

我们证明了，在定理的条件下，Newton 方法局部收敛到 $\hat{\boldsymbol{x}}$，且具有二次收敛速度。 □

定理说明：这个定理是单变量情况下相应结果的推广。条件 (1) 保证了 $\hat{\boldsymbol{x}}$ 确实是方程组的解；条件 (2) 要求 Jacobi 矩阵在解附近满足 Lipschitz 连续性，这保证了线性近似的精度；条件 (3) 保证了线性方程组有唯一解。

收敛速度：证明过程清楚地表明，Newton 方法具有二次收敛速度，即存在常数 $C > 0$，使得

$$\|\boldsymbol{x}^{(k+1)} - \hat{\boldsymbol{x}}\| \le C\|\boldsymbol{x}^{(k)} - \hat{\boldsymbol{x}}\|^2. \tag{3.43}$$

这意味着每次迭代后，误差的平方与上一次误差成正比，收敛速度非常快。一旦迭代值接近解，误差会以平方的速度快速衰减。

与单变量情况的比较：多变量 Newton 方法的收敛性分析与单变量情况在本质上相同，只是将标量替换为向量，将导数替换为 Jacobi 矩阵。二次收敛速度的特性在多变量情况下仍然保持，这使得 Newton 方法成为求解非线性方程组的最有效方法之一。

Chapter 4

插值法

在数值计算中，我们常常需要在离散数据点之间重建一条“足够好”的曲线，从而在保持计算代价可控的前提下近似原函数的行为。本章围绕这一类插值与逼近问题，讨论用多项式和样条函数来构造近似，并从误差、稳定性与收敛性的角度理解这些方法的适用范围与局限性。

本章路线图　本章将依次讨论以下内容：

- **插值与逼近的基本分类**：从函数逼近的视角出发，区分插值、最小二乘逼近和积分意义下的逼近，并澄清它们在数值分析中的角色。
- **多项式插值的理论与算法**：给出多项式插值的存在唯一性定理，引入 Lagrange 形式与 Newton 形式，并分析插值误差的表达式及其对收敛性的影响。
- **数值实现与计算复杂度**：通过差商表与 Horner 算法等工具，说明如何高效而稳定地实现插值多项式的构造与求值。
- **三次样条插值作为改进方案**：在认识多项式插值局限性的基础上，引入三次样条插值，展示它如何在保持光滑性的同时减弱全局振荡，为后续的数值逼近和数值微分提供基础工具。

4.1　引言

4.1.1　函数的逼近

本小节的目标是：从函数逼近的整体视角出发，给出几种常见的逼近准则，并解释插值在这一体系中的位置，为后续专门讨论插值方法做好认知铺垫。

在数值计算中，我们经常遇到以下情况：需要计算函数 $f(t)$ 在大量 t 值处的值，但我们不知道 f 的显式表达式。这种情况通常发生在两种情形：第一种情形是函数 f 只在某些实验点 $t_0, t_1, \ldots, t_n$ 处已知，这些点可能是通过实验测量或观测得到的；第二种情形是函数 f 由计算成本高昂的计算代码进行数值计算，虽然理论上可以计算，但每次计算都需要大量的计算资源。

为了解决这个问题，我们希望用一个计算简单的函数来近似表示 f。换句话说，这是用另一个更容易计算的函数来逼近函数 f。这个近似函数 $\tilde{f}$ 选自一个计算成本不太高的函数类 $\tilde{\mathscr{F}}$。常见的函数类包括多项式集合、有理分式集合、三角多项式集合等。

问题的关键在于我们希望以何种意义逼近 f。有很多种可能性，我们列举最常见的几种：

第一种可能性是寻找一个函数 $\tilde{f} \in \tilde{\mathscr{F}}$，使得对所有插值点 t_i（$i = 0, 1, \ldots, n$），都有 $\tilde{f}(t_i) = f(t_i)$。这是插值技术，也是本章主要讨论的内容。

第二种可能性是寻找一个函数 $\tilde{f} \in \tilde{\mathscr{F}}$，它最小化两条曲线在横坐标为 t_i（$0 \le i \le n$）的点处的差值平方和：

$$\sum_{i=0}^{n}(\tilde{f}(t_i) - f(t_i))^2 = \min_{g \in \tilde{\mathscr{F}}} \sum_{i=0}^{n}(g(t_i) - f(t_i))^2.$$

这是最小二乘最小化技术，将在其他章节中研究。

第三种可能性是寻找一个函数 $\tilde{f} \in \tilde{\mathscr{F}}$，使得

$$\int_a^b |f(x) - \tilde{f}(x)|^2 \,\mathrm{d}x = \min_{g \in \tilde{\mathscr{F}}} \int_a^b |f(x) - g(x)|^2 \,\mathrm{d}x.$$

这是二次逼近技术，也称为 L^2 范数意义下的逼近。

最后一种可能性是寻找一个函数 $\tilde{f} \in \tilde{\mathscr{F}}$，使得

$$\max_{x \in [a,b]} |f(x) - \tilde{f}(x)| = \min_{g \in \tilde{\mathscr{F}}} \max_{x \in [a,b]} |f(x) - g(x)|.$$

这是一致逼近技术，也称为 Chebyshev 逼近或 L^∞ 范数意义下的逼近。

在本章的后续内容中，我们只处理插值技术，即要求近似函数在给定的插值点处精确地等于被插值函数的值。

在目前阶段，只需记住：插值是“在有限个给定点上零误差”的逼近方式，而最小二乘和积分意义下的逼近则允许在每个点上有误差，但在整体指标上控制误差，这些方法将在后续章节系统讨论。

4.1.2　插值问题的表述

本小节关心的问题是：如何用精确通过给定数据点的函数来数学化“插值”的概念，并在这一框架下限定我们允许采用的函数类。

在以下所有内容中，我们假设给定两个包含 $n+1$ 个实数的族：第一个是横坐标集合 $\{t_0, t_1, \ldots, t_n\}$，假设它们都互不相同；第二个是对应的纵坐标集合 $\{y_0, y_1, \ldots, y_n\}$。这些值可以是函数 f 在点 t_i 处的值，即 $y_i = f(t_i)$，其中 f 是我们希望表示的函数。

我们然后寻找一个函数 h，它在每个点 t_i 处取值 y_i，这对 $i = 0, \ldots, n$ 成立。换句话说，我们要求

$$h(t_i) = y_i, \quad i = 0, 1, \ldots, n.$$

从构造的角度看，这个问题通常有无穷多个解。例如，我们可以构造一个分段常数函数，在每个区间 $[t_i, t_{i+1})$ 上取值为 y_i；或者我们可以构造一个分段线性函数，通过所有点 (t_i, y_i)。因此，必须明确我们将在其中寻找 h 的函数类 $\tilde{\mathscr{F}}$。

在以下内容中，我们将考虑两个特殊的函数类：第一个是次数最多为 n 的多项式集合 $\mathscr{P}_n$，这是本章主要讨论的内容；第二个是三次样条集合，它们是定义为分段多项式的函数，每个多项式的次数为 3，这将在后续章节中讨论。

还存在其他插值方法，例如使用 f 在点 $t_0, t_1, \ldots, t_n$ 处的导数信息的方法。这是 Hermite 插值，我们在这里不详细讨论。我们只给出一个非常简单的例子来说明这种思想。假设已知 f 及其直到 n 阶的导数在点 t_0 处的值。则满足

$$P_n^{(i)}(t_0) = f^{(i)}(t_0), \quad i = 0, 1, \ldots, n$$

的唯一 n 次多项式 P_n 由下式给出：

$$P_n(t) = f(t_0) + (t - t_0)f'(t_0) + \cdots + \frac{(t - t_0)^n}{n!} f^{(n)}(t_0).$$

这个多项式实际上就是 f 在点 $t = t_0$ 处的 n 阶 Taylor 展开的多项式部分。Taylor 多项式是一种特殊的 Hermite 插值，它只使用一个点的函数值和导数值。

4.2 多项式插值

本节的目标是：在一般插值问题的框架下，专门研究用多项式进行插值的情形，回答“多项式插值何时存在且唯一”这一基础问题，并为后续的具体算法与误差分析提供支撑。

4.2.1 Lagrange 基中的插值多项式

本小节的目标是：引入 Lagrange 基这一具体构造，用显式公式给出插值多项式，从而把存在唯一性定理变成可以直接计算的表达式。

多项式插值的核心问题是：给定 $n+1$ 个互异的实数 $(t_0, t_1, \ldots, t_n)$ 和 $n+1$ 个实数 $(y_0, y_1, \ldots, y_n)$，是否存在唯一的多项式 $p_n \in \mathscr{P}_n$（次数 $\leq n$ 的多项式集合），使得对所有 $i = 0, 1, \ldots, n$，都有 $p_n(t_i) = y_i$？

答案是肯定的，这就是下面的基本定理：

下面的定理表明：在给定互异节点的情况下，多项式插值不仅总是存在，而且解是唯一的，这是后续一切多项式插值算法的理论起点。

定理 4.2.1 (多项式插值的存在唯一性)**.** 给定 $n+1$ 个互异的实数 $(t_0, t_1, \ldots, t_n)$ 和 $n+1$ 个实数 $(y_0, y_1, \ldots, y_n)$，存在唯一的多项式 $p_n \in \mathscr{P}_n$（次数 $\leq n$ 的多项式集合），使得

$$p_n(t_i) = y_i, \quad i = 0, 1, \ldots, n.$$

证明. **存在性**：通过直接展示所求的多项式来证明。我们构造的多项式为

$$p_n(t) = \sum_{i=0}^{n} y_i L_i(t), \tag{4.1}$$

其中多项式 L_i 是 Lagrange 基多项式，其形式为

$$L_i(t) = \prod_{\substack{k=0 \\ k \neq i}}^{n} \frac{t - t_k}{t_i - t_k}. \tag{4.2}$$

第一步：验证 $L_i(t)$ 是多项式。由于 $L_i(t)$ 是 n 个线性因子的乘积，每个因子 $\frac{t-t_k}{t_i-t_k}$ 是 t 的一次多项式（因为 $t_i \neq t_k$，分母不为零），所以 $L_i(t)$ 是 n 次多项式，属于 $\mathscr{P}_n$。因此，由公式 (4.1) 定义的 $p_n(t)$ 是 n 个 n 次多项式的线性组合，其次数不超过 n，即 $p_n \in \mathscr{P}_n$。

第二步：验证 Lagrange 基多项式的插值性质。通过 $L_i(t)$ 的构造，我们可以验证：

$$L_i(t_j) = \begin{cases} 1, & \text{当} j = i, \\ 0, & \text{当} j \neq i. \end{cases}$$

详细验证如下：

- **情况 1**：$j = i$。当 $j = i$ 时，我们需要计算 $L_i(t_i)$。在乘积 $\prod_{\substack{k=0 \\ k \neq i}}^{n} \frac{t_i - t_k}{t_i - t_k}$ 中，每个因子 $\frac{t_i - t_k}{t_i - t_k} = 1$（因为 $t_i \neq t_k$，分母不为零）。因此，$L_i(t_i) = 1 \times 1 \times \cdots \times 1 = 1$。
- **情况 2**：$j \neq i$。当 $j \neq i$ 时，我们需要计算 $L_i(t_j)$。在乘积 $\prod_{\substack{k=0 \\ k \neq i}}^{n} \frac{t_j - t_k}{t_i - t_k}$ 中，当 $k = j$ 时，因子为 $\frac{t_j - t_j}{t_i - t_j} = \frac{0}{t_i - t_j} = 0$（因为 $t_i \neq t_j$，分母不为零）。因此，整个乘积中有一个因子为零，所以 $L_i(t_j) = 0$。

第三步：验证 $p_n(t)$ 满足插值条件。现在我们来验证由公式 (4.1) 定义的多项式 p_n 确实满足插值条件。对任

意 $j \in \{0,1,\ldots,n\}$，有

$$\begin{aligned} p_n(t_j) &= \sum_{i=0}^{n} y_i L_i(t_j) \\ &= y_0 L_0(t_j) + y_1 L_1(t_j) + \cdots + y_j L_j(t_j) + \cdots + y_n L_n(t_j) \\ &= y_0 \cdot 0 + y_1 \cdot 0 + \cdots + y_j \cdot 1 + \cdots + y_n \cdot 0 \\ &= y_j. \end{aligned} \tag{4.3}$$

这里我们使用了第二步中证明的性质：$L_i(t_j) = 0$ 当 $i \neq j$，$L_j(t_j) = 1$。因此，$p_n(t_j) = y_j$ 对所有 $j = 0,1,\ldots,n$ 成立，这证明了存在性。

唯一性：我们采用反证法来证明唯一性。假设存在两个不同的多项式 p_n 和 q_n 都属于 $\mathscr{P}_n$，且都满足插值条件：

$$p_n(t_i) = q_n(t_i) = y_i, \quad i = 0,1,\ldots,n.$$

第一步：构造差多项式。定义差多项式 $d_n(t) = p_n(t) - q_n(t)$。由于 p_n 和 q_n 都是次数不超过 n 的多项式，它们的差 d_n 也是次数不超过 n 的多项式，即 $d_n \in \mathscr{P}_n$。

第二步：分析差多项式的零点。对于任意 $i \in \{0,1,\ldots,n\}$，有

$$d_n(t_i) = p_n(t_i) - q_n(t_i) = y_i - y_i = 0.$$

因此，$d_n(t)$ 在 $n+1$ 个互异的点 $t_0, t_1, \ldots, t_n$ 处都为零，即 d_n 至少有 $n+1$ 个不同的根。

第三步：应用代数基本定理。根据代数基本定理，一个 n 次多项式最多有 n 个根（重根按重数计算）。更精确地说，如果 d_n 不是零多项式，设其次数为 m（$0 \le m \le n$），则 d_n 最多有 $m \le n$ 个根。但我们已经证明 d_n 至少有 $n+1$ 个不同的根，这与代数基本定理矛盾。

第四步：得出结论。因此，我们的假设（存在两个不同的多项式满足插值条件）是错误的。唯一可能的情况是 $d_n(t) \equiv 0$，即 $p_n(t) = q_n(t)$ 对所有 t 成立。这证明了唯一性。

我们证明了存在唯一的多项式 $p_n \in \mathscr{P}_n$ 满足插值条件 $p_n(t_i) = y_i$（$i = 0,1,\ldots,n$）。 □

例 4.2.1 (Lagrange 插值多项式的详细计算). 考虑函数 $f(x) = e^x$，已知其在以下 4 个点处的值：

x	-1	0	1	2
$f(x)$	$e^{-1} \approx 0.3679$	1	$e \approx 2.7183$	$e^2 \approx 7.3891$

我们希望构造一个 3 次 Lagrange 插值多项式 $p_3(x)$，使得 $p_3(x_i) = f(x_i)$ 对所有 $i = 0,1,2,3$ 成立，其中 $x_0 = -1$，$x_1 = 0$，$x_2 = 1$，$x_3 = 2$。

第一步：计算 Lagrange 基多项式。根据公式 (4.2)，我们需要计算 4 个基多项式。以 $L_0(x)$ 为例：

$$L_0(x) = \frac{(x-0)(x-1)(x-2)}{(-1-0)(-1-1)(-1-2)} = \frac{x(x-1)(x-2)}{-6} = -\frac{x(x-1)(x-2)}{6}.$$

类似地，其他基多项式为：

$$L_1(x) = \frac{(x+1)(x-1)(x-2)}{2}, \quad L_2(x) = -\frac{(x+1)x(x-2)}{2}, \quad L_3(x) = \frac{(x+1)x(x-1)}{6}.$$

第二步：验证基多项式的性质。我们可以验证：

- $L_0(-1) = -\dfrac{(-1)(-2)(-3)}{6} = -\dfrac{-6}{6} = 1$，$L_0(0) = 0$，$L_0(1) = 0$，$L_0(2) = 0$。
- $L_1(0) = \dfrac{(1)(-1)(-2)}{2} = 1$，$L_1(-1) = 0$，$L_1(1) = 0$，$L_1(2) = 0$。
- $L_2(1) = -\dfrac{(2)(1)(-1)}{2} = 1$，$L_2(-1) = 0$，$L_2(0) = 0$，$L_2(2) = 0$。
- $L_3(2) = \dfrac{(3)(2)(1)}{6} = 1$，$L_3(-1) = 0$，$L_3(0) = 0$，$L_3(1) = 0$。

这验证了基多项式的插值性质：$L_i(x_j) = \delta_{ij}$（Kronecker delta）。

第二步：构造插值多项式。根据公式 (4.1)，插值多项式为：

$$p_3(x) = 0.3679L_0(x) + 1 \cdot L_1(x) + 2.7183L_2(x) + 7.3891L_3(x).$$

展开并合并同类项后，得到标准形式：

$$p_3(x) = 0.3110x^3 + 0.5421x^2 + 0.8643x + 1.0.$$

第三步：验证插值条件。可以验证 $p_3(-1) \approx 0.3678 \approx e^{-1}$，$p_3(0) = 1.0 = e^0$，$p_3(1) \approx 2.7174 \approx e$，$p_3(2) \approx 7.3850 \approx e^2$，插值条件满足。

第六步：计算插值值。假设我们希望估计 $f(0.5) = e^{0.5} \approx 1.6487$。使用插值多项式：

$$p_3(0.5) \approx 0.3110(0.125) + 0.5421(0.25) + 0.8643(0.5) + 1.0 = 0.0389 + 0.1355 + 0.4322 + 1.0 = 1.6066.$$

误差为 $|e^{0.5} - p_3(0.5)| \approx |1.6487 - 1.6066| = 0.0421$。

注意事项：

- Lagrange 插值多项式的计算需要构造所有基多项式，计算量较大。
- 当插值点数量增加时，基多项式的次数也增加，计算变得更加复杂。
- 如果需要添加新的插值点，需要重新计算所有基多项式，这是 Lagrange 形式的一个缺点。

从数值实现的角度看，Lagrange 形式提供了一个结构清晰的显式公式，但在插值点数量增加或需要逐步添加新节点时，计算量和数值稳定性都并不理想。接下来，我们在误差分析与收敛性的框架下评估多项式插值的可靠性，并为后续改进方法做准备。

4.2.2 插值误差的估计

本小节的目标是：通过推导插值误差的精确表达式，理解多项式插值在什么条件下是可靠的，以及误差如何随节点分布和多项式次数的变化而变化。

设 $f : [a, b] \to \mathbb{R}$ 是一个给定的函数。我们构造多项式 $p_n(t)$，它在点 $t_0, t_1, \ldots, t_n$（$t_i \in [a, b]$）处插值 f 的值，这导致设 $p_n(t_i) = f(t_i)$，$i = 0, \ldots, n$。我们因此用多项式 p_n 逼近了函数 f。问题是：当我们用 p_n 逼近 f 时，我们犯了什么误差？

假设和记号：在以下内容中，我们假设 f 在 $[a, b]$ 上 $(n+1)$ 次连续可导，我们记 $e_n(t)$ 为插值误差，定义为：

$$e_n(t) = f(t) - p_n(t).$$

设 $\mathrm{Int}(t, t_0, \ldots, t_n)$ 为包含点 $t, t_0, \ldots, t_n$ 的最小闭区间，$\pi_n(t)$ 为定义的函数：

$$\pi_n(t) = (t - t_0)(t - t_1)(t - t_2) \cdots (t - t_n). \tag{4.4}$$

定理 4.2.2 (插值误差公式)**.** 对任意 $t \in [a, b]$，存在 $\xi_t \in \mathrm{Int}(t, t_0, \ldots, t_n)$，依赖于 t，使得

$$e_n(t) = \frac{1}{(n+1)!} f^{(n+1)}(\xi_t)\pi_n(t). \tag{4.5}$$

证明. **第一步：处理插值点的情况**。如果存在 $j \in \{0, 1, \ldots, n\}$ 使得 $t = t_j$，则根据插值条件，$p_n(t_j) = f(t_j)$，因此误差 $e_n(t_j) = f(t_j) - p_n(t_j) = 0$。另一方面，由于 $\pi_n(t) = (t - t_0)(t - t_1) \cdots (t - t_n)$，当 $t = t_j$ 时，$\pi_n(t_j) = 0$。因此，公式 (4.5) 的两边都为零，公式成立（此时可以取 $\xi_t = t_j$，虽然 $f^{(n+1)}(\xi_t)$ 的值可能不确定，但乘以零后结果仍为零）。

第二步：构造辅助函数。现在假设对任意 $j \in \{0, 1, \ldots, n\}$，都有 $t \neq t_j$。在这种情况下，$\pi_n(t) \neq 0$（因为 $\pi_n(t)$ 是 $n+1$ 个非零因子的乘积）。我们定义一个辅助函数 $g_n : \mathrm{Int}(t, t_0, \ldots, t_n) \to \mathbb{R}$：

$$g_n(z) = f(z) - p_n(z) - \frac{e_n(t)}{\pi_n(t)}\pi_n(z). \tag{4.6}$$

第三步：分析辅助函数的零点。我们需要验证 $g_n(z)$ 在 $n+2$ 个点处为零：

- **在点 t 处**：由于 $e_n(t) = f(t) - p_n(t)$，我们有

$$g_n(t) = f(t) - p_n(t) - \frac{e_n(t)}{\pi_n(t)}\pi_n(t) = e_n(t) - e_n(t) = 0.$$

- **在插值点 t_j 处**（$j = 0, 1, \ldots, n$）：由于 $p_n(t_j) = f(t_j)$，我们有 $e_n(t_j) = 0$。同时，$\pi_n(t_j) = 0$（因为 $\pi_n(t)$ 包含因子 $(t - t_j)$）。因此，

$$g_n(t_j) = f(t_j) - p_n(t_j) - \frac{e_n(t)}{\pi_n(t)} \cdot 0 = 0 - 0 = 0.$$

因此，$g_n(z)$ 在区间 $\mathrm{Int}(t, t_0, \ldots, t_n)$ 中的 $n+2$ 个互异点处为零：点 t 和 $n+1$ 个插值点 $t_0, t_1, \ldots, t_n$。

第四步：应用 Rolle 定理。由于 f 在 $[a, b]$ 上 $(n+1)$ 次连续可导，p_n 是多项式（无限次可导），π_n 也是多项式（无限次可导），因此 g_n 在 $\mathrm{Int}(t, t_0, \ldots, t_n)$ 上 $(n+1)$ 次连续可导。

现在我们将这 $n+2$ 个零点按大小顺序排列，记为 $z_0 < z_1 < \cdots < z_{n+1}$。根据 Rolle 定理，由于 g_n 在 $[z_i, z_{i+1}]$ 上连续，在 (z_i, z_{i+1}) 内可导，且 $g_n(z_i) = g_n(z_{i+1}) = 0$，存在 $\xi_i^{(1)} \in (z_i, z_{i+1})$ 使得 $g_n'(\xi_i^{(1)}) = 0$。因此，g_n' 在区间 $\mathrm{Int}(t, t_0, \ldots, t_n)$ 中至少有 $n+1$ 个零点。

类似地，对 g_n' 应用 Rolle 定理，我们得到 g_n'' 至少有 n 个零点。继续这个过程，我们得到：

- g_n' 至少有 $n+1$ 个零点；
- g_n'' 至少有 n 个零点；
- g_n''' 至少有 $n-1$ 个零点；
- $\ldots$
- $g_n^{(n)}$ 至少有 2 个零点；
- $g_n^{(n+1)}$ 至少有 1 个零点。

第五步：计算 $g_n^{(n+1)}(z)$。记 $\xi_t \in \mathrm{Int}(t, t_0, \ldots, t_n)$ 为使得 $g_n^{(n+1)}(\xi_t) = 0$ 的点（或其中之一）。由于函数 g_n 依赖于 t，点 ξ_t 也依赖于 t。

现在我们需要计算 $g_n^{(n+1)}(z)$。对公式 (4.6) 两边求 $(n+1)$ 阶导数：

$$g_n^{(n+1)}(z) = f^{(n+1)}(z) - p_n^{(n+1)}(z) - \frac{e_n(t)}{\pi_n(t)}\pi_n^{(n+1)}(z).$$

计算各项的 $(n+1)$ 阶导数：

- $p_n^{(n+1)}(z)$：由于 p_n 是 n 次多项式，$p_n^{(n+1)}(z) = 0$（n 次多项式的 $(n+1)$ 阶导数恒为零）。
- $\pi_n^{(n+1)}(z)$：由于 $\pi_n(t) = (t - t_0)(t - t_1)\cdots(t - t_n)$，这是一个 $n+1$ 次多项式。展开后，最高次项是 t^{n+1}，系数为 1。因此，$\pi_n^{(n+1)}(z) = (n+1)!$（这是因为 t^{n+1} 的 $(n+1)$ 阶导数是 $(n+1)!$，而所有低次项的 $(n+1)$ 阶导数都为零）。

因此，

$$g_n^{(n+1)}(z) = f^{(n+1)}(z) - \frac{e_n(t)}{\pi_n(t)}(n+1)!.$$

第六步：应用零点条件。由于 $g_n^{(n+1)}(\xi_t) = 0$，我们有

$$f^{(n+1)}(\xi_t) - \frac{e_n(t)}{\pi_n(t)}(n+1)! = 0.$$

解这个方程，得到

$$\frac{e_n(t)}{\pi_n(t)}(n+1)! = f^{(n+1)}(\xi_t),$$

即

$$e_n(t) = \frac{1}{(n+1)!}f^{(n+1)}(\xi_t)\pi_n(t).$$

这完成了定理的证明。注意到 $\xi_t \in \mathrm{Int}(t, t_0, \ldots, t_n)$，且依赖于 t。 □

误差公式的说明：首先注意到，这个误差表达式只有在被插值的函数 f 是 $n+1$ 次连续可导时才有意义。如果不是这种情况，前面的误差表达式就不再定义。此外，可以通过数值实验验证，在这种情况下，用 n 次多项式插值 f 往往得不到有效的近似。这对应了数值分析中的一般规则：当要逼近的函数没有所需的正则性时，使用为正则函数构造的高阶方法通常是无效的。

还注意到，在上面定理中得到的误差表达式是所谓的误差估计。它的价值在于它给出了误差的形式，并显示了当我们增加插值点的数量时，这个误差如何变化（除了 ξ 之外）。相反，它不允许我们计算给定点 t 处的误差 $e(t)$ 的值。事实上，点 ξ 是未知的（我们只知道它存在），更糟糕的是，我们通常不知道如何计算 f 的 $(n+1)$ 阶导数。事实上，如果我们逼近 f，那是因为我们不知道如何计算它，或者至少它的计算成本非常高。这通常排除了以可承受的成本计算 $f^{(n+1)}$ 的任何可能性。

从误差与稳定性的角度看，这一结果表明：即使理论上可以通过增加节点数减小误差，上述条件一旦不满足（例如函数不够光滑、导数不易获得或节点分布不当），插值误差仍可能被显著放大，引发数值不稳定。接下来，我们将通过 Newton 形式和差商把这一误差视角与具体的插值算法联系起来。

4.2.3 Newton 基中的插值多项式

本小节的目标是：在保持多项式插值唯一性的前提下，引入一种适合逐步添加插值点、便于实际计算机实现的 Newton 插值形式，并由此改进 Lagrange 形式在计算上的局限。

我们引入了次数小于或等于 n 的多项式空间 $\mathscr{P}_n$ 的一个新基：Lagrange 基。这使我们能够通过给出插值多项式在该基中的分量来证明插值多项式的存在性。然而，Lagrange 基在计算插值多项式在一个或多个点处的值时非常不方便。标准基 $\{1, t, t^2, \ldots, t^n\}$ 也不更合适，因此我们将引入第三个基。

给定 $n+1$ 个点 $t_0, t_1, \ldots, t_n$，Newton 基定义为：

$$\{1, (t-t_0), (t-t_0)(t-t_1), \ldots, (t-t_0)(t-t_1)\cdots(t-t_{n-1})\}. \tag{4.7}$$

我们将展示如何计算次数小于或等于 n 的多项式 p_n 的系数，该多项式在点 $t_0, t_1, \ldots, t_n$ 处插值函数 f。该多项式写成：

$$p_n(t) = c_0 + c_1(t-t_0) + \cdots + c_n(t-t_0)(t-t_1)\cdots(t-t_{n-1}). \tag{4.8}$$

注意到系数 c_n 是 p_n 中 t^n 的系数。这就是所谓的 p_n 的首项系数。

用 Newton 基表示的多项式 p_n 的一个优点是：如果我们记 p_k 为截断多项式，即：

$$p_k(t) = c_0 + c_1(t-t_0) + \cdots + c_k(t-t_0)(t-t_1)\cdots(t-t_{k-1}), \quad k = 0, \ldots, n,$$

则 p_k 是在点 $t_0, \ldots, t_k$ 处插值 f 的次数小于或等于 k 的多项式。事实上，我们有

$$p_n(t) = p_k(t) + c_{k+1}(t-t_0)(t-t_1)\cdots(t-t_k) + \cdots + c_n(t-t_0)(t-t_1)\cdots(t-t_{n-1}),$$

因此 $p_k(t_0) = p_n(t_0) = f(t_0)$，$p_k(t_1) = p_n(t_1) = f(t_1)$，$\ldots$，$p_k(t_k) = p_n(t_k) = f(t_k)$。

我们因此特别有：

$$p_n(t) = p_{n-1}(t) + c_n(t-t_0)(t-t_1)\cdots(t-t_{n-1}), \tag{4.9}$$

因此如果我们知道多项式 p_{n-1} 并且添加一个插值点 t_n，系数 $c_0, \ldots, c_{n-1}$ 是相同的，只需计算系数 c_n 并在多项式 p_{n-1} 上添加项 $c_n(t-t_0)(t-t_1)\cdots(t-t_{n-1})$。这个性质使得 Newton 形式的多项式在逐步添加插值点时非常方便。

4.2.4 差商

本小节的目标是：引入差商这一递推量，作为 Newton 形式系数的统一表示，为后续算法化计算打下基础。

为了计算 Newton 形式的多项式系数，我们需要引入差商的概念。

定义 4.2.1 (差商)**.** 设函数 f 在互异点 $t_0, t_1, t_2, \ldots$ 处的值已知。我们称差商（或差分商）为以下迭代定义的

表达式：

$$f[t_0] = f(t_0), \quad f[t_1] = f(t_1), \quad f[t_2] = f(t_2), \tag{4.10}$$

$$f[t_0, t_1] = \frac{f[t_0] - f[t_1]}{t_0 - t_1}, \quad f[t_1, t_2] = \frac{f[t_1] - f[t_2]}{t_1 - t_2}, \tag{4.11}$$

$$f[t_0, t_1, t_2] = \frac{f[t_0, t_1] - f[t_1, t_2]}{t_0 - t_2}, \tag{4.12}$$

一般地，如果 X 是一个互异点的有限集合，如果 $a \notin X$ 且 $b \notin X$，如果 $a \neq b$，则

$$f[a, X, b] = \frac{f[a, X] - f[X, b]}{a - b}.$$

差商是插值理论中的一个重要概念，它提供了计算插值多项式系数的一种系统方法。

定理 4.2.3 (差商与 Newton 形式系数的关系)**.** 设函数 f 在 $n+1$ 个互异点 t_i（$0 \leq i \leq n$）处的值已知，设 p_n 为插值多项式：

$$p_n(t) = c_0 + c_1(t - t_0) + c_2(t - t_0)(t - t_1) + \cdots + c_n(t - t_0)(t - t_1)\cdots(t - t_{n-1}).$$

则

$$\forall k = 0, \ldots, n, \quad f[t_0, t_1, \ldots, t_k] = c_k, \tag{4.13}$$

且

$$e_n(t) = f[t_0, t_1, \ldots, t_n, t](t - t_0)(t - t_1)(t - t_2)\cdots(t - t_n). \tag{4.14}$$

证明. 证明通过对插值点的数量进行归纳来完成。

基础情况：

- **情况 1：** $n = 0$。如果只有一个插值点 t_0，则 p_0 是在 t_0 处插值 f 的常数多项式。根据 Newton 形式，$p_0(t) = c_0$。由于 $p_0(t_0) = f(t_0)$，我们有 $c_0 = f(t_0)$。根据差商的定义，$f[t_0] = f(t_0)$。因此，$c_0 = f[t_0]$，公式 (4.13) 对 $k = 0$ 成立。
- **情况 2：** $n = 1$。如果有两个插值点 t_0 和 t_1，则 p_1 是在 t_0, t_1 处插值 f 的次数小于或等于 1 的多项式。根据 Newton 形式，我们写成

  $$p_1(t) = c_0 + c_1(t - t_0).$$

  由插值条件 $p_1(t_0) = f(t_0)$ 和 $p_1(t_1) = f(t_1)$，我们得到：

  $$\begin{aligned} p_1(t_0) &= c_0 = f(t_0), \\ p_1(t_1) &= c_0 + c_1(t_1 - t_0) = f(t_1). \end{aligned} \tag{4.15}$$

  由第一个等式，我们得到 $c_0 = f(t_0) = f[t_0]$。由第二个等式，我们得到

  $$c_1 = \frac{f(t_1) - f(t_0)}{t_1 - t_0} = \frac{f(t_0) - f(t_1)}{t_0 - t_1} = f[t_0, t_1].$$

  因此，公式 (4.13) 对 $k = 0, 1$ 都成立。

归纳步骤：现在假设公式对在 $k+1$ 个点处插值的多项式成立（$k \geq 1$）。具体地，假设 p_k 是在 $t_0, \ldots, t_k$ 处插值 f 的次数小于或等于 k 的多项式，且它在 Newton 基中的表示为

$$p_k(t) = f[t_0] + f[t_0, t_1](t - t_0) + \cdots + f[t_0, \ldots, t_k](t - t_0)(t - t_1)\cdots(t - t_{k-1}). \tag{4.16}$$

第一步：构造另一个插值多项式。同样，考虑在 $t_1, \ldots, t_{k+1}$ 处插值 f 的次数小于或等于 k 的多项式 q_k。根据归纳假设（将点重新编号为 $t_1, \ldots, t_{k+1}$），q_k 在 Newton 基中的表示为

$$q_k(t) = f[t_1] + f[t_1, t_2](t - t_1) + \cdots + f[t_1, \ldots, t_{k+1}](t - t_1)(t - t_2)\cdots(t - t_k). \tag{4.17}$$

（注意：这里我们使用了归纳假设，即对于 $k+1$ 个点的情况，公式成立。由于 q_k 是在 $k+1$ 个点 $t_1,\ldots,t_{k+1}$ 处插值的 k 次多项式，根据归纳假设，它的系数确实是 $f[t_1], f[t_1,t_2],\ldots,f[t_1,\ldots,t_{k+1}]$。）

第二步：构造 p_{k+1} 的候选多项式。我们希望通过 p_k 和 q_k 构造在 $t_0,t_1,\ldots,t_{k+1}$ 处插值 f 的次数小于或等于 $k+1$ 的多项式 p_{k+1}。考虑以下形式的线性组合：

$$p(t)=\alpha(t)p_k(t)+\beta(t)q_k(t),$$

其中 $\alpha(t)$ 和 $\beta(t)$ 是待定的函数。我们希望 $p(t)$ 满足插值条件：

$$\begin{aligned} p(t_0)&=f(t_0),\\ p(t_1)&=f(t_1),\\ &\vdots\\ p(t_{k+1})&=f(t_{k+1}). \end{aligned} \tag{4.18}$$

第三步：确定 $\alpha(t)$ 和 $\beta(t)$。为了使 $p(t)$ 在 t_0 处等于 $f(t_0)$，我们希望 $\alpha(t_0)=1$ 且 $\beta(t_0)=0$（因为 $p_k(t_0)=f(t_0)$ 但 $q_k(t_0)$ 不一定等于 $f(t_0)$）。类似地，为了使 $p(t)$ 在 t_{k+1} 处等于 $f(t_{k+1})$，我们希望 $\alpha(t_{k+1})=0$ 且 $\beta(t_{k+1})=1$。

一个简单的选择是：

$$\alpha(t)=\frac{t-t_{k+1}}{t_0-t_{k+1}},\quad \beta(t)=\frac{t-t_0}{t_{k+1}-t_0}=-\frac{t-t_0}{t_0-t_{k+1}}.$$

验证：$\alpha(t_0)=\dfrac{t_0-t_{k+1}}{t_0-t_{k+1}}=1$，$\alpha(t_{k+1})=0$；$\beta(t_0)=0$，$\beta(t_{k+1})=1$。

因此，我们定义多项式 p 为：

$$p(t)=\frac{(t-t_{k+1})p_k(t)-(t-t_0)q_k(t)}{t_0-t_{k+1}}. \tag{4.19}$$

第四步：验证 p 是 p_{k+1}。我们需要验证 p 满足以下条件：

1. p 是次数小于或等于 $k+1$ 的多项式；
2. $p(t_i)=f(t_i)$ 对所有 $i=0,1,\ldots,k+1$ 成立。

验证条件 1：由于 p_k 是 k 次多项式，$(t-t_{k+1})p_k(t)$ 是 $k+1$ 次多项式；类似地，$(t-t_0)q_k(t)$ 也是 $k+1$ 次多项式。因此，$p(t)$ 是次数小于或等于 $k+1$ 的多项式。

验证条件 2：我们逐一验证插值条件：

- **在 t_0 处**：
$$p(t_0)=\frac{(t_0-t_{k+1})p_k(t_0)-(t_0-t_0)q_k(t_0)}{t_0-t_{k+1}}=\frac{(t_0-t_{k+1})f(t_0)-0}{t_0-t_{k+1}}=f(t_0).$$
- **在 t_{k+1} 处**：
$$p(t_{k+1})=\frac{(t_{k+1}-t_{k+1})p_k(t_{k+1})-(t_{k+1}-t_0)q_k(t_{k+1})}{t_0-t_{k+1}}=\frac{0-(t_{k+1}-t_0)f(t_{k+1})}{t_0-t_{k+1}}=f(t_{k+1}).$$
- **在 t_i 处**（$i=1,\ldots,k$）：由于 $p_k(t_i)=f(t_i)$ 且 $q_k(t_i)=f(t_i)$，我们有
$$\begin{aligned} p(t_i)&=\frac{(t_i-t_{k+1})p_k(t_i)-(t_i-t_0)q_k(t_i)}{t_0-t_{k+1}}\\ &=\frac{(t_i-t_{k+1})f(t_i)-(t_i-t_0)f(t_i)}{t_0-t_{k+1}}\\ &=\frac{[(t_i-t_{k+1})-(t_i-t_0)]f(t_i)}{t_0-t_{k+1}}\\ &=\frac{(t_0-t_{k+1})f(t_i)}{t_0-t_{k+1}}\\ &=f(t_i). \end{aligned} \tag{4.20}$$

因此，p 确实是在 $t_0,t_1,\ldots,t_{k+1}$ 处插值 f 的次数小于或等于 $k+1$ 的多项式。根据插值多项式的唯一性（定理 4.2.1），我们有 $p=p_{k+1}$。

第五步：计算 p_{k+1} 的首项系数。现在我们需要计算 p_{k+1} 在 Newton 基中的系数 c_{k+1}，即 t^{k+1} 的系数。

根据归纳假设（公式 (4.16)），$p_k(t)$ 的首项（t^k 的项）的系数是 $f[t_0,\ldots,t_k]$。因此，$(t-t_{k+1})p_k(t)$ 的首项（t^{k+1} 的项）的系数也是 $f[t_0,\ldots,t_k]$（因为 $t\times t^k=t^{k+1}$，且 t^k 的系数是 $f[t_0,\ldots,t_k]$）。

类似地，根据归纳假设（公式 (4.17)），$q_k(t)$ 的首项（t^k 的项）的系数是 $f[t_1,\ldots,t_{k+1}]$。因此，$(t-t_0)q_k(t)$ 的首项（t^{k+1} 的项）的系数也是 $f[t_1,\ldots,t_{k+1}]$。

因此，$p(t)$ 的首项（t^{k+1} 的项）的系数是

$$\frac{f[t_0,\ldots,t_k]-f[t_1,\ldots,t_{k+1}]}{t_0-t_{k+1}}.$$

第六步：应用 Newton 形式的递归关系。另一方面，根据 Newton 形式的递归关系（公式 (4.9)），我们有

$$p_{k+1}(t)=p_k(t)+c_{k+1}(t-t_0)(t-t_1)\cdots(t-t_k).$$

展开 $(t-t_0)(t-t_1)\cdots(t-t_k)$，其首项是 t^{k+1}，系数为 1。因此，$p_{k+1}(t)$ 的首项（t^{k+1} 的项）的系数是 c_{k+1}。由于 $p=p_{k+1}$，我们得到

$$c_{k+1}=\frac{f[t_0,\ldots,t_k]-f[t_1,\ldots,t_{k+1}]}{t_0-t_{k+1}}=f[t_0,\ldots,t_{k+1}],$$

这里最后一个等式是差商的定义。

第七步：完成归纳。因此，根据归纳假设和 Newton 形式的递归关系，我们有

$$p_{k+1}(t)=f[t_0]+f[t_0,t_1](t-t_0)+\cdots+f[t_0,\ldots,t_{k+1}](t-t_0)(t-t_1)\cdots(t-t_k).$$

这完成了对公式 (4.13) 的归纳证明。

关于误差公式的证明：公式 (4.14) 的证明可以通过考虑在 $t_0,\ldots,t_n,t$ 处插值 f 的 $n+1$ 次多项式 p_{n+1} 来完成。由于 $p_{n+1}(t)=f(t)$，且 p_{n+1} 和 p_n 在 $t_0,\ldots,t_n$ 处相等，我们有

$$f(t)-p_n(t)=p_{n+1}(t)-p_n(t)=f[t_0,\ldots,t_n,t](t-t_0)(t-t_1)\cdots(t-t_n),$$

这给出了误差公式。详细证明从略。

□

定理 4.2.4 (差商的对称性)**.** 差商 $f[t_0,t_1,\ldots,t_k]$ 是其自变量的对称函数。

证明. **第一步：理解差商的含义**。根据定理 4.2.3，差商 $f[t_0,t_1,\ldots,t_k]$ 等于在点 $\{t_0,t_1,\ldots,t_k\}$ 处插值 f 的次数小于或等于 k 的多项式 p_k 在 Newton 基中的系数 c_k，即 p_k 的首项（t^k 的项）的系数。

第二步：应用插值多项式的唯一性。根据定理 4.2.1（多项式插值的存在唯一性），在点 $\{t_0,t_1,\ldots,t_k\}$ 处插值 f 的次数小于或等于 k 的多项式是唯一的。这个唯一性不依赖于我们给点编号的顺序，只依赖于点的集合 $\{t_0,t_1,\ldots,t_k\}$ 和函数值 $\{f(t_0),f(t_1),\ldots,f(t_k)\}$。

第三步：考虑点的重新编号。设 $\sigma:\{0,1,\ldots,k\}\to\{0,1,\ldots,k\}$ 是任意一个置换（双射）。考虑重新编号后的点 $t_{\sigma(0)},t_{\sigma(1)},\ldots,t_{\sigma(k)}$。根据定理 4.2.1，存在唯一的多项式 $q_k\in\mathscr{P}_k$，使得

$$q_k(t_{\sigma(i)})=f(t_{\sigma(i)}),\quad i=0,1,\ldots,k.$$

由于 σ 是双射，这等价于

$$q_k(t_i)=f(t_i),\quad i=0,1,\ldots,k.$$

第四步：应用唯一性。由于 p_k 和 q_k 都在点 $\{t_0,t_1,\ldots,t_k\}$ 处插值 f，且都属于 $\mathscr{P}_k$，根据插值多项式的唯一性，我们有 $p_k=q_k$。

第五步：比较首项系数。现在，p_k 和 q_k 是同一个多项式，因此它们的首项系数必须相等。但是，我们需要小心：p_k 的首项系数是在 Newton 基 $\{1,(t-t_0),(t-t_0)(t-t_1),\ldots\}$ 中定义的，而 q_k 的首项系数是在 Newton 基 $\{1,(t-t_{\sigma(0)}),(t-t_{\sigma(0)})(t-t_{\sigma(1)}),\ldots\}$ 中定义的。

第六步：统一基的表示。为了比较首项系数，我们需要将 p_k 和 q_k 都表示为标准形式（按 t 的幂次排列）。由于 $p_k = q_k$，它们在标准基 $\{1, t, t^2, \ldots, t^k\}$ 中的表示是相同的。特别地，t^k 的系数必须相等。

第七步：计算首项系数。根据定理 4.2.3，p_k 在 Newton 基中的系数 c_k（对应基函数 $(t-t_0)(t-t_1)\cdots(t-t_{k-1})$）等于 $f[t_0, \ldots, t_k]$。展开这个基函数，t^k 的系数是 1，因此 p_k 在标准基中 t^k 的系数也是 $f[t_0, \ldots, t_k]$。

类似地，q_k 在 Newton 基中的系数（对应基函数 $(t-t_{\sigma(0)})(t-t_{\sigma(1)})\cdots(t-t_{\sigma(k-1)})$）等于 $f[t_{\sigma(0)}, \ldots, t_{\sigma(k)}]$。展开这个基函数，$t^k$ 的系数也是 1，因此 q_k 在标准基中 t^k 的系数也是 $f[t_{\sigma(0)}, \ldots, t_{\sigma(k)}]$。

第八步：得出结论。由于 $p_k = q_k$，它们在标准基中 t^k 的系数必须相等，因此

$$f[t_{\sigma(0)}, t_{\sigma(1)}, \ldots, t_{\sigma(k)}] = f[t_0, t_1, \ldots, t_k].$$

由于 σ 是任意置换，这证明了差商是其自变量的对称函数。 □

4.2.5 差商的实际计算

本小节的目标是：把差商的递推定义转化为具体的表格和算法形式，使 Newton 插值多项式的系数计算更适合程序实现与复杂度分析。

回顾定义 4.2.1 中给出的差商表达式：

$$f[t_i] = f(t_i), \quad f[t_0, t_1, \ldots, t_k] = \frac{f[t_0, t_1, \ldots, t_{k-1}] - f[t_1, t_2, \ldots, t_k]}{t_0 - t_k}.$$

使用这个公式可以逐步计算 $f[t_0, t_1, \ldots, t_k]$，使用下表（每一列从前一列推导）：

$k=0$	$k=1$	$k=2$	$\cdots$	$k=n$
$f[t_0]$				
$f[t_1]$	$f[t_0, t_1]$			
$f[t_2]$	$f[t_1, t_2]$	$f[t_0, t_1, t_2]$		
$\vdots$	$\vdots$	$\vdots$	$\ddots$	
$f[t_n]$	$f[t_{n-1}, t_n]$	$f[t_{n-2}, t_{n-1}, t_n]$	$\cdots$	$f[t_0, t_1, \ldots, t_n]$

计算特点：

- 我们看到，确定系数 c_k 需要 $\dfrac{n(n+1)}{2}$ 次除法。
- 我们还看到，我们在表的对角线上得到系数 $f[t_0]$、$f[t_0, t_1]$、$f[t_0, t_1, t_2]$，最后是 $f[t_0, t_1, \ldots, t_n]$，即 p_n 在 Newton 基中的系数 c_0, c_1, c_2，最后是 c_n。

以下是计算差商 d_i（$0 \le i \le n$）的简洁算法：

Algorithm 14 计算差商

Require: 函数值 $f(t_i)$，$i = 0, \ldots, n$ 和插值点 t_i，$i = 0, \ldots, n$

Ensure: 差商 $d_i = f[t_0, \ldots, t_i]$，$i = 0, \ldots, n$

```
for i = 0 到 n do
    d_i ← f(t_i)
end for
for k = 1 到 n do
    for i = n 到 k 步长 −1 do
        d_i ← (d_i − d_{i−1})/(t_i − t_{i−k})
    end for
end for
```

注意到，算法结束时的元素 d_i 确实对应于在 Newton 基中写出的插值多项式的系数 c_i。

从计算的角度看，这一差商算法把抽象的递推定义变成了一种内存局部性良好、复杂度清晰的一维更新过程。接下来，通过一个具体例子，我们将把差商表与 Newton 插值多项式的实际构造过程联系起来。

例 4.2.2 (Newton 插值多项式的详细计算：差商与插值). 考虑函数 $f(x) = \ln(x+2)$，已知其在以下 5 个点处的值：

x	-1	0	1	2	3
$f(x)$	$\ln(1) = 0$	$\ln(2) \approx 0.6931$	$\ln(3) \approx 1.0986$	$\ln(4) \approx 1.3863$	$\ln(5) \approx 1.6094$

我们希望构造一个 4 次 Newton 插值多项式 $p_4(x)$，使得 $p_4(x_i) = f(x_i)$ 对所有 $i = 0, 1, 2, 3, 4$ 成立。

第一步：计算差商表。根据算法 14，我们逐步计算差商。首先初始化：

$$d_0 = f(-1) = 0, \quad d_1 = f(0) = 0.6931, \quad d_2 = f(1) = 1.0986, \quad d_3 = f(2) = 1.3863, \quad d_4 = f(3) = 1.6094.$$

第二步：计算差商表。逐步计算各阶差商，结果如下表：

$k=0$	$k=1$	$k=2$	$k=3$	$k=4$
0				
0.6931	0.6931			
1.0986	0.4055	-0.1438		
1.3863	0.2877	-0.0589	0.0283	
1.6094	0.2231	-0.0323	0.0089	-0.00485

计算过程：一阶差商如 $f[-1,0] = (0.6931 - 0)/(0 - (-1)) = 0.6931$；二阶差商如 $f[-1,0,1] = (0.6931 - 0.4055)/(-1-1) = -0.1438$；依此类推。

第三步：构造 Newton 插值多项式。根据定理 4.2.3，Newton 插值多项式为：

$$p_4(x) = 0.6931(x+1) - 0.1438(x+1)x + 0.0283(x+1)x(x-1) - 0.00485(x+1)x(x-1)(x-2).$$

第四步：验证插值条件。可以验证 $p_4(-1) = 0$，$p_4(0) = 0.6931$，$p_4(1) = 1.0986$，$p_4(2) = 1.3863$，$p_4(3) \approx 1.6096$，插值条件满足。

第八步：逐步添加插值点的优势。假设我们现在需要添加一个新的插值点 $x_5 = 4$，$f(4) = \ln(6) \approx 1.7918$。根据 Newton 形式的递归性质（公式 (4.9)），我们只需要计算新的差商 $f[-1,0,1,2,3,4]$，而不需要重新计算所有系数。

首先计算一阶差商 $f[3,4]$：

$$f[3,4] = \frac{f(4) - f(3)}{4-3} = \frac{1.7918 - 1.6094}{1} = 0.1824.$$

然后计算二阶差商 $f[2,3,4]$：

$$f[2,3,4] = \frac{f[2,3] - f[3,4]}{2-4} = \frac{0.2231 - 0.1824}{-2} = -0.02035.$$

继续计算更高阶差商，最终得到：

$$f[-1,0,1,2,3,4] = -0.00162.$$

因此，新的 5 次插值多项式为：

$$p_5(x) = p_4(x) - 0.00162(x+1)x(x-1)(x-2)(x-3).$$

Newton 形式的优势：

- 当添加新的插值点时，只需要计算新的差商，而不需要重新计算所有系数。
- 差商的计算可以逐步进行，每一步都利用前一步的结果。
- 差商表的结构清晰，便于理解和实现。

4.2.6 Horner 算法

本小节的目标是：利用 Horner 思想高效计算 Newton 形式多项式的函数值，并比较这种实现与直接展开计算在运算量和数值稳定性上的差异。

首先要注意，形式

$$p_n(t) = c_0 + c_1(t-t_0) + c_2(t-t_0)(t-t_1) + \cdots + c_n(t-t_0)(t-t_1)\cdots(t-t_{n-1})$$

非常适合使用 Horner 方案进行数值计算：

Algorithm 15 Horner 算法计算 Newton 形式的多项式

Require: 系数 $c_0, \ldots, c_n$，插值点 $t_0, \ldots, t_{n-1}$，以及要计算的点 t

Ensure: 多项式值 $p_n(t)$

```
p ← c_n
for k = n − 1 到 0 步长 −1 do
    p ← c_k + (t − t_k) × p
end for
```

例如，对于 $n=3$，我们从最内层的括号开始依次计算每个括号：

$$p_3(t) = c_0 + (t-t_0)[c_1 + (t-t_1)[c_2 + (t-t_2)[c_3]]].$$

经典写法会给出

$$p_3(t) = c_0 + c_1(t-t_0) + c_2(t-t_0)(t-t_1) + c_3(t-t_0)(t-t_1)(t-t_2),$$

这对应于 5 次乘法（$c_1(t-t_0)$ 需要 1 次乘法，$c_2(t-t_0)(t-t_1)$ 需要 2 次乘法，$c_3(t-t_0)(t-t_1)(t-t_2)$ 需要 3 次乘法，总共 6 次乘法，但 c_0 不需要乘法，所以是 5 次）。

在练习中，我们将证明，一般来说，对于 p_n 的计算，Horner 算法包含 n 次乘法，而经典计算包含 $2n-1$ 次乘法。Horner 算法的优势在于它减少了计算量，特别是在需要多次计算多项式值的情况下。

例 4.2.3 (Horner 算法计算 Newton 形式的多项式)**.** 考虑例 4.2.2 中构造的 Newton 插值多项式 $p_4(x)$：

$$p_4(x) = 0.6931(x+1) - 0.1438(x+1)x + 0.0283(x+1)x(x-1) - 0.00485(x+1)x(x-1)(x-2).$$

我们希望计算 $p_4(1.5)$ 的值。根据算法 15，我们使用 Horner 方案。

第一步：识别系数和插值点。Newton 形式的系数为：

$$c_0 = 0, \quad c_1 = 0.6931, \quad c_2 = -0.1438, \quad c_3 = 0.0283, \quad c_4 = -0.00485.$$

插值点为：$t_0 = -1$，$t_1 = 0$，$t_2 = 1$，$t_3 = 2$。

第二步：应用 Horner 算法。从最内层开始逐步计算：

$$p \leftarrow -0.00485, \quad p \leftarrow 0.0283 + 0.5 \times (-0.00485) = 0.025875,$$

$$p \leftarrow -0.1438 + 1.5 \times 0.025875 = -0.1049875, \quad p \leftarrow 0.6931 + 2.5 \times (-0.1049875) = 0.4306.$$

因此，$p_4(1.5) \approx 0.4306$。

第三步：比较计算复杂度。Horner 算法需要 $n=4$ 次乘法，而经典方法需要 $2n-1=7$ 次乘法（如果存储中间结果）。

Horner 算法的优势：

- 计算量更少：只需要 n 次乘法，而经典方法需要 $2n-1$ 次乘法。
- 数值稳定性更好：减少了中间计算步骤，降低了舍入误差的累积。
- 代码实现简单：算法结构清晰，易于编程实现。

通过差商表与 Horner 算法的配合，我们已经得到了一条从“给定数据点”到“高效评估插值多项式”的完整计算路径。接下来，我们将结合误差公式，通过具体例子考察插值误差的大小和分布，从而更直观地理解多项式插值在实际应用中的效果与局限。

例 4.2.4 (插值误差的估计与应用)**.** 考虑函数 $f(x) = \sin x$，已知其在区间 $[0, \pi]$ 上的以下 4 个等距点处的值：

x	0	$\pi/3$	$2\pi/3$	π
$f(x)$	0	$\sqrt{3}/2 \approx 0.8660$	$\sqrt{3}/2 \approx 0.8660$	0

我们希望估计在 $x = \pi/6$ 处的插值误差。

第一步：构造插值多项式。使用 Newton 插值方法（过程从略），我们得到 3 次插值多项式 $p_3(x)$。

第二步：应用误差公式。根据定理 4.2.2，插值误差为：

$$e_3(x) = f(x) - p_3(x) = \frac{1}{4!} f^{(4)}(\xi_x)\pi_3(x),$$

其中 $\pi_3(x) = (x-0)(x-\pi/3)(x-2\pi/3)(x-\pi)$，$\xi_x \in \mathrm{Int}(x, 0, \pi/3, 2\pi/3, \pi)$。

第三步：估计误差上界。对于 $f(x) = \sin x$，有 $f^{(4)}(x) = \sin x$，因此 $|f^{(4)}(x)| \le 1$。在 $x = \pi/6$ 处，$\pi_3(\pi/6) = -5\pi^4/1296$，因此误差上界为：

$$|e_3(\pi/6)| \le \frac{1}{24} \cdot 1 \cdot \frac{5\pi^4}{1296} = \frac{5\pi^4}{31104} \approx 0.0157.$$

第四步：实际误差。实际计算 $f(\pi/6) = \sin(\pi/6) = 0.5$，而 $p_3(\pi/6) \approx 0.5$（由于对称性，3 次插值多项式在 $\pi/6$ 处的值应该非常接近真实值）。实际误差远小于上界，这是因为误差公式给出的是最坏情况的估计。

第五步：误差分析的意义。

- 误差公式告诉我们误差的形式：$\dfrac{1}{(n+1)!} f^{(n+1)}(\xi)\pi_n(x)$。
- 当插值点数量增加时，$(n+1)!$ 增大，这有助于减小误差。
- 但是，$\pi_n(x)$ 的大小也依赖于插值点的分布。对于等距节点，$\pi_n(x)$ 在区间端点附近可能很大，导致误差增大（Runge 现象）。
- 误差公式中的 ξ 是未知的，我们只能估计误差的上界，而不能精确计算误差。

应用建议：

- 对于光滑函数，增加插值点数量通常能提高精度。
- 对于等距节点，在区间端点附近误差可能较大，应考虑使用非等距节点（如 Chebyshev 节点）。
- 误差公式可以帮助我们选择合适的插值点数量和分布。

4.2.7　插值多项式的收敛性

本小节的目标是:利用前面得到的误差表达式分析多项式插值随节点数增加时的收敛或发散行为,并通过 Runge 现象揭示多项式插值方法在某些情形下的失败机制。

误差表达式：

$$e_n(t) = \frac{1}{(n+1)!}\pi_n(t) f^{(n+1)}(\xi),$$

之前得到的这个表达式已经允许我们在一些有利的情况下获得有趣的结果。

设 $[a, b]$ 是包含所有插值点 t_i 的区间。我们有：

$$E_n = \sup_{t\in[a,b]} |e_n(t)| \le \frac{1}{(n+1)!} \sup_{t\in[a,b]} |\pi_n(t)| \sup_{t\in[a,b]} |f^{(n+1)}(t)|.$$

由此得出上界：

$$E_n \le \frac{(b-a)^{n+1}}{(n+1)!} \sup_{t\in[a,b]} |f^{(n+1)}(t)|.$$

由于

$$\lim_{n\to\infty}\frac{(b-a)^{n+1}}{(n+1)!}=0,$$

因此对于所有 $f\in C^\infty([a,b])$ 且其导数在 n 上一致有界的函数，有 $\lim\limits_{n\to\infty}E_n=0$。这类函数相当有限，但包含像 e^t、$\sin t$、$\cos t$ 这样的函数。

收敛性的限制：然而，需要注意的是，即使函数 f 是无限次可导的，插值多项式也不一定一致收敛到 f。一个经典的例子是 Runge 现象：对于函数

$$f(t)=\frac{1}{1+t^2}$$

在区间 $[-5,5]$ 上进行等距节点插值时，当插值点的数量增加时，插值多项式在区间端点附近会出现剧烈的振荡，导致插值多项式不一致收敛到 f。这表明，即使函数非常光滑，等距节点的多项式插值也可能失败。

这个例子说明了多项式插值的一个重要局限性：虽然理论上我们可以通过增加插值点的数量来提高精度，但在实践中，如果插值点的选择不当（例如等距节点），插值多项式可能不会收敛，甚至可能发散。因此，在实际应用中，需要谨慎选择插值点的位置，或者使用其他插值方法（如样条插值）来避免这些问题。

从更高一层的视角看，本小节的结果表明：单纯依赖高次全局多项式不足以解决所有逼近问题，节点分布与函数特性共同决定了插值的成败。在下一节中，我们将顺势引入三次样条插值，通过分段多项式的局部性来缓解多项式插值在收敛性和稳定性方面的困难。

4.3 三次样条插值 *

4.3.1 问题的提出

多项式插值虽然具有许多优良的性质，但在实际应用中存在几个明显的缺点：

第一个缺点：对陡峭函数的处理能力差。当被插值函数在某些区域变化非常剧烈（即导数很大）时，高次多项式插值往往会在这些区域产生剧烈的振荡。这是因为多项式是全局函数，一个局部区域的剧烈变化会影响整个定义域上的插值结果。

第二个缺点：收敛性较差。虽然理论上，对于某些非常光滑的函数（如指数函数、三角函数），当插值点数量趋于无穷时，插值多项式会一致收敛到被插值函数。但在实践中，对于许多常见的函数，即使增加插值点的数量，误差的改善也是有限的。特别地，当插值点的数量超过某个阈值后，进一步增加插值点可能反而使误差增大，这就是所谓的 Runge 现象。

第三个缺点：数值稳定性问题。高次多项式的系数对插值点的微小扰动非常敏感，这导致插值多项式在数值计算中可能不稳定。

问题的根源：多项式的刚性。从定性的角度来看，这些困难源于多项式的刚性过大。多项式的刚性是指它在整个定义域上具有高度的相关性：一个局部区域的特性会影响整个函数的行为。由于这种刚性，从 0 到 1 的突然跳跃会在整个定义区间上产生振荡，而不是仅仅局限在跳跃发生的区域。

解决思路：分段多项式。为了克服多项式插值的这些缺点，我们必须使用刚性较小的近似函数。换句话说，我们将要求近似函数具有较少的全局正则性。除了多项式的良好性质（如光滑性、易于计算）外，这将引导我们使用分段多项式。

具体来说，在接下来的内容中，我们考虑区间 $[a,b]$ 的一个分割：

$$\Delta=\{a=t_0<t_1<t_2<\cdots<t_{n-1}<t_n=b\},\tag{4.21}$$

我们称分割点 t_i 为节点。假设给定一个实数族 $\{y_0,\ldots,y_n\}$，它们对应插值值。一个相当自然的想法是分别考虑每个子区间 $[t_i,t_{i+1}]$ 并构造分段近似：这样我们得到一个满足插值条件 $g(t_i)=y_i$ 的函数 g，它在每个区间 $[t_i,t_{i+1}]$ 内与一个次数或多或少较高的多项式 $g_i(t)$ 重合。

最简单的分段插值：分段线性插值。在选择 $g_i(t)$ 为 1 次的情况下，这种技术包括用直线段连接点 (t_i,y_i)：这是使用所谓的样条函数的最基本方式。在这种情况下，方程 $y=g(t)$ 的曲线是一条折线，因此有角点，函数 g 在横坐标 t_i 处不可导。虽然分段线性插值简单直观，但它缺乏光滑性，这在许多应用中是不理想的。

更高次数的分段多项式。我们清楚地感觉到，如果采用更高次数的多项式，$g_i(t)$ 的选择就不再唯一。这个想法是利用这些额外的自由度来确保 g 在点 t_i 处具有一定的正则性。例如，可以要求 g 一次或多次连续可导。在本课程中，我们将限制在这些多项式为 3 次的情况，即三次样条插值。选择三次多项式的原因在于：三次多项式是能够保证二阶导数连续的最低次数的多项式，而二阶导数的连续性在许多物理和工程应用中具有重要意义（例如，在曲线设计中，二阶导数与曲率相关）。

4.3.2　三次样条插值的定义

定义 4.3.1 (三次样条函数与三次样条插值函数). 设 $a = x_0 < x_1 < \cdots < x_n = b$，若函数 $S(x)$ 满足条件：

1. $S''(x) \in C[a,b]$（S 的二阶导数连续），
2. 在每个小区间 $[x_k, x_{k+1}]$（$k = 0,1,\ldots,n-1$）上，$S(x)$ 为三次多项式，

则称 $S(x)$ 为关于节点 $x_0, x_1, \ldots, x_n$ 的**三次样条函数**。

若给定 $f_j = f(x_j)$（$j = 0,1,\ldots,n$），且三次样条函数 $S(x)$ 满足 $S(x_j) = f_j$（$j = 0,1,\ldots,n$），则称 $S(x)$ 为 $f(x)$ 的**三次样条插值函数**。

定义的解释：定义中的三次样条函数可以看成是一个新的函数类，其中的函数由分段三次多项式 “拼接” 而成，并且具有整体的 2 阶光滑性。事实上，由于各个小区间内是三次多项式，二阶导数显然连续，因此整体光滑性的要求主要针对节点处：在内部节点 x_j（$j = 1,2,\ldots,n-1$）处，左右两边的函数值、一阶导数值和二阶导数值必须相等，即

$$S(x_j - 0) = S(x_j + 0), \quad S'(x_j - 0) = S'(x_j + 0), \quad S''(x_j - 0) = S''(x_j + 0).$$

“样条”（spline）一词源自早期工程师绘图所用的薄木条，将它固定在一些给定的数据点上，可绘出一条连接各点的光滑曲线。从物理上讲，样条满足插值点约束，同时使势能达到最小，因此对应的曲线必然有连续的二阶导数。在数学上，这就是应用广泛的三次样条插值函数。

唯一性分析：下面分析定义 4.3.1 是否能唯一地确定三次样条插值函数。假设共有 $n+1$ 个插值节点 $x_0 < x_1 < \cdots < x_n$，它们形成 n 个小区间 $[x_j, x_{j+1}]$（$j = 0,1,\ldots,n-1$）。每个小区间为三次多项式，表示插值函数 $S(x)$ 共需 $4n$ 个系数。这些系数满足的条件有以下两种：

条件类型一：插值条件。对每个小区间，两端处函数值都知道，共 $2n$ 个条件，它们也保证了插值函数整体上是连续的。

条件类型二：节点处的连续性条件。在中间节点 x_j（$j = 1,2,\ldots,n-1$）处，一阶导数连续、二阶导数连续，即

$$S'(x_j - 0) = S'(x_j + 0), \quad S''(x_j - 0) = S''(x_j + 0),$$

它对应 $2(n-1)$ 个条件。

因此，根据上述条件一共可列出 $2n + 2(n-1) = 4n - 2$ 个方程，无法唯一确定 $4n$ 个系数。在实际问题中，通常在数据区间的两端点处增加一些条件，称为**边界条件**。

边界条件。常用的边界条件有以下 4 种：

1. **已知两端点处的一阶导数**：
$$S'(x_0) = f_0', \quad S'(x_n) = f_n',$$
其中 f_0' 和 f_n' 是给定的值。这个条件适用于我们知道函数在端点处的导数值的情况。
2. **已知两端点处的二阶导数**：
$$S''(x_0) = f_0'', \quad S''(x_n) = f_n''.$$
若 $f_0'' = f_n'' = 0$，通常称之为**自然边界条件**，对应的样条称为**自然三次样条**。这个条件称为 “自然” 是因为它使得样条在端点处的曲率为零，这在某些物理问题中具有自然的意义。
3. **周期边界条件**：假定被插值函数 $f(x)$ 是以 $x_n - x_0$ 为周期的周期函数（同时要求 $f_0 = f_n$），则要求 $S(x)$ 也是相同周期的二阶导数连续的函数，即
$$S'(x_0) = S'(x_n), \quad S''(x_0) = S''(x_n).$$

这个条件适用于周期函数的情况。

4. **非节点（not-a-knot）条件**：假定在区间 $[x_0, x_2]$ 上使用统一的三次多项式，在区间 $[x_{n-2}, x_n]$ 上使用统一的三次多项式（$n \geq 3$）。这意味着在 x_1 和 x_{n-1} 处，三阶导数也连续。这是 MATLAB 软件中 `spline` 命令采用的边界条件，它不需要额外设置参数，也不假设被插值函数具有周期性。

易知上述 4 种边界条件的任何一个都带来两个方程，因此使求解三次样条插值时的方程数与变量数匹配。对于给定的边界条件，三次样条插值函数存在并且唯一（这将在后续的构造方法中说明）。

4.3.3 三次样条插值的构造方法

为了构造三次样条插值函数，我们需要求解一个线性方程组来确定所有系数。虽然可以直接求解这个 $4n$ 阶的线性方程组，但这种方法计算量大且数值稳定性差。更有效的方法是使用所谓的 “三对角方程组” 方法，它利用了样条函数的特殊结构。

方法思想：我们不是直接求解所有系数 a_i, b_i, c_i, d_i，而是先求解二阶导数在节点处的值 $M_i = g''(t_i)$（$i = 0, 1, \ldots, n$），然后利用这些值来确定每个子区间上的三次多项式系数。

第一步：建立样条函数的表达式。求三次样条插值函数的常用方法是以插值节点的二阶导数值作为参数构造每个小区间上的表达式，然后利用已知条件列出这些参数满足的线性方程组并求解。

设 $S''(x_j) = M_j$（$j = 0, 1, \ldots, n$）。由于在各小区间上 $S''(x)$ 为一次多项式，则

$$S''(x) = M_j \frac{x_{j+1} - x}{h_j} + M_{j+1} \frac{x - x_j}{h_j}, \quad x \in [x_j, x_{j+1}], \quad (j = 0, 1, \ldots, n-1),$$

其中 $h_j = x_{j+1} - x_j$。

对上式做两次积分，得

$$S(x) = -\frac{M_j}{6h_j}(x - x_{j+1})^3 + \frac{M_{j+1}}{6h_j}(x - x_j)^3 + a_j x + b_j, \quad x \in [x_j, x_{j+1}],$$

其中 a_j 和 b_j 是积分常数。

第二步：确定积分常数。再根据 $S(x_j) = f_j$，$S(x_{j+1}) = f_{j+1}$ 定常数 a_j, b_j，得到

$$a_j = \frac{f_{j+1} - f_j}{h_j} - \frac{M_{j+1} - M_j}{6} h_j, \quad b_j = \frac{f_j x_{j+1} - f_{j+1} x_j}{h_j} + \frac{M_{j+1} x_j - M_j x_{j+1}}{6} h_j.$$

将上述代入并整理得到

$$\begin{aligned} S(x) =& M_j \frac{(x_{j+1} - x)^3}{6h_j} + M_{j+1} \frac{(x - x_j)^3}{6h_j} + \left(f_j - \frac{M_j h_j^2}{6}\right)\left(\frac{x_{j+1} - x}{h_j}\right) \\ &+ \left(f_{j+1} - \frac{M_{j+1} h_j^2}{6}\right)\left(\frac{x - x_j}{h_j}\right), \quad x \in [x_j, x_{j+1}], \quad (j = 0, 1, \ldots, n-1). \end{aligned} \tag{4.22}$$

在上述构造过程中，利用 $S(x_j) = f_j$（$j = 0, 1, \ldots, n$）的条件，设置节点处二阶导数为 M_j（$j = 0, 1, \ldots, n$），也隐含了 $S(x)$ 二阶导数连续。因此，接下来应利用节点处一阶导数连续的条件以及边界条件确定 M_j 的值。

第三步：利用一阶导数连续性建立方程组。对公式 (4.22) 求一阶导数，得

$$S'(x) = -M_j \frac{(x_{j+1} - x)^2}{2h_j} + M_{j+1} \frac{(x - x_j)^2}{2h_j} + \frac{f_{j+1} - f_j}{h_j} - \frac{M_{j+1} - M_j}{6} h_j, \quad x \in [x_j, x_{j+1}].$$

考虑节点 x_j 处的左导数，它通过区间 $[x_{j-1}, x_j]$ 上的函数表达式计算，即

$$S'(x_j - 0) = \frac{h_{j-1}}{6} M_{j-1} + \frac{h_{j-1}}{3} M_j + \frac{f_j - f_{j-1}}{h_{j-1}}. \tag{4.23}$$

同理，利用 $[x_j, x_{j+1}]$ 上的函数表达式计算节点 x_j 处的右导数

$$S'(x_j + 0) = -\frac{h_j}{3} M_j - \frac{h_j}{6} M_{j+1} + \frac{f_{j+1} - f_j}{h_j}. \tag{4.24}$$

节点处一阶导数连续的条件意味着公式 (4.23) 和 (4.24) 相等，因此得到方程

$$\mu_j M_{j-1} + 2M_j + \lambda_j M_{j+1} = d_j, \quad (j = 1, 2, \ldots, n-1), \tag{4.25}$$

其中

$$\mu_j = \frac{h_{j-1}}{h_{j-1}+h_j}, \quad \lambda_j = \frac{h_j}{h_{j-1}+h_j}, \quad d_j = 6\left[\frac{f_{j+1}-f_j}{h_j(h_{j-1}+h_j)} - \frac{f_j - f_{j-1}}{h_{j-1}(h_{j-1}+h_j)}\right].$$

注意：公式 (4.25) 给出了 $n-1$ 个线性方程，但未知量的数目是 $n+1$，所以应再利用边界条件。由于最终得到的联立线性方程组的未知量为节点处的二阶导数，在力学上的意义为 “弯矩”，并且每个方程最多包含 3 个 “弯矩” 变量，所以求参数 M_j（$j = 0, 1, \ldots, n$）的方程常常被称为**三弯矩方程**。

第四步：处理边界条件。下面分别考虑不同边界条件对应的方程。

边界条件类型一：已知两端点处的一阶导数。根据条件 $S'(x_0) = f'_0$，$S'(x_n) = f'_n$，结合公式 (4.24) 和 (4.23)，得到方程

$$2M_0 + M_1 = \frac{6}{h_0}\left(\frac{f_1 - f_0}{h_0} - f'_0\right), \tag{4.26}$$

以及

$$M_{n-1} + 2M_n = \frac{6}{h_{n-1}}\left(f'_n - \frac{f_n - f_{n-1}}{h_{n-1}}\right). \tag{4.27}$$

结合方程 (4.25)、(4.26) 和 (4.27)，可得到完整的 $(n+1)$ 阶线性方程组

$$\begin{bmatrix} 2 & \lambda_0 & & & \\ \mu_1 & 2 & \lambda_1 & & \\ & \ddots & \ddots & \ddots & \\ & & \mu_{n-1} & 2 & \lambda_{n-1} \\ & & & \mu_n & 2 \end{bmatrix} \begin{bmatrix} M_0 \\ M_1 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} d_0 \\ d_1 \\ \vdots \\ d_{n-1} \\ d_n \end{bmatrix}, \tag{4.28}$$

其中 $\lambda_0 = \mu_n = 1$，$d_0 = \dfrac{6}{h_0}\left(\dfrac{f_1 - f_0}{h_0} - f'_0\right)$，$d_n = \dfrac{6}{h_{n-1}}\left(f'_n - \dfrac{f_n - f_{n-1}}{h_{n-1}}\right)$。

边界条件类型二：已知两端点处的二阶导数（自然边界条件）。若 $M_0 = f''_0$，$M_n = f''_n$ 已知，可由公式 (4.25) 直接整理得到方程

$$\begin{bmatrix} 2 & \lambda_1 & & & \\ \mu_2 & 2 & \lambda_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \mu_{n-2} & 2 & \lambda_{n-2} \\ & & & \mu_{n-1} & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} d_1 - \mu_1 f''_0 \\ d_2 \\ \vdots \\ d_{n-2} \\ d_{n-1} - \lambda_{n-1} f''_n \end{bmatrix}. \tag{4.29}$$

这是一个 $(n-1)$ 阶的线性方程组。特别地，当 $f''_0 = f''_n = 0$ 时，这就是自然三次样条。

系数矩阵的性质：注意到，$\mu_j + \lambda_j = 1$（$j = 1, 2, \ldots, n-1$），且 $0 < \mu_j, \lambda_j < 1$，因此三弯矩方程的系数矩阵为按行严格对角占优的三对角矩阵，必定非奇异，可采用 “追赶法”（Thomas 算法）有效地求解，且不需要选主元。这说明了给定边界条件后三次样条插值函数存在并且唯一。

第五步：确定系数。一旦求得 M_j（$j = 0, 1, \ldots, n$），我们就可以确定每个子区间上的系数。将公式 (4.22) 展开为标准形式 $S(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$，通过比较系数可以得到：

$$a_j = f_j, \quad b_j = \frac{f_{j+1} - f_j}{h_j} - \frac{h_j}{6}(M_{j+1} + 2M_j), \quad c_j = \frac{M_j}{2}, \quad d_j = \frac{M_{j+1} - M_j}{6h_j}.$$

例 4.3.1 (自然三次样条插值的详细计算). 考虑函数 $f(x) = \sin(\pi x)$，已知其在以下 4 个等距点处的值：

x	0	1/3	2/3	1
$f(x)$	0	$\sqrt{3}/2 \approx 0.8660$	$\sqrt{3}/2 \approx 0.8660$	0

我们希望构造一个自然三次样条插值函数 $S(x)$。

第一步：确定节点和区间长度。节点为 $x_0 = 0, x_1 = 1/3, x_2 = 2/3, x_3 = 1$。区间长度为 $h_0 = h_1 = h_2 = 1/3$。

第二步：计算系数。由于所有区间长度相等，$\mu_j = \lambda_j = 1/2$（$j = 1, 2$）。计算右端项：

$$d_1 = 6\left[\frac{f_2 - f_1}{h_1(h_0 + h_1)} - \frac{f_1 - f_0}{h_0(h_0 + h_1)}\right] = 6\left[\frac{0.8660 - 0.8660}{(1/3)(2/3)} - \frac{0.8660 - 0}{(1/3)(2/3)}\right] = -23.382.$$

类似地，$d_2 = -23.382$。

第三步：建立三弯矩方程组。对于自然样条，$M_0 = M_3 = 0$。根据公式 (4.29)，我们得到：

$$\begin{bmatrix} 2 & 1/2 \\ 1/2 & 2 \end{bmatrix}\begin{bmatrix} M_1 \\ M_2 \end{bmatrix} = \begin{bmatrix} -23.382 \\ -23.382 \end{bmatrix}.$$

第四步：求解方程组。使用追赶法求解，得到 $M_1 = M_2 \approx -11.691$。

第五步：确定系数。对于子区间 $[0, 1/3]$，根据公式计算系数：

$$a_0 = 0, \quad b_0 = \frac{0.8660 - 0}{1/3} - \frac{1/3}{6}(-11.691 + 0) = 3.2475, \quad c_0 = 0, \quad d_0 = \frac{-11.691 - 0}{6 \times 1/3} = -5.8455.$$

因此，在 $[0, 1/3]$ 上：$S_0(x) = 3.2475x - 5.8455x^3$。类似地可以确定其他子区间上的多项式 $S_1(x)$ 和 $S_2(x)$。

第六步：验证。可以验证 $S(0) = 0$，$S(1/3) \approx 0.8660$，$S'(1/3)$ 在左右两边相等，$S''(1/3)$ 在左右两边相等，且 $S''(0) = S''(1) = 0$。

样条插值的优势：与高次多项式插值相比，三次样条插值在保持光滑性的同时，避免了全局振荡，特别适合处理具有局部剧烈变化的函数。

多项式插值是数值分析中的一个基本工具，它允许我们用多项式来近似表示函数。Lagrange 形式和 Newton 形式提供了两种不同的表示方法，各有其优势。Newton 形式特别适合逐步添加插值点的情况，而 Horner 算法使得 Newton 形式的多项式计算非常高效。然而，多项式插值也有其局限性，特别是在高次插值或等距节点的情况下可能出现 Runge 现象。在实际应用中，需要根据具体问题选择合适的插值方法和节点分布。通过本章的例子，我们展示了如何在实际问题中应用这些方法，包括差商的计算、Horner 算法的使用以及误差的分析。

三次样条插值作为多项式插值的重要补充，通过使用分段三次多项式并保证二阶导数连续，克服了高次多项式插值的许多缺点。它在保持光滑性的同时，避免了全局振荡，特别适合处理具有局部剧烈变化的函数。自然样条是最常用的样条类型，其构造方法基于求解三对角线性方程组（三弯矩方程），计算效率高且数值稳定。样条插值的核心思想是“局部化”：每个子区间上的三次多项式只依赖于相邻节点的信息，这使得样条函数能够更好地适应函数的局部特性，避免了高次多项式的全局振荡问题。与分段低次插值相比，三次样条插值具备它们的优点，而且有更好的整体光滑性（二阶导数连续），这使得它在许多实际应用中成为首选的插值方法。

Chapter 5

积分

数值积分（也称为数值求积）是数值分析中的一个重要主题，它涉及使用数值方法近似计算定积分的值。在实际应用中，我们经常遇到无法通过解析方法计算积分的情况，这时数值积分方法就显得尤为重要。本章将系统介绍数值积分的基本原理、经典方法及其误差分析，为实际应用提供理论基础和计算方法。

本章路线图 本章的主要内容和逻辑结构如下：

- **数值积分的动机与基本思想**：说明为什么在实际计算中需要数值积分，给出典型应用场景，并用简单例子建立“加权求和近似定积分”的整体图景。
- **插值型求积公式**：通过插值多项式构造求积公式，解释节点与权重的含义，并分析这种方法的误差来源。
- **Newton-Cotes 公式与复合公式**：在等距节点上构造经典的 Newton-Cotes 公式，讨论低阶公式与高阶公式在精度和稳定性上的差异，并引入复合公式以平衡局部精度和全局成本。
- **误差的先验与后验估计**：给出数值积分误差的上界估计和基于两种公式比较的后验估计，为后续关于自适应积分和误差控制的方法提供基础。

5.1 数值积分方法的动机和原理

本节的目标是：从整体上理解数值积分问题的来源与基本思想，说明在什么情形下必须用数值方法近似定积分，并为后续各种具体求积公式的构造提供直观背景。

5.1.1 问题的提出

本小节关心的问题是：在什么样的实际场景中，我们无法通过原函数计算定积分，只能依赖函数在有限个点处的取值来近似积分。

在数学分析和实际应用中，常常需要计算定积分

$$I(f) = \int_a^b f(t)\,\mathrm{d}t, \tag{5.1}$$

其中 f 是定义在区间 $[a,b]$ 上的函数。虽然微积分基本定理告诉我们，如果 f 有原函数 F，则

$$\int_a^b f(t)\,\mathrm{d}t = F(b) - F(a),$$

但在实际应用中，存在两种情况使得我们无法直接使用这个公式：

第一种情况：我们只知道函数 f 在某些点 $t_0, t_1, \ldots, t_n$ 处的值，并且无法获得除这些值之外的其他值。这是当函数 f 被制成表格时的情况，常见于实验数据、观测数据或数值模拟的结果。在这种情况下，我们无法获得函数的解析表达式，因此无法通过求原函数来计算积分。

第二种情况：虽然可以计算任意 t 处的 $f(t)$，但 f 的原函数未知，或者 f 的解析表达式过于复杂而无法显式表达。例如，$f(t)$ 可能是过于复杂的计算代码的结果，或者原函数无法用初等函数表示（如 $\int_0^x e^{-t^2}\,\mathrm{d}t$ 这样的积分）。

在这两种情况下，只能尝试使用函数 f 在有限个点 $t_0, t_1, \ldots, t_n$ 处的值来近似 $I(f)$。这些点要么是给定的（第一种情况），要么是为了使近似尽可能良好而经过合理选择的（第二种情况）。

例 5.1.1 (一个简单的数值积分公式). 为了先形成直观印象，假设我们只知道某个函数在区间 $[0,1]$ 上的三个函数值

$$f(0) = 1, \quad f(0.5) \approx 1.65, \quad f(1) \approx 2.72.$$

此时，我们已经可以用一个简单的数值积分公式来近似

$$\int_0^1 f(x)\,\mathrm{d}x \approx \frac{1}{6}\big[f(0) + 4f(0.5) + f(1)\big].$$

本章后面将系统推导此类公式，并解释它们的理论依据。从形式上看，数值积分方法的基本思想可以概括为：**在若干节点上取函数值，乘以适当的权重并求和，以近似定积分**。

5.1.2　数值积分公式的一般形式

本小节的目标是：用统一的数学语言刻画一般数值积分公式的形式，明确“节点”和“权重”的概念，并为后续具体公式提供抽象框架。

更准确地说，我们将使用被积函数在积分区间内某些点处的值的线性组合来近似积分，即

$$\int_a^b f(t)\,\mathrm{d}t \approx \sum_{i=0}^{n} w_i f(t_i), \tag{5.2}$$

其中点 t_i 称为公式的**节点**（nodes），w_i 是**权重**（weights），有时也称为公式的系数。

映射

$$f \mapsto \sum_{i=0}^{n} w_i f(t_i) \tag{5.3}$$

它将函数 f 映射到其在所考虑区间上的积分的 “近似” 值，定义了一个**数值积分公式**（或方法），也称为**求积公式**或**求积方法**。

求积公式的构造原则：理想情况下，我们希望对我们可能感兴趣的所有函数（实际上是对所有 Riemann 可积函数）都有精确值。这是不可能的，但这个想法给出了构造最常用方法的原则。因此，我们首先要求求积公式对尽可能广泛的函数类精确。由于每个公式只有有限个自由度，我们很快想到多项式。实际上，一方面它们确实允许构造公式，因为我们知道它们的原函数；另一方面，它们在连续函数空间中稠密，这意味着在实践中，任何连续函数都可以被多项式任意接近地逼近，这使得我们可以期望这样构造的公式对这个广泛的函数类具有良好的性质。

当被积函数 f 不属于上述“具有精确公式”的函数类时，更合理的要求是存在一族公式 $(Q_n)_{n\in\mathbb{N}}$ 使得

$$\lim_{n\to\infty} \sum_{i=0}^{n} w_i^{(n)} f(t_i^{(n)}) = \int_a^b f(t)\,\mathrm{d}t.$$

后文将构造对任何 Riemann 可积函数都具有此性质的公式族，并说明也存在一些公式族，虽然对多项式精确，但不具有这种收敛性质。

5.2 使用插值多项式的方法

本节的目标是：说明如何利用插值多项式近似被积函数，并通过对插值多项式积分来构造求积公式，从而把前一章的插值理论与本章的数值积分问题联系起来。

5.2.1 方法的原理

本小节的目标是：在“已知有限个节点上的函数值”的前提下，用插值多项式替代原函数，并解释由此得到的求积公式的结构。

为了通过函数 f 在点 $t_0, t_1, \ldots, t_n$ 处的值的线性组合来近似积分 $I(f)$，第一种方法是用

$$J(f) = \int_a^b p_f(t)\,\mathrm{d}t \tag{5.4}$$

来近似 $I(f)$，其中 $p_f(t)$ 是在互异点 $t_0, t_1, \ldots, t_n$ 处插值 f 的多项式。这些点通常选择在区间 $[a, b]$ 内。然而，在关于微分方程的章节中，我们将遇到使用积分区间外节点的求积公式。其理由非常简单：这允许使用在被积函数值已经计算过的点处的函数值。

从概念上看，这一方法的基本思想是：在仅已知函数在有限个节点处取值的情况下，用插值多项式 p_f 近似被积函数 f，并以其积分 $\int_a^b p_f(t)\,\mathrm{d}t$ 作为原积分 $\int_a^b f(t)\,\mathrm{d}t$ 的近似值。

前一章已经给出，插值多项式在 Lagrange 基中表示为

$$p_f(t) = \sum_{i=0}^{n} f(t_i) L_i(t), \tag{5.5}$$

其中 $L_i(t)$ 是第 i 个 Lagrange 基多项式。我们因此直接得到

$$J(f) = \sum_{i=0}^{n} w_i f(t_i), \quad \text{其中} \quad w_i = \int_a^b L_i(t)\,\mathrm{d}t. \tag{5.6}$$

公式 (5.6) 称为**求积公式**。

等价构造方法：等价地，我们也可以通过要求公式对所有次数小于或等于 n 的多项式精确，即等价地对 $\mathscr{P}_n$ 的规范基的每个单项式精确，来获得系数 w_i。这种等价性很容易证明：如果公式对所有次数小于或等于 n 的多项式精确，则特别地，它对 Lagrange 基多项式 $L_i(t)$ 精确，因此

$$\int_a^b L_i(t)\,\mathrm{d}t = \sum_{j=0}^{n} w_j L_i(t_j) = w_i,$$

这正是公式 (5.6) 中的权重。

因此，我们可以通过要求 $I(p) = J(p)$ 对 $p \in \{p_0, p_1, \ldots, p_n\}$ 成立来计算系数 $w_0, \ldots, w_n$，其中 $\{p_0, p_1, \ldots, p_n\}$ 是 $\mathscr{P}_n$ 的规范基，即 $p_k(t) = t^k$（$k = 0, 1, \ldots, n$）。我们得到线性方程组

$$\begin{pmatrix} 1 & 1 & \cdots & 1 \\ t_0 & t_1 & \cdots & t_n \\ t_0^2 & t_1^2 & \cdots & t_n^2 \\ \vdots & \vdots & \ddots & \vdots \\ t_0^n & t_1^n & \cdots & t_n^n \end{pmatrix} \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix} = \begin{pmatrix} v_0 \\ v_1 \\ \vdots \\ v_n \end{pmatrix}, \tag{5.7}$$

其中

$$v_k = \int_a^b t^k\,\mathrm{d}t = \frac{b^{k+1} - a^{k+1}}{k+1}. \tag{5.8}$$

该系统的矩阵是 Vandermonde 矩阵。当且仅当点 t_i 两两互异时，其行列式非零。因此，系统 (5.7) 确实有且仅有一个解。

注意 (本节小结). 本节的核心问题是：**在给定节点 t_i 的前提下，如何系统地求出权重 w_i**。主要思路包括：

- 利用 Lagrange 基多项式 L_i，直接计算 $w_i = \int_a^b L_i(t)\,\mathrm{d}t$；
- 要求公式对 $\mathscr{P}_n$ 中的单项式精确，得到线性方程组 (5.7)。

在应用层面，能够**利用上述任一思路，在简单情形下计算出权重 w_i**（如例 5.2.1），即可支撑后续对具体数值积分公式的使用。

从误差与稳定性的角度看，本节表明：一旦节点位置固定，求积公式的性质（包括对多项式的精确度和对一般函数的误差表现）就完全由权重 w_i 决定。接下来，我们通过把积分统一映射到参考区间，简化节点与权重的表达形式，为构造具体的 Newton-Cotes 公式做准备。

例 5.2.1 (求积公式权重的计算). 考虑区间 $[0,1]$ 上的两点求积公式，节点为 $t_0=0$，$t_1=1$。

方法一：使用 Lagrange 基多项式。Lagrange 基多项式为

$$L_0(t) = \frac{t-1}{0-1} = 1-t, \quad L_1(t) = \frac{t-0}{1-0} = t.$$

因此权重为

$$w_0 = \int_0^1 L_0(t)\,\mathrm{d}t = \int_0^1 (1-t)\,\mathrm{d}t = \frac{1}{2}, \quad w_1 = \int_0^1 L_1(t)\,\mathrm{d}t = \int_0^1 t\,\mathrm{d}t = \frac{1}{2}.$$

方法二：使用规范基。要求公式对 $p_0(t)=1$ 和 $p_1(t)=t$ 精确：

$$\int_0^1 1\,\mathrm{d}t = w_0\cdot 1 + w_1\cdot 1 = 1, \quad \int_0^1 t\,\mathrm{d}t = w_0\cdot 0 + w_1\cdot 1 = \frac{1}{2}.$$

求解得到 $w_0 = w_1 = 1/2$，与方法一的结果一致。

因此，两点求积公式为

$$\int_0^1 f(t)\,\mathrm{d}t \approx \frac{1}{2}(f(0)+f(1)),$$

这正是梯形公式。

5.2.2　参考区间

本小节的目标是：通过简单的仿射变换，把任意有限区间上的积分统一转换为参考区间 $[-1,1]$ 上的积分，从而使节点和权重的计算不再依赖具体区间端点。

一旦节点 t_i 选定，权重 w_i 不依赖于要积分的函数 f。然而，它们依赖于 a 和 b。我们现在将看到，可以无限制地将任意区间上的积分近似归结为在唯一一个称为参考区间的区间上的积分。我们选择区间 $[-1,1]$，因为它关于 0 的对称性简化了计算。然而，我们也可以选择任何其他区间，例如 $[0,1]$。

因此，我们将使用仿射变量变换，将区间 $[a,b]$ 上的积分计算归结为区间 $[-1,1]$ 上的积分。实际上，如果设

$$t = \frac{a+b+(b-a)u}{2}, \tag{5.9}$$

我们可以写成

$$\int_a^b f(t)\,\mathrm{d}t = \frac{b-a}{2}\int_{-1}^1 g(u)\,\mathrm{d}u, \tag{5.10}$$

其中

$$g(u) = f\left(\frac{a+b+(b-a)u}{2}\right). \tag{5.11}$$

因此，我们可以在 $[-1,1]$ 上定义求积公式，其中节点和权重都不依赖于 a 或 b。这些公式就是我们在数值积分书籍的表格中找到的公式。

当我们必须计算二重或三重积分时，情况就不那么简单了。实际上，不存在简单的坐标变换能够将任意二维或三维域变换到参考域。我们有一系列域形状族（三角形、四边形、四面体等），对于每个族，都有一个参考域。从一维到二维或三维的过渡改变了求积问题的复杂度水平。

在目前阶段，只需记住：一维情形下，我们总可以通过仿射变换把积分化为参考区间上的积分，这使得后续构造和查表使用具体求积公式变得简单。接下来，我们将利用插值误差公式给出求积误差的上界，并引入“阶数”这一刻画公式精度的基本概念。

5.2.3 求积误差的上界与阶数

本小节的目标是：利用插值误差公式推导出求积误差的上界表达式，并通过“对多项式精确的最高次数”来定义求积公式的阶数，从而把误差分析与具体公式的代数性质对应起来。

求积误差由下式给出：

$$E(f) = I(f) - J(f) = \int_a^b (f(t) - p_f(t))\,\mathrm{d}t. \tag{5.12}$$

如果 f 是 $n+1$ 次连续可导的函数，且 p_f 是其次数为 n 的插值多项式，我们在关于插值的章节中看到：

$$f(t) - p_f(t) = (t-t_0)(t-t_1)\cdots(t-t_n)\frac{f^{(n+1)}(\xi(t))}{(n+1)!} = \pi_n(t)\frac{f^{(n+1)}(\xi(t))}{(n+1)!}, \tag{5.13}$$

其中 $\xi(t) \in \mathrm{Int}(t, t_0, \ldots, t_n)$，$\pi_n(t) = (t-t_0)(t-t_1)\cdots(t-t_n)$。

因此有

$$|E(f)| \le \int_a^b |\pi_n(t)| \frac{|f^{(n+1)}(\xi(t))|}{(n+1)!}\,\mathrm{d}t = \frac{|f^{(n+1)}(\eta)|}{(n+1)!}\int_a^b |\pi_n(t)|\,\mathrm{d}t,$$

其中 $\eta \in \mathrm{Int}(a, b, t_0, \ldots, t_n)$（我们应用了第二积分中值定理：如果 f 和 g 是 $[a,b]$ 上的两个连续函数，且 g 在 $[a,b]$ 上不变号，则存在 $c \in (a,b)$ 使得 $\int_a^b f(x)g(x)\,\mathrm{d}x = f(c)\int_a^b g(x)\,\mathrm{d}x$）。

我们还可以进一步上界：

$$|E(f)| \le \frac{M}{(n+1)!}\int_a^b |\pi_n(t)|\,\mathrm{d}t, \tag{5.14}$$

其中 $M > 0$ 是 $|f^{(n+1)}|$ 在 $\mathrm{Int}(a, b, t_0, \ldots, t_n)$ 上的一个上界。

我们确实发现，如果 f 是次数小于或等于 n 的多项式，则 $E(f) = 0$，即近似是精确的。我们将在后面看到，特别是在上面引用的示例和练习中，可以获得更精细的误差估计。

定义 5.2.1 (求积公式的阶数). 对所有次数小于或等于 n（n 为最大值）的多项式精确的求积公式称为**阶数为 n** 或**精确度为 n**。

5.2.4 Newton-Cotes 公式

本小节的目标是：在等距节点上构造一类经典的插值型求积公式——Newton-Cotes 公式，并分析其低阶形式的优点以及高阶形式在稳定性上的局限。

在积分区间上构造等距节点的多项式插值，对应的插值型求积公式为牛顿-柯特斯（Newton-Cotes）公式。这是使用固定等距点 t_i 构造的经典公式。

假设将积分区间 n 等分，步长 $h = (b-a)/n$，插值节点为 $t_k = a + kh$（$k = 0, 1, 2, \ldots, n$），其中 $t_0 = a$，$t_n = b$，且点 t_i 等距：

$$t_{j+1} - t_j = h = \frac{b-a}{n}, \quad \forall j = 0, 1, \ldots, n-1.$$

根据式 (5.6) 的推导，用 $J(f) = \displaystyle\int_a^b p_f(t)\,\mathrm{d}t = \sum_{k=0}^n A_k f(t_k)$ 来近似 $I(f)$，其中 p_f 是在点 $t_0, \ldots, t_n$ 处插值 f 的拉格朗日插值多项式，权重系数为

$$A_k = \int_a^b L_k(t)\,\mathrm{d}t = \int_a^b \frac{(t-t_0)\cdots(t-t_{k-1})(t-t_{k+1})\cdots(t-t_n)}{(t_k-t_0)\cdots(t_k-t_{k-1})(t_k-t_{k+1})\cdots(t_k-t_n)}\,\mathrm{d}t, \tag{5.15}$$

这就是 n 阶牛顿-柯特斯（Newton-Cotes）公式的系数。

柯特斯系数的推导

为了简化计算并揭示系数的本质特征，我们引入变量代换 $t = a + sh$，其中 $s \in [0, n]$。注意到 $t_k = a + kh$，因此 $t - t_j = (s - j)h$，$t_k - t_j = (k - j)h$。代入式 (5.15)，得到

$$A_k = \int_0^n \prod_{\substack{j=0 \\ j\neq k}}^{n} \frac{(s-j)h}{(k-j)h} h \,\mathrm{d}s = \int_0^n \prod_{\substack{j=0 \\ j\neq k}}^{n} \frac{(s-j)}{(k-j)} \frac{b-a}{n} \,\mathrm{d}s. \tag{5.16}$$

令

$$C_k^{(n)} = \frac{1}{n} \int_0^n \prod_{\substack{j=0 \\ j\neq k}}^{n} \frac{(s-j)}{(k-j)} \,\mathrm{d}s, \quad (k = 0, 1, 2, \ldots, n) \tag{5.17}$$

它仅与阶数 n 有关，与区间大小无关，则积分系数

$$A_k = (b - a) C_k^{(n)}. \tag{5.18}$$

式 (5.17) 中的 $C_k^{(n)}$ 常被称为**柯特斯系数**（Cotes coefficients），可以预先计算出不同 n 值对应的柯特斯系数，制成一个表（见表 5.1），根据它方便地写出各阶牛顿-柯特斯公式。

表 5.1　柯特斯系数表

n	$C_0^{(n)}$	$C_1^{(n)}$	$C_2^{(n)}$	$C_3^{(n)}$	$C_4^{(n)}$	$C_5^{(n)}$	$C_6^{(n)}$	$C_7^{(n)}$	$C_8^{(n)}$
1	$\frac{1}{2}$	$\frac{1}{2}$							
2	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$						
3	$\frac{1}{8}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{1}{8}$					
4	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$				
5	$\frac{19}{288}$	$\frac{25}{96}$	$\frac{25}{144}$	$\frac{25}{144}$	$\frac{25}{96}$	$\frac{19}{288}$			
6	$\frac{41}{840}$	$\frac{9}{35}$	$\frac{9}{280}$	$\frac{34}{105}$	$\frac{9}{280}$	$\frac{9}{35}$	$\frac{41}{840}$		
7	$\frac{751}{17280}$	$\frac{3577}{17280}$	$\frac{1323}{17280}$	$\frac{2989}{17280}$	$\frac{2989}{17280}$	$\frac{1323}{17280}$	$\frac{3577}{17280}$	$\frac{751}{17280}$	
8	$\frac{989}{28350}$	$\frac{5888}{28350}$	$\frac{-928}{28350}$	$\frac{10496}{28350}$	$\frac{-4540}{28350}$	$\frac{10496}{28350}$	$\frac{-928}{28350}$	$\frac{5888}{28350}$	$\frac{989}{28350}$

从表 5.1 可以看出，当 $n = 8$ 时，柯特斯系数 $C_k^{(n)}$ 出现负值（如 $C_2^{(8)} = -928/28350$，$C_4^{(8)} = -4540/28350$），因此对应的求积公式是不稳定的。这是因为当 n 较大时，插值多项式在区间端点附近会出现强烈的振荡（Runge 现象），导致某些权重系数为负，从而在数值计算中可能放大舍入误差。

表 5.1 也显示出柯特斯系数的对称性，即 $C_k^{(n)} = C_{n-k}^{(n)}$。这一性质可以通过变量代换 $s \mapsto n - s$ 来证明：由于节点关于区间中点对称，且被积函数具有相应的对称性，因此权重系数也具有对称性。详细的证明留给读者作为练习。

从失败机制的角度看，本小节给出的结论是：单纯提高 Newton-Cotes 公式的阶数并不能保证数值效果变好，高阶插值多项式在端点附近的振荡会通过负权重放大误差并破坏稳定性。因此，在实际应用中，我们更倾向于使用低阶（但稳定）的基本公式，并通过复合思想在大区间上实现高精度，这正是下一节要讨论的内容。

几个低阶公式

下面介绍三种常用的低阶牛顿-柯特斯公式（对应于 $n=1$、$n=2$ 或 $n=4$ 的情况），它们在计算量、精度和稳定性之间取得了较好的平衡，是实际计算中最常用的基本构件：

（1）梯形公式（对应于 $n=1$ 的情况，$h=b-a$）：

$$T(f)=\frac{b-a}{2}\left[f(a)+f(b)\right]=\frac{h}{2}(f(t_0)+f(t_1)); \tag{5.19}$$

误差估计：$\exists\eta\in[a,b]$，使得 $E(f)=-\dfrac{h^3}{12}f''(\eta)$。

（2）辛普森（Simpson）公式（对应于 $n=2$ 的情况，$h=(b-a)/2$）：

$$S(f)=\frac{b-a}{6}\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right]=\frac{h}{3}(f(t_0)+4f(t_1)+f(t_2)); \tag{5.20}$$

误差估计：$\exists\eta\in[a,b]$，使得 $E(f)=-\dfrac{h^5}{90}f^{(4)}(\eta)$。

（3）柯特斯公式（对应于 $n=4$ 的情况，$h=(b-a)/4$）：

$$C(f)=\frac{b-a}{90}\left[7f(x_0)+32f(x_1)+12f(x_2)+32f(x_3)+7f(x_4)\right], \tag{5.21}$$

其中 $x_k=a+kh$（$k=0,1,2,3,4$）。误差估计：$\exists\eta\in[a,b]$，使得 $E(f)=-\dfrac{8h^7}{945}f^{(6)}(\eta)$。

（4）辛普森 3/8（Simpson 3/8）公式（对应于 $n=3$ 的情况（$h=(b-a)/3$）：

$$J(f)=\frac{3h}{8}(f(t_0)+3f(t_1)+3f(t_2)+f(t_3));\quad \exists\eta\in[a,b],\quad E(f)=-\frac{3h^5}{80}f^{(4)}(\eta). \tag{5.22}$$

一般理论结果

我们有以下一般结果：

定理 5.2.1 (带余项的 Newton-Cotes 公式 *)**.** 设 n 阶 Newton-Cotes 公式的求积公式为

$$I_n(f)=\sum_{k=0}^{n}A_kf(x_k)=(b-a)\sum_{k=0}^{n}C_k^{(n)}f(x_k),$$

其中 $x_k=a+kh$，$h=(b-a)/n$，$C_k^{(n)}$ 为柯特斯系数。

如果 n 是偶数且 f 是 $n+2$ 次连续可导的，则存在 $\eta\in[a,b]$ 使得

$$\int_a^b f(t)\,\mathrm{d}t=I_n(f)+\frac{h^{n+3}f^{(n+2)}(\eta)}{(n+2)!}\int_0^n s^2(s-1)\cdots(s-n)\,\mathrm{d}s. \tag{5.23}$$

如果 n 是奇数且 f 是 $n+1$ 次连续可导的，则存在 $\eta\in[a,b]$ 使得

$$\int_a^b f(t)\,\mathrm{d}t=I_n(f)+\frac{h^{n+2}f^{(n+1)}(\eta)}{(n+1)!}\int_0^n s(s-1)\cdots(s-n)\,\mathrm{d}s. \tag{5.24}$$

系数 A_k（或等价地，$C_k^{(n)}$）可以通过求解系统 (5.7) 或通过计算 $\displaystyle\int_a^b L_k(t)\,\mathrm{d}t$ 获得，其中 $L_k(t)$ 是第 k 个 Lagrange 基多项式。

证明. 本定理给出 Newton-Cotes 公式在奇数阶和偶数阶情形下的精确误差形式，属于**偏理论、偏进阶**的内容。初学者在第一次阅读时，可以**只记住结论的结构**（误差由高阶导数与若干积分常数组成），把下面的详细证明作为选读或日后复习时的参考。

证明采用两种不同的策略：对于奇数 n，使用 Hermite 插值方法；对于偶数 n，利用对称性和 Peano 核定理。

情况 1：n 为奇数

当 n 为奇数时，我们使用 Hermite 插值来避免直接处理变号的权函数。设 $p_n(t)$ 是在节点 $x_k = a + kh$（$k = 0, 1, \ldots, n$）处插值 f 的 n 次 Lagrange 插值多项式。

关键观察：多项式 $\omega(s) = s(s-1)\cdots(s-n)$ 在区间 $[0, n]$ 内有 $n+1$ 个零点（位于 $s = 0, 1, \ldots, n$），因此**会变号**。例如，当 $n = 1$ 时，$\omega(s) = s(s-1)$ 在 $[0, 1]$ 上先负后正；当 $n = 3$ 时，$\omega(s)$ 在 $[0, 3]$ 上多次变号。因此，**不能直接应用第二积分中值定理**，因为该定理要求权函数不变号。

标准证明方法：使用 Peano 核定理。定义线性泛函

$$R(f) = \int_a^b f(t)\,\mathrm{d}t - I_n(f) = \int_a^b f(t)\,\mathrm{d}t - \sum_{k=0}^{n} A_k f(x_k).$$

由于 R 对 n 次多项式为零（因为 Newton-Cotes 公式对 n 次多项式精确），且 R 是线性有界泛函，由 **Peano 核定理**，对 $n+1$ 次连续可导的函数 f，有

$$R(f) = \int_a^b f^{(n+1)}(t) K(t)\,\mathrm{d}t,$$

其中 Peano 核为

$$K(t) = \frac{1}{n!} R_x[(x-t)_+^n],$$

这里 $(x-t)_+^n$ 表示 $(x-t)^n$ 的正部，即当 $x \geq t$ 时为 $(x-t)^n$，否则为 0。

为什么 Peano 核不变号？通过直接计算 $K(t)$（需要分区间讨论 t 相对于节点的位置），并利用节点的等距性和公式的精确性，可以证明：当 n 为奇数时，$K(t)$ 在 $[a, b]$ 上**不变号**（实际上恒非负或恒非正）。这一结论的证明较为技术性，涉及对 $K(t)$ 的符号分析。

因此，由积分中值定理，存在 $\eta \in [a, b]$ 使得

$$E(f) = f^{(n+1)}(\eta) \int_a^b K(t)\,\mathrm{d}t.$$

通过变量代换 $t = a + sh$ 并计算 Peano 核的积分，最终得到

$$E(f) = \frac{h^{n+2} f^{(n+1)}(\eta)}{(n+1)!} \int_0^n s(s-1)\cdots(s-n)\,\mathrm{d}s.$$

注：虽然 $\omega(s) = s(s-1)\cdots(s-n)$ 会变号，但通过 Peano 核定理，我们将问题转化为计算 $K(t)$ 的积分，而 $K(t)$ 是不变号的，从而可以应用积分中值定理。这是处理变号权函数的标准技巧。

情况 2：n 为偶数

当 n 为偶数时，证明分为两步：首先证明公式对 $n+1$ 次多项式精确，然后使用 Peano 核定理推导误差公式。

步骤 1：证明公式对 $n+1$ 次多项式精确

设 $q(t)$ 是任意 $n+1$ 次多项式，$p_n(t)$ 是在节点 $x_0, \ldots, x_n$ 处插值 q 的 n 次插值多项式。则 $q(t) - p_n(t)$ 是 $n+1$ 次多项式，且在 $n+1$ 个节点处为零，因此可以写成

$$q(t) - p_n(t) = c \prod_{j=0}^{n} (t - x_j) = c h^{n+1} \omega(s),$$

其中 c 是常数，$t = a + sh$，$\omega(s) = s(s-1)\cdots(s-n)$。

关键：利用对称性。由于节点关于区间中点对称，即 $x_k + x_{n-k} = a + b$，等价地 $k + (n-k) = n$。通过变量代换 $s \mapsto n - s$，有

$$\omega(n-s) = (n-s)(n-s-1)\cdots(n-s-n) = (n-s)(n-s-1)\cdots(-s).$$

重新排列因子，令 $j = n - i$，则

$$\omega(n-s) = (-1)^{n+1}(s)(s-1)\cdots(s-n) = (-1)^{n+1}\omega(s).$$

由于 n 为偶数，$n+1$ 为奇数，因此 $(-1)^{n+1} = -1$，即

$$\omega(n-s) = -\omega(s).$$

这说明 $\omega(s)$ 关于 $s=n/2$ 是**奇函数**。因此

$$\int_0^n \omega(s)\,\mathrm{d}s = \int_0^{n/2} \omega(s)\,\mathrm{d}s + \int_{n/2}^n \omega(s)\,\mathrm{d}s = \int_0^{n/2} \omega(s)\,\mathrm{d}s + \int_0^{n/2} \omega(n-u)\,\mathrm{d}u = \int_0^{n/2} [\omega(s)-\omega(s)]\,\mathrm{d}s = 0,$$

其中第二个积分使用了代换 $s=n-u$。

因此

$$\int_a^b [q(t)-p_n(t)]\,\mathrm{d}t = ch^{n+2}\int_0^n \omega(s)\,\mathrm{d}s = 0,$$

即 $I_n(q)=\int_a^b q(t)\,\mathrm{d}t$，说明公式对 $n+1$ 次多项式精确。

步骤 2：使用 Peano 核定理推导误差公式

定义线性泛函

$$R(f)=\int_a^b f(t)\,\mathrm{d}t - I_n(f) = \int_a^b f(t)\,\mathrm{d}t - \sum_{k=0}^n A_k f(x_k).$$

由于 R 对 $n+1$ 次多项式为零（由步骤 1），且 R 是线性有界泛函，由 **Peano 核定理**，对 $n+2$ 次连续可导的函数 f，有

$$R(f)=\int_a^b f^{(n+2)}(t)K(t)\,\mathrm{d}t,$$

其中 Peano 核为

$$K(t)=\frac{1}{(n+1)!}R_x[(x-t)_+^{n+1}],$$

这里 $(x-t)_+^{n+1}$ 表示 $(x-t)^{n+1}$ 的正部，即

$$(x-t)_+^{n+1}=\begin{cases}(x-t)^{n+1}, & x\ge t,\\ 0, & x<t.\end{cases}$$

为什么引入 Peano 核？ Peano 核定理是数值分析中处理线性泛函误差的标准工具。它允许我们将误差表示为高阶导数的加权积分，其中权重（Peano 核）不依赖于被积函数，只依赖于求积公式本身。

计算 Peano 核：

$$K(t)=\frac{1}{(n+1)!}\left[\int_a^b (x-t)_+^{n+1}\,\mathrm{d}x - \sum_{k=0}^n A_k(x_k-t)_+^{n+1}\right].$$

通过直接计算（这里需要分区间讨论 t 的位置），并利用公式对 $n+1$ 次多项式的精确性，可以证明：当 n 为偶数时，$K(t)$ 在 $[a,b]$ 上**不变号**（实际上恒非负或恒非正）。这一结论的证明需要利用节点的对称性和公式的精确性。

因此，由积分中值定理，存在 $\eta\in[a,b]$ 使得

$$R(f)=f^{(n+2)}(\eta)\int_a^b K(t)\,\mathrm{d}t.$$

通过变量代换 $t=a+sh$ 并计算 Peano 核的积分，最终得到

$$E(f)=\frac{h^{n+3}f^{(n+2)}(\eta)}{(n+2)!}\int_0^n s^2(s-1)\cdots(s-n)\,\mathrm{d}s.$$

为什么出现 s^2 项？ 这是因为 Peano 核的计算涉及 $(x-t)_+^{n+1}$ 的积分，而由于公式对 $n+1$ 次多项式精确，在计算 $R_x[(x-t)^{n+1}]$ 时会出现额外的因子，导致最终表达式中出现 s^2 项，这反映了公式的更高阶精度。 □

对于 n 为偶数（因此是奇数个点），Newton-Cotes 公式对次数小于或等于 $n+1$ 的多项式精确，而对于 n 为奇数，这些公式对次数小于或等于 n 的多项式精确。因此，n 为偶数的情况给出了比预期更高阶的公式。这有一个直接后果：当 n 为偶数时，添加单个点不会改善积分的近似，必须添加 2 个点。

稳定性考虑： 从表 5.1 可以看出，当 $n\ge 8$ 时，某些柯特斯系数变为负值，这会导致数值不稳定。因此，在实际应用中，通常只使用低阶公式（$n\le 4$），对于大区间则采用复合求积公式（见下一节）。

例 5.2.2 (Simpson 公式的应用)**.** 计算积分 $I = \int_0^1 e^x \, \mathrm{d}x$。

精确值： $I = e - 1 \approx 1.71828$。

使用 Simpson 公式： 取 $n = 2$，$h = (1-0)/2 = 0.5$，节点为 $t_0 = 0$，$t_1 = 0.5$，$t_2 = 1$。

计算函数值：

$$f(0) = e^0 = 1, \quad f(0.5) = e^{0.5} \approx 1.64872, \quad f(1) = e^1 \approx 2.71828.$$

Simpson 公式给出：

$$J(f) = \frac{0.5}{3}(1 + 4 \times 1.64872 + 2.71828) = \frac{0.5}{3} \times 10.31316 \approx 1.71886.$$

误差分析： 误差为 $E(f) = |I - J(f)| \approx 0.00058$。

根据误差公式，$E(f) = -\frac{h^5}{90} f^{(4)}(\eta)$，其中 $f^{(4)}(x) = e^x$。由于 e^x 在 $[0,1]$ 上单调递增，$f^{(4)}(\eta) \in [1, e]$，因此

$$|E(f)| \le \frac{0.5^5}{90} \times e \approx 0.00095,$$

这与实际误差一致。

与梯形公式的比较： 梯形公式给出

$$J_T(f) = \frac{0.5}{2}(1 + 2.71828) = 0.92957,$$

误差为 $|I - J_T(f)| \approx 0.78871$，远大于 Simpson 公式的误差。这说明在**同样只用一个子区间**的情况下，Simpson 公式比简单梯形公式精度高得多。需要强调的是，如果改用**复合梯形公式**并适当细分区间，梯形方法的精度也可以显著提高，只是通常仍不及同阶的 Simpson 方法。

5.2.5　复合数值积分

本小节的目标是：说明如何在大区间上采用“分段 + 低阶公式”的策略构造复合求积公式，从而同时兼顾局部精度和全局稳定性。

在实践中，我们很少超过 $n = 2$，原因与偏好样条而非插值多项式相同，因为当 n 变大时，插值多项式倾向于强烈振荡。为了处理大区间，我们使用局部方法：首先将区间 $[a, b]$ 分割，然后在子区间上应用低阶公式。例如，对于梯形方法：

复合梯形公式：

1. 使用分割点 $t_i = a + ih$（$i = 0, \ldots, N$，其中 $h = (b-a)/N$）将区间 $[a, b]$ 细分为 N 个子区间，
2. 然后在每个子区间 $[t_i, t_{i+1}]$ 上应用梯形方法。

我们因此得到以下复合公式：

$$J(f) = h\left[\frac{f(t_0) + f(t_1)}{2} + \frac{f(t_1) + f(t_2)}{2} + \cdots + \frac{f(t_{N-1}) + f(t_N)}{2}\right] = h\left[\frac{1}{2}f(t_0) + \sum_{i=1}^{N-1} f(t_i) + \frac{1}{2}f(t_N)\right]. \tag{5.25}$$

对于误差的计算，我们对每个子区间上的误差求和，即

$$E(f) = \sum_{i=0}^{N-1} E_i(f), \quad \text{其中} \quad E_i(f) = -\frac{h^3}{12} f''(\eta_i),$$

其中 $\eta_i \in [t_i, t_{i+1}]$。然后我们应用中值定理来引用存在一个数 $\eta \in [a, b]$ 使得

$$f''(\eta) = \frac{1}{N} \sum_{i=0}^{N-1} f''(\eta_i),$$

全局误差写成

$$E(f) = -\frac{b-a}{12} h^2 f''(\eta). \tag{5.26}$$

同样地，对于 Simpson 方法：假设 N 是偶数，即 $N=2M$。得到的公式如下：

$$J(f)=\frac{h}{3}\left[f(t_0)+4\sum_{i=0}^{M-1}f(t_{2i+1})+2\sum_{i=1}^{M-1}f(t_{2i})+f(t_{2M})\right], \tag{5.27}$$

其中我们总是设 $h=(b-a)/N$，并且通过与之前相同的技术，我们得到误差估计：

$$E(f)=-\frac{b-a}{180}h^4f^{(4)}(\eta),\quad \eta\in[a,b]. \tag{5.28}$$

例 5.2.3 (复合 Simpson 公式的应用)**.** 计算积分 $I=\int_0^1 e^x\,\mathrm{d}x$，使用 $N=4$ 个子区间（即 $M=2$）。

精确值： $I=e-1\approx 1.71828$。

使用复合 Simpson 公式： $h=(1-0)/4=0.25$，节点为 $t_0=0$，$t_1=0.25$，$t_2=0.5$，$t_3=0.75$，$t_4=1$。计算函数值：

$$f(0)=1,\quad f(0.25)\approx 1.28403,\quad f(0.5)\approx 1.64872,\quad f(0.75)\approx 2.11700,\quad f(1)\approx 2.71828.$$

复合 Simpson 公式给出：

$$J(f)=\frac{0.25}{3}[1+4(1.28403+2.11700)+2\times 1.64872+2.71828]=\frac{0.25}{3}\times 20.61944\approx 1.71829.$$

误差分析： 误差为 $E(f)=|I-J(f)|\approx 0.00001$，非常小。

根据误差公式，$|E(f)|\le(1/180)\times 0.25^4\times e\approx 0.00002$，这与实际误差一致。

与简单 Simpson 公式的比较： 简单 Simpson 公式（$n=2$）的误差约为 0.00058，而复合 Simpson 公式（$N=4$）的误差约为 0.00001，精度提高了约 58 倍。这说明通过细分区间，我们可以显著提高数值积分的精度。

从误差控制的角度看，复合公式将“高阶导数与步长幂次”的误差结构转化为对步长 h 的显式依赖，使得我们可以通过调整划分精细程度来达到预期精度。接下来，我们进一步讨论如何利用先验误差估计和基于多个公式比较的后验误差估计，在实际计算中自动选择合适的步长和分段数。

5.3 求积误差的先验和后验估计

5.3.1 误差估计的基本思想

本小节的目标是：在已有误差公式的基础上，区分“先验估计”和“后验估计”，并说明如何利用这两类估计在实际计算中评估和控制数值积分误差。

练习中中点方法得到的误差和段落（Newton-Cotes 积分公式）中梯形方法得到的误差是同一类型的。它们表明，例如，在子区间 $[t_i,t_{i+1}]$ 上的误差以 h_i^3 的速度趋于 0，这是有趣的。然而，形式为 $f''(\xi_{M,i})$（或 $f''(\xi_{T,i})$）的项的存在禁止了误差值的任何评估。相反，如果我们同时有两个估计，即

$$E_i^M(f)=\frac{h_i^3}{24}f''(\xi_{M,i})\quad 和\quad E_i^T(f)=-\frac{h_i^3}{12}f''(\xi_{T,i}),$$

我们注意到，对于足够小的 h_i，我们可以写成

$$E_i^T(f)\approx -2E_i^M(f).$$

设

$$I_i^{\text{exact}}(f)=\int_{t_i}^{t_{i+1}}f(t)\,\mathrm{d}t,\quad I_i^M(f)=h_if\left(\frac{t_i+t_{i+1}}{2}\right),\quad I_i^T(f)=\frac{h_i}{2}(f(t_i)+f(t_{i+1})).$$

则有

$$I_i^{\text{exact}}(f)-I_i^T(f)\approx -2(I_i^{\text{exact}}(f)-I_i^M(f)).$$

我们因此推导出 $I_i^{\text{exact}}(f)$ 的改进近似值：

$$I_i^{\text{exact}}(f) \approx \frac{I_i^T(f) + 2I_i^M(f)}{3} = \frac{h_i}{6}\left[f(t_i) + 4f\left(\frac{t_i + t_{i+1}}{2}\right) + f(t_{i+1})\right], \tag{5.29}$$

这正是 Simpson 公式。

尤其是误差估计：

$$E_i^M(f) = I_i^{\text{exact}}(f) - I_i^M(f) \approx \frac{I_i^T(f) + 2I_i^M(f)}{3} - I_i^M(f) = \frac{I_i^T(f) - I_i^M(f)}{3}. \tag{5.30}$$

上述估计是**后验估计**：一旦获得了近似值 $I_i^M(f)$ 和 $I_i^T(f)$，它就给出误差的 ‘‘可计算值’’。我们将在下一段看到如何使用这些估计来获得给定精度的近似值。

让我们通过回顾所有获得的估计只对足够正则的被积函数有意义（具体来说，对于中点和梯形公式，需要 C^2 类函数）来结束这一段。此外，先验估计只对足够小的 h_i 有意义，使得当我们将 $\xi_{M,i}$ 和 $\xi_{T,i}$ 等同起来时可以忽略高阶项，但也不能太小，否则机器的截断误差变得占主导地位，并且在不考虑它们的情况下使这些估计失去意义。

Chapter 6

微分方程

微分方程是描述自然现象和工程问题的重要数学工具。从物理学的运动方程到生物学的种群模型，从工程中的控制系统到经济学的增长模型，微分方程无处不在。然而，大多数微分方程无法通过解析方法求解，必须依赖数值方法获得近似解。本章系统介绍微分方程数值求解的基本原理、经典方法及其理论分析，为实际应用提供坚实的数值计算基础。

本章路线图 本章围绕常微分方程初值问题的数值求解，按如下结构展开：

- **一般求解原理**：回顾解的存在唯一性理论，阐明数值方法构造的理论前提，并引入时间离散化思想。
- **单步格式**：从数值积分视角推导 Euler、梯形和 Runge-Kutta 等单步格式，分析其局部截断误差、阶数、稳定性与收敛性。
- **多步格式**：构造 Adams-Bashforth 与 Adams-Moulton 等多步格式，说明其在计算代价与实现难度上的特点，并给出阶数与误差形式。
- **Runge-Kutta 理论与自适应步长**：在一般框架下刻画 Runge-Kutta 方法，讨论误差估计与自适应步长策略，为后续刚性问题与高精度算法奠定基础。

引入性例子：一个无法显式求解的问题 在深入学习数值方法之前，先考察一个看似简单却无法用初等函数求解的例子。考虑初值问题：

$$\begin{cases} y'(t) = y(t)^2 - t, \\ y(0) = 0. \end{cases} \tag{6.1}$$

这是一个简单的一阶非线性微分方程，但无法通过初等函数或常见的解析方法（如分离变量、积分因子等）给出显式解。然而，在物理建模、化学反应动力学或金融模型中，这类方程经常出现。

自然出现的问题是：*即使写不出解的公式，是否仍然可以数值地获得解的近似值？*例如，$y(1)$ 和 $y(2)$ 约为多少？

数值方法正是围绕这一类问题发展起来的。本章后续的理论和方法，可以理解为对以下问题的系统回答：*在何种条件和分析框架下，计算出的近似值可以保证接近真实解，并且误差行为可控？*

值得强调的是，即使在许多**线性**微分方程中，当伴随复杂的边界条件、高维状态空间或复杂系数时，同样很难得到显式解析解，必须依赖数值方法。因此，数值方法并非仅用于“解析上无解”的个别方程，而是现代微分方程分析与计算中的基础工具。

本章的主线模型：非齐次线性方程　虽然从一个非线性问题出发，但在本章的理论分析中，主要围绕一个更一般、且结构清晰的模型来建立理论：

$$\begin{cases} y'(t) = -\lambda y(t) + g(t), \quad \lambda > 0, \\ y(0) = y_0, \end{cases} \tag{6.2}$$

其中 $g(t)$ 是给定的函数（称为**源项**或**强迫项**）。

注意 (学习定位). 在首次阅读本章时，无需立即掌握模型 (6.2) 的全部理论含义。可以将它视为本章的“运行示例”：后续所有数值格式（Euler 方法、Runge-Kutta 方法、多步格式等）都会在这个模型上进行测试和分析，类似于在编程课程中使用同一个示例程序来演示不同语法结构。随着对方法本身理解的加深，再回顾该模型在稳定性、误差传播和刚性分析中的作用，会更加自然。

为什么选择这个模型？这个非齐次线性方程具有以下关键优势：

- **包含测试方程：**当 $g(t) \equiv 0$ 时，它退化为纯指数衰减 $y' = -\lambda y$，这是我们熟悉的稳定性测试方程。因此，所有基于测试方程的稳定性分析都完全保留。
- **线性问题的一般形式：**任何一维线性初值问题都可以写成这种形式。更重要的是，通过线性化，一般非线性问题 $y' = f(t, y)$ 在点 (t_n, y_n) 附近可以近似为这种形式，其中 $\lambda = -f_y(t_n, y_n)$，$g(t) = f(t_n, y_n) + f_y(t_n, y_n)y_n$。
- **有解析解，便于误差分析：**该问题的精确解为

$$y(t) = e^{-\lambda t} y_0 + \int_0^t e^{-\lambda(t-s)} g(s)\,\mathrm{d}s. \tag{6.3}$$

  这个公式揭示了数值 ODE 的本质：解 = 衰减核（稳定性）+ 积分算子（数值积分误差）。第一项体现了齐次解的衰减行为，第二项体现了强迫项的影响，这正是数值误差分析的核心结构。

本章结构：在 Euler 方法、稳定性分析、Runge-Kutta 方法等各个部分中，将反复使用这一模型，以建立统一的数值分析框架。在章末，将回到最初的非线性问题，说明如何通过局部线性化将本章的理论应用到一般问题。

本章结构说明　本章将围绕初值问题的数值求解展开。首先回顾常微分方程的基本理论，说明数值求解存在的合理性；随后引入时间离散化思想，并从最基本的 Euler 方法出发，系统讨论单步格式的构造、误差、稳定性与收敛性理论；最后介绍高阶 Runge-Kutta 方法，作为工程和科学计算中的标准工具。

6.1　一般求解原理

6.1.1　微分方程理论要素

本小节的目标是回顾常微分方程解的存在唯一性和高阶方程的标准形式，为数值方法的构造与误差分析提供连续问题层面的理论前提。

在讨论数值方法之前，需要回答一个看似基础但极其重要的问题：*给定一个初值问题，它是否真的“有解”？这个解是否唯一？*

这个问题的重要性在于：如果一个问题本身可能有多个解，那么任何数值算法都无法判断它计算出来的结果究竟逼近的是哪一个解。更糟糕的是，如果问题本身没有解，那么数值方法计算出的“解”就完全没有意义。

因此，存在性与唯一性理论为数值计算提供了逻辑前提。只有在连续问题具有唯一解的前提下，关于“如何用数值方法逼近这个解”的讨论才有意义。本节介绍保证解存在唯一的经典理论——Cauchy-Lipschitz 定理（也称为 Picard-Lindelöf 定理）。

为了建立数值方法的理论基础，我们首先回顾微分方程的基本概念和理论。

定义 6.1.1 (一阶常微分方程). 设 I 是 $\mathbb{R}$ 的一个区间，$f : I \times \mathbb{R} \to \mathbb{R}$ 是一个连续可微函数。如果对于所有

$t \in I$ 有

$$\hat{y}'(t) = f(t, \hat{y}(t)),$$

则称连续可导函数 $\hat{y} : I \to \mathbb{R}$ 是微分方程

$$y' = f(t, y) \tag{6.4}$$

的解。

单独的微分方程通常不足以定义唯一解。通常，我们给它添加一个初始条件：

定义 6.1.2 (初值问题)**.** 微分方程 (6.4) 与初始条件

$$y(t_0) = y_0, \tag{6.5}$$

其中 $t_0 \in I$ 和 $y_0 \in \mathbb{R}$ 是给定的，构成一个**初值问题**（Initial Value Problem, IVP）。

定理 6.1.1 (Picard-Lindelöf 存在唯一性定理 *)**.** 设 $f : I \times \mathbb{R} \to \mathbb{R}$ 在区域 $D = \{(t, y) : |t - t_0| \le a, |y - y_0| \le b\}$ 上连续，且关于 y 满足 Lipschitz 条件：

$$|f(t, y_1) - f(t, y_2)| \le L|y_1 - y_2|, \quad \forall (t, y_1), (t, y_2) \in D,$$

其中 $L > 0$ 是 Lipschitz 常数。则存在 $\delta > 0$，使得初值问题

$$\begin{cases} y' = f(t, y), \\ y(t_0) = y_0 \end{cases}$$

在区间 $[t_0 - \delta, t_0 + \delta] \subset I$ 上存在唯一解。

注：δ 可以通过 f 在 D 上的界和 Lipschitz 常数显式给出。例如，设 $M = \max\limits_{(t,y)\in D} |f(t, y)|$，则 δ 可以取为 $\min\{a, b/M\}$。

注意 (数值分析视角下的关键结论)*.* 从数值计算的角度看，本节的主要作用在于说明以下三点：

- 在常见的正则性假设（如 Lipschitz 条件）下，初值问题在某个邻域内**确实存在唯一解**；
- Lipschitz 条件保证了解对初始值和右端项的**连续依赖性**，即对微小扰动不会过分敏感；
- 这些性质构成了数值方法可以稳定逼近真实解的**理论前提**：如果连续问题本身不适定，再好的格式也无法给出可靠的数值结果。

至于定理的完整证明，只需了解其核心思想是：通过将微分方程改写为积分方程，并构造一个逐步逼近真解的迭代序列，利用不动点理论证明解的存在性与唯一性。

证明. **证明思路说明**：证明将分为三个关键步骤：

- **步骤 1**：将微分方程改写为等价的积分方程；
- **步骤 2-3**：构造一个函数序列（Picard 迭代序列），并证明它在适当区间上一致收敛；
- **步骤 4-5**：证明极限函数满足原微分方程，并且是唯一的。

读者在第一次阅读时，可以暂时忽略技术细节，只需把握每一步的逻辑目标：*我们试图通过构造一个“越来越接近解”的函数序列，来证明解的存在性。*

证明采用 Picard 迭代方法。设 $M = \max\limits_{(t,y)\in D} |f(t, y)|$，取 $\delta = \min\{a, b/M\}$（如果 $M = 0$，则 $f \equiv 0$，解为常数，结论显然成立）。

步骤 1：将微分方程转化为积分方程

初值问题等价于积分方程

$$y(t) = y_0 + \int_{t_0}^{t} f(s, y(s))\,\mathrm{d}s, \quad t \in [t_0 - \delta, t_0 + \delta]. \tag{6.6}$$

步骤 2：定义 Picard 迭代序列

定义函数序列 $\{y_n(t)\}_{n=0}^{\infty}$：

$$\begin{aligned} y_0(t) &= y_0, \\ y_{n+1}(t) &= y_0 + \int_{t_0}^{t} f(s, y_n(s))\,\mathrm{d}s, \quad n \geq 0. \end{aligned} \tag{6.7}$$

步骤 3：证明序列的一致有界性

首先证明：对于所有 $n \geq 0$ 和 $t \in [t_0 - \delta, t_0 + \delta]$，有 $|y_n(t) - y_0| \leq b$。

当 $n = 0$ 时，$y_0(t) = y_0$，结论显然成立。

假设对某个 n，$|y_n(t) - y_0| \leq b$ 对所有 $t \in [t_0 - \delta, t_0 + \delta]$ 成立。则

$$|y_{n+1}(t) - y_0| = \left| \int_{t_0}^{t} f(s, y_n(s))\,\mathrm{d}s \right| \leq M|t - t_0| \leq M\delta \leq b,$$

由归纳法，结论对所有 n 成立。

步骤 4：证明序列的一致收敛性

定义 $d_n(t) = |y_{n+1}(t) - y_n(t)|$。我们有

$$d_0(t) = |y_1(t) - y_0| = \left| \int_{t_0}^{t} f(s, y_0)\,\mathrm{d}s \right| \leq M|t - t_0| \leq M\delta, \tag{6.8}$$

且

$$\begin{aligned} d_n(t) = |y_{n+1}(t) - y_n(t)| &= \left| \int_{t_0}^{t} [f(s, y_n(s)) - f(s, y_{n-1}(s))]\,\mathrm{d}s \right| \\ &\leq L \int_{t_0}^{t} |y_n(s) - y_{n-1}(s)|\,\mathrm{d}s = L \int_{t_0}^{t} d_{n-1}(s)\,\mathrm{d}s. \end{aligned} \tag{6.9}$$

通过归纳法可以证明：

$$d_n(t) \leq \frac{ML^n|t - t_0|^{n+1}}{(n+1)!} \leq \frac{ML^n\delta^{n+1}}{(n+1)!}.$$

因此，对于 $m > n$，

$$|y_m(t) - y_n(t)| \leq \sum_{k=n}^{m-1} d_k(t) \leq \sum_{k=n}^{m-1} \frac{ML^k\delta^{k+1}}{(k+1)!} \leq M\delta \sum_{k=n}^{\infty} \frac{(L\delta)^k}{k!}.$$

由于级数 $\sum_{k=0}^{\infty} \frac{(L\delta)^k}{k!} = e^{L\delta}$ 收敛，当 $n \to \infty$ 时，$\sum_{k=n}^{\infty} \frac{(L\delta)^k}{k!} \to 0$。因此，$\{y_n(t)\}$ 在 $[t_0 - \delta, t_0 + \delta]$ 上一致收敛到某个函数 $y(t)$。

步骤 5：证明极限函数是解

由于 f 连续且关于 y 满足 Lipschitz 条件，$f(s, y_n(s))$ 一致收敛到 $f(s, y(s))$。因此，在积分方程 (6.6) 中取极限，得到

$$y(t) = y_0 + \int_{t_0}^{t} f(s, y(s))\,\mathrm{d}s,$$

这说明 $y(t)$ 是积分方程的解，从而是初值问题的解。

步骤 6：证明唯一性

设 $y_1(t)$ 和 $y_2(t)$ 都是初值问题的解。定义 $d(t) = |y_1(t) - y_2(t)|$。则

$$d(t) = \left| \int_{t_0}^{t} [f(s, y_1(s)) - f(s, y_2(s))]\,\mathrm{d}s \right| \leq L \int_{t_0}^{t} d(s)\,\mathrm{d}s.$$

由 Gronwall 引理（或直接通过迭代），可以证明 $d(t) = 0$ 对所有 $t \in [t_0 - \delta, t_0 + \delta]$ 成立，因此 $y_1(t) \equiv y_2(t)$。

注：Gronwall 引理指出，如果 $u(t) \geq 0$ 且 $u(t) \leq C + L \int_{t_0}^{t} u(s)\,\mathrm{d}s$，则 $u(t) \leq Ce^{L|t-t_0|}$。在我们的情况下，$C = 0$，因此 $d(t) = 0$。 □

注：虽然 Picard-Lindelöf 定理保证了在 Lipschitz 条件下初值问题有唯一解，但在实际应用中，我们可能会遇到不满足该定理条件的情况。例如，当 f 不满足 Lipschitz 条件时，初值问题可能有多解甚至无穷多解（见下面的例子）。

例 6.1.1 (多解问题)**.** 考虑初值问题

$$\begin{cases} y'(t) = \sqrt{y(t)}, & t > 0, \\ y(0) = 0. \end{cases}$$

这个问题有两个明显的解：

$$y_1(t) = 0, y_2(t) = \frac{t^2}{4}.$$

实际上，对于任意实数 $a > 0$，函数

$$y_a(t) = \begin{cases} 0, & 0 \le t \le a, \\ \dfrac{(t-a)^2}{4}, & a \le t \end{cases}$$

也是解。因此，这个问题有无穷多个解。

数值意义说明：对于上述问题，如果仅依赖数值格式而不额外施加选择条件，不同的初始扰动或舍入误差可能导致数值解趋向不同的真实解。例如，当数值格式在某个时刻产生一个微小的正值（由舍入误差引起）时，解的演化可能转向非零解 $y_2(t) = t^2/4$，而不是保持在零解上。

这说明：*数值计算的可靠性依赖于问题本身的适定性*。当一个问题的解不唯一时，数值方法无法自动判断应该逼近哪一个解，这可能导致数值结果的不确定性。因此，在应用数值方法之前，首先验证问题的适定性（存在性与唯一性）是至关重要的。

这一反例也表明：数值方法本身并不会“自动修正”不恰当的数学模型，它只能忠实反映所给定方程的性质。如果模型允许多解或不稳定行为，数值解也只能在这一前提下理解和使用。

高阶微分方程：到目前为止，我们只限于一阶方程。然而，在实际应用中经常遇到高阶方程。例如，在力学中经常遇到二阶方程：

$$y''(t) = g(t, y(t), y'(t)),$$

其中 $g : I \times \mathbb{R}^2 \to \mathbb{R}$。

二阶方程需要两个额外条件才能有唯一解。对于初值问题，这通常对应于变量 t 表示时间的情况，这些条件的形式为：

$$y(0) = y_0 \quad 和 \quad y'(0) = y_1,$$

即物理上，初始时刻的位置和速度是给定的。

高阶方程的标准形式：任何 m 阶微分方程都可以转化为一阶微分方程组。设

$$\boldsymbol{U}(t) = \begin{pmatrix} U_1(t) \\ U_2(t) \\ \vdots \\ U_m(t) \end{pmatrix} = \begin{pmatrix} y(t) \\ y'(t) \\ \vdots \\ y^{(m-1)}(t) \end{pmatrix}, \quad \boldsymbol{U}_0 = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{m-1} \end{pmatrix}.$$

则 m 阶方程

$$y^{(m)}(t) = g(t, y(t), y'(t), \ldots, y^{(m-1)}(t))$$

可以写成

$$\boldsymbol{U}'(t) = \begin{pmatrix} U_2(t) \\ U_3(t) \\ \vdots \\ U_m(t) \\ g(t, U_1(t), U_2(t), \ldots, U_m(t)) \end{pmatrix} = \boldsymbol{F}(t, \boldsymbol{U}(t)),$$

其中

$$\boldsymbol{F}(t,\boldsymbol{U}) = \begin{pmatrix} U_2 \\ U_3 \\ \vdots \\ U_m \\ g(t,U_1,U_2,\ldots,U_m) \end{pmatrix}.$$

因此，我们归结为问题：

$$\boldsymbol{U}'(t) = \boldsymbol{F}(t,\boldsymbol{U}(t)), \quad \boldsymbol{U}(0) = \boldsymbol{U}_0,$$

它与问题 (6.4)–(6.5) 具有相同的形式。我们将高阶微分方程化为标准形式。

例 6.1.2 (二阶方程的标准形式)**.** 考虑二阶初值问题

$$\begin{cases} y''(t) = -y(t), \quad t \geq 0, \\ y(0) = 1, \\ y'(0) = 0. \end{cases}$$

引入 $U_1(t) = y(t)$，$U_2(t) = y'(t)$，则

$$\boldsymbol{U}'(t) = \begin{pmatrix} U_2(t) \\ -U_1(t) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} U_1(t) \\ U_2(t) \end{pmatrix} = \boldsymbol{A}\boldsymbol{U}(t),$$

其中 $\boldsymbol{A} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ 是常数矩阵。初始条件为 $\boldsymbol{U}(0) = (1,0)^T$。这是一个一阶线性微分方程组。

边值问题：对于同一个微分方程，还存在另一种类型的问题：**边值问题**（Boundary Value Problem, BVP）。两个条件分别在区间的两个端点给出，例如：

$$y(0) = y_0 \quad 和 \quad y(T) = y_T.$$

这第二种类型的问题对应于完全不同的物理情况，导致完全不同的数值求解方法。本章只关注初值问题。

线性系统的情形　对于线性系统 $\boldsymbol{U}'(t) = \boldsymbol{A}\boldsymbol{U}(t)$，其中 $\boldsymbol{A}$ 是常数矩阵，数值格式的稳定性与矩阵 $\boldsymbol{A}$ 的谱性质密切相关。具体地，如果 $\boldsymbol{A}$ 的所有特征值都具有负实部（稳定系统），则显式格式可能需要较小的步长来保持稳定性，而隐式格式通常具有更好的稳定性性质。这一观察为后续学习刚性问题的数值方法提供了重要基础。

6.1.2　数值方法的原理

本小节的目标是从时间离散化的角度说明单步格式的基本构造思想，并给出显式与隐式 Euler 格式作为后续分析的基准模型。

设 $y(t)$ 是初值问题

$$\begin{cases} y'(t) = f(t,y(t)), \quad t \in [t_0, t_0+T], \\ y(t_0) = y_0 \end{cases} \tag{6.10}$$

的精确解。

离散化策略：一般原理是将区间 $I = [t_0, t_0+T]$ 离散化，引入点 $t_0, t_1, \ldots, t_N = t_0 + T$，这些点可以是等距的，但不是必须的。量 $h_n = t_{n+1} - t_n$ 称为**步长**。目标是在 $n = 1, \ldots, N$ 的各个网格点上，通过迭代过程计算数值解 $\{z_n\}$ 来逼近精确解 $\{y(t_n)\}$。

最基本的构造思想：首先写出 $y(t)$ 在 $t = t_n$ 处的 Taylor 展开。由于解至少对 t 可导一次，可以写成

$$y(t_{n+1}) = y(t_n + h) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(\xi), \quad \xi \in [t_n, t_{n+1}],$$

即

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + \mathscr{O}(h^2).$$

如果我们假设 h 足够小，那么可以提出以下迭代格式：

$$\begin{cases} z_0 = y_0, \\ z_{n+1} = z_n + hf(t_n, z_n), \quad 0 \le n \le N-1. \end{cases} \tag{6.11}$$

这个迭代过程称为**显式 Euler 格式**（或简单 Euler 格式）。

格式的分类：我们区分两大类用于初值问题的数值求解格式：

- **单步格式**：为了计算所求函数在点 t_{n+1} 处的值的近似值，我们只使用点 t_n 处的值。主要优点是允许在计算过程中根据误差估计非常容易地改变步长。
- **多步格式**：在这些方法中，为了计算所求函数在点 t_{n+1} 处的值的近似值，我们使用在 $t_n, t_{n-1}, \ldots$ 处计算的值。这允许较低的计算成本，但使值更加相互依赖，因此在计算过程中局部改变步长更加困难。

本章主要关注单步格式。

6.2 单步格式

单步格式的核心问题有三个：**如何构造？有多准？会不会发散？**

- **如何构造？**我们将从数值积分角度出发，介绍 Euler 格式和 Runge-Kutta 格式的构造方法。
- **有多准？**通过局部误差、阶和一致性的概念来量化格式的精度。
- **会不会发散？**通过稳定性和收敛性理论来保证数值解的正确性。

6.2.1 从数值积分出发的 Euler 格式

本小节的目标是从数值积分公式出发推导显式与隐式 Euler 格式，并在主线模型上刻画它们对误差传播和稳定性的影响。

直观解释：用切线预测未来 显式 Euler 方法的思想可以概括为：*用当前点的切线近似未来一小段时间内的解的变化。*

在时间 t_n，已知 $y(t_n)$ 和斜率 $y'(t_n) = f(t_n, y(t_n))$，于是用一条直线来近似曲线在接下来一小段时间内的行为。这条直线的方程是：

$$\ell(t) = y(t_n) + (t - t_n) f(t_n, y(t_n)),$$

它在 $t = t_n$ 处与曲线 $y(t)$ 相切。如果我们相信这条切线在短时间内是曲线的良好近似，那么下一个时刻 $t_{n+1} = t_n + h$ 的函数值可以近似为：

$$y(t_{n+1}) \approx \ell(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)).$$

这给出了 Euler 方法的几何直观：*沿着切线走一小步，然后在新的点上重复这一过程。*

从积分角度理解：为了定义某些微分方程数值求解格式，注意到精确解 $y(t)$ 满足 $y'(t) = f(t, y(t))$，这给出

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\,\mathrm{d}t.$$

然后可以考虑使用 $f(t, y(t))$ 在区间 $[t_n, t_{n+1}]$ 上的值来近似积分，尽管 $y(t)$ 在这个区间上未知。

为了简化书写，我们假设步长 $h_n = t_{n+1} - t_n$ 是常数，$h = T/N$，其中 N 是整数，但推广到非常数步长是显然的。

显式 Euler 格式：首先，使用“左”矩形方法进行积分的近似计算（这等价于在 $t = t_n$ 处对 $y(t)$ 应用 Taylor 公式）。因此可以提出以下格式：

$$z_{n+1} = z_n + hf(t_n, z_n), \quad 0 \le n \le N-1, \tag{6.12}$$

这就是**显式 Euler 格式**。

主线模型的应用：非齐次线性方程　让我们将显式 Euler 格式应用到主线模型 (6.2) 上。对于 $y' = -\lambda y + g(t)$，显式 Euler 格式给出：

$$z_{n+1} = z_n + h[-\lambda z_n + g(t_n)] = (1-\lambda h)z_n + hg(t_n). \tag{6.13}$$

这个公式揭示了几个重要特征：

- **数值解不再是纯几何级数**：与纯指数衰减 $y' = -\lambda y$ 的情况不同，这里出现了强迫项 $hg(t_n)$，使得数值解的结构更加丰富。
- **误差传播的叠加结构**：齐次部分的误差通过因子 $(1-\lambda h)$ 传播，而强迫项的数值误差在每个时间步都会引入新的贡献。这体现了误差传播的叠加性质。
- **稳定性与一致有界性的分离**：即使齐次部分满足稳定性条件 $|1-\lambda h| < 1$，如果强迫项 $g(t)$ 很大或变化剧烈，数值解仍可能无界。这说明了稳定性分析需要考虑齐次部分和强迫项的共同影响。

类似地，隐式 Euler 格式应用到主线模型给出：

$$z_{n+1} = z_n + h[-\lambda z_{n+1} + g(t_{n+1})], \tag{6.14}$$

整理后得到：

$$z_{n+1} = \frac{1}{1+\lambda h} z_n + \frac{h}{1+\lambda h} g(t_{n+1}). \tag{6.15}$$

注意到对于任意 $h > 0$，都有 $\left|\dfrac{1}{1+\lambda h}\right| < 1$，因此隐式 Euler 格式对主线模型是**无条件稳定的**，这是隐式方法的一个重要优势。

隐式 Euler 格式：也可以使用右矩形方法近似积分。等价的计算来自在 $t = t_{n+1}$ 处对 $y(t)$ 应用 Taylor 公式：

$$y(t_n) = y(t_{n+1}) - hf(t_{n+1}, y(t_{n+1})) + \frac{h^2}{2} y''(\xi), \quad \xi \in [t_n, t_{n+1}].$$

这导致格式

$$z_{n+1} = z_n + hf(t_{n+1}, z_{n+1}), \quad 0 \le n \le N-1, \tag{6.16}$$

我们称之为**隐式 Euler 格式**（或后向 Euler 格式）。

在每一步中，(6.16) 都要求解一个关于 z_{n+1} 的（通常是非线性）方程。在本章中，我们主要关注隐式格式在稳定性和精度上的性质，而不详细展开不动点迭代或 Newton 法等求解技术，暂时将其视为可以通过适当算法可靠求解的子问题。

这个格式称为**隐式的**，因为 z_{n+1} 被隐式定义为方程

$$x = z_n + hf(t_{n+1}, x)$$

的解，这个方程通常是非线性的。在实际计算中，可以使用不动点或 Newton 类型的方法求解。由于步长 h 很小，实践中所需的迭代次数通常不多。初始迭代值可以取为上一步的近似值 z_n。

例 6.2.1 (显式与隐式 Euler 格式的应用). 考虑初值问题

$$\begin{cases} y' = -2y(t), \quad t \ge 0, \\ y(0) = 1. \end{cases}$$

精确解为 $y(t) = e^{-2t}$。

使用步长 $h=0.1$，显式 Euler 格式给出：

$$z_{n+1}=z_n+h(-2z_n)=z_n(1-2h)=z_n\cdot 0.8,$$

因此 $z_n=0.8^n$。

隐式 Euler 格式给出：

$$z_{n+1}=z_n+h(-2z_{n+1})=z_n-2hz_{n+1},$$

因此 $z_{n+1}=\dfrac{z_n}{1+2h}=\dfrac{z_n}{1.2}$，即 $z_n=\left(\dfrac{1}{1.2}\right)^n$。

在 $t=1$ 处，精确解为 $y(1)=e^{-2}\approx 0.1353$，显式 Euler 给出 $z_{10}=0.8^{10}\approx 0.1074$，隐式 Euler 给出 $z_{10}=(1/1.2)^{10}\approx 0.1615$。

6.2.2 单步预测-校正格式

本小节的目标是说明预测-校正思想如何在单步格式中结合显式与隐式方法，从而在保持较好稳定性的同时控制计算代价。

重新考虑前面定义的隐式 Euler 格式 (6.16)：

$$z_{n+1}=z_n+hf(t_{n+1},z_{n+1}),\quad 0\le n\le N-1.$$

可以构造一个新的格式，称为**预测-校正格式**：

- 确定 z_{n+1} 的第一次粗略估计，记作 $\tilde{z}_{n+1}$，例如使用显式 Euler 方法；
- 通过借鉴隐式 Euler 格式来改进这个估计。

我们得到格式：

$$\begin{cases}\tilde{z}_{n+1}=z_n+hf(t_n,z_n),\\ z_{n+1}=z_n+hf(t_{n+1},\tilde{z}_{n+1}).\end{cases}\tag{6.17}$$

在经典术语中，首先使用显式格式进行**预测**（$\tilde{z}_{n+1}$），然后使用隐式格式进行**校正**。在需要时，可以对校正步骤进行迭代。

梯形格式：再看另一个例子。如果使用梯形方法计算积分

$$y(t_{n+1})=y(t_n)+\int_{t_n}^{t_{n+1}}f(t,y(t))\,\mathrm{d}t,$$

这给出表达式

$$y(t_{n+1})=y(t_n)+\frac{h}{2}(f(t_n,y(t_n))+f(t_{n+1},y(t_{n+1})))-\frac{h^3}{12}y^{(3)}(\xi),\quad \xi\in[t_n,t_{n+1}],$$

其中 $y^{(3)}$ 对应于 $t\mapsto f(t,y(t))$ 对 t 的二阶导数。这导致以下格式：

$$z_{n+1}=z_n+\frac{h}{2}(f(t_n,z_n)+f(t_{n+1},z_{n+1})),\quad 0\le n\le N-1,\tag{6.18}$$

它是隐式的，就像隐式 Euler 格式一样。

Euler-Cauchy 预测-校正格式：可以像之前一样，构造一个受此隐式格式启发的新的预测-校正格式，这就是 **Euler-Cauchy 预测-校正格式**：

$$\begin{cases}\tilde{z}_{n+1}=z_n+hf(t_n,z_n),\\ z_{n+1}=z_n+\dfrac{h}{2}(f(t_n,z_n)+f(t_{n+1},\tilde{z}_{n+1})).\end{cases}\tag{6.19}$$

6.2.3 Euler 方法的初步研究：稳定性与误差传播

让我们将 Euler 格式应用到主线模型 (6.2) 上，深入分析数值方法的稳定性与误差传播机制。

齐次情况：纯指数衰减　首先考虑特殊情况 $g(t) \equiv 0$，即纯指数衰减：

$$\begin{cases} y' = -\lambda y(t), \quad \lambda > 0, \\ y(0) = y_0. \end{cases} \tag{6.20}$$

这个问题有唯一精确解：$y(t) = y_0 e^{-\lambda t}$。

显式 Euler 格式应用到该问题给出：

$$z_{n+1} = (1 - \lambda h) z_n,$$

因此 $z_n = (1 - \lambda h)^n y_0$。

隐式 Euler 格式给出：

$$z_{n+1} = \frac{1}{1 + \lambda h} z_n,$$

因此 $z_n = y_0/(1 + \lambda h)^n$。

稳定性分析：我们首先注意到精确解满足 $\lim\limits_{t \to +\infty} y(t) = 0$。

如果我们对大的 n 值使用 Euler 格式，为简化书写，假设步长 h 是常数，则必须有 $\lim\limits_{n \to +\infty} z_n = 0$。对于显式 Euler，这个条件只有在 $|1 - \lambda h| < 1$ 时才会满足，即对于 $\lambda > 0$，需要 $h < 2/\lambda$。这个条件可能非常严格。

物理解释：显式 Euler 的不稳定性来源于对指数衰减的线性近似。当步长过大时，这种近似会将衰减误认为振荡甚至增长，导致数值解发散。具体地，当 $h > 2/\lambda$ 时，$|1 - \lambda h| > 1$，数值解会指数增长，与精确解的衰减行为完全相反。

工程语言解释：换言之，显式 Euler 方法并不是 ‘‘算错了’’，而是由于步长过大，它错误地理解了系统的物理行为，把一个应当衰减的过程数值上放大了。这就像用一条直线去近似一条快速下降的曲线：如果步长太大，直线会 ‘‘冲过头’’，导致预测值反而比当前值更大，从而产生不稳定的振荡甚至发散。因此，在使用显式 Euler 方法时，必须确保步长足够小，以满足稳定性条件 $h < 2/\lambda$。

对于隐式 Euler 格式，序列 z_n 在 n 趋于无穷时收敛到 0，对 h 没有任何条件（只要 $h > 0$）。因此，我们可以使用这种方法，而无需先验的严格步长限制条件。是期望的精度决定了步长：步长越小，精度越好。

非齐次情况：稳定性与一致有界性的分离　现在考虑完整的非齐次方程 (6.2)。精确解为

$$y(t) = e^{-\lambda t} y_0 + \int_0^t e^{-\lambda(t-s)} g(s) \, \mathrm{d}s.$$

显式 Euler 格式给出：

$$z_{n+1} = (1 - \lambda h) z_n + h g(t_n).$$

关键观察：即使齐次部分满足稳定性条件 $|1 - \lambda h| < 1$，如果强迫项 $g(t)$ 很大或变化剧烈，数值解仍可能无界。这说明了**稳定性与一致有界性的分离**。

具体地，即使 $|1 - \lambda h| < 1$（齐次部分稳定），如果 $g(t_n)$ 在某个时间步很大，那么 $hg(t_n)$ 项可能主导数值解，导致数值解偏离精确解。这体现了非齐次问题的一个重要特征：*即使数值格式对齐次问题是稳定的，不恰当的步长仍可能放大强迫项引入的误差。*

稳定性不是格式的 ‘‘附加性质’’：稳定性不是格式的 ‘‘附加性质’’，而是数值解能否反映真实动力学的核心。对于非齐次问题，我们需要同时考虑：

- **零稳定性**：齐次部分的衰减行为（由 $|1 - \lambda h|$ 控制）；
- **一致有界性**：强迫项误差的累积（由 $g(t)$ 的性质和步长 h 共同决定）。

这种分析框架为后续讨论刚性问题和隐式方法提供了自然的理论基础。

收敛性研究：这次，我们固定一个点 t。我们选择步长 h，使得总是存在一个整数 n 使得 $nh = t$。然后，我们希望 z_n 是 $y(t)$ 的近似值。更好的是，我们希望当 $h \to 0$ 时，在约束 $nh = t$ 下，z_n 收敛到 $y(t)$。

收敛性证明：对于 Euler 方法，这个性质是满足的。详细证明如下：

显式 Euler 格式：$z_n = (1 - \lambda h)^n y_0$，其中 $nh = t$。因此

$$z_n = y_0 (1 - \lambda h)^{t/h} = y_0 \left[(1 - \lambda h)^{-1/(\lambda h)} \right]^{-\lambda t}.$$

当 $h \to 0$ 时，$\lambda h \to 0$，且

$$\lim_{u\to 0}(1+u)^{1/u} = e,$$

因此

$$\lim_{h\to 0}(1-\lambda h)^{-1/(\lambda h)} = \lim_{u\to 0}(1+u)^{1/u} = e,$$

其中 $u = -\lambda h$。因此

$$\lim_{h\to 0, nh=t} z_n = y_0 e^{-\lambda t} = y(t).$$

隐式 Euler 格式：$z_n = y_0/(1+\lambda h)^n$，其中 $nh = t$。类似地，

$$z_n = y_0(1+\lambda h)^{-t/h} = y_0\left[(1+\lambda h)^{1/(\lambda h)}\right]^{-\lambda t} \to y_0 e^{-\lambda t} = y(t),$$

当 $h \to 0$ 时。

我们强调这是两个不同且互补的观点。第一个（稳定性）只在 t 大时才有意义。实际上，t 大可能很快达到。因此，当我们进行天气预报时，$t = 10$ 天是非常大的，正是稳定性问题在实践中限制了这些预测的范围。

相反，如果我们只关注 “小” 的 t，只有收敛性问题重要。

6.2.4 单步格式的阶和一致性

给定 f, t_0, y_0, T，设 $y(t)$ 是初值问题 (6.10) 的精确解。

注意 (局部误差、阶、一致性与收敛性的关系概览). 在后续讨论中，将反复出现若干基本概念：

- **局部误差**：假设前一步是精确解时，单步格式在一步中引入的误差，其阶数刻画了格式的**局部精度**；
- **阶与一致性**：局部误差为 $\mathscr{O}(h^{p+1})$ 意味着格式是 p 阶的，从而自动一致（局部误差相对于步长趋于 0）；
- **稳定性**：描述全局误差和扰动在迭代过程中是否被放大，是从“局部精度”走向“全局可靠性”的关键桥梁；
- **收敛性**：当步长 $h \to 0$ 时，全局误差趋于 0。对单步格式而言，**一致性 + 稳定性**蕴含收敛性（定理 6.2.2）。

可以将这一关系粗略概括为：局部误差阶数 $\to$ 阶与一致性 $\to$ 在稳定性条件下推得收敛性。

为了简化说明，我们假设使用常数步长 $h = T/N$ 进行离散化，设 $t_n = t_0 + nh$。显式或预测-校正的单步格式可以写成一般形式

$$\begin{cases} z_0 = y_0, \\ z_{n+1} = z_n + h\varphi(t_n, z_n, h), \quad 0 \le n \le N-1. \end{cases} \tag{6.21}$$

我们可能想知道如何量化用离散序列 z_n 近似精确解 $y(t)$ 时产生的误差。以下定义给出了这个问题的初步答案。

定义 6.2.1 (局部误差). 我们称相对于格式 (6.21) 的**局部误差**为量

$$\tau_{n+1}(h) = (y(t_{n+1}) - y(t_n)) - h\varphi(t_n, y(t_n), h), \quad n = 0, \ldots, N-1,$$

其中 $y(t)$ 是微分方程 (6.10) 的精确解。

换句话说，$\tau_{n+1}(h) = y(t_{n+1}) - z_{n+1}$ 如果我们假设 $z_n = y(t_n)$。

重要说明：需要强调的是，局部误差并不是计算过程中真实发生的误差，而是假设前一步是精确解时，在单步中引入的误差。实际计算中的**全局误差** $e_n = y(t_n) - z_n$ 会累积局部误差，并可能被格式的稳定性性质放大或控制。

更精确地，设 $\{z_n\}$ 是由格式生成的数值解，$\{y(t_n)\}$ 是精确解在网格点上的值。全局误差满足递推关系：

$$e_{n+1} = e_n + h[\varphi(t_n, y(t_n), h) - \varphi(t_n, z_n, h)] + \tau_{n+1}(h).$$

如果格式是稳定的（即 φ 关于第二个参数满足 Lipschitz 条件），则全局误差的增长是可控的。实际上，如果格式是 p 阶的且稳定的，则全局误差为 $\mathscr{O}(h^p)$（见定理 6.2.2 的证明）。

定义 6.2.2 (格式的阶)**.** 当存在常数 $K>0$ 使得

$$\max_{1\le n\le N}\frac{|\tau_n(h)|}{h}\le Kh^p,$$

格式称为**阶数为** p。

定义 6.2.3 (一致性)**.** 当

$$\lim_{h\to 0}\max_{1\le n\le N}\frac{|\tau_n(h)|}{h}=0$$

时，格式称为**一致的**。

因此，严格正阶的格式是一致的。

Euler 格式的阶数分析：对于简单 Euler 格式，有

$$\tau_{n+1}(h)=(y(t_{n+1})-y(t_n))-hf(t_n,y(t_n)).$$

由于 $y(t)$ 是精确解，$y'(t)=f(t,y(t))$。由 Taylor 展开，存在 $\xi_n\in(t_n,t_{n+1})$ 使得

$$y(t_{n+1})=y(t_n)+hy'(t_n)+\frac{h^2}{2}y''(\xi_n)=y(t_n)+hf(t_n,y(t_n))+\frac{h^2}{2}y''(\xi_n).$$

因此

$$\tau_{n+1}(h)=\frac{h^2}{2}y''(\xi_n).$$

假设 y'' 在 $[t_0,t_0+T]$ 上连续（这要求 f 充分光滑），设 $M=\max\limits_{t\in[t_0,t_0+T]}|y''(t)|$，则有

$$|\tau_{n+1}(h)|\le\frac{M}{2}h^2,$$

因此

$$\frac{|\tau_{n+1}(h)|}{h}\le\frac{M}{2}h.$$

根据定义 6.2.2，简单 Euler 格式是 1 阶的（$p=1$，$K=M/2$）。

例 6.2.2 (Euler-Cauchy 格式的阶)**.** Euler-Cauchy 格式 (6.19) 是 2 阶的。详细证明如下：

设 $y(t)$ 是精确解，$y'(t)=f(t,y(t))$。Euler-Cauchy 格式为：

$$z_{n+1}=z_n+\frac{h}{2}[f(t_n,z_n)+f(t_{n+1},z_n+hf(t_n,z_n))].$$

假设 $z_n=y(t_n)$（局部误差分析），则

$$\tilde{z}_{n+1}=y(t_n)+hf(t_n,y(t_n))=y(t_n)+hy'(t_n).$$

由 Taylor 展开：

$$y(t_{n+1})=y(t_n)+hy'(t_n)+\frac{h^2}{2}y''(t_n)+\frac{h^3}{6}y'''(\xi_1),$$

因此

$$\tilde{z}_{n+1}=y(t_{n+1})-\frac{h^2}{2}y''(t_n)+\mathscr{O}(h^3).$$

又

$$f(t_{n+1},\tilde{z}_{n+1})=f(t_{n+1},y(t_{n+1}))+\frac{\partial f}{\partial y}(t_{n+1},y(t_{n+1}))(\tilde{z}_{n+1}-y(t_{n+1}))+\mathscr{O}((\tilde{z}_{n+1}-y(t_{n+1}))^2).$$

注意到 $f(t_{n+1},y(t_{n+1}))=y'(t_{n+1})$，且

$$y'(t_{n+1})=y'(t_n)+hy''(t_n)+\mathscr{O}(h^2),$$

因此

$$f(t_{n+1}, \tilde{z}_{n+1}) = y'(t_n) + hy''(t_n) - \frac{h^2}{2}y''(t_n)\frac{\partial f}{\partial y}(t_{n+1}, y(t_{n+1})) + \mathscr{O}(h^3).$$

Euler-Cauchy 格式给出：

$$z_{n+1} = y(t_n) + \frac{h}{2}[y'(t_n) + y'(t_n) + hy''(t_n) + \mathscr{O}(h^2)] = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \mathscr{O}(h^3).$$

而精确解满足：

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \frac{h^3}{6}y'''(\xi_2).$$

因此局部误差为：

$$\tau_{n+1}(h) = y(t_{n+1}) - z_{n+1} = \mathscr{O}(h^3),$$

即格式是 2 阶的。

6.2.5 单步格式的稳定性和收敛性

格式的一致性或阶数只是误差的局部指示。测量用 z_i 近似 $y(t_i)$ 的误差的更现实的方法是考虑 $i = 1, \ldots, N$ 时的最大误差，并查看当 $h \to 0$ 时该误差是否趋于零。

定义 6.2.4 (收敛性)**.** 格式 (6.21) 相对于微分方程 (6.10) 称为**收敛的**，如果

$$\lim_{h\to 0} \max_{1\le i\le N} |y(t_i) - z_i| = 0.$$

格式的一致性并不意味它是收敛的，我们将看到这最多是一个必要条件。一个额外条件涉及稳定性的概念：

定义 6.2.5 (稳定性)**.** 格式 (6.21) 称为**稳定的**，如果存在常数 M，使得对于所有 z_0，所有 u_0，所有 $h \le h^*$ 和所有序列 $\{\varepsilon_i\}$，由关系

$$\begin{aligned} z_{i+1} &= z_i + h\varphi(t_i, z_i, h), \\ u_{i+1} &= u_i + h\varphi(t_i, u_i, h) + \varepsilon_i \end{aligned} \tag{6.22}$$

定义的序列 $\{z_i\}$ 和 $\{u_i\}$ 满足条件

$$\forall i = 1, \ldots, N, \quad |z_i - u_i| \le M\left(|z_0 - u_0| + \sum_{k=0}^{i-1}|\varepsilon_k|\right).$$

说明：该定义体现的是对初始误差和扰动的整体控制，在单步格式理论中是常用的稳定性概念。大致上，这意味着稳定格式既不放大初始条件上的误差，也不放大格式中引入的误差：这是一个连续性的概念。可以注意到，稳定性可能依赖于 h。上述定义表明，可能存在一个极限步长 h^*，超过它，稳定格式会变得不稳定。

我们还有一个稳定性的充分条件：

定理 6.2.1 (稳定性的充分条件)**.** 为了使格式稳定，只要存在常数 Λ 使得

$$\forall t \in [t_0, t_0 + T], \quad \forall z, u \in \mathbb{R}, \quad \forall h \in [0, h^*], \quad |\varphi(t, z, h) - \varphi(t, u, h)| \le \Lambda|z - u|.$$

这意味着函数 φ 必须在其第二个参数上满足 Lipschitz 条件。通常我们证明 f 满足 Lipschitz 条件来获得 φ 的这个性质。

证明. 设序列 $\{z_i\}$ 和 $\{u_i\}$ 由定义 6.2.5 中的关系定义：

$$\begin{aligned} z_{i+1} &= z_i + h\varphi(t_i, z_i, h), \\ u_{i+1} &= u_i + h\varphi(t_i, u_i, h) + \varepsilon_i. \end{aligned} \tag{6.23}$$

定义 $e_i = z_i - u_i$。则

$$\begin{aligned} e_{i+1} &= z_{i+1} - u_{i+1} \\ &= z_i + h\varphi(t_i, z_i, h) - u_i - h\varphi(t_i, u_i, h) - \varepsilon_i \\ &= e_i + h[\varphi(t_i, z_i, h) - \varphi(t_i, u_i, h)] - \varepsilon_i. \end{aligned} \tag{6.24}$$

因此

$$|e_{i+1}| \le |e_i| + h|\varphi(t_i, z_i, h) - \varphi(t_i, u_i, h)| + |\varepsilon_i| \le |e_i| + h\Lambda|e_i| + |\varepsilon_i| = (1 + h\Lambda)|e_i| + |\varepsilon_i|.$$

通过递推，我们得到

$$|e_i| \le (1 + h\Lambda)^i |e_0| + \sum_{k=0}^{i-1} (1 + h\Lambda)^{i-1-k} |\varepsilon_k|.$$

由于 $h \le h^*$ 且 $i \le N$，其中 N 是使得 $Nh \le T$ 的最大整数，我们有

$$(1 + h\Lambda)^i \le (1 + h\Lambda)^N \le e^{\Lambda N h} \le e^{\Lambda T}.$$

因此

$$|e_i| \le e^{\Lambda T}|e_0| + e^{\Lambda T} \sum_{k=0}^{i-1} |\varepsilon_k| \le M \left(|e_0| + \sum_{k=0}^{i-1} |\varepsilon_k| \right),$$

其中 $M = e^{\Lambda T}$。这证明了格式是稳定的。 □

在这种情况下，简单 Euler 格式显然是稳定的，因为 $\varphi(t, z, h) = f(t, z)$，如果 f 关于 y 满足 Lipschitz 条件，则 φ 也满足。

以下定理简单而重要；它通常被称为**一致性加稳定性蕴含收敛性**。

定理 6.2.2 (一致性与稳定性蕴含收敛性)**.** 设 φ 是 $t \in [t_0, t_0 + T]$，$z \in \mathbb{R}$ 和 h 的连续函数，定义单步格式 (6.21)。

如果这个单步格式是一致的且它是稳定的，那么它相对于微分方程 (6.10) 是收敛的。

注：这是 Lax 等价思想在常微分方程单步格式中的对应版本。在偏微分方程的数值方法中，Lax 等价定理指出：对于线性初值问题，格式的稳定性等价于收敛性（在一致性条件下）。对于常微分方程的单步格式，我们也有类似的结论：一致性加稳定性蕴含收敛性。

证明. 设 $y(t)$ 是初值问题 (6.10) 的精确解，$\{z_n\}$ 是由格式 (6.21) 生成的数值解。

步骤 1：建立误差递推关系

定义全局误差 $e_n = y(t_n) - z_n$。我们有 $e_0 = y(t_0) - z_0 = 0$。

对于 $n \ge 0$，精确解满足

$$y(t_{n+1}) = y(t_n) + h\varphi(t_n, y(t_n), h) + \tau_{n+1}(h),$$

其中 $\tau_{n+1}(h)$ 是局部误差。数值解满足

$$z_{n+1} = z_n + h\varphi(t_n, z_n, h).$$

因此

$$\begin{aligned} e_{n+1} &= y(t_{n+1}) - z_{n+1} \\ &= y(t_n) + h\varphi(t_n, y(t_n), h) + \tau_{n+1}(h) - z_n - h\varphi(t_n, z_n, h) \\ &= e_n + h[\varphi(t_n, y(t_n), h) - \varphi(t_n, z_n, h)] + \tau_{n+1}(h). \end{aligned} \tag{6.25}$$

步骤 2：应用稳定性定义

定义序列 $\{u_n\}$ 如下：

$$\begin{aligned} u_0 &= y(t_0) = y_0, \\ u_{n+1} &= u_n + h\varphi(t_n, u_n, h) + \tau_{n+1}(h). \end{aligned} \tag{6.26}$$

注意到 $u_n = y(t_n)$（因为这是精确解满足的递推关系），且 z_n 满足

$$z_{n+1} = z_n + h\varphi(t_n, z_n, h).$$

将 $\tau_{n+1}(h)$ 视为扰动 ε_n，由稳定性的定义 6.2.5，存在常数 M 使得

$$|e_n| = |y(t_n) - z_n| = |u_n - z_n| \le M\left(|u_0 - z_0| + \sum_{k=0}^{n-1}|\tau_{k+1}(h)|\right) = M\sum_{k=0}^{n-1}|\tau_{k+1}(h)|,$$

因为 $u_0 = z_0 = y_0$。

步骤 3：利用一致性

由于格式是一致的，存在函数 $\omega(h)$ 使得 $\omega(h) \to 0$ 当 $h \to 0$，且

$$\max_{1\le k\le N}\frac{|\tau_k(h)|}{h} \le \omega(h).$$

因此，对于 $k = 1, \ldots, N$，有 $|\tau_k(h)| \le h\omega(h)$。

步骤 4：完成收敛性证明

对于 $n = 1, \ldots, N$，我们有

$$|e_n| \le M\sum_{k=0}^{n-1}|\tau_{k+1}(h)| \le Mnh\omega(h) \le MT\omega(h),$$

因为 $nh \le Nh = T$。

因此

$$\max_{1\le n\le N}|y(t_n) - z_n| \le MT\omega(h).$$

当 $h \to 0$ 时，$\omega(h) \to 0$，因此

$$\lim_{h\to 0}\max_{1\le n\le N}|y(t_n) - z_n| = 0,$$

这证明了格式是收敛的。

注：如果格式是 p 阶的，则 $\omega(h) = \mathscr{O}(h^p)$，因此全局误差为 $\mathscr{O}(h^p)$。这表明局部误差的阶数决定了全局误差的阶数（在稳定性条件下）。 □

6.2.6 Runge-Kutta 格式

这些格式允许恢复 Taylor 格式的良好性质（高阶），而不呈现其缺点（计算 f 的连续导数）。通常有两种构造方法。我们在这里介绍一种。

这些是单步格式，因此可以写成形式

$$\begin{cases} z_0 = y_0, \\ z_{n+1} = z_n + h\varphi(t_n, z_n, h), \quad n \ge 0, \end{cases} \tag{6.27}$$

其中 φ 采用以下特殊形式：

$$\varphi(t, z, h) = \sum_{i=1}^{q}\gamma_i k_i, \tag{6.28}$$

k_i 按以下方式递归定义：

$$\begin{cases} k_1 = f(t, z), \\ k_2 = f(t + h\alpha_1, z + h\beta_{11}k_1), \\ k_3 = f(t + h\alpha_2, z + h\beta_{21}k_1 + h\beta_{22}k_2), \\ \vdots \\ k_q = f\left(t + h\alpha_{q-1}, z + h\sum_{p=1}^{q-1}\beta_{q-1,p}k_p\right). \end{cases} \tag{6.29}$$

引入这 q 个中间值的目的是近似一个 q 阶 Taylor 格式。为此，必须确定未知量：

$$[\gamma_i]_{i=1,\ldots,q}, \quad [\alpha_i]_{i=1,\ldots,q-1}, \quad [\beta_{ij}]_{i=1,\ldots,q-1,j=1,\ldots,i}.$$

很难制定一种方法来确定任意 q 值的这些未知量，这就是为什么我们将只在一个例子上进行推理，这里对于 $q=2$，然后我们将不加证明地给出 $q=4$ 时获得的值，这是实践中最常用的值。

2 阶 Runge-Kutta 格式：一个 2 阶 Taylor 格式写成

$$z_{n+1}=z_n+h\psi(t_n,z_n,h),$$

其中

$$\psi(t,z,h)=f(t,z)+\frac{h}{2}\left(\frac{\partial f}{\partial t}(t,z)+f(t,z)\frac{\partial f}{\partial y}(t,z)\right). \tag{6.30}$$

对于 $q=2$，由 (6.28)–(6.29) 定义的函数 $\varphi(t,z,h)$ 写成

$$\varphi(t,z,h)=\gamma_1k_1+\gamma_2k_2=\gamma_1f(t,z)+\gamma_2f(t+h\alpha_1,z+h\beta_1f(t,z)),$$

其中设 $\beta_1=\beta_{11}$。我们将尝试识别未知系数 $\gamma_1,\gamma_2,\alpha_1,\beta_1$，使得 $\varphi(t,z,h)$ 和 $\psi(t,z,h)$ 尽可能接近。

为此，我们使用两个变量的 Taylor 展开展开 $f(t+h\alpha_1,z+h\beta_1f(t,z))$，这给出到 2 阶：

$$f(t+u,z+v)=f(t,z)+u\frac{\partial f}{\partial t}(t,z)+v\frac{\partial f}{\partial y}(t,z)+\mathscr{O}(h^2),$$

因此

$$f(t+h\alpha_1,z+h\beta_1f(t,z))=f(t,z)+h\alpha_1\frac{\partial f}{\partial t}(t,z)+h\beta_1f(t,z)\frac{\partial f}{\partial y}(t,z)+\mathscr{O}(h^2),$$

这允许我们写成

$$\varphi(t,z,h)=(\gamma_1+\gamma_2)f(t,z)+\gamma_2h\left(\alpha_1\frac{\partial f}{\partial t}(t,z)+\beta_1f(t,z)\frac{\partial f}{\partial y}(t,z)\right)+\mathscr{O}(h^2).$$

如果忽略余项，那么可以将此表达式与由 (6.30) 给出的 $\psi(t,z,h)$ 等同；我们因此得到以下方程：

$$\begin{cases}\gamma_1+\gamma_2=1,\\ \gamma_2\alpha_1=\frac{1}{2},\\ \gamma_2\beta_1=\frac{1}{2}.\end{cases} \tag{6.31}$$

由于有四个未知量的三个方程，系数选择有无穷多种可能性；我们这里只给出最流行的：

- $\gamma_1=0$，$\gamma_2=1$，$\alpha_1=\beta_1=\frac{1}{2}$，这给出格式

$$z_{n+1}=z_n+hf\left(t_n+\frac{h}{2},z_n+\frac{h}{2}f(t_n,z_n)\right), \tag{6.32}$$

  称为**中点格式**。

- $\gamma_1=\gamma_2=\frac{1}{2}$，$\alpha_1=\beta_1=1$，这给出众所周知的格式

$$z_{n+1}=z_n+\frac{h}{2}(f(t_n,z_n)+f(t_{n+1},z_n+hf(t_n,z_n))),$$

  因为它是 Euler-Cauchy 格式 (6.19)。

这些格式当然是 2 阶的，因为

$$\varphi(t_n,z_n,h)+\mathscr{O}(h^2)=\psi(t_n,z_n,h),$$

这允许保持所用 Taylor 格式的局部误差的阶。

4 阶 Runge-Kutta 格式（RK4）：最常用的选择是 $q=4$。这给出 4 阶 Runge-Kutta 格式，或简称为 RK4，在工程科学中几乎普遍使用：

$$\begin{cases}k_1=f(t_n,z_n),\\ k_2=f\left(t_n+\frac{h}{2},z_n+\frac{h}{2}k_1\right),\\ k_3=f\left(t_n+\frac{h}{2},z_n+\frac{h}{2}k_2\right),\\ k_4=f(t_{n+1},z_n+hk_3),\\ z_{n+1}=z_n+\dfrac{h}{6}(k_1+2k_2+2k_3+k_4).\end{cases} \tag{6.33}$$

Runge-Kutta 方法在主线模型上的应用 让我们将 RK4 方法应用到主线模型 (6.2) $y' = -\lambda y + g(t)$ 上。对于这个线性问题，RK4 方法可以写成：

$$z_{n+1} = R(-\lambda h)z_n + \text{加权的}g\text{ 项},$$

其中 $R(z)$ 是**稳定函数**（stability function）。

对于 RK4，稳定函数为：

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24} = \sum_{k=0}^{4} \frac{z^k}{k!},$$

这正是 e^z 的前 5 项 Taylor 展开。

稳定函数的意义：稳定函数并不是 ‘‘抽象概念’’，而是直接控制误差传播的数值量。稳定性条件 $|R(-\lambda h)| < 1$ 决定了方法是否稳定。对于 RK4，当 $|-\lambda h|$ 较小时，$R(-\lambda h) \approx e^{-\lambda h}$，这与精确解的衰减因子一致。

与 Euler 方法的对比：

- 显式 Euler：$R(z) = 1 + z$，稳定域较小（$|1 + z| < 1$，即 $z \in (-2, 0)$）；
- RK4：稳定域更大，可以容忍更大的步长，同时保持高阶精度。

这个分析框架自然地引出了稳定域、A-稳定性、L-稳定性等重要概念，而这些概念在处理刚性问题时至关重要。当 $\lambda \gg 1$ 且 $g(t)$ 平缓时，问题呈现刚性特征，显式方法（包括 RK4）可能需要非常小的步长才能稳定，此时隐式方法（如隐式 Runge-Kutta）的优势就体现出来了。

例 6.2.3 (RK4 格式的应用). 考虑初值问题

$$\begin{cases} y' = y - t^2 + 1, \quad 0 \le t \le 2, \\ y(0) = 0.5. \end{cases}$$

使用步长 $h = 0.5$，RK4 格式的计算过程如下：

在 $t_0 = 0$，$z_0 = 0.5$：

$$\begin{aligned} k_1 &= f(0, 0.5) = 0.5 - 0 + 1 = 1.5, \\ k_2 &= f(0.25, 0.5 + 0.25 \times 1.5) = f(0.25, 0.875) = 0.875 - 0.0625 + 1 = 1.8125, \\ k_3 &= f(0.25, 0.5 + 0.25 \times 1.8125) = f(0.25, 0.953125) = 0.953125 - 0.0625 + 1 = 1.890625, \\ k_4 &= f(0.5, 0.5 + 0.5 \times 1.890625) = f(0.5, 1.4453125) = 1.4453125 - 0.25 + 1 = 2.1953125, \\ z_1 &= 0.5 + \frac{0.5}{6}(1.5 + 2 \times 1.8125 + 2 \times 1.890625 + 2.1953125) \approx 1.4251302. \end{aligned} \tag{6.34}$$

继续这个过程可以得到后续时间点的近似值。

6.3 多步格式

在单步格式中，为了计算 z_{n+1}，我们只使用 z_n 的值。多步格式则利用多个历史值 $z_n, z_{n-1}, \ldots, z_{n-q}$ 来构造更高阶的格式。多步格式的主要优势在于：在相同阶数下，它们通常比单步格式需要更少的函数求值次数，从而具有更高的计算效率。然而，多步格式需要额外的启动过程（通常使用单步格式计算前几个值），且在计算过程中改变步长更加困难。

6.3.1 Adams-Bashforth 格式

Adams-Bashforth 格式是一类显式多步格式，其构造思想基于数值积分。与单步格式不同，多步格式利用多个历史点的信息来构造更高阶的格式，从而在相同阶数下减少函数求值次数。

构造原理：微分方程的精确解 $y(t)$ 满足

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\,\mathrm{d}t. \tag{6.35}$$

Adams-Bashforth 格式基于对积分

$$\int_{t_n}^{t_{n+1}} f(t, y(t))\,\mathrm{d}t \tag{6.36}$$

使用插值多项式求积公式的近似。具体地，我们用数量

$$\int_{t_n}^{t_{n+1}} p(t)\,\mathrm{d}t \tag{6.37}$$

来近似这个积分，其中 $p(t)$ 是满足插值条件

$$p(t_i) = f(t_i, y(t_i)), \quad i = n, n-1, \ldots, n-q$$

的唯一 q 次多项式。

如果在 Lagrange 基上写出这个多项式，有

$$p(t) = \sum_{k=0}^{q} L_k(t) f(t_{n-k}, y(t_{n-k})),$$

其中 $L_k(t)$ 是第 k 个 Lagrange 基多项式。因此

$$\int_{t_n}^{t_{n+1}} p(t)\,\mathrm{d}t = h \sum_{k=0}^{q} b_k f(t_{n-k}, y(t_{n-k})),$$

其中

$$b_k = \frac{1}{h} \int_{t_n}^{t_{n+1}} L_k(t)\,\mathrm{d}t. \tag{6.38}$$

因此有以下格式，称为 $(q+1)$ 步 Adams-Bashforth 格式：

$$z_{n+1} = z_n + h \sum_{k=0}^{q} b_k f(t_{n-k}, z_{n-k}), \quad n \geq q. \tag{6.39}$$

这个 $(q+1)$ 步格式涉及 $q+1$ 个点，在这些点上评估函数 f（(t_{n-k}, z_{n-k})，对于 $k = 0, \ldots, q$）。我们说这是一个 $(q+1)$ 步和 $(q+1)$ 节点的格式。

启动问题：这个格式只在 $n \geq q$ 时有效，这就是为什么在实践中需要用足够高阶的单步格式（例如 Runge-Kutta）来近似前 q 个值 $z_1, \ldots, z_q$。

系数的计算：注意到由公式 (6.38) 给出的系数 b_k 与 h 无关。实际上有

$$b_k = \frac{1}{h} \int_{t_n}^{t_{n+1}} \prod_{\substack{j=0 \\ j \neq k}}^{q} \frac{t - t_{n-j}}{t_{n-k} - t_{n-j}}\,\mathrm{d}t.$$

如果设 $s = \dfrac{t - t_n}{h}$，则 $t = t_n + sh$，且当 t 从 t_n 到 t_{n+1} 时，s 从 0 到 1。注意到 $t_{n-j} = t_n - jh$，因此 $t - t_{n-j} = h(s+j)$，$t_{n-k} - t_{n-j} = h(j-k)$。代入后得到

$$b_k = \int_0^1 \prod_{\substack{j=0 \\ j \neq k}}^{q} \frac{s+j}{j-k}\,\mathrm{d}s. \tag{6.40}$$

这个表达式清楚地表明 h 不出现，系数 b_k 只依赖于 q 和 k。

例 6.3.1 (两步 Adams-Bashforth 格式)**.** 对于 $q=1$，我们计算系数 b_0 和 b_1。根据公式 (6.40)，需要计算两个 Lagrange 基多项式的积分。

系数 b_0 的计算：

$$b_0=\int_0^1\prod_{\substack{j=0\\j\neq 0}}^{1}\frac{s+j}{j-0}\,\mathrm{d}s=\int_0^1\frac{s+1}{1-0}\,\mathrm{d}s=\int_0^1(s+1)\,\mathrm{d}s=\left[\frac{s^2}{2}+s\right]_0^1=\frac{3}{2}.$$

系数 b_1 的计算：

$$b_1=\int_0^1\prod_{\substack{j=0\\j\neq 1}}^{1}\frac{s+j}{j-1}\,\mathrm{d}s=\int_0^1\frac{s+0}{0-1}\,\mathrm{d}s=-\int_0^1 s\,\mathrm{d}s=-\frac{1}{2}.$$

格式的最终形式：因此两步 Adams-Bashforth 格式采用以下形式：

$$z_{n+1}=z_n+h\left[\frac{3}{2}f(t_n,z_n)-\frac{1}{2}f(t_{n-1},z_{n-1})\right],\quad n\geq 1. \tag{6.41}$$

几何解释：这个格式可以理解为：使用通过点 $(t_{n-1},f(t_{n-1},z_{n-1}))$ 和 $(t_n,f(t_n,z_n))$ 的线性插值多项式来近似 $f(t,y(t))$，然后在区间 $[t_n,t_{n+1}]$ 上积分。系数 $\frac{3}{2}$ 和 $-\frac{1}{2}$ 正是这个线性插值在积分中的权重。

定理 6.3.1 (Adams-Bashforth 格式的阶)**.** Adams-Bashforth 格式

$$z_{n+1}=z_n+h\sum_{k=0}^{q}b_kf(t_{n-k},z_{n-k}),\quad n\geq q,$$

其中 b_k 由 (6.40) 给出，是 $q+1$ 阶的。

证明. 为了证明 Adams-Bashforth 格式是 $(q+1)$ 阶的，我们需要证明：如果精确解 $y(t)$ 足够光滑，且假设 $z_{n-k}=y(t_{n-k})$ 对于 $k=0,1,\ldots,q$ 成立，则局部截断误差满足

$$\tau_{n+1}(h)=y(t_{n+1})-y(t_n)-h\sum_{k=0}^{q}b_kf(t_{n-k},y(t_{n-k}))=\mathscr{O}(h^{q+2}).$$

步骤 1：精确解的积分表示

精确解 $y(t)$ 满足微分方程 $y'(t)=f(t,y(t))$，因此

$$y(t_{n+1})=y(t_n)+\int_{t_n}^{t_{n+1}}f(t,y(t))\,\mathrm{d}t.$$

步骤 2：插值多项式的构造

设 $p(t)$ 是满足插值条件

$$p(t_{n-k})=f(t_{n-k},y(t_{n-k})),\quad k=0,1,\ldots,q$$

的唯一 q 次多项式。由 Lagrange 插值公式，有

$$p(t)=\sum_{k=0}^{q}L_k(t)f(t_{n-k},y(t_{n-k})),$$

其中 $L_k(t)$ 是第 k 个 Lagrange 基多项式。

步骤 3：插值误差的估计

由于 $f(t,y(t))$ 是 $(q+1)$ 次可微的（这要求 f 和 y 都足够光滑），由 Lagrange 插值误差公式，插值误差为

$$f(t,y(t))-p(t)=\frac{f^{(q+1)}(\xi,y(\xi))}{(q+1)!}\prod_{k=0}^{q}(t-t_{n-k}),$$

其中 $\xi \in [t_{n-q}, t_{n+1}]$ 依赖于 t。这里 $f^{(q+1)}$ 表示 f 关于 t 的 $(q+1)$ 阶导数（通过链式法则，这涉及到 y 的各阶导数）。

步骤 4：局部截断误差的分解

局部截断误差可以写成

$$\begin{aligned}\tau_{n+1}(h) &= y(t_{n+1}) - y(t_n) - h\sum_{k=0}^{q} b_k f(t_{n-k}, y(t_{n-k})) \\ &= \int_{t_n}^{t_{n+1}} f(t, y(t))\,\mathrm{d}t - h\sum_{k=0}^{q} b_k f(t_{n-k}, y(t_{n-k})) \\ &= \int_{t_n}^{t_{n+1}} [f(t, y(t)) - p(t)]\,\mathrm{d}t + \int_{t_n}^{t_{n+1}} p(t)\,\mathrm{d}t - h\sum_{k=0}^{q} b_k f(t_{n-k}, y(t_{n-k})).\end{aligned}$$

关键观察：注意到由系数 b_k 的定义 (6.40)，有

$$\int_{t_n}^{t_{n+1}} p(t)\,\mathrm{d}t = \int_{t_n}^{t_{n+1}} \sum_{k=0}^{q} L_k(t) f(t_{n-k}, y(t_{n-k}))\,\mathrm{d}t = h\sum_{k=0}^{q} b_k f(t_{n-k}, y(t_{n-k})),$$

其中第二个等号来自 b_k 的定义。因此，插值多项式的积分精确等于数值格式中的加权和，从而

$$\tau_{n+1}(h) = \int_{t_n}^{t_{n+1}} [f(t, y(t)) - p(t)]\,\mathrm{d}t.$$

这说明局部截断误差完全来自于用插值多项式 $p(t)$ 近似 $f(t, y(t))$ 所产生的误差。

步骤 5：误差阶数的确定

由于 $t_{n-k} = t_n - kh$（等距网格），我们有

$$\prod_{k=0}^{q}(t - t_{n-k}) = \prod_{k=0}^{q}(t - t_n + kh).$$

当 $t \in [t_n, t_{n+1}]$ 时，设 $t = t_n + sh$，其中 $s \in [0, 1]$，则

$$\prod_{k=0}^{q}(t - t_{n-k}) = \prod_{k=0}^{q}(sh + kh) = h^{q+1}\prod_{k=0}^{q}(s + k).$$

误差上界的估计：因此

$$|f(t, y(t)) - p(t)| \le \frac{M_{q+1}}{(q+1)!} h^{q+1} \prod_{k=0}^{q} |s + k|,$$

其中 $M_{q+1} = \max_{t\in[t_{n-q}, t_{n+1}]} |f^{(q+1)}(t, y(t))|$ 是 $(q+1)$ 阶导数的上界。

由于 $s \in [0, 1]$，有 $s + k \in [k, k+1]$，因此 $\prod_{k=0}^{q} |s + k| \le \prod_{k=0}^{q}(k+1) = (q+1)!$。这给出

$$|f(t, y(t)) - p(t)| \le M_{q+1} h^{q+1}.$$

最终误差估计：对误差进行积分，得到

$$|\tau_{n+1}(h)| = \left|\int_{t_n}^{t_{n+1}} [f(t, y(t)) - p(t)]\,\mathrm{d}t\right| \le \int_{t_n}^{t_{n+1}} M_{q+1} h^{q+1}\,\mathrm{d}t = M_{q+1} h^{q+2}.$$

因此 $\tau_{n+1}(h) = \mathscr{O}(h^{q+2})$，这证明了 Adams-Bashforth 格式是 $(q+1)$ 阶的（因为局部误差是 h^{q+2} 阶，格式的阶数比局部误差的阶数低一阶）。 □

多步格式的阶、一致性和稳定性的概念与单步格式略有不同。我们不会在本课程中详细讨论它们，它们在专门的数值分析教材中得到了很好的处理[1]。

[1]参见 BURDEN, R.L. 和 FAIRES, J.D.：*Numerical Analysis*, PWS-Kent, Boston, p. 257-268。

6.3.2 Adams-Bashforth 格式的一些例子

以下是 Adams-Bashforth 格式对应于某些 q 值的情况：

- **两步 Adams-Bashforth 格式**（两个节点）

$$z_{n+1} = z_n + h\left[\frac{3}{2}f(t_n, z_n) - \frac{1}{2}f(t_{n-1}, z_{n-1})\right], \quad n \geq 1, \tag{6.42}$$

局部误差 $\tau_{n+1}(h) = \dfrac{5}{12}y^{(3)}(\eta)h^3$，$\eta \in [t_{n-1}, t_{n+1}]$。

- **三步 Adams-Bashforth 格式**（三个节点）

$$z_{n+1} = z_n + \frac{h}{12}[23f(t_n, z_n) - 16f(t_{n-1}, z_{n-1}) + 5f(t_{n-2}, z_{n-2})], \quad n \geq 2, \tag{6.43}$$

局部误差 $\tau_{n+1}(h) = \dfrac{3}{8}y^{(4)}(\eta)h^4$，$\eta \in [t_{n-2}, t_{n+1}]$。

- **四步 Adams-Bashforth 格式**（四个节点）

$$z_{n+1} = z_n + \frac{h}{24}[55f(t_n, z_n) - 59f(t_{n-1}, z_{n-1}) + 37f(t_{n-2}, z_{n-2}) - 9f(t_{n-3}, z_{n-3})], \quad n \geq 3, \tag{6.44}$$

局部误差 $\tau_{n+1}(h) = \dfrac{251}{720}y^{(5)}(\eta)h^5$，$\eta \in [t_{n-3}, t_{n+1}]$。

6.3.3 Adams-Moulton 隐式格式

我们将采用与 Adams-Bashforth 格式相同的方法。然而，这次我们将用数量

$$\int_{t_n}^{t_{n+1}} p(t)\,\mathrm{d}t$$

来近似积分 (6.36)，其中 $p(t)$ 是满足插值条件

$$p(t_i) = f(t_i, y(t_i)), \quad i = n+1, n, n-1, \ldots, n-q$$

的唯一 $(q+1)$ 次多项式，即我们将 $y(t_{n+1})$ 包含在 “已知” 值中。这就是为什么我们因此得到一族称为 $(q+1)$ 步 Adams-Moulton 格式的多步隐式格式：

$$z_{n+1} = z_n + h\sum_{k=-1}^{q} b_k f(t_{n-k}, z_{n-k}), \quad n \geq q. \tag{6.45}$$

这个格式是一个 $(q+1)$ 步和 $(q+2)$ 节点的格式（(t_{n-k}, z_{n-k})，对于 $k = -1, \ldots, q$）。
系数 b_k，其计算方式与 Adams-Bashforth 格式相同，由公式

$$b_k = \int_0^1 \prod_{\substack{j=-1 \\ j\neq k}}^{q} \frac{s+j}{j-k}\,\mathrm{d}s, \quad k = -1, 0, \ldots, q \tag{6.46}$$

获得。

例 6.3.2 (两步 Adams-Moulton 格式（Euler-Centré 格式）). 对于 $q = 1$，我们得到 Euler-Centré 格式：

$$z_{n+1} = z_n + \frac{h}{2}[f(t_{n+1}, z_{n+1}) + f(t_n, z_n)], \quad n \geq 0, \tag{6.47}$$

局部误差 $\tau_{n+1}(h) = -\dfrac{1}{12}y^{(3)}(\eta)h^3$，$\eta \in [t_n, t_{n+1}]$。

预测-校正格式：在实践中，Adams-Moulton 格式不直接使用，因为它们是隐式的，因此尝试使用不动点或 Newton 方法获得 z_{n+1} 会失去通过重用 f 的先前值获得的优势。相反，我们联合使用相同阶的 Adams-Bashforth

格式和 Adams-Moulton 格式来构造**预测-校正格式**，在 Adams-Moulton 格式中使用 Adams-Bashforth 格式预测的值作为 z_{n+1} 的初始估计。

最常用的预测-校正格式在 4 阶使用这两种格式：

预测：

$$\tilde{z}_{n+1} = z_n + \frac{h}{24}[55f(t_n, z_n) - 59f(t_{n-1}, z_{n-1}) + 37f(t_{n-2}, z_{n-2}) - 9f(t_{n-3}, z_{n-3})], \tag{6.48}$$

校正：

$$z_{n+1} = z_n + \frac{h}{24}[9f(t_{n+1}, \tilde{z}_{n+1}) + 19f(t_n, z_n) - 5f(t_{n-1}, z_{n-1}) + f(t_{n-2}, z_{n-2})]. \tag{6.49}$$

可以证明，两个格式中每一个的局部误差的阶（这里是 4）都得到保持。

6.3.4 Adams-Moulton 格式的一些例子

以下是 Adams-Moulton 格式对应于某些 q 值的情况：

- **两步 Adams-Moulton 格式**（三个节点）

$$z_{n+1} = z_n + \frac{h}{12}[5f(t_{n+1}, z_{n+1}) + 8f(t_n, z_n) - f(t_{n-1}, z_{n-1})], \quad n \geq 1, \tag{6.50}$$

  局部误差 $\tau_{n+1}(h) = -\frac{1}{24}y^{(4)}(\eta)h^4$，$\eta \in [t_{n-1}, t_{n+1}]$。

- **三步 Adams-Moulton 格式**（四个节点）

$$z_{n+1} = z_n + \frac{h}{24}[9f(t_{n+1}, z_{n+1}) + 19f(t_n, z_n) - 5f(t_{n-1}, z_{n-1}) + f(t_{n-2}, z_{n-2})], \quad n \geq 2, \tag{6.51}$$

  局部误差 $\tau_{n+1}(h) = -\frac{19}{720}y^{(5)}(\eta)h^5$，$\eta \in [t_{n-2}, t_{n+1}]$。

- **四步 Adams-Moulton 格式**（五个节点）

$$z_{n+1} = z_n + \frac{h}{720}[251f(t_{n+1}, z_{n+1}) + 646f(t_n, z_n) - 264f(t_{n-1}, z_{n-1}) + 106f(t_{n-2}, z_{n-2}) - 19f(t_{n-3}, z_{n-3})], \quad n \geq 3, \tag{6.52}$$

  局部误差 $\tau_{n+1}(h) = -\frac{3}{160}y^{(6)}(\eta)h^6$，$\eta \in [t_{n-3}, t_{n+1}]$。

6.4 Runge-Kutta 方法的理论基础与自适应步长

6.4.1 Runge-Kutta 方法的定义

单步方法只使用点 (t_n, y_n) 来计算 (t_{n+1}, y_{n+1})。因此，为了获得足够高阶的方法，必须引入中间点。考虑下面的 Cauchy 问题

$$\begin{cases} y' = f(t, y), \quad t \in [t_0, t_0 + T], \\ y(t_0) = y_0. \end{cases} \tag{6.53}$$

我们试图相对于分割 $t_0 < t_1 < \cdots < t_N = t_0 + T$ 对其进行离散化。因此，想法是使用中间点 $(t_{n,i}, y_{n,i})$ 从点 (t_n, y_n) 计算点 (t_{n+1}, y_{n+1})。

在每个这样的点，我们关联“斜率”

$$p_{n,i} = f(t_{n,i}, y_{n,i}),$$

这实际上是函数 y 在点 $t_{n,i}$ 处的切线斜率 $y'(t_{n,i})$ 的近似。

这些中间点将是数值积分公式 (Q)（Q 表示求积）的节点：

$$\int_0^1 g(\tau)\,\mathrm{d}\tau \approx \sum_{i=1}^{q} b_i g(c_i) \quad (Q)$$

它将用于近似精确表达式

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t))\,\mathrm{d}t,$$

即，在从参考区间 $[0,1]$ 转换到区间 $[t_n, t_{n+1}]$ 之后：

$$t_{n,i} = t_n + c_i h_n, \quad c_i \in [0,1],$$

$$y_{n+1} = y_n + h_n \sum_{i=1}^{q} b_i f(t_{n,i}, y_{n,i}).$$

为了获得数值格式，还需要构造近似 $y_{n,i}$。为此，我们再次使用数值积分，但为了避免无限引入新的中间点，这些新的求积公式将只使用先前的节点 c_i。

然后，对于每个点 $(t_{n,i}, y_{n,i})$，我们给出一个数值积分方法，在每个参考区间上定义，这里是 $[0, c_i]$：

$$\int_0^{c_i} g(\tau)\,\mathrm{d}\tau \approx \sum_{j=1}^{q} a_{ij} g(c_j) \quad (Q_i)$$

设 $y(t)$ 是 (6.53) 的精确解。有

$$y(t_{n,i}) = y(t_n) + \int_{t_n}^{t_{n,i}} f(t, y(t))\,\mathrm{d}t = y(t_n) + h_n \int_0^{c_i} f(t_n + uh_n, y(t_n + uh_n))\,\mathrm{d}u,$$

其中我们做了变量替换 $t = t_n + uh_n$。同样，

$$y(t_{n+1}) = y(t_n) + h_n \int_0^1 f(t_n + uh_n, y(t_n + uh_n))\,\mathrm{d}u.$$

当我们将这些方法应用于 $g(u) = f(t_n + uh_n, y(t_n + uh_n))$ 时，有

$$\begin{cases} y(t_{n,i}) \approx y(t_n) + h_n \displaystyle\sum_{j=1}^{q} a_{ij} f(t_{n,j}, y(t_{n,j})), \quad i = 1, \ldots, q, \\ y(t_{n+1}) \approx y(t_n) + h_n \displaystyle\sum_{i=1}^{q} b_i f(t_{n,i}, y(t_{n,i})). \end{cases}$$

因此对应的 Runge-Kutta 方法：

$$\begin{cases} t_{n,i} = t_n + \tau_i h_n, \\ y_{n,i} = y_n + h_n \displaystyle\sum_{j=1}^{q} a_{ij} p_{n,j}, \\ p_{n,i} = f(t_{n,i}, y_{n,i}), \\ t_{n+1} = t_n + h_n, \\ y_{n+1} = y_n + h_n \displaystyle\sum_{i=1}^{q} b_i p_{n,i}. \end{cases} \tag{6.54}$$

我们看到这确实是单步方法，其中

$$\Phi(t, y, h) = \sum_{i=1}^{q} b_i f(t + c_i h, y_i), \quad \text{其中} y_i = y + h \sum_{j=1}^{q} a_{ij} f(t + c_j h, y_j), \quad 1 \le i \le q.$$

对应的 Runge-Kutta 方法是**显式的**：y_{n+1} 的计算从 y_n 的值开始，无需求解方程。如果对 j 的求和从 1 到 i，则方法称为**半隐式的**。如果它超过（最多到 q），则称为**隐式的**。那么，必须在每一步求解一个方程（通常是非线性的）来从 y_n 计算 y_{n+1}。

6.4.2 Lipschitz 条件与解的存在唯一性

在讨论数值方法的收敛性之前，我们需要确保微分方程本身有唯一解。这需要 Lipschitz 条件。

定义 6.4.1 (Lipschitz 条件). 设 g 是定义在 $D \subset \mathbb{R}^n$ 上取值于 $\mathbb{R}^p$ 的函数，如果存在常数 $L > 0$ 使得

$$\forall \boldsymbol{x}, \boldsymbol{y} \in D, \quad \|g(\boldsymbol{x}) - g(\boldsymbol{y})\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|,$$

则称 g 满足 **Lipschitz 条件**（或它是 Lipschitz 的）。

定理 6.4.1 (Cauchy-Lipschitz 定理). 假设函数 f 在 y 上是 Lipschitz 的，即存在常数 $L > 0$ 使得

$$\forall t \in I, \quad \forall x, y \in \mathbb{R}, \quad |f(t,x) - f(t,y)| \leq L|x - y|,$$

那么问题

$$\begin{cases} y'(t) = f(t, y(t)), & t \in I, \\ y(t_0) = y_0, & t_0 \in I, \end{cases} \tag{6.55}$$

有唯一解。

阅读导航：这个定理（也称为 Picard-Lindelöf 定理）的完整证明需要用到 Banach 不动点定理和压缩映射原理。我们在前面已经给出了证明的主要思路。读者在第一次阅读时，可以暂时忽略技术细节，只需把握证明的核心思想：*通过将微分方程转化为积分方程，并构造一个“越来越接近解”的函数序列，来证明解的存在性和唯一性。*

这个定理实际上就是 Picard-Lindelöf 定理，我们在前面已经证明过了。

因此，只要能够使局部误差不超过某个容差阈值，就可以控制全局误差。例如，这将允许使用最少的步长，同时遵守此容差。不幸的是，这并不令人惊讶，如果步长 h 保持常数，在控制全局误差的同时拥有最少的步长是一个不可能实现的目标。

局部误差的估计：可以按以下方式估计局部误差：假设我们有一个 p 阶的单步格式

$$\begin{cases} z_0 = y_0, \\ z_{n+1} = z_n + h\varphi(t_n, z_n, h), & n \geq 0. \end{cases} \tag{6.56}$$

与此格式相关的局部误差由量

$$\tau_{n+1}(h) = y(t_{n+1}) - y(t_n) - h\varphi(t_n, y(t_n), h) = \mathscr{O}(h^{p+1})$$

给出，其中 y 表示原始微分方程的解。

误差估计的基本思想：为了估计局部误差 $\tau_{n+1}(h)$，我们使用一个更高阶的格式作为“参考”。具体地，我们给出第二个格式

$$\begin{cases} \tilde{z}_0 = y_0, \\ \tilde{z}_{n+1} = \tilde{z}_n + h\tilde{\varphi}(t_n, z_n, h), & n \geq 0, \end{cases} \tag{6.57}$$

它是 $p+1$ 阶的，其局部误差 $\tilde{\tau}_{n+1}(h)$ 因此满足

$$\tilde{\tau}_{n+1}(h) = y(t_{n+1}) - y(t_n) - h\tilde{\varphi}(t_n, y(t_n), h) = \mathscr{O}(h^{p+2}).$$

误差估计的推导：假设我们对于给定的 $n \geq 0$ 有 z_n，并应用两个格式来获得 $y(t_{n+1})$ 的近似：有

$$z_{n+1} = z_n + h\varphi(t_n, z_n, h), \tag{6.58}$$

$$\tilde{z}_{n+1} = z_n + h\tilde{\varphi}(t_n, z_n, h). \tag{6.59}$$

关键假设：如果我们假设 $z_n = y(t_n)$（对于 $n = 0$ 为真！），则局部误差可以写成

$$\tau_{n+1}(h) = y(t_{n+1}) - z_n - h\varphi(t_n, z_n, h) = y(t_{n+1}) - z_n - (z_{n+1} - z_n) = y(t_{n+1}) - z_{n+1}.$$

误差分解：如果我们在最后这个量上加上和减去 $\tilde{z}_{n+1}$，得到

$$\tau_{n+1}(h) = y(t_{n+1}) - \tilde{z}_{n+1} + \tilde{z}_{n+1} - z_{n+1} = \tilde{\tau}_{n+1}(h) + \tilde{z}_{n+1} - z_{n+1}.$$

误差估计的合理性：由于 $\tilde{\tau}_{n+1}(h) = \mathscr{O}(h^{p+2})$ 和 $\tau_{n+1}(h) = \mathscr{O}(h^{p+1})$，当 h 很小时，$\tilde{\tau}_{n+1}(h)$ 相对于 $\tau_{n+1}(h)$ 可以忽略。因此，我们可以通过以下方式估计 $\tau_{n+1}(h)$：

$$\hat{\tau}_{n+1}(h) = \tilde{z}_{n+1} - z_{n+1}. \tag{6.60}$$

误差估计的精度：这个估计的误差为

$$|\tau_{n+1}(h) - \hat{\tau}_{n+1}(h)| = |\tilde{\tau}_{n+1}(h)| = \mathscr{O}(h^{p+2}),$$

相对于 $\tau_{n+1}(h) = \mathscr{O}(h^{p+1})$ 来说，这是一个高阶误差项，因此估计是合理的。

自适应步长算法：我们按以下方式使用此估计器：从初始步长开始，通常是 $h = (T - t_0)/100$，在每次迭代 n 时，我们计算 $\tilde{z}_{n+1} - z_{n+1}$ 并因此计算 $\hat{\tau}_{n+1}(h)$。然后我们考虑以下情况：

- 如果 $\dfrac{|\hat{\tau}_{n+1}(h)|}{h} \in [\varepsilon/10, \varepsilon]$，则我们接受 z_{n+1}，为下一次迭代保持相同的步长。
- 如果 $\dfrac{|\hat{\tau}_{n+1}(h)|}{h} \in [0, \varepsilon/10)$，我们接受 z_{n+1} 并为下一次迭代增加步长（例如 $h \leftarrow 2h$）。
- 如果 $\dfrac{|\hat{\tau}_{n+1}(h)|}{h} > \varepsilon$，我们拒绝 z_{n+1}，减小步长（例如 $h \leftarrow h/2$）并重新计算 z_{n+1}、$\tilde{z}_{n+1}$ 然后 $\hat{\tau}_{n+1}(h)$。

通常，我们对步长施加最大值（通常是其初始值）以及最小值。当达到最小步长时，这意味着要么最小值相对于要求的容差 ε 太大，要么微分方程的解在考虑的瞬时 t_n 处存在奇点。

最常用的方法基于两个 Runge-Kutta 格式，分别是 4 阶和 5 阶（这是 MATLAB 函数 `ode45` 中使用的方法）。这种方法能够自动平衡精度和效率，是现代科学计算软件的核心技术。

Chapter 7

特征值和特征向量的确定

特征值和特征向量的计算是数值线性代数的核心问题之一，在科学计算、工程应用和数据分析中具有重要地位。例如，在结构力学中，特征值对应系统的固有频率；在量子力学中，特征值对应可观测量的可能取值；在主成分分析（PCA）中，特征值反映了数据的主要变化方向。本章系统介绍计算矩阵特征值和特征向量的主要数值方法，重点讨论幂迭代方法、逆幂迭代方法以及收缩方法。

本章路线图　本章围绕有限维矩阵的特征值与特征向量的数值计算，按如下结构展开：

- **回顾和记号**：形式化特征值问题，明确特征多项式与特征值的关系，并约定本章使用的基本记号。
- **幂迭代方法**：在严格主特征值假设下，分析幂迭代的收敛机制与误差衰减速度，说明其适用场景和局限性。
- **逆幂迭代方法**：通过位移与逆矩阵的特征值关系，构造用于计算给定近似值附近特征值的迭代方法。
- **收缩方法**：在已知部分特征对的前提下，介绍如何通过收缩（降维）构造新的矩阵以继续提取其余特征值。
- **方法总结与比较**：从误差、收敛性与计算代价的角度，对本章方法进行汇总比较，并与更通用的特征值算法建立接口。

7.1　回顾和记号

本节的目标是用统一的符号语言形式化特征值与特征向量问题，并说明特征多项式在理论上的角色，从而为后续数值方法的构造提供背景。

7.1.1　问题陈述

定义 7.1.1 (特征值和特征向量)**.** 设 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ 是一个 n 阶方阵。如果存在标量 $\lambda \in \mathbb{C}$ 和非零向量 $\boldsymbol{y} \in \mathbb{C}^n$，使得

$$\boldsymbol{A}\boldsymbol{y} = \lambda \boldsymbol{y}, \tag{7.1}$$

则称 λ 为矩阵 $\boldsymbol{A}$ 的**特征值**（eigenvalue），$\boldsymbol{y}$ 为对应于特征值 λ 的**特征向量**（eigenvector）。

该定义刻画的是：在特征向量方向上，线性变换 $\boldsymbol{A}$ 仅产生尺度伸缩而不改变方向，因此特征值描述了这一伸缩因子。在数值计算中，许多数值方法试图通过迭代过程逼近这样的“方向-尺度对”，从而获得矩阵的谱信息。

方程 (7.1) 可以改写为

$$(\boldsymbol{A} - \lambda \boldsymbol{I})\boldsymbol{y} = \boldsymbol{0},$$

其中 $\boldsymbol{I}$ 是 n 阶单位矩阵。由于 $\boldsymbol{y} \neq \boldsymbol{0}$，这意味着矩阵 $\boldsymbol{A} - \lambda \boldsymbol{I}$ 是奇异的，即

$$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0.$$

定义 7.1.2 (特征多项式). 设 $\boldsymbol{A} \in \mathbb{R}^{n \times n}$，多项式

$$P_{\boldsymbol{A}}(s) = \det(s\boldsymbol{I} - \boldsymbol{A}) \tag{7.2}$$

称为矩阵 $\boldsymbol{A}$ 的**特征多项式**（characteristic polynomial）。

特征多项式将矩阵特征值问题转化为一元多项式求根问题，其根即为特征值本身。在数值分析中，直接利用特征多项式求根通常并不稳定，但这一对应关系为后续以特征值问题形式重述多项式求根提供了理论基础。

特征多项式 $P_{\boldsymbol{A}}(s)$ 是 n 次多项式，其 n 个根（可能重复，可能为复数）就是矩阵 $\boldsymbol{A}$ 的 n 个特征值。因此，从理论上讲，确定矩阵的特征值等价于求特征多项式的根。

7.1.2　记号约定

在本章中，我们采用以下记号约定：

- 记 $\lambda_1, \lambda_2, \ldots, \lambda_n$ 为矩阵 $\boldsymbol{A}$ 的 n 个特征值（可能重复），并假设它们按模长从大到小排列：

$$|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_n|. \tag{7.3}$$

- λ_1 称为 $\boldsymbol{A}$ 的**主特征值**（dominant eigenvalue），如果 $|\lambda_1| > |\lambda_2|$，则称 λ_1 为**严格主特征值**。
- 我们假设矩阵 $\boldsymbol{A}$ 是可对角化的，即存在 n 个线性无关的特征向量 $\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, \ldots, \boldsymbol{y}^{(n)}$，它们构成 $\mathbb{R}^n$（或 $\mathbb{C}^n$）的一组基。

7.1.3　特征值计算与多项式求根的关系

本小节的目标是说明多项式求根问题与矩阵特征值问题之间的等价关系，从而突出特征值计算在更广泛数值问题中的基础地位。

在理论上，特征值的数值计算似乎可以归结为特征多项式的数值根计算。然而，在数值实践中，更常见的是反向的转换：**多项式求根问题可以转化为特征值问题**。

命题 7.1.1 (友矩阵). 给定一个 n 次首一多项式

$$p(t) = t^n + a_n t^{n-1} + a_{n-1} t^{n-2} + \cdots + a_2 t + a_1,$$

则 $p(t)$ 是以下矩阵的特征多项式：

$$\boldsymbol{A} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ -a_1 & -a_2 & \cdots & -a_{n-1} & -a_n \end{pmatrix}. \tag{7.4}$$

具有上述形式的矩阵 $\boldsymbol{A}$ 称为**友矩阵**（companion matrix）。这一结果表明，多项式求根问题可以转化为特征值问题，因此特征值计算方法在数值分析中具有基础性地位。

从数值计算的角度看，这一构造说明：一旦能够稳定地求解矩阵的特征值，就可以间接解决多项式求根问题。反之，直接针对高次特征多项式进行求根往往在数值稳定性上更为脆弱。

例 7.1.1 (二次多项式的友矩阵). 考虑二次多项式 $p(t) = t^2 + at + b$，其友矩阵为

$$\boldsymbol{A} = \begin{pmatrix} 0 & 1 \\ -b & -a \end{pmatrix}.$$

可以验证：

$$\det(s\boldsymbol{I} - \boldsymbol{A}) = \det \begin{pmatrix} s & -1 \\ b & s+a \end{pmatrix} = s(s+a) + b = s^2 + as + b = p(s).$$

因此，$p(t)$ 的两个根就是 $\boldsymbol{A}$ 的两个特征值。

7.2 幂迭代方法

本节的目标是构造并分析幂迭代方法这一最基本的特征值迭代格式，说明在严格主特征值假设下，它如何以可控的速度逼近主特征值及其对应特征向量。

7.2.1 方法陈述

幂迭代方法（Power Iteration Method）是计算矩阵主特征值和对应特征向量的经典迭代方法。其基本思想是：从一个初始向量开始，反复用矩阵 $\boldsymbol{A}$ 作用并对结果归一化，在合适假设下，得到的向量序列收敛到主特征值对应的特征向量。

Algorithm 16 幂迭代方法

Require: 矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$，初始向量 $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$，最大迭代次数 K

Ensure: 主特征值的近似值 λ_1 和对应的特征向量 $\boldsymbol{x}^{(K)}$

```
k ← 0
while k < K do
    x^(k+1) ← A x^(k)                          ▷ 矩阵-向量乘法
    x^(k+1) ← x^(k+1) / ||x^(k+1)||            ▷ 归一化
    k ← k + 1
end while
计算 λ_1 ≈ [A x^(K)]_p / [x^(K)]_p，其中 p 是使得 [x^(K)]_p ≠ 0 的指标
返回 λ_1 和 x^(K)
```

算法的迭代格式可以写成：

$$\begin{cases} \boldsymbol{x}^{(0)} \text{ 给定在} \mathbb{R}^n \text{ 中}, \\ \boldsymbol{x}^{(k+1)} = \dfrac{\boldsymbol{A}\boldsymbol{x}^{(k)}}{\|\boldsymbol{A}\boldsymbol{x}^{(k)}\|}, \quad k \geq 0, \end{cases} \tag{7.5}$$

其中 $\|\cdot\|$ 表示任意向量范数（通常使用 2-范数或 ∞-范数）。

7.2.2 假设条件

为了确保幂迭代方法的收敛性，我们需要以下假设：

1. **主特征值的唯一性**：假设

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|. \tag{7.6}$$

这意味着 λ_1 是严格主特征值。在练习中可以证明，如果 $\boldsymbol{A}$ 是实矩阵且满足 (7.6)，则 λ_1 是实数。

2. **初始向量的选择**：假设初始向量 $\boldsymbol{x}^{(0)}$ 不属于由特征向量 $\{\boldsymbol{y}^{(2)}, \boldsymbol{y}^{(3)}, \ldots, \boldsymbol{y}^{(n)}\}$ 生成的子空间。换句话说，$\boldsymbol{x}^{(0)}$ 在特征向量基 $\{\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, \ldots, \boldsymbol{y}^{(n)}\}$ 下的展开中，$\boldsymbol{y}^{(1)}$ 的系数非零。
3. **特征向量的非零分量**：设 p 是一个指标，使得 $\boldsymbol{y}^{(1)}$ 的第 p 个分量 $y_p^{(1)} \neq 0$。这个条件用于计算特征值的近似值。

7.2.3 收敛性定理

定理 7.2.1 (幂迭代方法的收敛性)**.** 在假设条件 (7.6) 下，由关系 (7.5) 生成的序列 $\{\boldsymbol{x}^{(k)}\}_{k\in\mathbb{N}}$ 具有以下性质：

1. **特征值的收敛**：
$$\lim_{k\to\infty} \|\boldsymbol{A}\boldsymbol{x}^{(k)}\| = |\lambda_1|, \tag{7.7}$$
更精确地，有
$$\lim_{k\to\infty} \frac{[\boldsymbol{A}\boldsymbol{x}^{(k)}]_p}{[\boldsymbol{x}^{(k)}]_p} = \lambda_1, \tag{7.8}$$
其中 p 是使得 $y_p^{(1)} \neq 0$ 的指标。
2. **特征向量的收敛**：序列 $\{\boldsymbol{x}^{(k)}\}_{k\in\mathbb{N}}$ 以下列方式收敛到与 λ_1 相关的特征向量：
$$\lim_{k\to\infty} [\operatorname{sgn}(\lambda_1)]^k \boldsymbol{x}^{(k)} = \gamma \boldsymbol{y}^{(1)}, \tag{7.9}$$
其中 γ 是一个实常数。

7.2.4 收敛性证明

我们现在证明定理 7.2.1。

第一步：我们首先证明
$$\boldsymbol{x}^{(k)} = \frac{\boldsymbol{A}^k \boldsymbol{x}^{(0)}}{\|\boldsymbol{A}^k \boldsymbol{x}^{(0)}\|}. \tag{7.10}$$
实际上，这个性质对于 $k = 1$ 由 $\boldsymbol{x}^{(1)}$ 的构造本身为真。如果我们假设该性质对于 k 为真，那么有
$$\boldsymbol{x}^{(k+1)} = \frac{\boldsymbol{A}\boldsymbol{x}^{(k)}}{\|\boldsymbol{A}\boldsymbol{x}^{(k)}\|} = \frac{\boldsymbol{A} \cdot \dfrac{\boldsymbol{A}^k \boldsymbol{x}^{(0)}}{\|\boldsymbol{A}^k \boldsymbol{x}^{(0)}\|}}{\left\| \boldsymbol{A} \cdot \dfrac{\boldsymbol{A}^k \boldsymbol{x}^{(0)}}{\|\boldsymbol{A}^k \boldsymbol{x}^{(0)}\|} \right\|} = \frac{\boldsymbol{A}^{k+1} \boldsymbol{x}^{(0)}}{\|\boldsymbol{A}^{k+1} \boldsymbol{x}^{(0)}\|}.$$
由数学归纳法，公式 (7.10) 对所有 $k \geq 1$ 成立。

第二步：在特征向量基上展开初始向量。由于 $\{\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, \ldots, \boldsymbol{y}^{(n)}\}$ 构成 $\mathbb{R}^n$ 的一组基，可以写成
$$\boldsymbol{x}^{(0)} = \sum_{i=1}^{n} \xi_i \boldsymbol{y}^{(i)}, \quad \xi_1 \neq 0,$$
其中 $\xi_1 \neq 0$ 由假设条件保证。

第三步：计算 $\boldsymbol{A}^k \boldsymbol{x}^{(0)}$。由于 $\boldsymbol{A}\boldsymbol{y}^{(i)} = \lambda_i \boldsymbol{y}^{(i)}$，有
$$\boldsymbol{A}^k \boldsymbol{x}^{(0)} = \sum_{i=1}^{n} \xi_i \boldsymbol{A}^k \boldsymbol{y}^{(i)} = \sum_{i=1}^{n} \xi_i \lambda_i^k \boldsymbol{y}^{(i)} = \lambda_1^k \left[\xi_1 \boldsymbol{y}^{(1)} + \sum_{i=2}^{n} \xi_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{y}^{(i)} \right] = \lambda_1^k \boldsymbol{w}^{(k)},$$
其中设
$$\boldsymbol{w}^{(k)} = \xi_1 \boldsymbol{y}^{(1)} + \sum_{i=2}^{n} \xi_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{y}^{(i)}.$$

第四步：证明特征向量的收敛性。使用 (7.10)，得到
$$\boldsymbol{x}^{(k)} = \frac{\lambda_1^k}{|\lambda_1^k|} \cdot \frac{\boldsymbol{w}^{(k)}}{\|\boldsymbol{w}^{(k)}\|} = [\operatorname{sgn}(\lambda_1)]^k \frac{\boldsymbol{w}^{(k)}}{\|\boldsymbol{w}^{(k)}\|}.$$

由于 $|\lambda_i/\lambda_1| < 1$ 对所有 $i > 1$ 成立，有

$$\lim_{k\to\infty} \boldsymbol{w}^{(k)} = \xi_1 \boldsymbol{y}^{(1)},$$

因此

$$\lim_{k\to\infty} [\mathrm{sgn}(\lambda_1)]^k \boldsymbol{x}^{(k)} = \gamma \boldsymbol{y}^{(1)}, \quad 其中\gamma = \frac{\xi_1}{|\xi_1|\|\boldsymbol{y}^{(1)}\|}.$$

第五步：证明特征值的收敛性。根据 (7.10)，有

$$\boldsymbol{A}\boldsymbol{x}^{(k)} = \frac{\boldsymbol{A}^{k+1}\boldsymbol{x}^{(0)}}{\|\boldsymbol{A}^k\boldsymbol{x}^{(0)}\|} = \frac{\lambda_1^{k+1}\boldsymbol{w}^{(k+1)}}{|\lambda_1^k|\|\boldsymbol{w}^{(k)}\|} = \lambda_1[\mathrm{sgn}(\lambda_1)]^k \frac{\boldsymbol{w}^{(k+1)}}{\|\boldsymbol{w}^{(k)}\|},$$

因此

$$\|\boldsymbol{A}\boldsymbol{x}^{(k)}\| = |\lambda_1| \frac{\|\boldsymbol{w}^{(k+1)}\|}{\|\boldsymbol{w}^{(k)}\|},$$

从而

$$\lim_{k\to\infty} \|\boldsymbol{A}\boldsymbol{x}^{(k)}\| = |\lambda_1|.$$

此外，使用 (7.8) 的表达式，有

$$\frac{[\boldsymbol{A}\boldsymbol{x}^{(k)}]_p}{[\boldsymbol{x}^{(k)}]_p} = \lambda_1 \frac{[\boldsymbol{w}^{(k+1)}]_p}{[\boldsymbol{w}^{(k)}]_p}.$$

由于 $\lim_{k\to\infty}[\boldsymbol{w}^{(k)}]_p = \xi_1 y_p^{(1)} \neq 0$（由假设），因此

$$\lim_{k\to\infty} \frac{[\boldsymbol{A}\boldsymbol{x}^{(k)}]_p}{[\boldsymbol{x}^{(k)}]_p} = \lambda_1.$$

7.2.5 收敛速度与注意事项

注 (收敛速度). 幂迭代方法的收敛速度由比值 $|\lambda_2/\lambda_1|$ 决定。如果 $|\lambda_2/\lambda_1|$ 接近 1，则收敛会很慢；如果 $|\lambda_2/\lambda_1|$ 远小于 1，则收敛较快。具体地，误差的衰减速度约为 $\mathscr{O}(|\lambda_2/\lambda_1|^k)$。

注 (重特征值的情况). 如果 $\lambda_1 = \lambda_2$ 且 $|\lambda_1| > |\lambda_3| \geq \cdots \geq |\lambda_n|$，即如果主特征值是重根，证明仍然有效，只需写成

$$\boldsymbol{x}^{(0)} = \xi_1 \boldsymbol{y}^{(1)} + \sum_{i=3}^{n} \xi_i \boldsymbol{y}^{(i)},$$

其中 $\xi_1 \boldsymbol{y}^{(1)}$ 是 $\boldsymbol{x}^{(0)}$ 在与 λ_1 相关的特征子空间上的分量。如果 λ_1 是多重特征值，这仍然有效。

相反，如果我们有 $\lambda_1 = \lambda_2$ 且 $|\lambda_1| = |\lambda_2|$，那么证明不成立，这特别发生在复特征值的情况下（例如，$\lambda_1 = \lambda_2^*$，其中 $*$ 表示共轭）。

注 (符号振荡问题). 在公式 (7.9) 中出现 λ_1 的符号，这意味着，如果 λ_1 是负数，那么向量 $\boldsymbol{x}^{(k)}$ 在 $+\gamma\boldsymbol{y}^{(1)}$ 和 $-\gamma\boldsymbol{y}^{(1)}$ 两个向量之间振荡。在练习中将看到幂迭代方法的一个变体，它巧妙地使用无穷范数以避免这些振荡。

例 7.2.1 (幂迭代方法的应用). 考虑矩阵

$$\boldsymbol{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

矩阵 $\boldsymbol{A}$ 的特征值为 $\lambda_1 = 3$ 和 $\lambda_2 = 1$，对应的特征向量为 $\boldsymbol{y}^{(1)} = (1,1)^T$ 和 $\boldsymbol{y}^{(2)} = (1,-1)^T$（归一化后）。取初始向量 $\boldsymbol{x}^{(0)} = (1,0)^T$，使用 2-范数进行归一化。

迭代 1：

$$\boldsymbol{A}\boldsymbol{x}^{(0)} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad \|\boldsymbol{A}\boldsymbol{x}^{(0)}\|_2 = \sqrt{5}, \quad \boldsymbol{x}^{(1)} = \frac{1}{\sqrt{5}}\begin{pmatrix} 2 \\ 1 \end{pmatrix} \approx \begin{pmatrix} 0.8944 \\ 0.4472 \end{pmatrix}.$$

迭代 2：

$$\boldsymbol{A}\boldsymbol{x}^{(1)} = \frac{1}{\sqrt{5}}\begin{pmatrix} 2.2361 \\ 2.2361 \end{pmatrix}, \quad \|\boldsymbol{A}\boldsymbol{x}^{(1)}\|_2 = 2.2361, \quad \boldsymbol{x}^{(2)} \approx \begin{pmatrix} 0.7071 \\ 0.7071 \end{pmatrix}.$$

迭代 3：

$$\boldsymbol{A}\boldsymbol{x}^{(2)} \approx \begin{pmatrix} 2.1213 \\ 2.1213 \end{pmatrix}, \quad \|\boldsymbol{A}\boldsymbol{x}^{(2)}\|_2 = 3.0, \quad \boldsymbol{x}^{(3)} \approx \begin{pmatrix} 0.7071 \\ 0.7071 \end{pmatrix}.$$

可以看到，$\boldsymbol{x}^{(k)}$ 快速收敛到 $\boldsymbol{y}^{(1)}/\|\boldsymbol{y}^{(1)}\|_2 = (1/\sqrt{2}, 1/\sqrt{2})^T$，且 $\|\boldsymbol{A}\boldsymbol{x}^{(k)}\|_2$ 收敛到 $|\lambda_1| = 3$。

7.3 逆幂迭代方法

本节的目标是说明如何通过位移与逆矩阵的特征值关系，构造能够逼近给定实数 q 附近特征值的迭代方法，从而在局部范围内细化谱信息。

7.3.1 方法原理

逆幂迭代方法（Inverse Power Iteration Method）是幂迭代方法的一个重要变体，用于计算最接近某个给定值的特征值。其基本思想基于以下观察：

命题 7.3.1 (逆矩阵的特征值)**.** 如果 λ 是 $\boldsymbol{A}$ 的特征值且 $\boldsymbol{A}$ 是可逆的，那么 λ^{-1} 是 $\boldsymbol{A}^{-1}$ 的特征值，相关的特征向量是相同的。

证明. 实际上有

$$\boldsymbol{A}\boldsymbol{y} = \lambda\boldsymbol{y} \quad \Leftrightarrow \quad \boldsymbol{y} = \lambda\boldsymbol{A}^{-1}\boldsymbol{y} \quad \Leftrightarrow \quad \boldsymbol{A}^{-1}\boldsymbol{y} = \lambda^{-1}\boldsymbol{y}.$$

□

位移技巧：对于给定的标量 q，定义矩阵

$$\boldsymbol{B} = \boldsymbol{A} - q\boldsymbol{I}. \tag{7.11}$$

这个矩阵的特征值为 $\lambda_i - q$，其中 λ_i 是 $\boldsymbol{A}$ 的特征值。如果 $\boldsymbol{B}$ 可逆（即 q 不是 $\boldsymbol{A}$ 的特征值），则 $\boldsymbol{B}^{-1}$ 的特征值为

$$\mu_i = \frac{1}{\lambda_i - q}. \tag{7.12}$$

实际上，如果 $\boldsymbol{z}$ 是与 $(\lambda_i - q)^{-1}$ 相关的 $\boldsymbol{B}^{-1}$ 的特征向量，它是 $\boldsymbol{B}$ 的特征向量，因此是 $\boldsymbol{A}$ 的特征向量，显然它与特征值 λ_i 相关。

逆幂迭代的思想：设 $\boldsymbol{A}$ 是在 $\mathbb{R}$ 上可对角化的矩阵，q 是一个实数，λ_j 是 $\boldsymbol{A}$ 的一个特征值，满足

$$0 < |q - \lambda_j| < |q - \lambda_i|, \quad \forall i \neq j, \tag{7.13}$$

即 q 不是 $\boldsymbol{A}$ 的特征值，且 λ_j 是最接近 q 的特征值。

由 (7.12) 得出，矩阵 $\boldsymbol{B}^{-1} = (\boldsymbol{A} - q\boldsymbol{I})^{-1}$ 以

$$\mu_1 = \frac{1}{\lambda_j - q}$$

作为主特征值（因为 $|\mu_1| > |\mu_i|$ 对所有 $i \neq j$ 成立）。

因此，对于给定的 q，可以借鉴应用于 $\boldsymbol{B}^{-1}$ 的幂迭代算法，这允许获得 μ_1，我们通过设

$$\lambda_j = \frac{1}{\mu_1} + q$$

来恢复 λ_j。

该方法称为**逆幂迭代方法**。

7.3.2 算法描述

设 $\boldsymbol{B} = \boldsymbol{A} - q\boldsymbol{I}$。给定 $\boldsymbol{x}^{(0)}$ 任意（或几乎任意！），并通过以下方式定义序列 $\{\boldsymbol{x}^{(k)}\}$：

$$\begin{cases} \boldsymbol{B}\boldsymbol{u}^{(k+1)} = \boldsymbol{x}^{(k)}, \\ \boldsymbol{x}^{(k+1)} = \dfrac{\boldsymbol{u}^{(k+1)}}{\|\boldsymbol{u}^{(k+1)}\|}, \quad k \geq 0. \end{cases} \tag{7.14}$$

注意，每一步迭代都需要求解一个线性方程组 $\boldsymbol{B}\boldsymbol{u}^{(k+1)} = \boldsymbol{x}^{(k)}$。如果 $\boldsymbol{A}$ 是稀疏矩阵，可以使用高效的稀疏求解器。

Algorithm 17 逆幂迭代方法

Require: 矩阵 $\boldsymbol{A} \in \mathbb{R}^{n\times n}$，位移 $q \in \mathbb{R}$（q 不是 $\boldsymbol{A}$ 的特征值），初始向量 $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$，最大迭代次数 K

Ensure: 最接近 q 的特征值 λ_j 和对应的特征向量 $\boldsymbol{x}^{(K)}$

```
B ← A − qI
k ← 0
while k < K do
    求解线性方程组 Bu^(k+1) = x^(k)                    ▷ 例如使用 LU 分解
    x^(k+1) ← u^(k+1) / ‖u^(k+1)‖                      ▷ 归一化
    k ← k + 1
end while
计算 μ_1 ≈ [u^(K)]_p / [x^(K−1)]_p，其中 p 是使得 [x^(K−1)]_p ≠ 0 的指标
λ_j ← 1/μ_1 + q
返回 λ_j 和 x^(K)
```

7.3.3 收敛性定理

定理 7.3.1 (逆幂迭代方法的收敛性)**.** 设给定的 $q \in \mathbb{R}$ 使得 $q \neq \lambda_i$ 对所有 i 成立，且设 $\boldsymbol{B} = \boldsymbol{A} - q\boldsymbol{I}$。如果初始向量 $\boldsymbol{x}^{(0)}$ 不属于由 $\{\boldsymbol{y}^{(i)}\}_{i=1,\ldots,n,i\neq j}$ 生成的子空间，那么由方法 (7.14) 生成的序列 $\{\boldsymbol{x}^{(k)}\}_{k\in\mathbb{N}}$ 具有以下性质：

1. **特征值的收敛：**
$$\lim_{k\to\infty} \|\boldsymbol{u}^{(k)}\| = \left|\frac{1}{\lambda_j - q}\right|,$$
更精确地，如果 $y_p^{(j)} \neq 0$，有
$$\lim_{k\to\infty} \frac{[\boldsymbol{u}^{(k+1)}]_p}{[\boldsymbol{x}^{(k)}]_p} = \frac{1}{\lambda_j - q}.$$
2. **特征向量的收敛：**序列 $\{\boldsymbol{x}^{(k)}\}_{k\in\mathbb{N}}$ 以下列方式收敛到与 λ_j 相关的 $\boldsymbol{A}$ 的特征向量 $\boldsymbol{y}^{(j)}$：
$$\lim_{k\to\infty} \operatorname{sgn}\left(\frac{1}{\lambda_j - q}\right)^k \boldsymbol{x}^{(k)} = \gamma \boldsymbol{y}^{(j)},$$
其中 γ 是一个实常数。

注 (收敛速度)**.** 可以证明，方法收敛得越快，$|q - \lambda_j|$ 相对于 $|q - \lambda_i|$（$\forall i \neq j$）越小。因此，如果我们对 λ_j 有一个好的初始估计，逆幂迭代方法可以非常快速地收敛。这一性质使得逆幂迭代方法在计算已知特征值的精确近似时特别有用。

例 7.3.1 (逆幂迭代方法的应用)**.** 考虑矩阵
$$\boldsymbol{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix},$$

其特征值为 $\lambda_1 = 3$ 和 $\lambda_2 = 1$。

如果我们想计算较小的特征值 $\lambda_2 = 1$，可以选择 $q = 0.5$（接近 λ_2）。则

$$\boldsymbol{B} = \boldsymbol{A} - 0.5\boldsymbol{I} = \begin{pmatrix} 1.5 & 1 \\ 1 & 1.5 \end{pmatrix}.$$

取初始向量 $\boldsymbol{x}^{(0)} = (1, 0)^T$。

迭代 1：求解 $\boldsymbol{B}\boldsymbol{u}^{(1)} = \boldsymbol{x}^{(0)}$，即

$$\begin{pmatrix} 1.5 & 1 \\ 1 & 1.5 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

得到 $\boldsymbol{u}^{(1)} = (1.2, -0.8)^T$，归一化后 $\boldsymbol{x}^{(1)} \approx (0.8321, -0.5547)^T$。

继续迭代，$\boldsymbol{x}^{(k)}$ 将收敛到 $\boldsymbol{y}^{(2)}/\|\boldsymbol{y}^{(2)}\| = (1/\sqrt{2}, -1/\sqrt{2})^T$，且可以恢复 $\lambda_2 = 1$。

7.4 收缩方法

本节的目标是说明在已经获得部分特征对的前提下，如何通过构造新的矩阵实现“收缩”，从而在降维后的问题上继续提取其余特征值。

前面的方法允许确定一个特征值和相关的特征向量。例如，幂迭代方法允许计算主特征值，逆幂迭代方法允许计算最接近给定实数的特征值（在适当唯一性条件下）。在此基础上，可以考虑构造一个矩阵，其中刚刚确定的特征值不再起主导作用，而其余特征值保持不变，然后在这个新矩阵上再次应用迭代方法，从而逐步提取全部谱信息。

这一思路构成了所谓的**收缩方法**（Deflation Method）。

7.4.1 对称矩阵的情况

假设我们知道与特征值 λ 相关的特征向量 $\boldsymbol{y}$，且 $\|\boldsymbol{y}\|_2 = 1$。

然后考虑矩阵

$$\boldsymbol{B} = \boldsymbol{A} - \lambda \boldsymbol{y}\boldsymbol{y}^T. \tag{7.15}$$

命题 7.4.1 (对称矩阵的收缩). 设 $\boldsymbol{A}$ 是对称矩阵，λ 是其特征值，$\boldsymbol{y}$ 是对应的归一化特征向量（$\|\boldsymbol{y}\|_2 = 1$）。则矩阵 $\boldsymbol{B} = \boldsymbol{A} - \lambda \boldsymbol{y}\boldsymbol{y}^T$ 具有以下性质：

1. $\boldsymbol{B}\boldsymbol{y} = \boldsymbol{0}$，即 $\boldsymbol{y}$ 是 $\boldsymbol{B}$ 的零特征值对应的特征向量。
2. 如果 $\boldsymbol{z}$ 是与另一个特征值 μ 相关的 $\boldsymbol{A}$ 的另一个特征向量，且 $\boldsymbol{z}$ 与 $\boldsymbol{y}$ 正交，则 $\boldsymbol{B}\boldsymbol{z} = \mu\boldsymbol{z}$，即 $\boldsymbol{z}$ 也是 $\boldsymbol{B}$ 的与特征值 μ 相关的特征向量。

证明. **性质 1**：由定义直接得到

$$\boldsymbol{B}\boldsymbol{y} = \boldsymbol{A}\boldsymbol{y} - \lambda \boldsymbol{y}\boldsymbol{y}^T\boldsymbol{y} = \lambda\boldsymbol{y} - \lambda\boldsymbol{y} \cdot 1 = \boldsymbol{0},$$

因为 $\|\boldsymbol{y}\|_2 = 1$ 意味着 $\boldsymbol{y}^T\boldsymbol{y} = 1$。

性质 2：如果 $\boldsymbol{z}$ 是与另一个特征值 μ 相关的 $\boldsymbol{A}$ 的另一个特征向量，且 $\boldsymbol{z}$ 与 $\boldsymbol{y}$ 正交（即 $\boldsymbol{y}^T\boldsymbol{z} = 0$），则

$$\boldsymbol{B}\boldsymbol{z} = \boldsymbol{A}\boldsymbol{z} - \lambda \boldsymbol{y}\boldsymbol{y}^T\boldsymbol{z} = \mu\boldsymbol{z} - \lambda\boldsymbol{y} \cdot 0 = \mu\boldsymbol{z},$$

因为根据特征向量的正交性，$\boldsymbol{y}^T\boldsymbol{z} = 0$。 □

因此，矩阵 $\boldsymbol{B}$ 保留了 $\boldsymbol{A}$ 的所有其他特征值和对应的特征向量，只是将 λ 替换为 0。这样，我们可以对 $\boldsymbol{B}$ 应用幂迭代方法，找到 $\boldsymbol{A}$ 的第二个最大特征值。

例 7.4.1 (对称矩阵的收缩). 考虑对称矩阵

$$\boldsymbol{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix},$$

其特征值为 $\lambda_1 = 3$ 和 $\lambda_2 = 1$，对应的归一化特征向量为 $\boldsymbol{y}^{(1)} = (1/\sqrt{2}, 1/\sqrt{2})^T$ 和 $\boldsymbol{y}^{(2)} = (1/\sqrt{2}, -1/\sqrt{2})^T$。

假设我们已经通过幂迭代方法找到了 $\lambda_1 = 3$ 和 $\boldsymbol{y}^{(1)}$，则收缩矩阵为

$$\boldsymbol{B} = \boldsymbol{A} - 3\boldsymbol{y}^{(1)}(\boldsymbol{y}^{(1)})^T = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} - 3 \cdot \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1/2 & -1/2 \\ -1/2 & 1/2 \end{pmatrix}.$$

可以验证，$\boldsymbol{B}\boldsymbol{y}^{(1)} = \boldsymbol{0}$，且 $\boldsymbol{B}\boldsymbol{y}^{(2)} = \boldsymbol{y}^{(2)}$，因此 $\boldsymbol{B}$ 的特征值为 0 和 1，对应 $\boldsymbol{A}$ 的特征值 3 和 1。

7.4.2 一般矩阵的情况：Duncan 和 Collar 方法

设 $\boldsymbol{A}$ 是任意矩阵（不一定对称），假设我们获得了特征对 $(\lambda, \boldsymbol{y})$，并假设为简化起见，$\boldsymbol{y}$ 的第一个分量等于 1，即 $y_1 = 1$。

记 $\boldsymbol{A}_1$ 为矩阵 $\boldsymbol{A}$ 的第一行，根据特征对 $(\lambda, \boldsymbol{y})$ 的定义，有

$$\boldsymbol{A}_1\boldsymbol{y} = \lambda y_1 = \lambda.$$

然后通过以下方式定义矩阵 $\boldsymbol{B}$：

$$\boldsymbol{B} = \boldsymbol{A} - \boldsymbol{y}\boldsymbol{A}_1. \tag{7.16}$$

命题 7.4.2 (一般矩阵的收缩). 设 $\boldsymbol{A}$ 是任意矩阵，$(\lambda, \boldsymbol{y})$ 是其特征对，且 $y_1 = 1$。则矩阵 $\boldsymbol{B} = \boldsymbol{A} - \boldsymbol{y}\boldsymbol{A}_1$ 具有以下性质：

1. $\boldsymbol{B}$ 的第一行是零，即 $\boldsymbol{B}_1 = \boldsymbol{0}^T$。
2. $\boldsymbol{B}\boldsymbol{y} = \boldsymbol{0}$，即 $\boldsymbol{y}$ 是 $\boldsymbol{B}$ 的零特征值对应的特征向量。
3. 如果 $(\mu, \boldsymbol{z})$ 是 $\boldsymbol{A}$ 的另一个特征对（$\mu \neq \lambda$），且 $z_1 \neq 0$，则 $(\mu, \boldsymbol{z} - \boldsymbol{y})$ 是 $\boldsymbol{B}$ 的特征对。

证明. **性质 1**：$\boldsymbol{B}$ 的第一行是

$$\boldsymbol{B}_1 = \boldsymbol{A}_1 - y_1\boldsymbol{A}_1 = \boldsymbol{A}_1 - 1 \cdot \boldsymbol{A}_1 = \boldsymbol{0}^T.$$

性质 2：根据 (7.16) 和特征对的定义：

$$\boldsymbol{B}\boldsymbol{y} = \boldsymbol{A}\boldsymbol{y} - \boldsymbol{y}\boldsymbol{A}_1\boldsymbol{y} = \lambda\boldsymbol{y} - \lambda\boldsymbol{y} = \boldsymbol{0}.$$

性质 3：如果我们假设 $\boldsymbol{A}$ 的其他特征向量的第一个分量非零，那么可以将它们归一化，使其等于 1。在这些条件下，如果 $(\mu, \boldsymbol{z})$ 是 $\boldsymbol{A}$ 的另一个特征对（$\mu \neq \lambda$），有 $\boldsymbol{A}\boldsymbol{z} = \mu\boldsymbol{z}$，因此

$$\boldsymbol{A}_1\boldsymbol{z} = \mu z_1 = \mu$$

（因为 $z_1 = 1$）。

现在考虑 $\boldsymbol{B}(\boldsymbol{z} - \boldsymbol{y})$：

$$\boldsymbol{B}(\boldsymbol{z} - \boldsymbol{y}) = \boldsymbol{B}\boldsymbol{z} - \boldsymbol{B}\boldsymbol{y} = \boldsymbol{B}\boldsymbol{z} - \boldsymbol{0} = \boldsymbol{B}\boldsymbol{z}.$$

由于 $\boldsymbol{B}$ 的第一行为零，且 $\boldsymbol{B} = \boldsymbol{A} - \boldsymbol{y}\boldsymbol{A}_1$，有

$$\boldsymbol{B}\boldsymbol{z} = \boldsymbol{A}\boldsymbol{z} - \boldsymbol{y}\boldsymbol{A}_1\boldsymbol{z} = \mu\boldsymbol{z} - \mu\boldsymbol{y} = \mu(\boldsymbol{z} - \boldsymbol{y}),$$

第一个等式是性质 2 的后果。因此，$\boldsymbol{B}$ 的其他特征值与 $\boldsymbol{A}$ 的特征值相同，相关的特征向量为 $\boldsymbol{z} - \boldsymbol{y}$。

相反，如果 $(\mu, \boldsymbol{u})$ 是 $\boldsymbol{B}$ 的特征对，如果我们假设 $\mu \neq 0$ 且 $\boldsymbol{A}_1\boldsymbol{u} = \mu - \lambda$（如果 $\boldsymbol{A}_1\boldsymbol{u} = 0$，这总是可能的），那么：

$$\boldsymbol{A}(\boldsymbol{u} + \boldsymbol{y}) = \boldsymbol{B}\boldsymbol{u} + \boldsymbol{y}\boldsymbol{A}_1\boldsymbol{u} + \boldsymbol{A}\boldsymbol{y} = \mu\boldsymbol{u} + (\mu - \lambda)\boldsymbol{y} + \lambda\boldsymbol{y} = \mu(\boldsymbol{u} + \boldsymbol{y}).$$

我们刚刚证明了 $(\boldsymbol{u}+\boldsymbol{y})$ 是与特征值 μ 相关的 $\boldsymbol{A}$ 的特征向量。为了获得除 $(\lambda,\boldsymbol{y})$ 之外的 $\boldsymbol{A}$ 的特征对，我们因此归结为寻找 $\boldsymbol{B}$ 的特征对（非零特征值）。考虑到 $\boldsymbol{B}$ 的特殊结构（其第一行为零），我们归结为在 $\mathbb{R}^{n-1}$ 中计算特征值的问题。 □

注 (收缩方法的计算复杂度). 收缩方法的主要优势在于，一旦找到一个特征对，可以将问题降维。对于对称矩阵，收缩后的矩阵仍然是 $n\times n$，但有一个已知的零特征值。对于一般矩阵，由于第一行为零，实际上可以将问题降为 $(n-1)\times(n-1)$ 的特征值问题，从而减少计算量。

例 7.4.2 (一般矩阵的收缩). 考虑矩阵

$$\boldsymbol{A}=\begin{pmatrix}2 & 1\\ 1 & 2\end{pmatrix},$$

其特征值为 $\lambda_1=3$ 和 $\lambda_2=1$，对应的特征向量为 $\boldsymbol{y}^{(1)}=(1,1)^T$ 和 $\boldsymbol{y}^{(2)}=(1,-1)^T$。

假设我们已经找到了 $\lambda_1=3$ 和 $\boldsymbol{y}^{(1)}=(1,1)^T$。归一化使得第一个分量为 1，得到 $\boldsymbol{y}=(1,1)^T$。

$\boldsymbol{A}$ 的第一行为 $\boldsymbol{A}_1=(2,1)$，因此收缩矩阵为

$$\boldsymbol{B}=\boldsymbol{A}-\boldsymbol{y}\boldsymbol{A}_1=\begin{pmatrix}2 & 1\\ 1 & 2\end{pmatrix}-\begin{pmatrix}1\\ 1\end{pmatrix}\begin{pmatrix}2 & 1\end{pmatrix}=\begin{pmatrix}0 & 0\\ -1 & 1\end{pmatrix}.$$

可以验证，$\boldsymbol{B}$ 的第一行为零，且 $\boldsymbol{B}\boldsymbol{y}=\boldsymbol{0}$。此外，$\boldsymbol{B}(\boldsymbol{y}^{(2)}-\boldsymbol{y})=\boldsymbol{B}(0,-2)^T=(0,-2)^T=1\cdot(0,-2)^T$，因此 $(1,(0,-2)^T)$ 是 $\boldsymbol{B}$ 的特征对，对应 $\boldsymbol{A}$ 的特征值 1。

7.5 方法总结与比较

本节的目标是从适用特征值类型、误差行为与计算复杂度等角度，对本章介绍的特征值迭代方法进行整体比较，并与后续更通用的特征值算法建立联系。

7.5.1 方法特点总结

表 7.1 特征值计算方法比较

方法	适用特征值	计算复杂度	收敛速度	主要优势
幂迭代	主特征值	$\mathscr{O}(n^2)$ 每次迭代	线性	简单，内存需求低
逆幂迭代	最接近 q 的特征值	$\mathscr{O}(n^3)$ 每次迭代	超线性	可精化已知估计
收缩方法	所有特征值	$\mathscr{O}(n^3)$ 每个特征值	取决于基础方法	系统化计算所有特征值

7.5.2 实际应用建议

- **幂迭代方法**：适用于只需要主特征值的场景，如计算矩阵的谱半径、判断线性系统的稳定性等。收敛速度主要由比值 $|\lambda_2/\lambda_1|$ 决定。
- **逆幂迭代方法**：适用于已经拥有某个特征值的粗略估计并希望获得高精度近似的情况。通过选择接近目标特征值的位移 q，可以显著加速收敛。
- **收缩方法**：当需要计算多个特征值时，可以结合幂迭代或逆幂迭代使用。需要注意的是，收缩过程会引入舍入误差，可能影响后续特征值的计算精度。
- **混合策略**：在实际应用中，一种常见做法是先使用幂迭代找到主特征值，然后应用收缩方法降维，再对降维后的矩阵继续使用幂迭代或逆幂迭代。对于高精度需求，可以在最后阶段使用逆幂迭代进行精化。

7.5.3 数值稳定性考虑

注 (数值稳定性). 在实际计算中，需要注意以下问题：

1. **归一化的选择**：使用 2-范数归一化通常比 ∞-范数更稳定，但计算成本稍高。
2. **线性方程组的求解**：在逆幂迭代中，需要求解 $\boldsymbol{B}\boldsymbol{u}^{(k+1)} = \boldsymbol{x}^{(k)}$。如果 $\boldsymbol{A}$ 是稀疏矩阵，应使用稀疏求解器；如果是稠密矩阵，可以使用 LU 分解并重复使用。
3. **收缩的数值误差**：收缩过程会引入舍入误差，特别是当特征值接近时。在实际应用中，可能需要使用更高级的技术（如 QR 算法）来获得更高的数值稳定性。